amrex Documentation

Release 26.00-dev

AMReX Team

Nov 11, 2026

CONTENTS:

1 AMReX Code of Conduct 3

2 AMReX Introduction 7

3 Getting Started 9

4 Building AMReX 15

5 Basics 29

6 Gridding and Load Balancing 77

7 AmrCore Source Code 83

8 Amr Source Code 93

9 Fork-Join 97

10 I/O (Plotfile, Checkpoint) 101

11 Linear Solvers 111

12 Particles 133

13 Fortran Interface 149

14 Python Interface 157

15 Embedded Boundaries 159

16 Discrete Fourier Transform 171

17 Time Integration 179

18 GPU 183

19 Visualization 217

20 Post-Processing 235

21 Debugging 243

22 Runtime Parameters 247

23 AMReX-based Profiling Tools 265

24 External Profiling Tools 275

25 External Frameworks 285

26 Regression Testing 287

27 Frequently Asked Questions 289

Bibliography 293

Index 295

AMReX is a software framework containing all the functionality needed to write massively parallel, block-structured adaptive mesh refinement (AMR) applications. AMReX is freely available on GitHub. The AMReX project website provides additional project information.

AMReX is a project of the High Performance Software Foundation (HPSF), which is part of the nonprofit Linux Foundation.

All of AMReX's development is done in the GitHub repository under the development branch; anyone can see the latest updates. A monthly release is tagged at the beginning of each month.

We are always happy to have users contribute to the AMReX source code. To contribute, issue a pull request against the development branch (details here). Changes at any level are welcome: documentation, bug fixes, new test problems, new solvers, etc. To obtain help, simply post a discussion or an issue on the AMReX GitHub webpage. All participation in the AMReX community is subject to our *Code of Conduct*.

To learn AMReX, there are small stand-alone tutorial codes that demonstrate how to use different parts of the AMReX functionality. Their documentation is available at AMReX Tutorials.

Besides this documentation, there is API documentation generated by Doxygen.

Documentation on migration from BoxLib is available in the AMReX repository at Docs/Migration.

CHAPTER

ONE

AMREX CODE OF CONDUCT

1.1 Our Pledge

We as members, contributors, and leaders pledge to make participation in our community a harassment-free experience for everyone, regardless of age, body size, visible or invisible disability, ethnicity, sex characteristics, gender identity and expression, level of experience, education, socioeconomic status, nationality, personal appearance, race, caste, color, religion, or sexual identity and orientation.

We pledge to act and interact in ways that contribute to an open, welcoming, diverse, inclusive, and healthy community.

1.2 Our Standards

Examples of behavior that contributes to a positive environment for our community include:

- Demonstrating empathy and kindness toward other people
- Being respectful of differing opinions, viewpoints, and experiences
- Giving and gracefully accepting constructive feedback
- Accepting responsibility and apologizing to those affected by our mistakes, and learning from the experience
- Focusing on what is best not just for us as individuals, but for the overall community

Examples of unacceptable behavior include:

- The use of sexualized language or imagery, and sexual attention or advances of any kind
- Trolling, insulting or derogatory comments, and personal or political attacks
- Public or private harassment
- Publishing others' private information, such as a physical or email address, without their explicit permission
- Other conduct which could reasonably be considered inappropriate in a professional setting

1.3 Enforcement Responsibilities

Community leaders are responsible for clarifying and enforcing our standards of acceptable behavior and will take appropriate and fair corrective action in response to any behavior that they deem inappropriate, threatening, offensive, or harmful.

Community leaders have the right and responsibility to remove, edit, or reject comments, commits, code, wiki edits, issues, and other contributions that are not aligned to this Code of Conduct, and will communicate reasons for moderation decisions when appropriate.

1.4 Scope

This Code of Conduct applies within all community spaces, and also applies when an individual is officially representing the community in public spaces. Examples of representing our community include using an official e-mail address, posting via an official social media account, or acting as an appointed representative at an online or offline event.

1.5 Enforcement

Instances of abusive, harassing, or otherwise unacceptable behavior may be reported to the community leaders responsible for enforcement at amrex@lbl.gov. All complaints will be reviewed and investigated promptly and fairly.

All community leaders are obligated to respect the privacy and security of the reporter of any incident. Anyone involved in the reported behavior will recuse themselves from the investigation and decision making about the resolution of the complaint.

1.6 Enforcement Guidelines

Community leaders will follow these Community Impact Guidelines in determining the consequences for any action they deem in violation of this Code of Conduct:

1.6.1 1. Correction

Community Impact: Use of inappropriate language or other behavior deemed unprofessional or unwelcome in the community.

Consequence: A private, written warning from community leaders, providing clarity around the nature of the violation and an explanation of why the behavior was inappropriate. A public apology may be requested.

1.6.2 2. Warning

Community Impact: A violation through a single incident or series of actions.

Consequence: A warning with consequences for continued behavior. No interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, for a specified period of time. This includes avoiding interactions in community spaces as well as external channels like social media. Violating these terms may lead to a temporary or permanent ban.

1.6.3 3. Temporary Ban

Community Impact: A serious violation of community standards, including sustained inappropriate behavior.

Consequence: A temporary ban from any sort of interaction or public communication with the community for a specified period of time. No public or private interaction with the people involved, including unsolicited interaction with those enforcing the Code of Conduct, is allowed during this period. Violating these terms may lead to a permanent ban.

1.6.4 4. Permanent Ban

Community Impact: Demonstrating a pattern of violation of community standards, including sustained inappropriate behavior, harassment of an individual, or aggression toward or disparagement of classes of individuals.

Consequence: A permanent ban from any sort of public interaction within the community.

1.7 Attribution

This Code of Conduct is adapted from the Contributor Covenant, version 2.1, available at https://www.contributor-covenant.org/version/2/1/code_of_conduct.html.

Community Impact Guidelines were inspired by Mozilla's code of conduct enforcement ladder.

For answers to common questions about this code of conduct, see the FAQ at https://www.contributor-covenant.org/faq. Translations are available at https://www.contributor-covenant.org/translations.

CHAPTER

TWO

AMREX INTRODUCTION

AMReX is a publicly available software framework designed for building massively parallel block-structured adaptive mesh refinement (AMR) applications.

Key features of AMReX include:

- C++ and Fortran interfaces
- 1-, 2- and 3-D support
- Support for cell-centered, face-centered, edge-centered, and nodal data
- Support for hyperbolic, parabolic, and elliptic solves on hierarchical adaptive grid structures
- Optional subcycling in time for time-dependent PDEs
- Support for particles
- Support for embedded boundary (cut cell) representations of complex geometries
- Parallelization via flat MPI, OpenMP, hybrid MPI/OpenMP, hybrid MPI/(CUDA or HIP or SYCL), or MPI/MPI
- Parallel I/O
- Plotfile format supported by AmrVis, VisIt, ParaView, and yt.

AMReX is developed at LBNL.

CHAPTER

THREE

GETTING STARTED

In this chapter, we will walk you through two simple examples. It is assumed here that your machine has GNU Make, Python, GCC (including gfortran), and MPI, although AMReX can be built with CMake and other compilers.

3.1 Downloading the Code

The source code is available at https://github.com/AMReX-Codes/amrex. The GitHub repo is our central repo for development. The development branch includes the latest state of the code, and it is tagged as a release on a monthly basis with version number YY.MM (e.g., 17.04). The MM part of the version is incremented every month, and the YY part every year. Bug fix releases are tagged with YY.MM.patch (e.g., 17.04.1).

Each release on GitHub carries a source tarball, `amrex-YY.MM.tar.gz`, together with a signed build-provenance attestation, `amrex-YY.MM.intoto.jsonl`, that records the commit it was made from. To check a downloaded tarball with the GitHub CLI, run

```
gh attestation verify amrex-YY.MM.tar.gz --repo AMReX-Codes/amrex
```

The release also carries `amrex-YY.MM.spdx.json`, an SBOM that lists the GitHub Actions used by AMReX's CI rather than the library's build dependencies.

AMReX can also be obtained using Spack (https://spack.io/). Assuming you have Spack installed, simply type, `spack install amrex`. For more information see the *Spack* section in Building AMReX.

3.2 Example: Hello World

The source code of this example is at `amrex-tutorials/Base/HelloWorld/` and is also shown below.

```
#include <AMReX.H>
#include <AMReX_Gpu.H>
#include <AMReX_OpenMP.H>
#include <AMReX_ParallelDescriptor.H>
#include <AMReX_Print.H>

int main (int argc, char* argv[])
{
    // Initialize MPI and the GPU device, if enabled.
    amrex::Initialize(argc, argv);
    {
        // amrex::Print only prints on the I/O process (rank 0).
        amrex::Print() << "Hello world from AMReX version "
                       << amrex::Version() << "\n";
```

(continues on next page)

(continued from previous page)

```
        amrex::Print() << "  MPI ranks:      " << amrex::ParallelDescriptor::NProcs() <<
↪"\n"
                       << "  OpenMP threads: " << amrex::OpenMP::get_max_threads() << "\n
↪"
                       << "  GPU backend:    "
#if defined(AMREX_USE_CUDA)
                       << "CUDA"
#elif defined(AMREX_USE_HIP)
                       << "HIP"
#elif defined(AMREX_USE_SYCL)
                       << "SYCL"
#else
                       << "none"
#endif
                       << "\n";

        // amrex::AllPrint prints on every process.
        amrex::AllPrint() << "  Rank " << amrex::ParallelDescriptor::MyProc()
                          << " says hello\n";
    }
    amrex::Finalize();
}
```

Usually the first and last statements of the `int main(...)` function of every program should be calling `amrex::Initialize` and `amrex::Finalize`, respectively. They set up and shut down MPI and the GPU device, if enabled. In between, `amrex::Print` prints a string that includes the AMReX version returned by the `amrex::Version` function, the number of MPI processes and OpenMP threads, and the GPU backend. `amrex::Print` prints only on the I/O process, whereas `amrex::AllPrint` prints on every process. AMReX objects must be destroyed before `amrex::Finalize` is called, hence the extra scope. Note that the name of all AMReX header files starts with `AMReX_` (or just AMReX in the case of AMReX.H). All AMReX C++ functions are in the `amrex` namespace.

3.2.1 Building the Code

You build the code in the `amrex-tutorials/Base/HelloWorld/` directory. The `GNUmakefile` there expects AMReX at `../../../amrex`, i.e., cloned next to `amrex-tutorials`; otherwise set the `AMREX_HOME` variable to the path of your AMReX source tree. Typing `make USE_MPI=FALSE` will start the compilation process and result in an executable named `main3d.gnu.ex`. The name shows that the GNU compiler is used and that the executable is built for 3D. Although this simple example code is dimension independent, dimensionality does matter for all non-trivial examples. The build process can be adjusted by modifying the `amrex-tutorials/Base/HelloWorld/GNUmakefile` file. More details on how to build AMReX can be found in *Building AMReX*. The tutorials can also be built with CMake; see Tutorials Getting Started in the tutorials documentation.

3.2.2 Running the Code

The example code can be run as follows,

```
./main3d.gnu.ex
```

The result may look like:

```
Initializing AMReX (26.11-6-gb33ab585fb22)...
AMReX (26.11-6-gb33ab585fb22) initialized
```

(continues on next page)

(continued from previous page)

```
Hello world from AMReX version 26.11-6-gb33ab585fb22
  MPI ranks:      1
  OpenMP threads: 1
  GPU backend:    none
  Rank 0 says hello
AMReX (26.11-6-gb33ab585fb22) finalized
```

The version string means the current commit b33ab585fb22 (note that the first letter g in gb33a.. is not part of the hash) is based on 26.11 with 6 additional commits. If the AMReX working tree is dirty (i.e., there are uncommitted changes), `-dirty` is appended.

In the GNUmakefile there are compilation options for DEBUG mode (less optimized code with more error checking), dimensionality, compiler type, and flags to enable MPI and/or OpenMP parallelism. If there are multiple instances of a parameter, the last instance takes precedence.

3.2.3 Parallelization

Now let's build with MPI by typing `make` (`USE_MPI=TRUE` is the default in the GNUmakefile). This should make an executable named `main3d.gnu.MPI.ex`. Note MPI in the file name. You can then run,

```
mpiexec -n 4 ./main3d.gnu.MPI.ex amrex.v=1
```

The result may look like:

```
Initializing AMReX (26.11-6-gb33ab585fb22)...
MPI initialized with 4 MPI processes
MPI initialized with thread support level 0
AMReX (26.11-6-gb33ab585fb22) initialized
Hello world from AMReX version 26.11-6-gb33ab585fb22
  MPI ranks:      4
  OpenMP threads: 1
  GPU backend:    none
  Rank 0 says hello
  Rank 1 says hello
  Rank 2 says hello
  Rank 3 says hello
AMReX (26.11-6-gb33ab585fb22) finalized
```

The lines printed by `amrex::AllPrint` may appear in any order. If the compilation fails, you are referred to *Building AMReX* for more details on how to configure the build system. The *optional* command line argument `amrex.v=1` sets the AMReX verbosity level to 1 to print the number of MPI processes used. The default verbosity level is 1, and you can pass `amrex.v=0` to turn it off. More details on how runtime parameters are handled can be found in section *ParmParse*.

If you want to build with OpenMP, type `make USE_OMP=TRUE USE_MPI=FALSE`. This should make an executable named `main3d.gnu.OMP.ex`. Note OMP in the file name. Make sure the `OMP_NUM_THREADS` environment variable is set on your system. You can then run,

```
OMP_NUM_THREADS=4 ./main3d.gnu.OMP.ex
```

The result may look like:

```
Initializing AMReX (26.11-6-gb33ab585fb22)...
OMP initialized with 4 OMP threads
```

(continues on next page)

(continued from previous page)

```
AMReX (26.11-6-gb33ab585fb22) initialized
Hello world from AMReX version 26.11-6-gb33ab585fb22
  MPI ranks:      1
  OpenMP threads: 4
  GPU backend:    none
  Rank 0 says hello
AMReX (26.11-6-gb33ab585fb22) finalized
```

Note that you can build with both `USE_MPI=TRUE` and `USE_OMP=TRUE`. You can then run,

```
OMP_NUM_THREADS=4 mpiexec -n 2 ./main3d.gnu.MPI.OMP.ex
```

The result may look like:

```
Initializing AMReX (26.11-6-gb33ab585fb22)...
MPI initialized with 2 MPI processes
MPI initialized with thread support level 0
OMP initialized with 4 OMP threads
AMReX (26.11-6-gb33ab585fb22) initialized
Hello world from AMReX version 26.11-6-gb33ab585fb22
  MPI ranks:      2
  OpenMP threads: 4
  GPU backend:    none
  Rank 0 says hello
  Rank 1 says hello
AMReX (26.11-6-gb33ab585fb22) finalized
```

3.3 Example: Heat Equation Solver

We now look at a more complicated example at `amrex-tutorials/Base/HeatEquation` and show how simulation results can be visualized. This example solves the heat equation,

$$\frac{\partial \phi}{\partial t} = D\nabla^2\phi$$

using forward Euler temporal integration on a periodic domain, with the standard 5-point (in 2D) or 7-point (in 3D) stencil for the Laplacian, e.g., in 2D,

$$\phi_{i,\,j}^{n+1} = \phi_{i,\,j}^{n} + D\Delta t \left(\frac{\phi_{i+1,\,j}^{n} - 2\phi_{i,\,j}^{n} + \phi_{i-1,\,j}^{n}}{\Delta x^2} + \frac{\phi_{i,\,j+1}^{n} - 2\phi_{i,\,j}^{n} + \phi_{i,\,j-1}^{n}}{\Delta y^2} \right).$$

The implementation details of the code are discussed in the Heat Equation tutorial. For now let's just build and run the code, and visualize the results.

3.3.1 Building and Running the Code

To build a 2D executable, go to `amrex-tutorials/Base/HeatEquation` and type `make DIM=2`. This will generate an executable named `main2d.gnu.MPI.ex`. To run it, type,

```
mpiexec -n 2 ./main2d.gnu.MPI.ex inputs plot_int=10
```

Note that the command takes the file `inputs`, followed by a runtime parameter that overrides the value in the file. The calculation solves the heat equation in 2D on a domain with 32×32 cells until $t = 0.01$ and makes a plotfile every 10 steps. When the run finishes, you will have a number of plotfiles, `plt00000, plt00010,` etc, in the directory where

you are running, and the code prints the error against the exact solution. You can control runtime parameters such as the number of cells, the stop time and how often to write plotfiles by setting them in `inputs` or on the command line.

3.4 Visualization

There are several visualization tools that can be used for AMReX plotfiles. One standard tool used within the AMReX community is Amrvis, a package developed and supported by CCSE that is designed specifically for highly efficient visualization of block-structured hierarchical AMR data. (Amrvis can also be used to visualize performance data; see the *AMReX-based Profiling Tools* chapter for further details.) Plotfiles can also be viewed using the VisIt, ParaView, and yt packages. Particle data can be viewed using ParaView. Refer to Chapter on *Visualization* for how to use each of these tools.

3.5 Tutorials

Users new to AMReX may be interested in the AMReX Tutorials, small stand-alone codes organized by topic, from introductory examples of the basic data structures to complete codes meant as starting points for new applications. All but the Python tutorials build with CMake or GNU Make, and each is documented together with its source code.

CHAPTER

FOUR

BUILDING AMREX

In this chapter, we discuss AMReX's build systems. It is also possible to install AMReX using Spack (https://spack.io/). For more information, see the *Spack* section.

There are three ways to use AMReX's build systems. Most AMReX developers use GNU Make. With this approach, there is no installation step; application codes adopt AMReX's build system and compile AMReX while compiling their own codes. This will be discussed in more detail in the section on *Building with GNU Make*.

The second approach is to build and install AMReX as a library using GNU Make (*Building libamrex*); an application code then uses its own build system and links to AMReX as an external library.

Finally, AMReX can also be built with CMake, as detailed in the section on *Building with CMake*.

AMReX requires a C++ compiler that supports the C++20 standard, a Fortran compiler that supports the Fortran 2003 standard, and a C compiler that supports the C99 standard. Prerequisites for building with GNU Make include Python (>= 2.7, including 3) and standard tools available in any Unix-like environment (e.g., Perl and sed). For building with CMake, the minimal requirement is version 3.25.

The minimum toolchain versions we target for C++20 builds are:

- GCC 11 or newer (for both host builds and CUDA host compilers).
- LLVM Clang 14 or newer, including AppleClang 14 on macOS.
- Microsoft Visual Studio 2022 (MSVC 19.44 / 17.14) or newer.
- NVIDIA CUDA Toolkit 12.2 or newer.
- AMD ROCm/HIP 6.0 or newer.
- Intel oneAPI DPC++ 2025.2 or newer.

Please note that we fully support AMReX for Linux systems in general and on the DOE supercomputers (e.g., Perlmutter, Frontier) in particular. Many of our users do build and use AMReX on Macs but we do not have the resources to fully support Mac users.

4.1 Building with GNU Make

In this build approach, you write your own makefiles defining a number of variables and rules. Then you invoke `make` to start the building process. This will result in an executable upon successful completion. The temporary files generated in the building process are stored in a temporary directory named `tmp_build_dir`.

4.1.1 Dissecting a Simple Make File

An example of building with GNU Make can be found in `amrex-tutorials/Base/HelloWorld`. Table 4.1 below shows a list of important variables.

Table 4.1: Important make variables

Variable	Value	Default
AMREX_HOME	Path to amrex	environment
COMP	gnu, cray, ibm, intel, intel-llvm, intel-classic, llvm, or pgi	none
CXXSTD	C++ standard (e.g., `c++20`)	compiler default, at least `c++20`
DEBUG	TRUE or FALSE	FALSE
DIM	1 or 2 or 3	3
PRECISION	DOUBLE or FLOAT	DOUBLE
TEST	TRUE or FALSE	FALSE
USE_ASSERTION	TRUE or FALSE	FALSE
USE_MPI	TRUE or FALSE	FALSE
USE_OMP	TRUE or FALSE	FALSE
USE_CUDA	TRUE or FALSE	FALSE
USE_HIP	TRUE or FALSE	FALSE
USE_SYCL	TRUE or FALSE	FALSE
USE_RPATH	TRUE or FALSE	FALSE
USE_MATH_ERRNO	TRUE or FALSE	FALSE

At the beginning of amrex-tutorials/Base/HelloWorld/GNUmakefile, AMREX_HOME is set to the path to the top directory of AMReX. Note that in the example ?= is a conditional variable assignment operator that only has an effect if AMREX_HOME has not been defined (including in the environment). One can also set AMREX_HOME as an environment variable. For example, in bash, one can set

```
export AMREX_HOME=/path/to/amrex
```

Alternatively, in tcsh, one can set

```
setenv AMREX_HOME /path/to/amrex
```

Note: when setting AMREX_HOME in the GNUmakefile, be aware that ~ does not expand, so AMREX_HOME=~/amrex/ will yield an error.

One must set the COMP variable to choose a compiler. Currently the list of supported compilers includes gnu, cray, ibm, intel, llvm, and pgi.

One could set the DIM variable to either 1, 2, or 3, depending on the dimensionality of the problem. The default dimensionality is 3. AMReX uses double precision by default. One can change to single precision by setting PRECISION=FLOAT. (Particles have an equivalent flag USE_SINGLE_PRECISION_PARTICLES=TRUE/FALSE.)

Variables DEBUG, TEST, USE_MPI and USE_OMP are optional and default to FALSE. The meaning of these variables should be obvious. When DEBUG=TRUE, aggressive compiler optimization flags are turned off and assertions in source code are turned on. For production runs, DEBUG should be set to FALSE. TEST and USE_ASSERTION are set by default in CI and provide slight debugging support, e.g., by initializing default values in FABs. An advanced variable, MPI_THREAD_MULTIPLE, can be set to TRUE to initialize MPI with support for concurrent MPI calls from multiple threads.

Variables USE_CUDA, USE_HIP and USE_SYCL are used for targeting Nvidia, AMD and Intel GPUs, respectively. At most one of the three can be TRUE.

The variable USE_RPATH controls the link mechanism to dependent libraries. If enabled, the library path at link time will be saved as a rpath hint in created binaries. When disabled, dynamic library paths could be provided via export LD_LIBRARY_PATH hints at runtime.

Unless USE_MATH_ERRNO is TRUE, non-debug GCC and Clang builds use -fno-math-errno for AMReX and the application, making math functions faster.

For GCC and Clang, the variable WARN_ALL controls the compiler's warning options. There is also a make variable WARN_ERROR (with default of FALSE) to turn warnings into errors.

When USE_CUDA is TRUE, the make system will try to detect what CUDA arch should be used by running $(CUDA_HOME)/extras/demo_suite/deviceQuery if your computer is unknown. If it fails to detect the CUDA arch, the default value of 70 will be used. The user can override it by make USE_CUDA=TRUE CUDA_ARCH=80 or make USE_CUDA=TRUE AMREX_CUDA_ARCH=80.

After defining these make variables, a number of files, Make.defs, Make.package and Make.rules, are included in the GNUmakefile. AMReX-based applications do not need to include all directories in AMReX; an application which does not use particles, for example, does not need to include files from the Particle directory in its build. In this simple example, we only need to include $(AMREX_HOME)/Src/Base/Make.package. An application code also has its own Make.package file (e.g., ./Make.package in this example) to append source files to the build system using operator +=. Variables for various source files are shown below.

CEXE_sources
: C++ source files. Note that C++ source files are assumed to have a .cpp extension.

CEXE_headers
: C++ headers with .h, .hpp, or .H extension.

cEXE_sources
C source files with .c extension.

cEXE_headers
C headers with .h extension.

f90EXE_sources
Free format Fortran source with .f90 extension.

F90EXE_sources
Free format Fortran source with .F90 extension. Note that these Fortran files will go through preprocessing.

In this simple example, the extra source file, `main.cpp`, is in the current directory that is already in the build system's search path. If this example has files in a subdirectory (e.g., `mysrcdir`), you will then need to add the following to `Make.package`.

```
VPATH_LOCATIONS += mysrcdir
INCLUDE_LOCATIONS += mysrcdir
```

Here `VPATH_LOCATIONS` and `INCLUDE_LOCATIONS` are the search paths for source and header files, respectively.

4.1.2 Tweaking the Make System

The GNU Make build system is located at `amrex/Tools/GNUMake`. You can read `README.md` and the makefiles there for more information. Here we will give a brief overview.

Besides building executables, other common make commands include:

`make cleanconfig`
This removes the executable, .o files, and the temporarily generated files for the given build. Note that one can add additional targets to this rule using the double colon (::)

`make clean` and `make realclean`
These remove all files generated by make for all builds.

`make help`
This shows the rules for compilation.

`make print-xxx`
This shows the value of variable xxx. This is very useful for debugging and tweaking the make system.

Compiler flags are set in `amrex/Tools/GNUMake/comps/`. Note that variables like `CXX` and `CXXFLAGS` are reset in that directory and their values in environment variables are disregarded. However, one could override them with make command-line arguments (e.g., `make CXX=/path/to/my/mpicxx`). Site-specific setups (e.g., the MPI installation) are in `amrex/Tools/GNUMake/sites/`, which includes a generic setup in `Make.unknown`. You can override the setup by having your own `sites/Make.$(host_name)` file, where variable `host_name` is your host name in the make system and can be found via `make print-host_name`. You can also have an `amrex/Tools/GNUMake/Make.local` file to override various variables. See `amrex/Tools/GNUMake/Make.local.template` for more examples of how to customize the build process.

If you need to pass macro definitions to the preprocessor, you can add them to your make file as follows:

```
DEFINES += -Dmyname1 -Dmyname2=mydefinition
```

To link to an additional library, say `foo`, with headers located at `foopath/include` and the library at `foopath/lib`, you can add the following to your make file before the line that includes AMReX's `Make.defs`,

```
INCLUDE_LOCATIONS += foopath/include
LIBRARY_LOCATIONS += foopath/lib
LIBRARIES += -lfoo
```

Alternatively, you can add the following to the end of your GNUmakefile,

```
includes += -I"foopath/include"
libraries += -L"foopath/lib" -lfoo
```

4.1.3 Specifying your own compiler

The amrex/Tools/GNUMake/Make.local file can also specify your own compile commands by setting the variables CXX, CC, FC, and F90. This might be necessary if your system contains non-standard names for compiler commands.

For example, the following amrex/Tools/GNUMake/Make.local builds AMReX using a specific compiler (in this case gcc-8) without MPI. Whenever USE_MPI is true, this configuration defaults to the appropriate mpixxx command:

```
ifeq ($(USE_MPI),TRUE)
  CXX = mpicxx
  CC  = mpicc
  FC  = mpif90
  F90 = mpif90
else
  CXX = g++-8
  CC  = gcc-8
  FC  = gfortran-8
  F90 = gfortran-8
endif
```

For building with MPI, we assume mpicxx, mpif90, etc. provide access to the correct underlying compilers.

4.1.4 MPI Wrapper

When building with MPI, users should usually use MPI compiler wrappers, such as mpicxx, mpicc, and mpif90. These wrappers provide the include and library flags needed by the MPI installation. The generic AMReX GNUmake setup queries these wrappers automatically when USE_MPI=TRUE.

If there are issues with using or querying the MPI wrappers, one can disable AMReX's MPI checking and provide the MPI flags explicitly. Add this before the line that includes AMReX's Make.defs:

```
NO_MPI_CHECKING = TRUE
```

Then add the MPI include and library flags at the end of the GNUmakefile:

```
includes += -I/path/to/mpi/include
libraries += -L/path/to/mpi/lib -lmpi
```

The exact flags can usually be obtained from the MPI C++ wrapper. For Open MPI-family wrappers, use mpicxx -showme. For MPICH-family wrappers, use mpicxx -compile_info for compile and include flags, and mpicxx -link_info for link flags. Compile and include flags belong in includes; library search paths, linker options such as -Xlinker or -Wl,..., and library flags such as -lmpi belong in libraries.

4.1.5 GCC on macOS

The example configuration above should also run on the latest macOS. On macOS the default cxx compiler is clang, whereas the default Fortran compiler is gfortran. Sometimes it is best to avoid mixing compilers; in that case, we can use `Make.local` to force GCC. However, macOS' Xcode ships with its own (woefully outdated) version of GCC (4.2.1). It is therefore recommended to install GCC using the homebrew package manager. Running `brew install gcc` installs gcc with names reflecting the version number. If GCC 8.2 is installed, homebrew installs it as `gcc-8`. AMReX can be built using `gcc-8` (with and without MPI) by using the following `amrex/Tools/GNUMake/Make.local`:

```
CXX = g++-8
CC  = gcc-8
FC  = gfortran-8
F90 = gfortran-8

INCLUDE_LOCATIONS += /usr/local/include
```

The additional `INCLUDE_LOCATIONS` are also installed using homebrew. Note that if you are building AMReX using homebrew's gcc, it is recommended that you use homebrew's mpich. Normally it is fine to simply install its binaries: `brew install mpich`. But if you are experiencing problems, we suggest building mpich using homebrew's gcc: `brew install mpich --cc=gcc-8`.

4.1.6 Fortran

If your code does not use Fortran, you can add `BL_NO_FORT=TRUE` to your makefile to disable Fortran.

4.1.7 ccache

If you use ccache, you can add `USE_CCACHE=TRUE` to your makefile.

4.2 Building libamrex

If an application code already has its own elaborate build system and wants to use AMReX, an external AMReX library can be created instead. In this approach, one runs `./configure`, followed by `make` and `make install`. Other make options include `make distclean` and `make uninstall`. In the top AMReX directory, one can run `./configure -h` to show the various options for the configure script. In particular, one can specify the installation path for the AMReX library using:

```
./configure --prefix=[AMReX library path]
```

This approach is built on the AMReX GNU Make system. Thus the section on *Building with GNU Make* is recommended if any fine-tuning is needed. The result of `./configure` is `GNUmakefile` in the AMReX top directory. One can modify the makefile for fine-tuning.

To compile an application code against the external AMReX library, it is necessary to set appropriate compiler flags and set the library paths for linking. To assist with this, when the AMReX library is built, a configuration file is created in `[AMReX library path]/lib/pkgconfig/amrex.pc`. This file contains the Fortran and C++ flags used to compile the AMReX library as well as the appropriate library and include entries.

The following sample GNU Makefile will compile a `main.cpp` source file against an external AMReX library, using the C++ flags and library paths used to build AMReX:

```
AMREX_LIBRARY_HOME ?= [AMReX library path]

LIBDIR := $(AMREX_LIBRARY_HOME)/lib
INCDIR := $(AMREX_LIBRARY_HOME)/include
```

(continues on next page)

(continued from previous page)

```
COMPILE_CPP_FLAGS ?= $(shell awk '/Cflags:/ {$$1=$$2=""; print $$0}' $(LIBDIR)/pkgconfig/
↪amrex.pc)
COMPILE_LIB_FLAGS ?= $(shell awk '/Libs:/ {$$1=$$2=""; print $$0}' $(LIBDIR)/pkgconfig/
↪amrex.pc)

CFLAGS := -I$(INCDIR) $(COMPILE_CPP_FLAGS)
LFLAGS := -L$(LIBDIR) $(COMPILE_LIB_FLAGS)

all:
        g++ -o main.exe main.cpp $(CFLAGS) $(LFLAGS)
```

4.3 Building with CMake

An alternative to the approach described in the section on *Building libamrex* is to install AMReX as an external library by using the CMake build system. A CMake build is a two-step process. First `cmake` is invoked to create configuration files and makefiles in a chosen directory (`builddir`). This is roughly equivalent to running `./configure` (see the section on *Building libamrex*). Next, the actual build and installation are performed by invoking `make install` from within `builddir`. This installs the library files in a chosen installation directory (`installdir`). If no installation path is provided by the user, AMReX will be installed in `/path/to/amrex/installdir`. The CMake build process is summarized as follows:

```
mkdir /path/to/builddir
cd    /path/to/builddir
cmake [options] -DCMAKE_BUILD_TYPE=[Debug|Release|RelWithDebInfo|MinSizeRel] -DCMAKE_
↪INSTALL_PREFIX=/path/to/installdir  /path/to/amrex
make  install
make  test_install  # optional step to test if the installation is working
```

In the above snippet, `[options]` indicates one or more options for the customization of the build, as described in the subsection on *Customization options*. If the option `CMAKE_BUILD_TYPE` is omitted, `CMAKE_BUILD_TYPE=Release` is assumed. Although the AMReX source could be used as build directory, we advise against doing so. After the installation is complete, `builddir` can be removed.

4.3.1 Customization options

The AMReX build can be customized by setting the value of suitable configuration variables on the command line via the `-D <var>=<value>` syntax, where `<var>` is the variable to set and `<value>` its desired value. For example, one can enable OpenMP support as follows:

```
cmake -DAMReX_OMP=YES -DCMAKE_INSTALL_PREFIX=/path/to/installdir  /path/to/amrex
```

In the example above `<var>=AMReX_OMP` and `<value>=YES`. Configuration variables requiring a boolean value are evaluated to true if they are assigned a value of `1`, `ON`, `YES`, `TRUE`, `Y`. Conversely they are evaluated to false if they are assigned a value of `0`, `OFF`, `NO`, `FALSE`, `N`. Boolean configuration variables are case-insensitive. The list of available options is reported in the *table* below.

Table 4.2: AMReX build options (refer to section *Building GPU Support* for GPU-related options).

Variable Name	Description	Default	Possible values
CMAKE_Fortran_COMPIL	User-defined Fortran compiler		user-defined
CMAKE_CXX_COMPILE	User-defined C++ compiler		user-defined
CMAKE_Fortran_FLAGS	User-defined Fortran flags		user-defined
CMAKE_CXX_FLAGS	User-defined C++ flags		user-defined
CMAKE_CXX_STANDAR	C++ standard	compiler/20	20, 23
AMReX_SPACEDIM	Dimension of AMReX build	3 ;-separated list	"1;2;3"
USE_XSDK_DEFAULTS	Use xSDK defaults settings	NO	YES, NO
AMReX_BUILD_SHARED_LI	Build as shared C++ library	NO (unless xSDK)	YES, NO
AMReX_FASTMATH	Enable fast-math optimizations	NO (CUDA is ON)	YES, NO
AMReX_MATH_ERRNO	Let math functions set errno in non-Debug builds	NO	YES, NO
AMReX_FORTRAN	Enable Fortran language	NO	YES, NO
AMReX_PRECISION	Set the precision of reals	DOUBLE	DOUBLE, SINGLE
AMReX_PIC	Build Position Independent Code	NO	YES, NO
AMReX_IPO	Interprocedural optimization (IPO/LTO)	NO	YES, NO
AMReX_MPI	Build with MPI support	YES	YES, NO
AMReX_SIMD	Enable SIMD Primitives (using vir::stdx::simd)	NO	YES, NO
AMReX_OMP	Build with OpenMP support	NO	YES, NO
AMReX_GPU_BACKEND	Build with on-node, accelerated GPU backend	NONE	NONE, SYCL, HIP, CUDA
AMReX_GPU_RDC	Build with Relocatable Device Code support	YES; NO with HIP and OpenMP	YES, NO
AMReX_FORTRAN_INTERFA	Build Fortran API	NO	YES, NO
AMReX_LINEAR_SOLVERS	Build AMReX linear solvers	YES	YES, NO
AMReX_LINEAR_SOLVERS_	Build AMReX linear solvers for incompressible flow	YES	YES, NO
AMReX_LINEAR_SOLVERS_	Build AMReX linear solvers for electromagnetic solvers	YES	YES, NO
AMReX_AMRDATA	Build data services	NO	YES, NO
AMReX_AMRLEVEL	Build AmrLevel class	YES	YES, NO
AMReX_EB	Build Embedded Boundary support	NO	YES, NO
AMReX_FFT	Build FFT support	NO	YES, NO
AMReX_PARTICLES	Build particle classes	YES	YES, NO

continues on next page

Table 4.2 – continued from previous page

Variable Name	Description	Default	Possible values
AMReX_PARTICLES_PRECIS	Set reals precision in particle classes	Same as AMReX_PRECISION	DOUBLE, SINGLE
AMReX_BASE_PROFILE	Build with basic profiling support	NO	YES, NO
AMReX_TINY_PROFILE	Build with tiny profiling support	NO	YES, NO
AMReX_TRACE_PROFILE	Build with trace-profiling support	NO	YES, NO
AMReX_COMM_PROFILE	Build with comm-profiling support	NO	YES, NO
AMReX_MEM_PROFILE	Build with memory-profiling support	NO	YES, NO
AMReX_TP_PROFILE	Third-party profiling options	IGNORE	CRAYPAT,FORGE,VTUNE
AMReX_TESTING	Build for testing –sets MultiFab initial data to NaN	NO	YES, NO
AMReX_MPI_THREAD_MUL	Concurrent MPI calls from multiple threads	NO	YES, NO
AMReX_PROFPARSER	Build with profile parser support	NO	YES, NO
AMReX_ROCTX	Build with roctx markup profiling support	NO	YES, NO
AMReX_FPE	Build with Floating Point Exceptions checks	NO	YES, NO
AMReX_ASSERTIONS	Build with assertions turned on	NO	YES, NO
AMReX_BOUND_CHECK	Enable bound checking in Array4 class	NO	YES, NO
AMReX_EXPORT_DYNAMIC	Enable backtrace on macOS	NO (unless Darwin)	YES, NO
AMReX_SENSEI	Enable the SENSEI in situ infrastructure	NO	YES, NO
AMReX_NO_SENSEI_AMR_I	Disables the instrumentation in amrex::Amr	NO	YES, NO
AMReX_CONDUIT	Enable Conduit support	NO	YES, NO
AMReX_CATALYST	Enable Catalyst support	NO	YES, NO
AMReX_ASCENT	Enable Ascent support	NO	YES, NO
AMReX_HYPRE	Enable HYPRE interfaces	NO	YES, NO
AMReX_PETSC	Enable PETSc interfaces	NO	YES, NO
AMReX_SUNDIALS	Enable SUNDIALS interfaces	NO	YES, NO
AMReX_HDF5	Enable HDF5-based I/O	NO	YES, NO
AMReX_HDF5_ZFP	Enable compression with ZFP in HDF5-based I/O	NO	YES, NO
AMReX_PLOTFILE_TOOLS	Build and install plotfile postprocessing tools	NO	YES, NO
AMReX_ENABLE_TESTS	Enable CTest suite	NO	YES, NO

continues on next page

Table 4.2 – continued from previous page

Variable Name	Description	Default	Possible values
AMReX_TEST_TYPE	Test type – affects the number of tests	All	All, Small
AMReX_DIFFERENT_COMP	Allow an app to use a different compiler	NO	YES, NO
AMReX_INSTALL	Generate Install Targets	YES	YES, NO
AMReX_PROBINIT	Enable support for probin file	Platform dependent	YES, NO
AMReX_FLATTEN_FOR	Enable flattening of ParallelFor and similar functions for host code	NO	YES, NO
AMReX_COMPILER_DEFAU	Use default inline behavior of compiler, so far relevant for GCC Only	NO for GCC YES otherwise	YES, NO
AMReX_INLINE_LIMIT	Inline limit. Relevant only when AMReX_COMPILER_DEFAU is NO.	43210	Non-negative number

The option `CMAKE_BUILD_TYPE=Debug` implies `AMReX_ASSERTIONS=YES`. In order to turn off assertions in debug mode, `AMReX_ASSERTIONS=NO` must be set explicitly while invoking CMake.

The `CMAKE_C_COMPILER`, `CMAKE_CXX_COMPILER`, and `CMAKE_Fortran_COMPILER` options are used to tell CMake which compiler to use for the compilation of C, C++, and Fortran sources respectively. If those options are not set by the user, CMake will use the system default compilers.

The options `CMAKE_Fortran_FLAGS` and `CMAKE_CXX_FLAGS` allow the user to set their own compilation flags for Fortran and C++ source files, respectively. If `CMAKE_Fortran_FLAGS`/ `CMAKE_CXX_FLAGS` are not set by the user, they will be initialized with the value of the environment variables `FFLAGS`/ `CXXFLAGS`. If neither `FFLAGS`/ `CXXFLAGS` nor `CMAKE_Fortran_FLAGS`/ `CMAKE_CXX_FLAGS` are defined, AMReX default flags are used.

For a detailed explanation of GPU support in AMReX CMake, refer to section *Building GPU Support*.

4.3.2 CMake and macOS

While not strictly necessary when using homebrew on macOS, it is highly recommended that the user specifies `-DCMAKE_C_COMPILER=$(which gcc-X) -DCMAKE_CXX_COMPILER=$(which g++-X)` (where X is the GCC version installed by homebrew) when using gfortran. This is because homebrew's CMake defaults to the Clang C/C++ compiler. Normally Clang plays well with gfortran, but if there are issues, we recommend telling CMake to use gcc for C/C++ as well.

4.3.3 Importing AMReX into your CMake project

In order to import AMReX into your CMake project, you need to include the following line in the appropriate CMakeLists.txt file:

```
find_package(AMReX)
```

Calls to `find_package(AMReX)` will find a valid installation of AMReX, if present, and import its settings and targets into your CMake project. Imported AMReX targets can be linked to any of your targets, after they have been made available following a successful call to `find_package(AMReX)`, by including the following line in the appropriate CMakeLists.txt file:

```
target_link_libraries( <your-target-name> PUBLIC AMReX::<amrex-target-name> )
```

In the above snippet, <amrex-target-name> is any of the targets listed in the table below.

Table 4.3: AMReX targets available for import.

Target name	Description
amrex_1d	AMReX library in 1D
amrex_2d	AMReX library in 2D
amrex_3d	AMReX library in 3D
amrex	AMReX library (alias, points to last dim)
Flags_CXX	C++ flags preset (interface)
Flags_Fortran	Fortran flags preset (interface)
Flags_FPE	Floating Point Exception flags (interface)

The options used to configure the AMReX build may result in certain parts, or components, of the AMReX source code to be excluded from compilation. For example, setting -DAMReX_LINEAR_SOLVERS=no at configure time prevents the compilation of AMReX linear solvers code. Your CMake project can check which components are included in the AMReX library via *find_package*:

```
find_package(AMReX REQUIRED <components-list>)
```

The keyword REQUIRED in the snippet above will cause a fatal error if AMReX is not found, or if it is found but the components listed in <components-list> are not included in the installation. A list of AMReX component names and related configure options are shown in the table below.

Table 4.4: AMReX components.

Option	Component
AMReX_SPACEDIM	1D, 2D, 3D
AMReX_PRECISION	DOUBLE, SINGLE
AMReX_FORTRAN	FORTRAN
AMReX_PIC	PIC
AMReX_MPI	MPI, NOMPI
AMReX_SIMD	SIMD
AMReX_OMP	OMP
AMReX_GPU_BACKEND	CUDA, HIP, SYCL
AMReX_FORTRAN_INTERFACES	FINTERFACES
AMReX_LINEAR_SOLVERS	LSOLVERS
AMReX_LINEAR_SOLVERS_INCFLO	LSOLVERS_INCFLO
AMReX_LINEAR_SOLVERS_EM	LSOLVERS_EM
AMReX_AMRDATA	AMRDATA
AMReX_AMRLEVEL	AMRLEVEL
AMReX_EB	EB
AMReX_FFT	FFT
AMReX_PARTICLES	PARTICLES
AMReX_PARTICLES_PRECISION	PDOUBLE, PSINGLE
AMReX_BASE_PROFILE	BASEP
AMReX_TINY_PROFILE	TINYP
AMReX_TRACE_PROFILE	TRACEP
AMReX_COMM_PROFILE	COMMP
AMReX_MEM_PROFILE	MEMP

continues on next page

Table 4.4 – continued from previous page

Option	Component
AMReX_PROFPARSER	PROFPARSER
AMReX_FPE	FPE
AMReX_ASSERTIONS	ASSERTIONS
AMReX_SENSEI	SENSEI
AMReX_CONDUIT	CONDUIT
AMReX_ASCENT	ASCENT
AMReX_HYPRE	HYPRE
AMReX_PLOTFILE_TOOLS	PFTOOLS

As an example, consider the following CMake code:

```
find_package(AMReX REQUIRED 3D EB)
target_link_libraries(Foo PUBLIC AMReX::amrex_3d)
```

The code in the snippet above checks whether an AMReX installation with 3D and Embedded Boundary support is available on the system. If so, AMReX is linked to target `Foo` and AMReX flags preset is used to compile `Foo`'s C++ sources. If no AMReX installation is found or if the available one was built without 3D or Embedded Boundary support, a fatal error is issued.

You can tell CMake to look for the AMReX library in non-standard paths by setting the environment variable `AMReX_ROOT` to point to the AMReX installation directory or by adding `-DAMReX_ROOT=<path/to/amrex/installation/directory>` to the `cmake` invocation. More details on `find_package` can be found here.

4.4 AMReX on Windows

The AMReX team does development on Linux machines, from laptops to supercomputers. Many people also use AMReX on Macs without issues.

We do not officially support AMReX on Windows, and many of us do not have access to any Windows machines. However, we believe there are no fundamental issues for it to work on Windows.

(1) AMReX mostly uses standard C++20. We run continuous integration tests on Windows with MSVC and Clang compilers.

(2) We use POSIX signal handling when floating-point exceptions, segmentation faults, etc. happen. This capability is not supported on Windows.

(3) Memory profiling is an optional feature in AMReX that is not enabled by default. It reads memory system information from the OS to give us a summary of our memory usage. This is not supported on Windows.

4.5 Spack

AMReX can be installed using the scientific software package manager Spack. Spack supports multiple versions and configurations of AMReX across a wide variety of platforms and environments. To learn more about Spack, visit http://www.spack.io. For system requirements and installation instructions please see https://spack.readthedocs.io/.

Once Spack has been downloaded and the Spack environment enabled, AMReX can be installed with the command:

```
spack install amrex
```

This will install the latest release of AMReX and required dependencies if needed.

AMReX can be built in several combinations of versions and configurations. Available options can be viewed by typing:

```
spack info amrex
```

For example, suppose we want to install the development version of AMReX for a two-dimensional simulation with CUDA support for CUDA architecture `sm_60`. Then we would use the install command:

```
spack install amrex@develop dimensions=2 +cuda cuda_arch=60
```

CHAPTER

FIVE

BASICS

In this chapter, we present the basics of AMReX. The implementation source code is in `amrex/Src/Base/`. Note that AMReX classes and functions are in namespace `amrex`. For clarity, we usually drop `amrex::` in the example codes here. It is also assumed that headers have been properly included. We recommend you study the tutorial in `amrex-tutorials/Base/HeatEquation` while reading this chapter. After reading this chapter, one should be able to develop single-level parallel codes using AMReX. It should also be noted that this is not a comprehensive reference manual.

5.1 Dimensionality

As we have mentioned in *Building AMReX*, the dimensionality of AMReX must be set at compile time. A macro, `AMREX_SPACEDIM`, is defined to be the number of spatial dimensions. C++ codes can also use the `amrex::SpaceDim` variable. Fortran codes can use either the macro and preprocessing or do

```
use amrex_fort_module, only : amrex_spacedim
```

The coordinate directions are zero-based.

5.2 Vector, Array, GpuArray, Array1D, Array2D, and Array3D

`Vector` class in `AMReX_Vector.H` is derived from `std::vector`. The main difference between `Vector` and `std::vector` is that `Vector::operator[]` provides bound checking when compiled with `DEBUG=TRUE`.

`Array` class in `AMReX_Array.H` is simply an alias to `std::array`. AMReX also provides `GpuArray`, a trivial type that works on both host and device. (It was added when the minimal requirement for C++ standard was C++11, for which `std::array` does not work on device.) It also works when compiled just for CPU. Besides `GpuArray`, AMReX also provides GPU-safe `Array1D`, `Array2D` and `Array3D` that are 1, 2 and 3-dimensional fixed size arrays, respectively. These three class templates can have non-zero-based indexing.

5.3 Real

AMReX can be compiled to use either double precision (which is the default) or single precision. `amrex::Real` is typedef'd to either `double` or `float`. C codes can use `amrex_real`. They are defined in `AMReX_REAL.H`. The data type is accessible in Fortran codes via

```
use amrex_fort_module, only : amrex_real
```

In C++, AMReX also provides a user literal `_rt` so that one can have a proper type for constants (e.g., `2.7_rt`).

5.4 Long

AMReX defines a 64-bit integer type `amrex::Long` that is an alias to `long` on Unix-like systems and `long long` on Windows. In C, the type alias is `amrex_long`. In Fortran, one can use `amrex_long` defined in `amrex_fort_module`.

5.5 ParallelDescriptor

AMReX users do not need to use MPI directly. Parallel communication is often handled by the data abstraction classes (e.g., MultiFab; see the section on *FabArray, MultiFab and iMultiFab*). In addition, AMReX has provided the namespace `ParallelDescriptor` in `AMReX_ParallelDescriptor.H`. The frequently used functions include

```
int myproc = ParallelDescriptor::MyProc();  // Return the rank

int nprocs = ParallelDescriptor::NProcs();  // Return the number of processes

if (ParallelDescriptor::IOProcessor()) {
    // Only the I/O process executes this
}

int ioproc = ParallelDescriptor::IOProcessorNumber();  // I/O rank

ParallelDescriptor::Barrier();

// Broadcast 100 ints from the I/O Processor
Vector<int> a(100);
ParallelDescriptor::Bcast(a.data(), a.size(),
                    ParallelDescriptor::IOProcessorNumber())

// See AMReX_ParallelDescriptor.H for many other Reduce functions
ParallelDescriptor::ReduceRealSum(x);
```

Additionally, `amrex_parallelldescriptor_module` in `Src/Base/AMReX_ParallelDescriptor_F.F90` provides a number of functions for Fortran.

5.6 ParallelContext

Users can also use groups of MPI subcommunicators to perform simultaneous physics calculations. These comms are managed by AMReX's `ParallelContext` in `AMReX_ParallelContext.H`. It maintains a stack of `MPI_Comm` handlers. A global comm is placed in the `ParallelContext` stack during AMReX's initialization and additional subcommunicators can be handled by adding comms with `push(MPI_Comm)` and removed using `pop()`. This creates a hierarchy of `MPI_Comm` objects that can be used to split work as the user sees fit. Note that `ParallelDescriptor` by default uses AMReX's base comm, independent of the status of the `ParallelContext` stack.

`ParallelContext` also tracks and returns information about the local (most recently added) and global `MPI_Comm`. The most common access functions are given below. See `AMReX_ParallelContext.H` for a full listing of the available functions.

```
MPI_Comm subCommA = ....;
MPI_Comm subCommB = ....;
// Add a communicator to ParallelContext.
// After these pushes, subCommB becomes the
```

(continues on next page)

(continued from previous page)

```
//     "local" communicator.
ParallelContext::push(subCommA);
ParallelContext::push(subCommB);

// Get Global and Local communicator (subCommB).
MPI_Comm globalComm = ParallelContext::CommunicatorAll();
MPI_Comm localComm  = ParallelContext::CommunicatorSub();

// Get local number of ranks and global IO Processor Number.
int localRanks = ParallelContext::NProcsSub();
int globalIO     = ParallelContext::IOProcessorNumberAll();

if (ParallelContext::IOProcessorSub()) {
    // Only the local I/O process executes this
}

// Translation of global rank to local communicator rank.
// Returns MPI_UNDEFINED if comms do not overlap.
int localRank = ParallelContext::global_to_local_rank(globalrank);

// Translations of MPI rank IDs using integer arrays.
// Returns MPI_UNDEFINED if comms do not overlap.
ParallelContext::global_to_local_rank(local_array, global_array, n);
ParallelContext::local_to_global_rank(global_array, local_array, n);

// Remove the last added subcommunicator.
// This would make "subCommA" the new local communicator.
// Note: The user still needs to free "subCommB".
ParallelContext::pop();
```

5.7 Print

AMReX provides classes in `AMReX_Print.H` for printing messages to standard output or any C++ `ostream`. The main reason one should use them instead of `std::cout` is that messages from multiple processes or threads do not get mixed up. Below are some examples.

```
Print() <<  "x = " << x << "\n"; // Print on I/O processor

Real pi = std::atan(1.0)*4.0;
// Print on rank 3 with precision of 17 digits
// SetPrecision does not modify cout's floating-point decimal precision setting.
Print(3).SetPrecision(17) << pi << "\n";

int oldprec = std::cout.precision(10);
Print() << pi << "\n";  // Print with 10 digits

AllPrint() << "Every process prints\n";  // Print on every process

std::ofstream ofs("my.txt", std::ofstream::out);
Print(ofs) << "Print to a file" << std::endl;
ofs.close();
```

(continues on next page)

(continued from previous page)

```
AllPrintToFile("file.") << "Each process appends to its own file (e.g., file.3)\n";
```

It should be emphasized that `Print()` without any argument only prints on the I/O process. A common mistake in using it for debug printing is that one forgets that for non-I/O processes to print we should use `AllPrint()` or `Print(rank)`.

5.8 Enum Class

Added in version 24.09: Enum class support.

AMReX provides `AMREX_ENUM` in `AMReX_Enum.H` for defining a reflected `enum class`. Use `AMREX_ENUM` at namespace scope.

```
AMREX_ENUM(MyColor, red, green, blue);

void f ()
{
    MyColor color = amrex::getEnum<MyColor>("red"); // MyColor::red
    std::string name = amrex::getEnumNameString(MyColor::blue); // "blue"
    std::vector<std::string> names = amrex::getEnumNameStrings<MyColor>();
    // names = {"red", "green", "blue"};
    std::string class_name = amrex::getEnumClassName<MyColor>(); // "MyColor"
}
```

An enumerator may be given an explicit value, which must be either the name of a preceding enumerator or a decimal, octal or hexadecimal integer literal. Binary literals and expressions such as `1 << 2` are not supported and result in a runtime error.

Use `AMREX_ENUM_IN_CLASS` for an enum class declared inside a class or class template.

Added in version 26.05: `AMREX_ENUM_IN_CLASS`.

```
struct Options {
    AMREX_ENUM_IN_CLASS(Solver, cg, bicgstab, gmres);
};

template <typename T>
struct State {
    AMREX_ENUM_IN_CLASS(Mode, init, run, stop);
};
```

`AMREX_ENUM_IN_CLASS` must be used inside a class definition. Use `AMREX_ENUM` at namespace scope.

5.9 ParmParse

`ParmParse` in AMReX_ParmParse.H is a class providing a database for the storage and retrieval of command-line and input-file arguments. When `amrex::Initialize(int& argc, char**& argv, ...)` is called, the first command-line argument after the executable name (if there is one, and it does not contain the character '=' or start with '-') is taken to be the inputs file, and the contents of the file are used to initialize the `ParmParse` database. The rest of the command-line arguments are also parsed by `ParmParse`, with the exception of those following a '--' which signals command-line sharing (see section *Sharing the Command Line*).

5.9.1 Inputs File

The format of the inputs file is a series of definitions in the form of `prefix.name = value value ...`. For each line, text after # is a comment. For values spanning multiple lines except for tables, one must use \ at the end of a line for continuation, otherwise it's a runtime error. Note that there must be at least one space before the continuation character \. Multiple lines inside a pair of double quotes are considered a single string containing \ns. Here is an example inputs file.

```
nsteps    = 100               # integer
nsteps    = 1000              # nsteps appears a second time
dt        = 0.03              # floating-point number
ncells    = 128 64 32         # a list of 3 ints
xrange    = -0.5 0.5          # a list of 2 reals
title     = "Three Kingdoms"  # a string
hydro.cfl = 0.8               # with prefix, hydro
my_2d_table = \
       # col 1      2      3
         {{ 11.0,  12.0,  13.0 }   # row 1
          { 21.0,  22.0,  23.0 }   # row 2
          { 31.0,  32.0,  33.0 }   # row 3
          { 41.0,  42.0,  43.0 } } # row 4
# or
my_2d_table = # col 1    2    3
                {{ 11,  12,  13 }   # row 1
                 { 21,  22,  23 }   # row 2
                 { 31,  32,  33 }   # row 3
                 { 41,  42,  43 } } # row 4
# or
my_2d_table = # col 1    2    3
                {{ 11,  12,  13 },   # row 1
                 { 21,  22,  23 },   # row 2
                 { 31,  32,  33 },   # row 3
                 { 41,  42,  43 }  } # row 4
```

The following code shows how to use `ParmParse` to get/query the values.

```
ParmParse pp;

int nsteps = 0;
pp.query("nsteps", nsteps);
amrex::Print() << nsteps << "\n";  // 1000

Real dt;
pp.get("dt", dt);  // runtime error if dt is not in inputs

Vector<int> numcells;
// The variable name 'numcells' can be different from parameter name 'ncells'.
pp.getarr("ncells", numcells);
amrex::Print() << numcells.size() << "\n";  // 3

Vector<Real> xr {-1.0, 1.0};
if (!pp.queryarr("xrange", xr)) {
    amrex::Print() << "Cannot find xrange in inputs, "
                   << "so the default {-1.0,1.0} will be used\n";
```

(continues on next page)

(continued from previous page)

```
}

std::string title;
pp.query("title", title);  // query string

ParmParse pph("hydro");  // with prefix 'hydro'
Real cfl;
pph.get("cfl", cfl);    // get parameter with prefix

std::vector<std::vector<double>> my_2d_table;
pp.gettable("my_2d_table", my_2d_table);
```

Note that when there are multiple definitions for a parameter `ParmParse` by default returns the last one. The difference between `query` and `get` should also be noted. It is a runtime error if `get` fails to get the value, whereas `query` returns an error code without generating a runtime error that will abort the run.

The `*arr` functions grow the vector if it is too small, but they never shrink it. If `xr` above held defaults `{-1.0, 0.0, 1.0}` and the inputs had `xrange = 2.0 3.0`, `xr` would become `{2.0, 3.0, 1.0}`. Pass an empty vector to get exactly the values in the inputs.

5.9.2 Math Expressions

Added in version 24.08: Math expression support in `ParmParse`.

`ParmParse` supports math expressions for integers and floating-point numbers. For example,

```
# three numbers. whitespaces inside `""` are okay.
f = 3+4  99  "5 + 6"

# two numbers. `\` is for continuation
g = 3.1+4.1 \
    5.0+6.6

# two numbers unless using [query|get]WithParser
w = 1 -2

my_constants.alpha = 5.
amrex.c = c

# must use [query|get]WithParser
amrex.foo = sin( pi/2 ) + alpha + -amrex.c**2/c^2

# either [query|get] or [query|get]WithParser is okay
amrex.bar = sin(pi/2)+alpha+-amrex.c**2/c^2

geom.prob_lo = 2*sin(pi/4)/sqrt(2)  sin(pi/2)+cos(pi/2)  -(sin(pi*3/2)+cos(pi*3/2))

# three numbers. `\` is for continuation
geom.prob_hi =  "2*sin(pi/4)/sqrt(2)" \
                "sin(pi/2) + cos(pi/2)" \
                -(sin(pi*3/2)+cos(pi*3/2))
```

can be processed by

```
{
    ParmParse::SetParserPrefix("physical_constants");
    ParmParse pp("physical_constants");
    pp.add("c", 299792458.);
    pp.add("pi", 3.14159265358979323846);
}
{
    ParmParse pp;

    double f0 = -1;
    pp.query("f", f0);
    std::cout << " double f = " << f0 << '\n';

    std::vector<int> f;
    pp.queryarr("f", f);
    std::cout << " int f[3] = {" << f[0] << ", " << f[1] << ", "
              << f[2] << "}\n";

    std::vector<double> g;
    pp.queryarr("g", g);
    std::cout << " double g[] = " << g[0] << " " << g[1] << '\n';

    double w;
    pp.query("w", w);
    std::cout << " w = " << w << " with query\n";
    pp.queryWithParser("w", w);
    std::cout << " w = " << w << " with queryParser\n";
}
{
    ParmParse pp("amrex", "my_constants");
    double foo = -1, bar;
    pp.getWithParser("foo", foo);
    pp.get("bar", bar);
    std::cout << " foo = " << foo << ", bar = " << bar << '\n';
}
{
    ParmParse pp;
    std::array<double,3> prob_lo, prob_hi;
    pp.get("geom.prob_lo", prob_lo);
    pp.get("geom.prob_hi", prob_hi);
    std::cout << " double prob_lo[] = {" << prob_lo[0] << ", "
              << prob_lo[1] << ", " << prob_lo[2] << "}\n"
              << " double prob_hi[] = {" << prob_hi[0] << ", "
              << prob_hi[1] << ", " << prob_hi[2] << "}\n";
}
```

The results will be

```
double f = 7
int f[3] = {7, 99, 11}
double g[] = 7.2 11.6
w = 1 with query
w = -1 with queryParser
```

(continues on next page)

(continued from previous page)

```
foo = 5, bar = 5
double prob_lo[] = {1, 1, 1}
double prob_hi[] = {1, 1, 1}
```

Note that spaces are significant for math expressions unless they are inside a pair of " or explicitly parsed by ParmParse::queryWithParser or ParmParse::getWithParser. If the expression contains another variable, it will be looked up by ParmParse. ParmParse's constructor accepts an optional second argument, parser_prefix. When a variable in a math expression is being looked up, it will first try to find it by using the exact name of the variable. If this attempt fails and the ParmParse object has a non-empty non-static member parser_prefix, it will try again, this time looking up the variable by prefixing its name with the value of parser_prefix followed by a .. If this attempt also fails and the ParmParse class has a non-empty static member ParserPrefix (which can be set by ParmParse::SetParserPrefix), it will try again, this time looking up the variable by prefixing its name with the value of ParserPrefix followed by a ..

The variables in ParmParse math expressions are not evaluated until they are referenced. If a variable is defined multiple times, the last occurrence will override previous ones even if it appears after the variable has been referenced. This behavior is demonstrated in the following example.

```
foo.a = 1
foo.b = foo.a
foo.a = 2
```

will become

```
foo.a = 2
foo.b = 2
```

5.9.3 Enum Class

Added in version 24.09: Enum class support in ParmParse.

ParmParse can read enum types declared with AMREX_ENUM or AMREX_ENUM_IN_CLASS (see *Enum Class*).

```
color1 = red
color2 = BLue
```

```
AMREX_ENUM(MyColor, none, red, green, blue);

ParmParse pp;
MyColor c1 = MyColor::none;
MyColor c2 = MyColor::none;

pp.query("color1", c1); // c1 becomes MyColor::red
pp.query_enum_case_insensitive("color2", c2); // c2 becomes MyColor::blue
```

5.9.4 Other Useful Functions

ParmParse provides several additional member functions:

- queryAdd(name, ref) queries the database. If the name is found, its value is stored in ref. If not, the current value of ref is added to the database as a default. This is useful for setting parameter defaults inside functions (see *Setting Parameter Values Inside Functions*).

- `getline(name, ref)` / `queryline(name, ref)` retrieve the entire value list as a single whitespace-joined string. For example, if the input contains `foo = a b c`, then `getline("foo", s)` sets `s` to `"a b c"`, whereas `get("foo", s)` would set `s` to `"a"` only.
- `queryAsDouble(name, ref)` / `getAsDouble(name, ref)` parse the value as a `double` math expression and then cast the result to the type of `ref` (which can be an integer type). This avoids integer truncation issues when the expression involves division.
- `eval(expr)` evaluates a math expression string directly, looking up any unknown symbols in the `ParmParse` database.

```
ParmParse pp;
pp.add("two", 2.0);
double result = pp.eval<double>("two * 3.14"); // 6.28
```

- `remove(name)` removes a parameter from the database.
- `contains(name)` returns `true` if the parameter exists, without retrieving its value.

5.9.5 TOML-Like Features

Our `ParmParse` format is somewhat similar to TOML. A subset of TOML can be processed by `ParmParse`. For a key/value pair, the key starts with an alphabetical letter (a-zA-Z) followed by zero or more allowed characters (alphabetical letters, numbers, `_`, `-`, and `.`).

In TOML, the same key cannot appear more than once. In `ParmParse`, this is allowed and the last one will overwrite previous ones.

```
# Allowed in ParmParse, but do NOT do this if TOML compatibility is needed.
a = 1
a = 2
```

In `ParmParse`, quotes (") are optional for strings, whereas in TOML they are required. In `ParmParse`, a basic string is a raw string. For compatibility, you should avoid special escape sequences in strings. UTF-8 strings are allowed, but it might be better to avoid them unless it's absolutely necessary.

In TOML, arrays are values inside square brackets and they can be nested. `ParmParse` supports TOML-like arrays and arrays of arrays, but not more deeply nested arrays. `ParmParse` also does not support mixed types in an array.

Although math expressions are allowed in `ParmParse`'s native array format, they are not allowed in arrays started by square brackets.

```
a = [3+4, 5+6] # Not allowed
b = 3+4  5+6  # Allowed in ParmParse, but not in TOML. Same as b = [7,11].
```

Tables in TOML are started by headers (e.g., `[amr]` on a line). By default, an entry before the table header is in the nameless top level table. Once a table header is defined, it will continue until another one is introduced.

```
k = 1
p.k = 2

[a]
k = 3   # the full key/value pair is a.k = 3
b.k = 4 # the full key/value pair is a.b.k = 4

[b.c]
k = 5     # the full key/value pair is b.c.k = 5
d.e.k = 6 # the full key/value pair is b.c.d.e.k = 6
```

The file above is the same as the following:

```
k = 1
p.k = 2

a.k = 3
a.b.k = 4

b.c.k = 5
b.c.d.e.k = 6
```

In TOML, it's not allowed to define a table more than once. But it's allowed in ParmParse. If you want compatibility with TOML, you should avoid it.

```
# Allowed in ParmParse, but do NOT do this if TOML compatibility is needed.
[a]
b = 1
[a.b]     # a.b already defined
c = 2
```

```
# Allowed in ParmParse, but do NOT do this if TOML compatibility is needed.
[a]
k = 1
[a]      # [a] already defined
b = 2
```

5.9.6 Including Another File

In ParmParse, you can use FILE = another_file to add the contents of another file to the ParmParse database of key/value pairs. Note that TOML-like table headers defined in the included file have no effect on the current environment. Likewise, the included file also does not inherit the active table header from the including file.

5.9.7 Overriding Parameters with Command-Line Arguments

It is sometimes convenient to override parameters with command-line arguments without modifying the inputs file. The command-line arguments after the inputs file are added later than the file to the database and are therefore used by default. For example, to change the value of ncells and hydro.cfl, one can run with:

```
myexecutable myinputsfile ncells="64 32 16" hydro.cfl=0.9 my_string=\"A String\"
```

Note that the shell strips the quoting characters before the arguments reach main(int argc, char** argv). The quotes in ncells="64 32 16" only ensure that the spaces stay inside a single argument; the literal " never arrives in argv. If you actually need to pass a single string to the code (e.g., for my_string), you must escape them as shown above. The example command line is equivalent to putting the following entries in an inputs file:

```
ncells = 64 32 16
hydro.cfl = 0.9
my_string = "A String"
```

You can also remove the effect of having defined an input parameter at all using the UNSET directive (parameters that are merely overridden will still be caught by pp.contains() checks in code). Specifying keyword = 5 in an input file and then UNSET = keyword subsequently in the input file or from the command line completely removes keyword from the ParmParse table. Multiple keywords can be removed simultaneously (UNSET = key1 key2 key3). if using the UNSET directive with TOML-like input files, note that full parameter names must be used even if the UNSET falls within a TOML table:

```
[x]
a = 1  # Same as x.a = 1 at the root level

UNSET = x.a # full name required to remove x.a entry
```

5.9.8 Setting Defaults via an Environment Variable

You can specify default parameter values using the environment variable `AMREX_DEFAULT_INIT`. This is useful for setting site-wide or machine-specific defaults in HPC job scripts without modifying application input files or command-line arguments.

The parameter value precedence, from highest to lowest, is:

1. Function pointer passed to `amrex::Initialize` (see *Setting Parameter Values Inside Functions* below)
2. Command-line arguments
3. Input file settings
4. `AMREX_DEFAULT_INIT` environment variable

The function pointer is called after all other sources have been parsed, so values it sets with `ParmParse::add` take effect unconditionally. However, the function can use `ParmParse::queryAdd` or check `ParmParse::contains` before calling `ParmParse::add`, effectively lowering its own precedence for that parameter.

Because `AMREX_DEFAULT_INIT` has the lowest precedence, it provides defaults that can always be overridden by any of the other sources.

For example, on a machine where GPU-aware MPI is available, you can add the following to your job script:

```
export AMREX_DEFAULT_INIT="amrex.use_gpu_aware_mpi=1"
```

Multiple parameters can be set in a single value:

```
export AMREX_DEFAULT_INIT="amrex.envfoo=0 amrex.envbar=1 amrex.envabc=1 2 3 amrex.
↪envstr=\"a b c\""
```

The above is equivalent to setting the following in the inputs file:

```
amrex.envfoo = 0
amrex.envbar = 1
amrex.envabc = 1 2 3
amrex.envstr = "a b c"
```

5.9.9 Setting Parameter Values Inside Functions

An application code may want to set values or defaults that differ from those in AMReX in a function. This is accomplished in two steps:

- First, define a function that sets the variable(s).
- Second, pass the name of that function to `amrex::Initialize`.

The example function below sets variable values using different approaches to highlight subtle differences in implementation:

```
void add_par () {
   ParmParse pp("eb2");

   // `variable_one` can be overridden by an inputs file and/or command-line argument.
   if(not pp.contains("variable_one")) {
      pp.add("variable_one",false);
   }

   // Equivalent shorthand for the above: query first, add the default only if not found.
   bool variable_one_v2 = false;
   pp.queryAdd("variable_one_v2", variable_one_v2);

   // The inputs file or command-line arguments for `variable_two` are ignored.
   pp.add("variable_two",false);
};
```

First, this function, `add_par`, declares a `ParmParse` object that will be used to set variables. In the next section of code, we check if the value for `variable_one` has already been set elsewhere before writing to it. This approach prevents the function from overriding a value set in the inputs file or at the command line. The `queryAdd` call for `variable_one_v2` does the same thing more concisely: it queries the database and, only if the parameter is not found, adds the value of its `ref` argument as a default. In the next section, we write a value to `variable_two` without a conditional statement. In this case, we will ignore values for `variable_two` set in the inputs file or as a command-line argument, effectively overriding them with the value set here in the function.

In the second step, we pass the name of the function we defined to `amrex::Initialize`. In the example above the function was called `add_par`, and therefore we write,

```
amrex::Initialize(argc, argv, add_par);
```

Now AMReX will use the user-defined function to appropriately set the desired values.

5.9.10 Sharing the Command Line

In some cases we want AMReX to read only some of the command-line arguments – this happens, for example, when we are going to use AMReX in cooperation with another code package and that code also takes arguments.

Consider:

```
main2d.gnu.exe inputs amrex.v=1 amrex.fpe_trap_invalid=1 -- -tao_monitor
```

In this example, AMReX will parse the inputs file and the optional AMReX command-line arguments, but will ignore arguments after the double dashes.

5.9.11 Command Line Flags

AMReX allows application codes to parse flags such as `-h` or `--help` while still making use of ParmParse for parsing other runtime parameters, but only if the flag is the first argument after the executable. If the first argument following the executable name begins with a dash, AMReX will initialize without reading any parameters and the application code may then parse the command line and handle those cases. Several built-in functions are available to help do this. They are briefly introduced in the table below.

Table 5.1: AMReX functions for parsing the command line.

Function	Type	Purpose
`amrex::get_command()`	String	Get the entire command line.
`amrex::get_argument_count()`	Int	Get the number of command line arguments after the executable.
`amrex::get_command_argument(int n)`	String	Returns the n-th argument after the executable.

5.10 Parser

AMReX provides a parser in `AMReX_Parser.H` that can be used at runtime to evaluate mathematical expressions given in the form of a string. The parser compiles expressions into a compact executable form that can be evaluated efficiently on both CPU and GPU. When `compile` is called, the parser automatically performs constant folding and algebraic simplification.

5.10.1 Supported Operators and Functions

Arithmetic operators: +, -, *, /, ** (power), ^ (power).

Basic math functions: `sqrt`, `abs`, `floor`, `ceil`, `fmod`, `pow`, `min`, `max`.

Exponential and logarithmic: `exp`, `log`, `log10`.

Trigonometric: `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2`.

Hyperbolic: `sinh`, `cosh`, `tanh`, `asinh`, `acosh`, `atanh`.

Special functions: `erf`, `jn(n,x)` (Bessel function of the first kind of order `n`), `yn(n,x)` (Bessel function of the second kind of order `n`), `comp_ellint_1(k)` and `comp_ellint_2(k)` (complete elliptic integrals of the first and second kind). In SYCL builds without the Intel math extension, `jn` and `yn` are host-only, so `compile` aborts for expressions using them; use `compileHost` instead.

The elliptic-integral argument `k` is the modulus, rather than the parameter `m = k*k`. At `k = -1` and `k = 1`, `comp_ellint_2(k)` returns `1`. For direct C++ calls, `amrex::Math::comp_ellint_1` returns positive infinity at these singular endpoints, and `amrex::Math::comp_ellint_2` returns `1`.

Heaviside step function: `heaviside(x1,x2)` returns `0` when `x1 < 0`, `x2` when `x1 = 0`, and `1` when `x1 > 0`.

Conditional: `if(a,b,c)` returns `b` if `a` is nonzero (true), `c` if `a` is zero (false).

Comparison operators: <, >, ==, !=, <=, >=. Comparisons return `1.0` for true and `0.0` for false. They can be chained (e.g., `a < x < b` is equivalent to `a < x and x < b`). Parentheses stop chaining, so `(a < x) < b` compares the result of `a < x` with `b`.

Logical operators: `and`, `or`. A value is considered true if it is nonzero. The precedence of operators follows the convention of the C and C++ programming languages.

5.10.2 Basic Usage

```
Parser parser("if(a<x<b, sin(x)*cos(y)*if(z<0, 1.0, exp(-z)), .3*c**2)");
parser.setConstant("a", ...);
parser.setConstant("b", ...);
parser.setConstant("c", ...);
parser.registerVariables({"x","y","z"});
auto f = parser.compile<3>();  // 3 because there are three variables.
```

(continues on next page)

(continued from previous page)

```
// ParserExecutor<3> f is thread-safe, and can be used in both host and
// device code. It takes 3 arguments in this example. The parser object
// must be alive for f to be valid.
for (int k = 0; ...) {
  for (int j = 0; ...) {
    for (int i = 0; ...) {
      my_array(i,j,k) = f(i*dx, j*dy, k*dz);
    }
  }
}
```

Constants are set with `setConstant` and are substituted into the expression when `compile` is called (i.e., at parser compile time, not C++ compile time). Variables are registered with `registerVariables` and their values are provided at evaluation time. The template parameter to `compile` must match the number of registered variables.

If the compiled executor is only needed on the host, `compileHost` can be used instead of `compile` to skip the GPU copy.

5.10.3 Local Variables

Local automatic variables can be defined in the expression. For example,

```
Parser parser("r2=x*x+y*y; r=sqrt(r2); cos(a+r2)*log(r)");
parser.setConstant("a", ...);
parser.registerVariables({"x","y"});
auto f = parser.compile<2>();  // 2 because there are two variables.
```

An assignment to a local variable must be terminated with `;`. The final expression in the string is the return value, and a trailing `;` after it is optional. One should avoid name conflicts between local variables and the constants set by `setConstant` or the variables registered by `registerVariables`.

5.10.4 User-Defined Functions

User-defined functions with one to four arguments can be registered with the parser. When the parser encounters an unknown function name in the expression, it is treated as a user-defined function. The user must then register a function pointer (for both host and device) before compiling.

```
Parser parser("my_fn(x, y) + z");
parser.registerVariables({"x","y","z"});

// Register host and device function pointers for my_fn (2 arguments).
parser.registerUserFn2("my_fn", my_host_fn, my_device_fn);

auto f = parser.compile<3>();
```

The registration functions are `registerUserFn1`, `registerUserFn2`, `registerUserFn3`, and `registerUserFn4` for functions with one, two, three, and four arguments, respectively. In CPU-only builds, either function pointer argument may be `nullptr` and the non-null one will be used.

Note that user-defined functions are not supported in device code in SYCL builds. `compile` aborts for such an expression; use `compileHost` and evaluate it on the host instead.

5.10.5 Querying the Parser

The `Parser` class provides several methods for introspection:

- `symbols()` returns a `std::set<std::string>` of all variable names found in the expression (excluding local variables and constants already set).
- `expr()` returns the original expression string.
- `print()` prints the abstract syntax tree (AST) of the expression.
- `printExe()` prints the compiled instruction sequence.

5.10.6 Integer Parser

Besides `amrex::Parser` for floating-point numbers, AMReX also provides `amrex::IParser` for integers. The two parsers have a lot of similarity, but floating-point-specific functions (e.g., `sqrt`, `sin`, etc.) are not supported in `IParser`. In addition to `/` whose result truncates towards zero, the integer parser also supports `//` whose result truncates towards negative infinity. Single quotes `'` are allowed as a separator for `IParser` numbers just like C++ integer literals. Additionally, a floating-point-like number with a positive exponent may be accepted as an integer if it is reasonable to do so. For example, it's okay to have `1.234e3`, but `1.234e2` is an error.

Added in version 24.08: Support for `'` and `e` in `IParser` integers.

5.10.7 Thread Safety

Different `Parser` (or `IParser`) objects may be constructed concurrently from multiple host threads. Once compiled, the `ParserExecutor` or `IParserExecutor` returned by `compile` or `compileHost` may be called concurrently from multiple threads or GPU kernels. However, it is usually unnecessary and less efficient to construct one parser per thread merely to evaluate an expression in parallel. Construct, configure, and compile the parser once, then share the compiled executor while keeping the owning parser object alive.

5.11 Initialize and Finalize

As we have mentioned, `Initialize` must be called to initialize the execution environment for AMReX and `Finalize` must be paired with `Initialize` to release the resources used by AMReX. There are three versions of `Initialize`.

```
void Initialize (MPI_Comm mpi_comm,
                 std::ostream& a_osout = std::cout,
                 std::ostream& a_oserr = std::cerr,
                 ErrorHandler a_errhandler = nullptr,
                 int a_device_id = -1);

AMReX* Initialize (int& argc, char**& argv,
                   const std::function<void()>& func_parm_parse,
                   std::ostream& a_osout = std::cout,
                   std::ostream& a_oserr = std::cerr,
                   ErrorHandler a_errhandler = nullptr,
                   int a_device_id = -1);

void Initialize (int& argc, char**& argv, bool build_parm_parse=true,
                 MPI_Comm mpi_comm = MPI_COMM_WORLD,
                 const std::function<void()>& func_parm_parse = {},
                 std::ostream& a_osout = std::cout,
                 std::ostream& a_oserr = std::cerr,
```

(continues on next page)

(continued from previous page)

```
                ErrorHandler a_errhandler = nullptr,
                int a_device_id = -1);
```

`Initialize` checks if MPI has been initialized. If it has, AMReX will duplicate the `MPI_Comm` argument provided by the user in the first and third versions or `MPI_COMM_WORLD` in the second version. If not, AMReX will initialize MPI and ignore the `MPI_Comm` argument. Since AMReX 25.06, MPI types are no longer placed in the global namespace in non-MPI builds to avoid potential conflicts with other libraries. If you want to use MPI types in non-MPI builds for convenience (e.g., calling `amrex::Initialize(MPI_COMM_WORLD)`), you could add `using namespace amrex::mpidatatypes;`.

All three versions accept two optional `std::ostream` parameters, one for standard output in `Print` (section *Print*) and the other for standard error. These streams can be accessed via functions `OutStream()` and `ErrorStream()`. Each version can also take an optional error handler function. If provided, AMReX will use it to handle errors and signals; otherwise, it will use its own function for error and signal handling.

The first version of `Initialize` does not parse the command-line options. The second version builds the ParmParse database (section *ParmParse*), and the third version does so as well unless the `build_parm_parse` parameter is set to `false`. In both the second and third versions, the user may also pass a function that adds parameters to the ParmParse database instead of reading from the command line or input file.

The last optional parameter, `int a_device_id = -1`, applies to GPU builds only. By default, when multiple GPU devices are visible, AMReX automatically selects one for you. In most cases, you should rely on this default behavior and omit the optional argument. However, if another library has already been initialized and assigned processes to specific devices, you may need AMReX to use a particular GPU. In that case, you can pass the desired device ID to `amrex::Initialize`. Conversely, if you want another library to use the device selected by AMReX, you can obtain the device ID by calling `int amrex::Gpu::Device::deviceId()`. If the application initializes MPI itself, see *Assigning GPUs to MPI Ranks*.

Because many AMReX classes and functions (including destructors inserted by the compiler) do not function properly after `amrex::Finalize` is called, it's best to put the code between `amrex::Initialize` and `amrex::Finalize` into its own scope (e.g., a pair of curly braces or a separate function) to make sure resources are properly freed.

5.12 Example of AMR Grids

In block-structured AMR, there is a hierarchy of logically rectangular grids. The computational domain on each AMR level is decomposed into a union of rectangular domains. Fig. 5.1 below shows an example of AMR with three total levels. In the AMReX numbering convention, the coarsest level is level 0. The coarsest grid (*black*) covers the domain with 16^2 cells. Bold lines represent grid boundaries. There are two intermediate resolution grids (*blue*) at level 1 and the cells are a factor of two finer than those at level 0. The two finest grids (*red*) are at level 2 and the cells are a factor of two finer than the level 1 cells. There are 1, 2 and 2 Boxes on levels 0, 1, and 2, respectively. Note that there is no direct parent-child connection. In this chapter, we will focus on single levels.

5.13 Box, IntVect and IndexType

`Box` in AMReX_Box.H is the data structure for representing a rectangular domain in indexing space. In Fig. 5.1, there are 1, 2 and 2 Boxes on levels 0, 1 and 2, respectively. `Box` is a dimension-dependent class. It has lower and upper corners (represented by `IntVect`) and an index type (represented by `IndexType`). A `Box` contains no floating-point data.

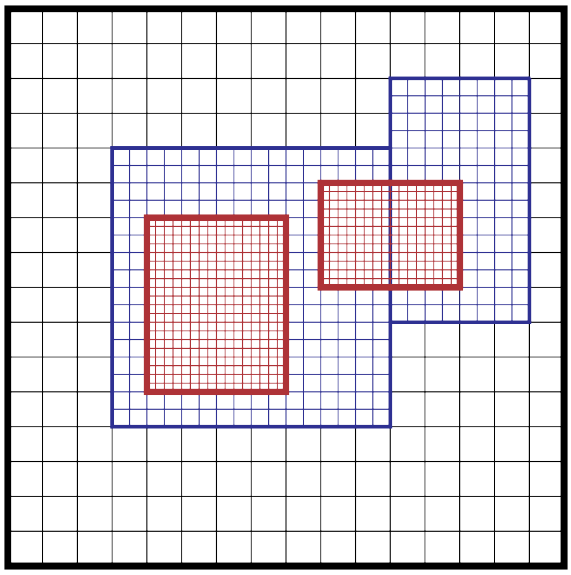


Fig. 5.1: Example of AMR grids. There are three levels in total. There are 1, 2 and 2 Boxes on levels 0, 1, and 2, respectively.

5.13.1 IntVect

`IntVect` is a dimension-dependent class representing an integer vector in `AMREX_SPACEDIM`-dimensional space. An `IntVect` can be constructed as follows:

```
IntVect iv(AMREX_D_DECL(19, 0, 5));
```

Here `AMREX_D_DECL` is a macro that expands `AMREX_D_DECL(19,0,5)` to either `19` or `19, 0` or `19, 0, 5` depending on the number of dimensions. The data can be accessed via `operator[]`, and the internal data pointer can be returned by the function `getVect`. For example

```
for (int idim = 0; idim < AMREX_SPACEDIM; ++idim) {
    amrex::Print() << "iv[" << idim << "] = " << iv[idim] << "\n";
}
const int * p = iv.getVect();  // This can be passed to Fortran/C as an array
```

The class has a static function `TheZeroVector()` returning the zero vector, `TheUnitVector()` returning the unit vector, and `TheDimensionVector (int dir)` returning a reference to a constant `IntVect` that is zero except in the `dir`-direction. Note the direction is zero-based. For index types, `TheCellVector()` and `TheNodeVector()` return the cell-centered and nodal types, `TheFaceVector (int dir)` the type of faces normal to `dir`, and `TheEdgeVector (int dir)` the type of edges parallel to `dir`. `IntVect` has a number of relational operators, ==, !=, <, <=, >, and >= that can be used for lexicographical comparison (e.g., key of `std::map`), and a class `IntVect::shift_hasher` that can be used as a hash function (e.g., for `std::unordered_map`). It also has various arithmetic operators. For example,

```
IntVect iv(AMREX_D_DECL(19, 0, 5));
IntVect iv2(AMREX_D_DECL(4, 8, 0));
iv += iv2;  // iv is now (23,8,5)
iv *= 2;    // iv is now (46,16,10);
```

In AMR codes, one often needs to do refinement and coarsening on `IntVect`. The refinement operation can be done with the multiplication operation. However, the coarsening requires care because of the rounding towards zero behavior of integer division in Fortran, C and C++. For example `int i = -1/2` gives `i = 0`, and what we want is usually `i =`

-1. Thus, one should use the coarsen functions:

```
IntVect iv(AMREX_D_DECL(127,127,127));
IntVect coarsening_ratio(AMREX_D_DECL(2,2,2));
iv.coarsen(2);                  // Coarsen each component by 2
iv.coarsen(coarsening_ratio);  // Component-wise coarsening
const auto& iv2 = amrex::coarsen(iv, 2); // Return an IntVect w/o modifying iv
IntVect iv3 = amrex::coarsen(iv, coarsening_ratio); // iv not modified
```

Finally, we note that `operator<<` is overloaded for `IntVect` and therefore one can call

```
amrex::Print() << iv << "\n";
std::cout << iv << "\n";
```

5.13.2 IndexType

This class defines an index as being cell-based or node-based in each dimension. The default constructor defines a cell-based type in all directions. One can also construct an `IndexType` with an `IntVect` with zero and one representing cell and node, respectively.

```
// Node in x-direction and cell-based in y and z-directions
// (i.e., x-face of numerical cells)
IndexType xface(IntVect{AMREX_D_DECL(1,0,0)});
```

The class provides various functions including

```
// True if the IndexType is cell-based in all directions.
bool cellCentered () const;

// True if the IndexType is cell-based in dir-direction.
bool cellCentered (int dir) const;

// True if the IndexType is node-based in all directions.
bool nodeCentered () const;

// True if the IndexType is node-based in dir-direction.
bool nodeCentered (int dir) const;
```

Index type is a very important concept in AMReX. It is a way of representing the notion of indices i and $i + 1/2$.

5.13.3 Box

A `Box` is an abstraction for defining discrete regions of `AMREX_SPACEDIM`-dimensional indexing space. Boxes have an `IndexType` and two `IntVect`s representing the lower and upper corners. Boxes can exist in positive and negative indexing space. Typical ways of defining a `Box` are

```
IntVect lo(AMREX_D_DECL(64,64,64));
IntVect hi(AMREX_D_DECL(127,127,127));
IndexType typ({AMREX_D_DECL(1,1,1)});
Box cc(lo,hi);        // By default, Box is cell-based.
Box nd(lo,hi+1,typ);  // Construct a nodal Box.
Print() << "A cell-centered Box " << cc << "\n";
Print() << "An all nodal Box    " << nd << "\n";
```

Depending on the dimensionality, the output of the code above is

```
A cell-centered Box ((64,64,64) (127,127,127) (0,0,0))
An all nodal Box    ((64,64,64) (128,128,128) (1,1,1))
```

For simplicity, we will assume it is 3D for the rest of this section. In the output, three integer tuples for each box are the lower corner indices, upper corner indices, and the index types. Note that 0 and 1 denote cell and node, respectively. For each tuple like (64,64,64), the 3 numbers are for 3 directions. The two Boxes in the code above represent different indexing views of the same domain of 64^3 cells. Note that in AMReX convention, the lower side of a cell has the same integer value as the cell centered index. That is, if we consider a cell-based index to represent i, the nodal index with the same integer value represents $i - 1/2$. Fig. 5.2 shows some of the different index types for 2D.

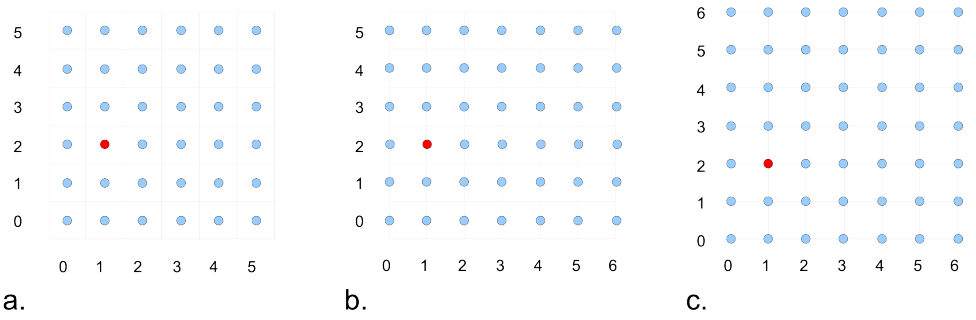


Fig. 5.2: Some of the different index types in two dimensions: (a) cell-centered, (b) x-face-centered (i.e., nodal in x-direction only), and (c) corner/nodal, i.e., nodal in all dimensions.

There are a number of ways of converting a Box from one type to another.

```
Box b0 ({64,64,64}, {127,127,127}); // Index type: (cell, cell, cell)

Box b1 = surroundingNodes(b0);  // A new Box with type (node, node, node)
Print() << b1;                  // ((64,64,64) (128,128,128) (1,1,1))
Print() << b0;                  // Still ((64,64,64) (127,127,127) (0,0,0))

Box b2 = enclosedCells(b1);     // A new Box with type (cell, cell, cell)
if (b2 == b0) {                 // Yes, they are identical.
   Print() << "b0 and b2 are identical!\n";
}

Box b3 = convert(b0, {0,1,0});  // A new Box with type (cell, node, cell)
Print() << b3;                  // ((64,64,64) (127,128,127) (0,1,0))

b3.convert({0,0,1});            // Convert b0 to type (cell, cell, node)
Print() << b3;                  // ((64,64,64) (127,127,128) (0,0,1))

b3.surroundingNodes();          //  Exercise for you
b3.enclosedCells();             //  Exercise for you
```

The internal data of Box can be accessed via various member functions. Examples include:

```
const IntVect& smallEnd () const&;  // Get the small end of the Box
int bigEnd (int dir) const;         // Get the big end in dir direction
const int* loVect () const&;        // Get a const pointer to the lower end
const int* hiVect () const&;        // Get a const pointer to the upper end
```

Boxes can be refined and coarsened. Refinement or coarsening does not change the index type. Some examples are shown below.

```
Box ccbx ({16,16,16}, {31,31,31});
ccbx.refine(2);
Print() << ccbx;                    // ((32,32,32) (63,63,63) (0,0,0))
Print() << ccbx.coarsen(2);         // ((16,16,16) (31,31,31) (0,0,0))

Box ndbx ({16,16,16}, {32,32,32}, {1,1,1});
ndbx.refine(2);
Print() << ndbx;                    // ((32,32,32) (64,64,64) (1,1,1))
Print() << ndbx.coarsen(2);         // ((16,16,16) (32,32,32) (1,1,1))

Box facebx ({16,16,16}, {32,31,31}, {1,0,0});
facebx.refine(2);
Print() << facebx;                  // ((32,32,32) (64,63,63) (1,0,0))
Print() << facebx.coarsen(2);       // ((16,16,16) (32,31,31) (1,0,0))

Box uncoarsenable ({16,16,16}, {30,30,30});
Print() << uncoarsenable.coarsen(2); // ((8,8,8), (15,15,15));
Print() << uncoarsenable.refine(2);  // ((16,16,16), (31,31,31));
                                     // Different from the original!
```

Note that the behavior of refinement and coarsening depends on the index type. A refined `Box` covers the same physical domain as the original `Box`, and a coarsened `Box` also covers the same physical domain if the original `Box` is coarsenable. `Box uncoarsenable` in the example above is considered uncoarsenable because its coarsened version does not cover the same physical domain in the AMR context.

Boxes can grow in one or all directions. There are a number of grow functions. Some are member functions of the `Box` class and others are free functions in the `amrex` namespace.

The `Box` class provides the following member functions testing if a `Box` or `IntVect` is contained within this `Box`. Note that it is a runtime error if the two Boxes have different types.

```
bool contains (const Box& b) const;
bool strictly_contains (const Box& b) const;
bool contains (const IntVect& p) const;
bool strictly_contains (const IntVect& p) const;
```

Another very common operation is the intersection of two Boxes like in the following examples.

```
Box b0 ({16,16,16}, {31,31,31});
Box b1 ({ 0, 0,30}, {23,23,63});
if (b0.intersects(b1)) {                  // true
    Print() << "b0 and b1 intersect.\n";
}

Box b2 = b0 & b1;     // b0 and b1 unchanged
Print() << b2;        // ((16,16,30) (23,23,31) (0,0,0))

Box b3 = surroundingNodes(b0) & surroundingNodes(b1); // b0 and b1 unchanged
Print() << b3;        // ((16,16,30) (24,24,32) (1,1,1))

b0 &= b2;             // b2 unchanged
Print() << b0;        // ((16,16,30) (23,23,31) (0,0,0))
```

(continues on next page)

(continued from previous page)

```
b0 &= b3;             // Runtime error because of type mismatch!
```

5.14 Dim3 and XDim3

`Dim3` and `XDim3` are plain structs with three fields,

```
struct Dim3 { int x; int y; int z; };
struct XDim3 { Real x; Real y; Real z; };
```

One can convert an `IntVect` to `Dim3`,

```
IntVect iv(...);
Dim3 d3 = iv.dim3();
```

`Dim3` always has three fields even when AMReX is built for 1D or 2D. For the example above, the extra fields are set to zero. Given a `Box`, one can get its lower and upper bounds and use them to write dimension agnostic loops.

```
Box bx(...);
Dim3 lo = lbound(bx);
Dim3 hi = ubound(bx);
for     (int k = lo.z; k <= hi.z; ++k) {
  for   (int j = lo.y; j <= hi.y; ++j) {
    for (int i = lo.x; i <= hi.x; ++i) {
    }
  }
}
```

One can also call function `Dim3 length(Box const&)` to return the length of a Box.

5.15 RealBox and Geometry

A `RealBox` stores the physical location in floating-point numbers of the lower and upper corners of a rectangular domain.

The `Geometry` class in AMReX_Geometry.H describes problem domain and coordinate system for rectangular problem domains. A `Geometry` object can be constructed with

```
explicit Geometry (const Box&     dom,
                   const RealBox* rb     = nullptr,
                   int            coord  = -1,
                   int*           is_per = nullptr) noexcept;

Geometry (const Box& dom, const RealBox& rb, int coord,
          Array<int,AMREX_SPACEDIM> const& is_per) noexcept;
```

Here the constructors take a cell-centered `Box` specifying the indexing space domain, a `RealBox` specifying the physical domain, an `int` specifying coordinate system type, and an `int` pointer or array specifying periodicity. If a `RealBox` is not given in the first constructor, AMReX will construct one based on `ParmParse` parameters, `geometry.prob_lo` / `geometry.prob_hi` / `geometry.prob_extent`, where each of the parameter is an array of `AMREX_SPACEDIM` real numbers. See the section on *Geometry* for more details about how to specify these.

The argument for coordinate system is an integer type with valid values being 0 (Cartesian), or 1 (cylindrical), or 2 (spherical). If it is invalid as in the case of the default argument value of the first constructor, AMReX will query the ParmParse database for geometry.coord_sys and use it if one is found. If it cannot find the parameter, the coordinate system is set to 0 (i.e., Cartesian coordinates).

The Geometry class has the concept of periodicity. An argument can be passed specifying periodicity in each dimension. If it is not given in the first constructor, the domain is assumed to be non-periodic unless there is the ParmParse integer array parameter geometry.is_periodic with 0 denoting non-periodic and 1 denoting periodic. Below is an example of defining a Geometry for a periodic rectangular domain of $[-1.0, 1.0]$ in each direction discretized with 64 numerical cells in each direction.

```
int n_cell = 64;

// This defines a Box with n_cell cells in each direction.
Box domain(IntVect{AMREX_D_DECL(       0,        0,        0)},
           IntVect{AMREX_D_DECL(n_cell-1, n_cell-1, n_cell-1)});

// This defines the physical box, [-1,1] in each direction.
RealBox real_box({AMREX_D_DECL(-1.0,-1.0,-1.0)},
                 {AMREX_D_DECL( 1.0, 1.0, 1.0)});

// This says we are using Cartesian coordinates
int coord = 0;

// This sets the boundary conditions to be doubly or triply periodic
Array<int,AMREX_SPACEDIM> is_periodic {AMREX_D_DECL(1,1,1)};

// This defines a Geometry object
Geometry geom(domain, real_box, coord, is_periodic);
```

A Geometry object can return various information of the physical domain and the indexing space domain. For example,

```
const auto problo = geom.ProbLoArray(); // Lower corner of the physical
                                        // domain.  The return type is
                                        // GpuArray<Real,AMREX_SPACEDIM>.
Real yhi = geom.ProbHi(1);              // y-direction upper corner
const auto dx = geom.CellSizeArray();   // Cell size for each direction.
const Box& domain = geom.Domain();      // Index domain
bool is_per = geom.isPeriodic(0);       // Is periodic in x-direction?
if (geom.isAllPeriodic()) {}            // Periodic in all direction?
if (geom.isAnyPeriodic()) {}            // Periodic in any direction?
```

5.16 BoxArray

BoxArray is a class in AMReX_BoxArray.H for storing a collection of Boxes on a single AMR level. One can make a BoxArray out of a single Box and then chop it into multiple Boxes.

```
Box domain(IntVect{0,0,0}, IntVect{127,127,127});
BoxArray ba(domain);  // Make a new BoxArray out of a single Box
Print() << "BoxArray size is " << ba.size() << "\n";  // 1
ba.maxSize(64);       // Chop into boxes of 64^3 cells
Print() << ba;
```

The output is like below,

```
(BoxArray maxbox(8)
       m_ref->m_hash_sig(0)
((0,0,0) (63,63,63) (0,0,0)) ((64,0,0) (127,63,63) (0,0,0))
((0,64,0) (63,127,63) (0,0,0)) ((64,64,0) (127,127,63) (0,0,0))
((0,0,64) (63,63,127) (0,0,0)) ((64,0,64) (127,63,127) (0,0,0))
((0,64,64) (63,127,127) (0,0,0)) ((64,64,64) (127,127,127) (0,0,0)) )
```

It shows that ba now has 8 Boxes, and it also prints out each Box.

In AMReX, BoxArray is a global data structure. It holds all the Boxes in a collection, even though a single process in a parallel run only owns some of the Boxes via domain decomposition. In the example above, a 4-process run may divide the work and each process owns say 2 Boxes (see section on *DistributionMapping*). Each process can then allocate memory for the floating-point data on the Boxes it owns (see sections on *FabArray, MultiFab and iMultiFab* & *BaseFab, FArrayBox, IArrayBox, and Array4*).

BoxArray has an indexing type, just like Box. Each Box in a BoxArray has the same type as the BoxArray itself. In the following example, we show how one can convert BoxArray to a different type.

```
BoxArray cellba(Box(IntVect{0,0,0}, IntVect{63,127,127}));
cellba.maxSize(64);
BoxArray faceba = cellba;       // Make a copy
faceba.convert(IntVect{0,0,1}); // convert to index type (cell, cell, node)
// Return an all node BoxArray
const BoxArray& nodeba = amrex::convert(faceba, IntVect{1,1,1});
Print() << cellba[0] << "\n";  // ((0,0,0) (63,63,63) (0,0,0))
Print() << faceba[0] << "\n";  // ((0,0,0) (63,63,64) (0,0,1))
Print() << nodeba[0] << "\n";  // ((0,0,0) (64,64,64) (1,1,1))
```

As shown in the example above, BoxArray has an operator[] that returns a Box given an index. It should be emphasized that there is a difference between its behavior and the usual behavior of a subscript operator one might expect. The subscript operator in BoxArray returns by **value instead of reference**. This means code like below is meaningless because it modifies a temporary return value.

```
ba[3].coarsen(2);  // DO NOT DO THIS!  Doesn't do what one might expect.
```

BoxArray has a number of member functions that allow the Boxes to be modified. For example,

```
BoxArray& refine (int refinement_ratio);   // Refine each Box in BoxArray
BoxArray& refine (const IntVect& refinement_ratio);
BoxArray& coarsen (int refinement_ratio);  // Coarsen each Box in BoxArray
BoxArray& coarsen (const IntVect& refinement_ratio);
```

We have mentioned at the beginning of this section that BoxArray is a global data structure storing Boxes shared by all processes. The operation of a deep copy is thus undesirable because it is expensive and the extra copy wastes memory. The implementation of the BoxArray class uses std::shared_ptr to an internal container holding the actual Box data. Thus making a copy of BoxArray is a quite cheap operation. The conversion of types and coarsening are also cheap because they can share the internal data with the original BoxArray. In our implementation, function refine does create a new deep copy of the original data. Also note that a BoxArray and its variant with a different type share the same internal data is an implementation detail. We discuss this so that the users are aware of the performance and resource cost. Conceptually we can think of them as completely independent of each other.

```
BoxArray ba(...);  // original BoxArray
BoxArray ba2 = ba; // a copy that shares the internal data with the original
ba2.coarsen(2);    // Modify the copy
// The original copy is unmodified even though they share internal data.
```

For advanced users, AMReX provides functions performing the intersection of a BoxArray and a Box. These functions are much faster than a naive implementation of performing intersection of the Box with each Box in the BoxArray. If one needs to perform those intersections, functions amrex::intersect, BoxArray::intersects and BoxArray::intersections should be used.

5.17 DistributionMapping

DistributionMapping is a class in AMReX_DistributionMapping.H that describes which process owns the data living on the domains specified by the Boxes in a BoxArray. Like BoxArray, there is an element for each Box in DistributionMapping, including the ones owned by other parallel processes. One can construct a DistributionMapping object given a BoxArray,

```
DistributionMapping dm {ba};
```

or by simply making a copy,

```
DistributionMapping dm {another_dm};
```

Note that this class is built using std::shared_ptr. Thus making a copy is relatively cheap in terms of performance and memory resources. This class has a subscript operator that returns the process ID at a given index.

By default, DistributionMapping uses an algorithm based on space filling curve to determine the distribution. One can change the default via the ParmParse parameter DistributionMapping.strategy. KNAPSACK is a common choice that is optimized for load balance. One can also explicitly construct a distribution. The DistributionMapping class allows the user to have complete control by passing an array of integers that represent the mapping of grids to processes.

```
DistributionMapping dm;   // empty object
Vector<int> pmap {...};
// The user fills the pmap array with the values specifying owner processes
dm.define(pmap);  // Build DistributionMapping given an array of process IDs.
```

5.18 BaseFab, FArrayBox, IArrayBox, and Array4

AMReX is a block-structured AMR framework. Although AMR introduces irregularity to the data and algorithms, there is regularity at the block/Box level because each is still logically rectangular, and the data structure at the Box level is conceptually simple. BaseFab is a class template for multi-dimensional array-like data structure on a Box. The template parameter is typically basic types such as Real, int or char. The dimensionality of the array is AMREX_SPACEDIM *plus one*. The additional dimension is for the number of components. The data are internally stored in a contiguous block of memory in Fortran array order (i.e., column-major order) for $(x, y, z, \text{component})$, and each component also occupies a contiguous block of memory because of the ordering. For example, a BaseFab<Real> with 4 components defined on a three-dimensional Box(IntVect{-4,8,32},IntVect{32,64,48}) is like a Fortran array of real(amrex_real), dimension(-4:32,8:64,32:48,0:3). Note that the convention in the C++ part of AMReX is that the component index is zero-based. The code for constructing such an object is as follows:

```
Box bx(IntVect{-4,8,32}, IntVect{32,64,48});
int numcomps = 4;
BaseFab<Real> fab(bx,numcomps);
```

Most applications do not use BaseFab directly, but utilize specialized classes derived from BaseFab. The most common types are FArrayBox in AMReX_FArrayBox.H derived from BaseFab<Real> and IArrayBox in AMReX_IArrayBox.H derived from BaseFab<int>.

These derived classes also obtain many BaseFab member functions via inheritance. We now show some common usages of these functions. To get the Box where a BaseFab or its derived object is defined, one can call

```
const Box& box() const;
```

To get the number of components, one can call

```
int nComp() const;
```

To get a pointer to the array data, one can call

```
T* dataPtr(int n=0);      // Data pointer to the nth component
                          // T is template parameter (e.g., Real)
const T* dataPtr(int n=0) const; // const version
```

The typical usage of the returned pointer is then to pass it to a Fortran or C function that works on the array data (see the section on *Fortran and C++ Kernels*). BaseFab has several functions that set the array data to a constant value. Two examples are as follows.

```
void setVal(T x);         // Set all data to x
// Set the sub-region specified by bx to value x starting from component
// nstart.  ncomp is the total number of component to be set.
void setVal(T x, const Box& bx, int nstart, int ncomp);
```

One can copy data from one BaseFab to another.

```
BaseFab<T>& copy (const BaseFab<T>& src, const Box& srcbox, int srccomp,
                  const Box& destbox, int destcomp, int numcomp);
```

Here the function copies the data from the region specified by srcbox in the source BaseFab src into the region specified by destbox in the destination BaseFab that invokes the function call. Note that although srcbox and destbox may be different, they must be the same size, shape and index type, otherwise a runtime error occurs. The user also specifies how many components (int numcomp) are copied starting at component srccomp in src and stored starting at component destcomp. BaseFab has functions returning the minimum or maximum value.

```
T min (int comp=0) const;  // Minimum value of given component.
T min (const Box& subbox, int comp=0) const; // Minimum value of given
                                             // component in given subbox.
T max (int comp=0) const;  // Maximum value of given component.
T max (const Box& subbox, int comp=0) const; // Maximum value of given
                                             // component in given subbox.
```

BaseFab also has many arithmetic functions. Here are some examples using FArrayBox.

```
Box box(IntVect{0,0,0}, IntVect{63,63,63});
int ncomp = 2;
FArrayBox fab1(box, ncomp);
FArrayBox fab2(box, ncomp);
fab1.setVal(1.0);    // Fill fab1 with 1.0
fab1.mult(10.0, 0);  // Multiply component 0 by 10.0
fab2.setVal(2.0);    // Fill fab2 with 2.0
Real a = 3.0;
fab2.saxpy(a, fab1); // For both components, fab2 <- a * fab1 + fab2
```

These floating-point operations are templated with the parameter RunOn specifying where they run, RunOn::Host or RunOn::Device. When AMReX is built just for CPU, the template parameter has a default value of RunOn::Host

so that the user does not need to specify it for backward compatibility, and if `RunOn::Device` is provided it will be ignored. However, when AMReX is built with GPU support, one must specify where to run for these `BaseFab` functions. For example,

```
fab1.setVal<RunOn::Host>(1.0);    // Fill fab1 with 1.0
fab1.mult<RunOn::Device>(10.0, 0);  // Multiply component 0 by 10.0
```

For more complicated expressions that are not supported, one can write Fortran or C/C++ functions for those (see the section on *Fortran and C++ Kernels*). In C++, one can use `Array4`, which is a class template for accessing `BaseFab` data in a more array like manner using `operator()`. Below is an example of using `Array4`.

```
FArrayBox afab(...), bfab(...);
IArrayBox ifab(...);
Array4<Real> const& a = afab.array();
Array4<Real const> const b = bfab.const_array();
Array4<int const> m = ifab.array();
Dim3 lo = lbound(a);
Dim3 hi = ubound(a);
int nc = a.nComp();
for (int n = 0; n < nc; ++n) {
  for     (int k = lo.z; k <= hi.z; ++k) {
    for   (int j = lo.y; j <= hi.y; ++j) {
      for (int i = lo.x; i <= hi.x; ++i) {
        if (m(i,j,k) > 0) {
          a(i,j,k,n) *= 2.0;
        } else {
          a(i,j,k,n) = 2.0*a(i,j,k,n) + 0.5*(b(i-1,j,k,n)+b(i+1,j,k,n));
        }
      }
    }
  }
}
```

Note that `operator()` of `Array4` takes either three or four arguments. The optional fourth argument has a default value of zero. The two `const`s in `Array4<Real const> const&` have different meaning. The first `const` inside `<>` means the data accessed via `Array4` is read-only, whereas the second `const` means the `Array4` object itself cannot be modified to point to other data. In the example above, neither `m(i,j,k) = 0` nor `b(i,j,k) = 0.0` is allowed. However one is allowed to do `m = ifab2.array()` to assign `m` again, but not to `b`. The behavior is in some sense similar to `double const * const p`.

`BaseFab` and its derived classes are containers for data on `Box`. Recall that `Box` has various types (see the section on *Box, IntVect and IndexType*). The examples in this section so far use the default cell-based type. However, some functions will result in a runtime error if the types mismatch. For example.

```
Box ccbx ({16,16,16}, {31,31,31});           // cell centered box
Box ndbx ({16,16,16}, {31,31,31}, {1,1,1});  // nodal box
FArrayBox ccfab(ccbx);
FArrayBox ndfab(ndbx);
ccfab.setVal(0.0);
ndfab.copy(ccfab);   // runtime error due to type mismatch
```

Because it typically contains a lot of data, BaseFab's copy constructor and copy assignment operator are disabled to prevent performance degradation. However, BaseFab does provide a move constructor. In addition, it also provides a constructor for making an alias of an existing object. Here is an example using `FArrayBox`.

```
FArrayBox orig_fab(box, 4);  // 4-component FArrayBox
// Make a 2-component FArrayBox that is an alias of orig_fab
// starting from component 1.
FArrayBox alias_fab(orig_fab, amrex::make_alias, 1, 2);
```

In this example, the alias FArrayBox has only two components even though the original one has four components. The alias has a sliced component view of the original FArrayBox. This is possible because of the array ordering. However, it is not possible to slice in the real space (i.e., the first AMREX_SPACEDIM dimensions). Note that no new memory is allocated in constructing the alias and the alias contains a non-owning pointer. It should be emphasized that the alias will contain a dangling pointer after the original FArrayBox reaches its end of life. One can also construct an alias BaseFab given an Array4,

```
Array4<Real> const a = orig_fab.array();
FArrayBox alias_fab(a);
```

5.19 FabArray, MultiFab and iMultiFab

FabArray<FAB> is a class template in AMReX_FabArray.H for a collection of FABs on the same AMR level associated with a BoxArray (see the section on *BoxArray*). The template parameter FAB is usually BaseFab<T> or its derived classes (e.g., FArrayBox). However, FabArray can also be used to hold other data structures. To construct a FabArray, a BoxArray must be provided because the FabArray is intended to hold *grid* data defined on a union of rectangular regions embedded in a uniform index space. For example, a FabArray object can be used to hold data for one level as in Fig. 5.1.

FabArray is a parallel data structure in which the data (i.e., FAB) are distributed among parallel processes. For each process, a FabArray contains only the FAB objects owned by that process, and the process operates only on its local data. For operations that require data owned by other processes, remote communications are involved. Thus, the construction of a FabArray requires a DistributionMapping (see the section on *DistributionMapping*) that specifies which process owns which Box. For level 2 (*red*) in Fig. 5.1, there are two Boxes. Suppose there are two parallel processes, and we use a DistributionMapping that assigns one Box to each process. Then the FabArray on each process is built on the BoxArray with both Boxes, but contains only the FAB associated with its process.

In AMReX, there are some specialized classes derived from FabArray. The iMultiFab class in AMReX_iMultiFab.H is derived from FabArray<IArrayBox>. The most commonly used FabArray kind class is MultiFab in AMReX_MultiFab.H derived from FabArray<FArrayBox>. In the rest of this section, we use MultiFab as example. However, these concepts are equally applicable to other types of FabArrays. There are many ways to define a MultiFab. For example,

```
// ba is BoxArray
// dm is DistributionMapping
int ncomp = 4;
int ngrow = 1;
MultiFab mf(ba, dm, ncomp, ngrow);
```

Here we define a MultiFab with 4 components and 1 ghost cell. A MultiFab contains a number of FArrayBoxes (see the section on *BaseFab, FArrayBox, IArrayBox, and Array4*) defined on Boxes grown by the number of ghost cells (1 in this example). That is the Box in the FArrayBox is not exactly the same as in the BoxArray. If the BoxArray has a Box{(7,7,7) (15,15,15)}, the one used for constructing FArrayBox will be Box{(6,6,6) (16,16,16)} in this example. For cells in FArrayBox, we call those in the original Box **valid cells** and the grown part **ghost cells**. Note that FArrayBox itself does not have the concept of ghost cells. Ghost cells are a key concept of MultiFab, however, that allows for local operations on ghost cell data originated from remote processes. We will discuss how to fill ghost cells with data from valid cells later in this section. MultiFab also has a default constructor. One can define an empty MultiFab first and then call the define function as follows.

```
MultiFab mf;
// ba is BoxArray
// dm is DistributionMapping
int ncomp = 4;
int ngrow = 1;
mf.define(ba, dm, ncomp, ngrow);
```

Given an existing MultiFab, one can also make an alias MultiFab as follows.

```
// orig_mf is an existing MultiFab
int start_comp = 3;
int num_comps = 1;
MultiFab alias_mf(orig_mf, amrex::make_alias, start_comp, num_comps);
```

Here the first integer parameter is the starting component in the original MultiFab that will become component 0 in the alias MultiFab and the second integer parameter is the number of components in the alias. It's a runtime error if the sum of the two integer parameters is greater than the number of the components in the original MultiFab. Note that the alias MultiFab has exactly the same number of ghost cells as the original MultiFab.

We often need to build new MultiFabs that have the same BoxArray and DistributionMapping as a given MultiFab. Below is an example of how to achieve this.

```
// mf0 is an already defined MultiFab
const BoxArray& ba = mf0.boxArray();
const DistributionMapping& dm = mf0.DistributionMap();
int ncomp = mf0.nComp();
int ngrow = mf0.nGrow();
MultiFab mf1(ba,dm,ncomp,ngrow);  // new MF with the same ncomp and ngrow
MultiFab mf2(ba,dm,ncomp,0);      // new MF with no ghost cells
// new MF with 1 component and 2 ghost cells
MultiFab mf3(mf0.boxArray(), mf0.DistributionMap(), 1, 2);
```

As we have repeatedly mentioned in this chapter that Box and BoxArray have various index types. Thus, MultiFab also has an index type that is obtained from the BoxArray used for defining the MultiFab. It should be noted again that index type is a very important concept in AMReX. Let's consider an example of a finite-volume code, in which the state is defined as cell averaged variables and the fluxes are defined as face averaged variables.

```
// ba is cell-centered BoxArray
// dm is DistributionMapping
int ncomp = 3;  // Suppose the system has 3 components
int ngrow = 0;  // no ghost cells
MultiFab state(ba, dm, ncomp, ngrow);
MultiFab xflux(amrex::convert(ba, IntVect{1,0,0}), dm, ncomp, 0);
MultiFab yflux(amrex::convert(ba, IntVect{0,1,0}), dm, ncomp, 0);
MultiFab zflux(amrex::convert(ba, IntVect{0,0,1}), dm, ncomp, 0);
```

Here all MultiFabs use the same DistributionMapping, but their BoxArrays have different index types. The state is cell-based, whereas the fluxes are on the faces. Suppose the cell-based BoxArray contains a Box{(8,8,16), (15,15,31)}. The state on that Box is conceptually a Fortran Array with the dimension of (8:15,8:15,16:31,0:2). The fluxes are arrays with slightly different indices. For example, the x-direction flux for that Box has the dimension of (8:16,8:15,16:31,0:2). Note there is an extra element in x-direction.

The MultiFab class provides many functions performing common arithmetic operations on a MultiFab or between MultiFabs built with the *same* BoxArray and DistributionMap. For example,

```
Real dmin = mf.min(3);   // Minimum value in component 3 of MultiFab mf
                         // no ghost cells included
Real dmax = mf.max(3,1); // Maximum value in component 3 of MultiFab mf
                         // including 1 ghost cell
mf.setVal(0.0);          // Set all values to zero including ghost cells

MultiFab::Add(mfdst, mfsrc, sc, dc, nc, ng);  // Add mfsrc to mfdst
MultiFab::Copy(mfdst, mfsrc, sc, dc, nc, ng); // Copy from mfsrc to mfdst
// MultiFab mfdst: destination
// MultiFab mfsrc: source
// int      sc   : starting component index in mfsrc for this operation
// int      dc   : starting component index in mfdst for this operation
// int      nc   : number of components for this operation
// int      ng   : number of ghost cells involved in this operation
//                 mfdst and mfsrc may have more ghost cells
```

The infinity norm, `mf.norminf(comp, ncomp, nghost)`, counts a NaN as infinity, so a NaN in the data always shows up in the norm. To test single values, use `amrex::isnan`, `amrex::isinf` and `amrex::isfinite`. Unlike the `std::` versions, they work in builds with fast-math optimizations (e.g., `AMReX_FASTMATH=ON`), where the compiler may assume that NaNs and infinities never occur.

We refer the reader to `amrex/Src/Base/AMReX_MultiFab.H` and `amrex/Src/Base/AMReX_FabArray.H` for more details. It should be noted again it is a runtime error if the two `MultiFab`s passed to functions like `MultiFab::Copy` are not built with the *same* `BoxArray` (including index type) and `DistributionMapping`.

It is usually the case that the Boxes in the `BoxArray` used for building a `MultiFab` are non-intersecting except that they can be overlapping due to nodal index type. However, `MultiFab` can have ghost cells, and in that case FArrayBoxes are defined on Boxes larger than the Boxes in the `BoxArray`. Parallel communication is then needed to fill the ghost cells with valid cell data from other FArrayBoxes possibly on other parallel processes. The function for performing this type of communication is `FillBoundary`.

```
MultiFab mf(...parameters omitted...);
Geometry geom(...parameters omitted...);
mf.FillBoundary();                    // Fill ghost cells for all components
                                      // Periodic boundaries are not filled.
mf.FillBoundary(geom.periodicity());  // Fill ghost cells for all components
                                      // Periodic boundaries are filled.
mf.FillBoundary(2, 3);        // Fill 3 components starting from component 2
mf.FillBoundary(geom.periodicity(), 2, 3);
```

Note that `FillBoundary` does not modify any valid cells. Also note that `MultiFab` itself does not have the concept of periodic boundary, but `Geometry` has, and we can provide that information so that periodic boundaries can be filled as well. You might have noticed that a ghost cell could overlap with multiple valid cells from different FArrayBoxes in the case of nodal index type. In that case, it is unspecified which valid cell's value is used to fill the ghost cell. It ought to be the case the values in those overlapping valid cells are the same up to roundoff errors. If a ghost cell does not overlap with any valid cells, its value will not be modified by `FillBoundary`.

Another type of parallel communication is copying data from one `MultiFab` to another `MultiFab` with a different `BoxArray` or the same `BoxArray` with a different `DistributionMapping`. The data copy is performed on the regions of intersection. The most generic interface for this is

```
mfdst.ParallelCopy(mfsrc, compsrc, compdst, ncomp, ngsrc, ngdst, period, op);
```

Here `mfdst` and `mfsrc` are destination and source MultiFabs, respectively. Parameters `compsrc`, `compdst`, and `ncomp` are integers specifying the range of components. The copy is performed on `ncomp` components starting from compo-

nent compsrc of mfsrc and component compdst of mfdst. Parameters ngsrc and ngdst specify the number of ghost cells involved for the source and destination, respectively. Parameter period is optional, and by default no periodic copy is performed. Like FillBoundary, one can use Geometry::periodicity() to provide the periodicity information. The last parameter is also optional and is set to FabArrayBase::COPY by default. One could also use FabArrayBase::ADD. This determines whether the function copies or adds data from the source to the destination. Similar to FillBoundary, if a destination cell has multiple cells as source, it is unspecified which source cell is used in FabArrayBase::COPY, and, for FabArrayBase::ADD, the multiple values are all added to the destination cell. This function has two variants, in which the periodicity and operation type are also optional.

```
mfdst.ParallelCopy(mfsrc, period, op);  // mfdst and mfsrc must have the same
                                        // number of components
mfdst.ParallelCopy(mfsrc, compsrc, compdst, ncomp, period, op);
```

Here the number of ghost cells involved is zero, and the copy is performed on all components if unspecified (assuming the two MultiFabs have the same number of components).

Both ParallelCopy(...) and FillBoundary(...) are blocking calls. They will only return when the communication is completed and the destination MultiFab is guaranteed to be properly updated. AMReX also provides non-blocking versions of these calls to allow users to overlap communication with calculation and potentially improve overall application performance.

The non-blocking calls are used by calling the ***_nowait(...) function to begin the comm operation, followed by the ***_finish() function at a later time to complete it. For example:

```
mfA.ParallelCopy_nowait(mfsrc, period, op);

// ... Any overlapping calc work here on other data, e.g.
mfB.setVal(0.0);

mfA.ParallelCopy_finish();

mfB.FillBoundary_nowait(period);
// ... Overlapping work here
mfB.FillBoundary_finish();
```

All function signatures of the blocking calls are also available in the non-blocking calls and should be used in the *nowait* function. The *finish* functions take no parameters, as the required data is stored during *nowait* and retrieved. Users that choose to use non-blocking calls must ensure the calls are properly used to avoid race conditions, which typically means not interacting with the MultiFab between the _nowait and _finish calls.

5.20 MFIter and Tiling

In this section, we will first show how MFIter works without tiling. Then we will introduce the concept of logical tiling. Finally we will show how logical tiling can be launched via MFIter.

5.20.1 MFIter without Tiling

In the section on *FabArray, MultiFab and iMultiFab*, we have shown some of the arithmetic functionalities of MultiFab, such as adding two MultiFabs together. In this section, we will show how you can operate on the MultiFab data with your own functions. AMReX provides an iterator, MFIter for looping over the FArrayBoxes in MultiFabs. For example,

```
for (MFIter mfi(mf); mfi.isValid(); ++mfi) // Loop over grids
{
```

(continues on next page)

(continued from previous page)

```
    // This is the valid Box of the current FArrayBox.
    // By "valid", we mean the original ungrown Box in BoxArray.
    const Box& box = mfi.validbox();

    // A reference to the current FArrayBox in this loop iteration.
    FArrayBox& fab = mf[mfi];

    // Obtain Array4 from FArrayBox.  We can also do
    //     Array4<Real> const& a = mf.array(mfi);
    Array4<Real> const& a = fab.array();

    // Call function f1 to work on the region specified by box.
    // Note that the whole region of the Fab includes ghost
    // cells (if there are any), and is thus larger than or
    // equal to "box".
    f1(box, a);
}
```

Here function `f1` might be something like below,

```
void f1 (Box const& bx, Array4<Real> const& a)
{
   const auto lo = lbound(bx);
   const auto hi = ubound(bx);
   for     (int k = lo.z; k <= hi.z; ++k) {
     for   (int j = lo.y; j <= hi.y; ++j) {
       for (int i = lo.x; i <= hi.x; ++i) {
         a(i,j,k) = ...
       }
     }
   }
}
```

`MFIter` only loops over grids owned by this process. For example, suppose there are 5 Boxes in total and processes 0 and 1 own 2 and 3 Boxes, respectively. That is the MultiFab on process 0 has 2 FArrayBoxes, whereas there are 3 FArrayBoxes on process 1. Thus the numbers of iterations of MFIter are 2 and 3 on processes 0 and 1, respectively.

In the example above, `MultiFab` is assumed to have a single component. If it has multiple components, we can call `int nc = mf.nComp()` or `int nc = a.nComp()` to get the number of components.

There is only one `MultiFab` in the example above. Below is an example of working with multiple MultiFabs. Note that these two MultiFabs are not necessarily built on the same `BoxArray`. But they must have the same `DistributionMapping`, and their BoxArrays are typically related (e.g., they are different due to index types).

```
// U and F are MultiFabs
for (MFIter mfi(F); mfi.isValid(); ++mfi) // Loop over grids
{
    const Box& box = mfi.validbox();

    Array4<Real const> const& u = U.const_array(mfi);
    Array4<Real      > const& f = F.array(mfi);

    f2(box, u, f);
}
```

Here function f2 might be something like below,

```
void f1 (Box const& bx, Array4<Real const> const& u,
         Array4<Real> const& f)
{
   const auto lo = lbound(bx);
   const auto hi = ubound(bx);
   const int nf = f.nComp();
   for (int n = 0; n < nf; ++n) {
     for     (int k = lo.z; k <= hi.z; ++k) {
       for   (int j = lo.y; j <= hi.y; ++j) {
         for (int i = lo.x; i <= hi.x; ++i) {
           f(i,j,k,n) = ... u(i,j,k,n) ...
         }
       }
     }
   }
}
```

5.20.2 MFIter with Tiling

Tiling, also known as cache blocking, is a well known loop transformation technique for improving data locality. This is often done by transforming the loops into tiling loops that iterate over tiles and element loops that iterate over the data elements within a tile. For example, the original loops might look like this in Fortran

```
do k = kmin, kmax
  do j = jmin, jmax
    do i = imin, imax
      A(i,j,k) = B(i+1,j,k)+B(i-1,j,k)+B(i,j+1,k)+B(i,j-1,k) &
                 +B(i,j,k+1)+B(i,j,k-1)-6.0d0*B(i,j,k)
    end do
  end do
end do
```

And the manually tiled loops might look like

```
jblocksize = 11
kblocksize = 16
jblocks = (jmax-jmin+jblocksize-1)/jblocksize
kblocks = (kmax-kmin+kblocksize-1)/kblocksize
do kb = 0, kblocks-1
  do jb = 0, jblocks-1
    do k = kb*kblocksize, min((kb+1)*kblocksize-1,kmax)
      do j = jb*jblocksize, min((jb+1)*jblocksize-1,jmax)
        do i = imin, imax
          A(i,j,k) = B(i+1,j,k)+B(i-1,j,k)+B(i,j+1,k)+B(i,j-1,k) &
                     +B(i,j,k+1)+B(i,j,k-1)-6.0d0*B(i,j,k)
        end do
      end do
    end do
  end do
end do
```

As we can see, to manually tile individual loops is very labor-intensive and error-prone for large applications. AMReX

has incorporated the tiling construct into `MFIter` so that the application codes can get the benefit of tiling easily. An `MFIter` loop with tiling is almost the same as the non-tiling version. The first example in (see the previous section on *MFIter without Tiling*) requires only two minor changes:

1. passing `true` when defining `MFIter` to indicate tiling;
2. calling `tilebox` instead of `validbox` to obtain the work region for the loop iteration.

```
//               * true *  turns on tiling
for (MFIter mfi(mf,true); mfi.isValid(); ++mfi) // Loop over tiles
{
    //                   tilebox() instead of validbox()
    const Box& box = mfi.tilebox();

    FArrayBox& fab = mf[mfi];
    Array4<Real> const& a = fab.array();
    f1(box, a);
}
```

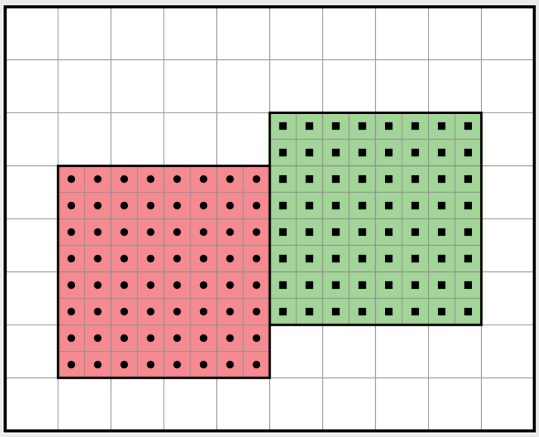
The second example in the previous section on *MFIter without Tiling* also requires only two minor changes.

```
//               * true *  turns on tiling
for (MFIter mfi(F,true); mfi.isValid(); ++mfi) // Loop over tiles
{
    //                   tilebox() instead of validbox()
    const Box& box = mfi.tilebox();

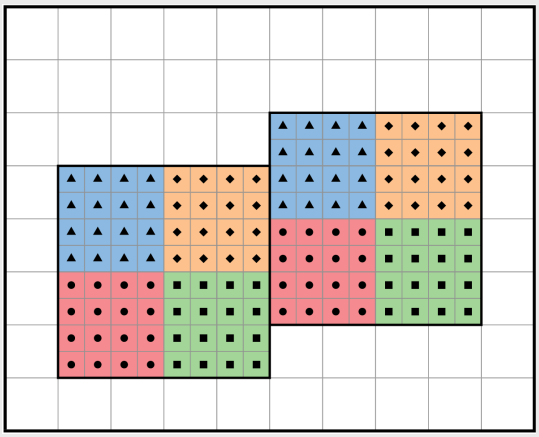
    Array4<Real const> const& u = U.const_array(mfi);
    Array4<Real      > const& f = F.array(mfi);
    f2(box, u, f);
}
```

The kernels functions like `f1` and `f2` in the two examples here usually require very little changes.

Table 5.2: Comparison of `MFIter` with (right) and without (left) tiling.



Example of cell-centered valid boxes. There are two valid boxes in this example. Each has 8^2 cells.



Example of cell-centered tile boxes. Each grid is *logically* broken into 4 tiles, and each tile as 4^2 cells. There are 8 tiles in total.

Table 5.2 shows an example of the difference between `validbox` and `tilebox`. In this example, there are two grids of cell-centered index type. The function `validbox` always returns a `Box` for the valid region of an `FArrayBox` no matter whether or not tiling is enabled, whereas the function `tilebox` returns a `Box` for a tile. (Note that when tiling is disabled, `tilebox` returns the same `Box` as `validbox`.) The number of loop iteration is 2 in the non-tiling version, whereas in the tiling version the kernel function is called 8 times.

It is important to use the correct `Box` when implementing tiling, especially if the box is used to define a work region inside of the loop. For example:

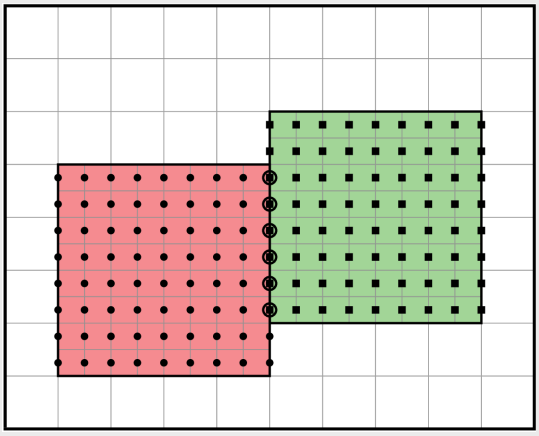
```
// MFIter loop with tiling on.
for (MFIter mfi(mf,true); mfi.isValid(); ++mfi)
{
    Box bx = mfi.validbox();     // Gets box of entire, untiled region.
    calcOverBox(bx);             // ERROR! Works on entire box, not tiled box.
                                 // Other iterations will redo many of the same cells.
}
```

The tile size can be explicitly set when defining `MFIter`.

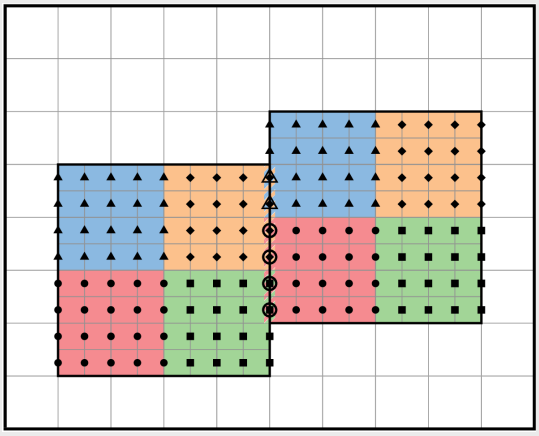
```
// No tiling in x-direction. Tile size is 16 for y and 32 for z.
for (MFIter mfi(mf,IntVect(1024000,16,32)); mfi.isValid(); ++mfi) {...}
```

An `IntVect` is used to specify the tile size for every dimension. A tile size larger than the grid size simply means tiling is disabled in that direction. AMReX has a default tile size `IntVect{1024000,8,8}` in 3D and no tiling in 2D. This is used when tile size is not explicitly set but the tiling flag is on. One can change the default size using `ParmParse` (section *ParmParse*) parameter `fabarray.mfiter_tile_size`.

Table 5.3: Comparison of `MFIter` with (right) and without (left) tiling, for face-centered nodal indexing.



Example of face valid boxes. There are two valid boxes in this example. Each has 9×8 points. Note that points in one `Box` may overlap with points in the other `Box`. However, the memory locations for storing floating-point data of those points do not overlap, because they belong to separate FArrayBoxes.



Example of face tile boxes. Each grid is *logically* broken into 4 tiles as indicated by the symbols. There are 8 tiles in total. Some tiles have 5×4 points, whereas others have 4×4 points. Points from different Boxes may overlap, but points from different tiles of the same Box do not.

Dynamic tiling, which runs one box per OpenMP thread, either with or without tiling the box, is also available. This is useful when the underlying work cannot benefit from thread parallelization. Dynamic tiling is implemented using the `MFItInfo` object and requires the `MFIter` loop to be defined in an OpenMP parallel region:

```
// Dynamic tiling, one box per OpenMP thread.
// No further tiling details,
//   so each thread works on a single tilebox.
#ifdef AMREX_USE_OMP
#pragma omp parallel
#endif
    for (MFIter mfi(mf,MFItInfo().SetDynamic(true)); mfi.isValid(); ++mfi)
    {
        const Box& bx = mfi.validbox();
        ...
    }
```

Dynamic tiling also allows explicit definition of a tile size:

```
// Dynamic tiling, one box per OpenMP thread.
// No tiling in x-direction. Tile size is 16 for y and 32 for z.
#ifdef AMREX_USE_OMP
#pragma omp parallel
#endif
    for (MFIter mfi(mf,MFItInfo().SetDynamic(true).EnableTiling(1024000,16,32)); mfi.
↪isValid(); ++mfi)
    {
        const Box& bx = mfi.tilebox();
        ...
    }
```

Note that `EnableTiling()`, with no argument, will use the default tile size.

Usually `MFIter` is used for accessing multiple MultiFabs, like the second example in the previous section on *MFIter without Tiling* in which two MultiFabs, `U` and `F`, use `MFIter` via `array()` and `const_array()` functions. These different MultiFabs may have different BoxArrays. For example, `U` might be cell-centered, whereas `F` might be nodal in x-direction and cell in other directions. The `MFIter::validbox` and `tilebox` functions return Boxes of the same type as the `MultiFab` used in defining the `MFIter` (`F` in this example). Table 5.3 illustrates an example of non-cell-centered valid and tile boxes. Besides `validbox` and `tilebox`, `MFIter` has a number of functions returning various Boxes. Examples include,

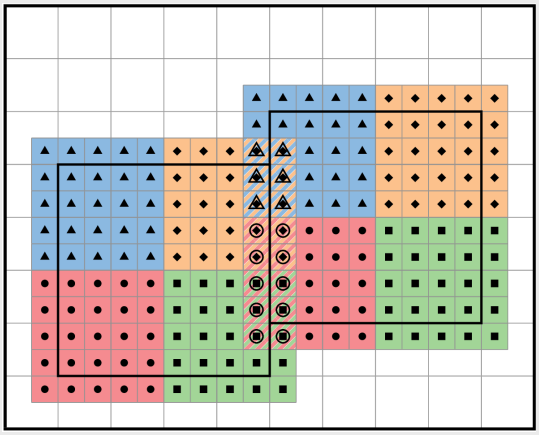
```
Box fabbox() const;       // Return the Box of the FArrayBox

// Return grown tile box.  By default it grows by the number of
// ghost cells of the MultiFab used for defining the MFIter.
Box growntilebox(int ng=-1000000) const;

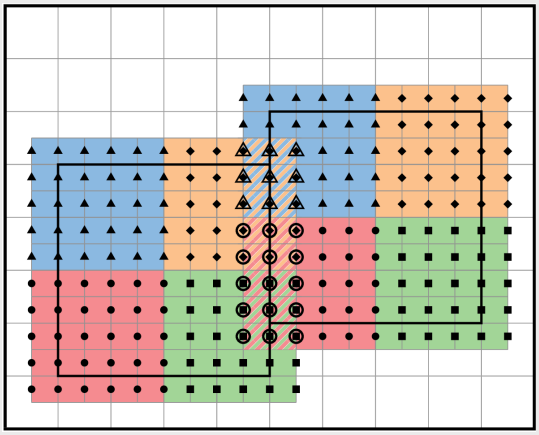
// Return tilebox with provided nodal flag as if the MFIter
// is constructed with MultiFab of such flag.
Box tilebox(const IntVect& nodal_flag);
```

It should be noted that the function `growntilebox` does not grow the tile Box like a normal `Box`. Growing a `Box` normally means the Box is extended in every face of every dimension. However, the function `growntilebox` only extends the tile Box in such a way that tiles from the same grid do not overlap. This is the basic design principle of these various tiling functions. Tiling is a way of domain decomposition for work sharing. Overlapping tiles is undesirable because work would be wasted and for multi-threaded codes race conditions could occur.

Table 5.4: Comparing growing cell-type and face-type tile boxes.



Example of cell-centered grown tile boxes. As indicated by symbols and colors, there are 4 tiles per grid in this example. Tiles from the same grid do not overlap. But tiles from different grids may overlap.



Example of face type grown tile boxes. As indicated by symbols and colors, there are 4 tiles per grid in this example. Tiles from the same grid do not overlap even though they have face index type.

Table 5.4 illustrates an example of `growntilebox`. These functions in `MFIter` return `Box` by value. There are three ways of using these functions.

```
const Box& bx = mfi.validbox();  // const& to temporary object is legal

// Make a copy if Box needs to be modified later.
// Compilers can optimize away the temporary object.
Box bx2 = mfi.validbox();
bx2.surroundingNodes();

Box&& bx3 = mfi.validbox(); // bound to the return value
bx3.enclosedCells();
```

But `Box& bx = mfi.validbox()` is not legal and will not compile.

Finally it should be emphasized that tiling should not be used when running on GPUs because of kernel launch overhead.

5.20.3 Multiple MFIters

To avoid some common bugs, it is not allowed to have multiple active `MFIter` objects like below by default.

```
for (MFIter mfi1(...); ...) {
    for (MFIter mfi2(...); ...) {
    }
}
```

```
call amrex_mfiter_build(mf1, ...)
call amrex_mfiter_build(mf2, ...)
```

This will result in an assertion failure at runtime. To disable the assertion, one could call

```
int old_flag = amrex::MFIter::allowMultipleMFIters(true);
```

```
logical :: old_flag
old_flag = amrex_mfiter_allow_multiple(.true.)
```

5.21 Fortran and C++ Kernels

In the section on *MFIter and Tiling*, we have shown that a typical pattern for working with MultiFabs is to use `MFIter` to iterate over the data. In each iteration, a kernel function is called to work on the data and the work region is specified by a `Box`. When tiling is used, the work region is a tile. The tiling is logical in the sense that there is no data layout transformation. The kernel function still gets the whole arrays in `FArrayBox`es, even though it is supposed to work on a tile region of the arrays. We have shown examples of writing kernels in C++ in the previous section. Fortran is also often used for writing these kernels because of its native multi-dimensional array support. To C++, these kernel functions are C functions, whose function signatures are typically declared in a header file named `*_f.H` or `*_F.H`. We recommend the users to follow this convention. Examples of these function declarations are as follows.

```
#include <AMReX_BLFort.H>
#ifdef __cplusplus
extern "C"
{
#endif
    void f1(const int*, const int*, amrex_real*, const int*, const int*);
    void f2(const int*, const int*,
            const amrex_real*, const int*, const int*, const int*
            amrex_real*, const int*, const int*, const int*);
#ifdef __cplusplus
}
#endif
```

These Fortran functions take C pointers and view them as multi-dimensional arrays of the shape specified by the additional integer arguments. Note that Fortran takes arguments by reference unless the **value** keyword is used. So an integer argument on the Fortran side matches an integer pointer on the C++ side. Thanks to Fortran 2003, function name mangling is easily achieved by declaring the Fortran function as **bind**(c).

AMReX provides many macros for passing an FArrayBox's data into Fortran/C. For example

```
for (MFIter mfi(mf,true); mfi.isValid(); ++mfi)
{
    const Box& box = mfi.tilebox();
    f(BL_TO_FORTRAN_BOX(box),
      BL_TO_FORTRAN_ANYD(mf[mfi]));
}
```

Here `BL_TO_FORTRAN_BOX` takes a `Box` and provides two **int** * s specifying the lower and upper bounds of the Box. `BL_TO_FORTRAN_ANYD` takes an `FArrayBox` returned by `mf[mfi]` and the preprocessor turns it into `Real *`, **int** *, **int** *, where `Real *` is the data pointer that matches real array argument in Fortran, the first **int** * (which matches an integer argument in Fortran) specifies the lower bounds, and the second **int** * the upper bounds of the spatial dimensions of the array. An example of the Fortran function is shown below,

```
subroutine f(lo, hi, u, ulo, uhi) bind(c)
  use amrex_fort_module, only : amrex_real
  integer, intent(in) :: lo(3),hi(3),ulo(3),uhi(3)
```

(continues on next page)

(continued from previous page)

```
  real(amrex_real),intent(inout)::u(ulo(1):uhi(1),ulo(2):uhi(2),ulo(3):uhi(3))
end subroutine f
```

Here, the size of the integer arrays is 3, the maximal number of spatial dimensions. If the actual spatial dimension is less than 3, the values in the degenerate dimensions are set to zero. So the Fortran function interface does not have to change according to the spatial dimensionality, and the bound of the third dimension of the data array simply becomes `0:0`. With the data passed by `BL_TO_FORTRAN_BOX` and `BL_FORTRAN_ANYD`, this version of Fortran function interface works for any spatial dimensions. If one wants to write a special version just for 2D and would like to use 2D arrays, one can use

```
subroutine f2d(lo, hi, u, ulo, uhi) bind(c)
  use amrex_fort_module, only : amrex_real
  integer, intent(in) :: lo(2),hi(2),ulo(2),uhi(2)
  real(amrex_real),intent(inout)::u(ulo(1):uhi(1),ulo(2):uhi(2))
end subroutine f2d
```

Note that this does not require any changes in the C++ part, because when C++ passes an integer pointer pointing to an array of three integers Fortran can treat it as a 2-element integer array.

Another commonly used macro is `BL_TO_FORTRAN`. This macro takes an `FArrayBox` and provides a real pointer for the floating-point data array and a number of integer scalars for the bounds. However, the number of the integers depends on the dimensionality. More specifically, there are 6 and 4 integers for 2D and 3D, respectively. The first half of the integers are the lower bounds for each spatial dimension and the second half the upper bounds. For example,

```
subroutine f2d(u, ulo1, ulo2, uhi1, uhi2) bind(c)
  use amrex_fort_module, only : amrex_real
  integer, intent(in) :: ulo1, ulo2, uhi1, uhi2
  real(amrex_real),intent(inout)::u(ulo1:uhi1,ulo2:uhi2)
end subroutine f2d

subroutine f3d(u, ulo1, ulo2, ulo3, uhi1, uhi2, uhi3) bind(c)
  use amrex_fort_module, only : amrex_real
  integer, intent(in) :: ulo1, ulo2, ulo3, uhi1, uhi2, uhi3
  real(amrex_real),intent(inout)::u(ulo1:uhi1,ulo2:uhi2,ulo3:uhi3)
end subroutine f3d
```

Here for simplicity we have omitted passing the tile Box.

Usually `MultiFabs` have multiple components. Thus we often also need to pass the number of component into Fortran functions. We can obtain the number by calling the `MultiFab::nComp()` function, and pass it to Fortran. We can also use the `BL_TO_FORTRAN_FAB` macro that is similar to `BL_TO_FORTRAN_ANYD` except that it provides an additional `int *` for the number of components. The Fortran function matching `BL_TO_FORTRAN_FAB(fab)` is then like below,

```
subroutine f(u, ulo, uhi,nu) bind(c)
  use amrex_fort_module, only : amrex_real
  integer, intent(in) :: lo(3),hi(3),ulo(3),uhi(3),nu
  real(amrex_real),intent(inout)::u(ulo(1):uhi(1),ulo(2):uhi(2),ulo(3):uhi(3),nu)
end subroutine f
```

There is a potential type safety issue when calling Fortran functions from C++. If there is a mismatch between the function declaration on the C++ side and the function definition in Fortran, the compiler cannot catch it. For example

```
// function declaration
extern "C" {
```

(continues on next page)

(continued from previous page)

```
    void f (amrex_real* x);
}

for (MFIter mfi(mf,true); mfi.isValid(); ++mfi)
{
    f(mf[mfi].dataPtr());
}

! Fortran definition
subroutine f(x,y) bind(c)
    implicit none
    integer x, y
end subroutine f
```

The code above will compile without errors even though the number of arguments and types don't match.

To help detect this kind of issue, AMReX provides a type check tool. Note that it only works when GCC is used. In the directory where an AMReX- based code is compiled, type

```
make typecheck
```

Extra arguments used in a usual AMReX build (e.g., USE_MPI=TRUE DIM=2) can be added. When it finishes, the output may look like,

```
Function my_f in main_F.H vs. Fortran procedure in f.f90
    number of arguments 1 does NOT match 2.
    arg #1: C type ['double', 'pointer'] does NOT match Fortran type ('INTEGER 4',
↪'pointer', 'x').
22 functions checked, 1 error(s) found.  More details can be found in tmp_build_dir/t/3d.
↪gnu.DEBUG.EXE/amrex_typecheck.ou.
```

It should be noted that Fortran by default passes argument by reference. In the example output above, `pointer` in `Fortran type ('INTEGER 4', 'pointer', 'x')` means it's a reference to argument (i.e., C pointer), not a Fortran pointer.

The type check tool has known limitations. For a function to be checked by the tool in the GNU Make build system, the declaration must be in a header file named `*_f.H` or `*_F.H`, and the header file must be in the `CEXE_headers` make variable. The headers are preprocessed first by cpp as C code, and then parsed by pycparser (https://pypi.python.org/pypi/pycparser) that needs to be installed on your system. Because pycparser is a C parser, C++ parts of the headers (e.g., `extern "C" {}`) need to be hidden with macro `#ifdef __cplusplus`. Headers like `AMReX_BLFort.H` can be used as a C header, but most other AMReX headers cannot and should be hidden by `#ifdef __cplusplus` if they are included. More details can be found at `amrex/Docs/Readme.typecheck`. Despite these limitations, it is recommended to use the type check tool and report issues to us.

Although Fortran has native multi-dimensional arrays, we recommend writing kernels in C++ because of performance portability for CPU and GPU. AMReX provides a multi-dimensional array syntax, similar to Fortran, that is readable and easy to implement. We have demonstrated how to use `Array4` in previous sections. Because of its importance, we will summarize its basic usage again with the example below.

```
void f (Box const& bx, FArrayBox const& sfab, FArrayBox& dfab)
{
    const Dim3 lo = amrex::lbound(bx);
    const Dim3 hi = amrex::ubound(bx);
```

(continues on next page)

(continued from previous page)

```
    Array4<Real const> const& src = sfab.const_array();
    Array4<Real      > const& dst = dfab2.array();

    for         (int k = lo.z; k <= hi.z; ++k) {
        for     (int j = lo.y; j <= hi.y; ++j) {
            AMREX_PRAGMA_SIMD
            for (int i = lo.x; i <= hi.x; ++i) {
                dst(i,j,k) = 0.5*(src(i,j,k)+src(i+1,j,k));
            }
        }
    }
}

for (MFIter mfi(mf1,true); mfi.isValid(); ++mfi)
{
    const Box& box = mfi.tilebox();
    f(box, mf1[mfi], mf2[mfi]);
}
```

A `Box` and two `FArrayBox`es are passed to a C++ kernel function. In the function, `amrex::lbound` and `amrex::ubound` are called to get the start and end of the loops from `Box::smallEnd()` and `Box::bigEnd` of `bx`. Both functions return a `amrex::Dim3`, a trivial type containing three integers. The individual components are accessed by using `.x`, `.y` and `.z`, as shown in the **`for`** loops.

`BaseFab::array()` is called to obtain an `Array4` object that is designed as an independent, **`operator()`** based accessor to the `BaseFab` data. `Array4` is an AMReX class that contains a pointer to the `FArrayBox` data and two `Dim3` structs that contain the bounds of the `FArrayBox`. The bounds are stored to properly translate the three dimensional coordinates to the appropriate location in the one-dimensional array. `Array4`'s **`operator()`** can also take a fourth integer to access across states of the `FArrayBox`. When AMReX is built for 1D or 2D, it can be used by passing *0* to the missing dimensions.

The `AMREX_PRAGMA_SIMD` macro is placed in the innermost loop to notify the compiler that loop iterations are independent and it is safe to vectorize the loop. This should be done whenever possible to achieve the best performance. Be aware: the macro generates a compiler dependent pragma, so their exact effect on the resulting code is also compiler dependent. It should be emphasized that using the `AMREX_PRAGMA_SIMD` macro on loops that are not safe for vectorization may lead to errors, so if unsure about the independence of the iterations of a loop, test and verify before adding the macro.

These loops should usually use `i <= hi.x`, not `i < hi.x`, when defining the loop bounds. If not, the highest index cells will be left out of the calculation.

5.22 ParallelFor

In the examples so far, we have explicitly written out the for loops when we iterate over a `Box`. AMReX also provides function templates for writing these in a concise and performance portable way like below,

```
#ifdef AMREX_USE_OMP
#pragma omp parallel if (Gpu::notInLaunchRegion())
#endif
  for (MFIter mfi(mfa,TilingIfNotGPU()); mfi.isValid(); ++mfi)
  {
    const Box& bx = mfi.tilebox();
    Array4<Real> const& a = mfa[mfi].array();
```

(continues on next page)

(continued from previous page)

```
    Array4<Real const> const& b = mfb[mfi].const_array();
    Array4<Real const> const& c = mfc[mfi].const_array();
    ParallelFor(bx, [=] AMREX_GPU_DEVICE (int i, int j, int k)
    {
      a(i,j,k) += b(i,j,k) * c(i,j,k);
    });
  }
```

Here, `ParallelFor` takes two arguments. The first argument is a `Box` specifying the iteration index space, and the second argument is a C++ lambda function that works on cell `(i,j,k)`. Variables a, b and c in the lambda function are captured by value from the enclosing scope. The code above is performance portable. It works with and without GPU support. When AMReX is built with GPU support, AMREX_GPU_DEVICE indicates that the lambda function is a device function and `ParallelFor` launches a GPU kernel to do the work. When it is built without GPU support, AMREX_GPU_DEVICE has no effects whatsoever. More details on `ParallelFor` will be presented in section *Launching C++ nested loops*. It should be emphasized that `ParallelFor` does not start an OpenMP parallel region. The OpenMP parallel region will be started by the pragma above the `MFIter` loop if it is built with OpenMP and without enabling GPU. Tiling is turned off if GPU is enabled so that more parallelism is exposed to GPU kernels. Also note that when tiling is off, `tilebox` returns `validbox`.

There are other versions of `ParallelFor`,

```
// 1D for loop
ParallelFor(N, [=] AMREX_GPU_DEVICE (int i) { ... });

// 4D for loop
ParallelFor(box, numcomps,
            [=] AMREX_GPU_DEVICE (int i, int j, int k, int n) { ... });
```

Note that on CPU `ParallelFor` marks the innermost loop with `AMREX_PRAGMA_SIMD`, i.e., it promises the compiler that the loop iterations are independent of each other. If the iterations are not independent – for example, when different iterations may update the same memory location, as in scatter-style accumulation with `Gpu::Atomic` operations, which are plain non-atomic updates on the host – use `amrex::For` instead. It provides the same set of signatures (including the `CompileTimeOptions` variants) and is identical to `ParallelFor` on GPU, but does not add the SIMD pragma on CPU. The SIMD tutorial shows how to write kernels that the compiler vectorizes, and the explicit SIMD version `ParallelForSIMD`.

To specialize kernels for a small set of runtime choices, see *Compile-time options for ParallelFor* for an explanation and example.

5.23 Ghost Cells

AMReX uses a `MultiFab` as a container for floating-point data on multiple Boxes at a single level of refinement. Each rectangular Box has its own boundaries on the low and high side in each coordinate direction. Each Box within a `MultiFab` can have ghost cells for storing data outside the Box's valid region. This allows us to, e.g., perform stencil-type operations on regular arrays. There are three basic types of boundaries:

1. interior boundary
2. coarse/fine boundary
3. physical boundary

Interior boundary is the border among the grid Boxes themselves. For example, in Fig. 5.1, the two blue grid Boxes on level 1 share an interior boundary that is 10 cells long. For a `MultiFab` with ghost cells on level 1, we can use the `MultiFab::FillBoundary` function introduced in the section on *FabArray, MultiFab and iMultiFab* to fill ghost

cells at the interior boundary with valid cell data from other Boxes. `MultiFab::FillBoundary` can optionally fill periodic boundary ghost cells as well.

A coarse/fine boundary is the border between two AMR levels. `FillBoundary` does not fill these ghost cells. These ghost cells on the fine level need to be interpolated from the coarse level data. This is a subject that will be discussed in the section on *FillPatchUtil and Interpolater*.

Note that periodic boundary is not considered a basic type in the discussion here because after periodic transformation it becomes either interior boundary or coarse/fine boundary.

The third type of boundary is the physical boundary at the physical domain. Note that both coarse and fine AMR levels could have grids touching the physical boundary. It is up to the application codes to properly fill the ghost cells at the physical boundary. However, AMReX does provide support for some common operations. See the section on *Boundary Conditions* for a discussion on domain boundary conditions in general, including how to implement physical (non-periodic) boundary conditions.

5.24 Boundary Conditions

This section describes how to implement domain boundary conditions in AMReX. A ghost cell that is outside of the valid region can be thought of as either "interior" (which includes periodic and coarse-fine ghost cells), or "physical". Physical boundary conditions can occur on domain boundaries and can be characterized as inflow, outflow, slip/no-slip walls, etc., and are ultimately linked to mathematical Dirichlet or Neumann conditions.

The basic idea behind physical boundary conditions is as follows:

- Create a `BCRec` object, which is essentially a multidimensional integer array of `2*DIM` components. Each component defines a boundary condition type for the lo/hi side of the domain, for each direction. See `amrex/Src/Base/AMReX_BC_TYPES.H` for common physical and mathematical types. Below is an example of setting up a `Vector<BCRec>` for multiple components before the call to ghost cell routines.

```
// Set up BC; see ``amrex/Src/Base/AMReX_BC_TYPES.H`` for supported types
Vector<BCRec> bc(phi.nComp());
for (int n = 0; n < phi.nComp(); ++n)
{
    for (int idim = 0; idim < AMREX_SPACEDIM; ++idim)
    {
        if (geom.isPeriodic(idim))
        {
            bc[n].setLo(idim, BCType::int_dir); // interior
            bc[n].setHi(idim, BCType::int_dir);
        }
        else
        {
            bc[n].setLo(idim, BCType::foextrap); // first-order extrapolation
            bc[n].setHi(idim, BCType::foextrap);
        }
    }
}
```

`amrex::BCType` has the following types,

int_dir
: Interior, including periodic boundary

ext_dir
: "External Dirichlet". It is the user's responsibility to write a routine to fill ghost cells (more details

below). The boundary location is on the domain face even when the data inside the domain are cell-centered.

ext_dir_cc
"External Dirichlet". It is the user's responsibility to write a routine to fill ghost cells (more details below). The boundary location is at the cell center of ghost cells outside the domain.

foextrap
"First Order Extrapolation" First order extrapolation from last cell in interior.

hoextrap
"High Order Extrapolation". The boundary location is on the domain face even when the data inside the domain are cell-centered.

hoextrapcc
"High Order Extrapolation" The boundary location is at the cell center of ghost cells outside the domain.

reflect_even
Reflection from interior cells with sign unchanged, $q(-i) = q(i)$.

reflect_odd
Reflection from interior cells with sign changed, $q(-i) = -q(i)$.

user_1, user_2 and user_3
"User". It is the user's responsibility to write a routine to fill ghost cells (more details below).

- For external Dirichlet and user boundaries, the user needs to provide a callable object like below.

```
struct MyExtBCFill {
    AMREX_GPU_DEVICE
    void operator() (const IntVect& iv, Array4<Real> const& dest,
                     const int dcomp, const int numcomp,
                     GeometryData const& geom, const Real time,
                     const BCRec* bcr, const int bcomp,
                     const int orig_comp) const
    {
        // external Dirichlet or user BC for cell iv
    }
};
```

  Here, for the CPU build, the AMREX_GPU_DEVICE macro has no effect whatsoever, whereas for the GPU build, this marks the operator as a GPU device function.

- It is the user's responsibility to have a consistent definition of what the ghost cells represent. A common option used in AMReX codes is to fill the domain ghost cells with the value that lies on the boundary (as opposed to another common option where the value in the ghost cell represents an extrapolated value based on the boundary condition type). Then in our stencil based "work" codes, we also pass in the `BCRec` object and use modified stencils near the domain boundary that know the value in the first ghost cell represents the value on the boundary.

Depending on the level of complexity of your code, there are various options for filling domain boundary ghost cells.

For single-level codes built from `amrex/Src/Base` (excluding the `amrex/Src/AmrCore` and `amrex/Src/Amr` source code directories), you will have single-level MultiFabs filled with data in the valid region where you need to fill the ghost cells on each grid.

```
MultiFab mf;
Geometry geom;
Vector<BCRec> bc;
```

(continues on next page)

(continued from previous page)

```
Real time;

// ...

// fills interior and periodic domain boundary ghost cells
mf.FillBoundary(geom.periodicity());

// fills physical domain boundary ghost cells for a cell-centered multifab
if (not geom.isAllPeriodic()) {
    GpuBndryFuncFab<MyExtBCFill> bf(MyExtBCFill{});
    PhysBCFunct<GpuBndryFuncFab<MyExtBCFill> > physbcf(geom, bc, bf);
    physbcf(mf, 0, mf.nComp(), mf.nGrowVector(), time, 0);
}
```

5.25 Masks

Given an index `(i,j,k)`, we often need to know its relationship with other points and levels (e.g., whether this point on a coarse level is covered by a fine level, whether this ghost point is outside coarse/fine boundary, etc.). AMReX provides various functions for creating masks for this type of purposes.

5.25.1 Owner Mask

AMReX supports various index types such as face, edge and node, besides cell centered type. For non-cell types, two boxes could overlap. For example, a nodal index `(i,j,k)` could exist in more than one `FArrayBox` of a nodal `MultiFab`. AMReX provides a function to create an owner mask, where the owner is the grid with the lowest grid number containing the data. This has a number of use cases. The nodal data for the same nodal point on different `FArrayBox`es may be out of sync. We can use `MultiFab::OverrideSync` and an owner mask to sync up the data with owners overriding non-owners.

```
MultiFab mf(...); // non-cell-centered
auto mask = amrex::OwnerMask(mf, geom.periodicity());
mf.OverrideSync(*mask, geom.periodicity());
```

This version of `OverrideSync` offers the flexibility of providing a custom mask. However, when a custom mask is not needed, we can synchronize the data also with

```
MultiFab mf(...); // non-cell-centered
mf.OverrideSync(geom.periodicity());
```

This version of `OverrideSync` has better performance than the previous one. The StaggeredGrids tutorial puts cell-centered, face-centered and nodal data on the same grids, averages between them and synchronizes the shared faces.

To compute the dot product of two nodal `MultiFab`s, we can use a mask to avoid double counting.

```
MultiFab mf1(...);
MultiFab mf2(...);
auto mask = amrex::OwnerMask(mf1, geom.periodicity());
Real result = MultiFab::Dot(*mask, mf1, 0, mf2, 0, 1, 0);
```

5.25.2 Overlap Mask

For the synchronization example mentioned previously, maybe instead of overriding, we want to do averaging. This can be achieved with an overlap mask indicating how many duplicates are in each point. The code below shows how the `MultiFab::AverageSync` function is implemented in AMReX.

```
MultiFab mf(...); // non-cell-centered
auto mask = mf.OverlapMask(geom.periodicity());
mask->invert(1.0, 0, 1);
mf.WeightedSync(*mask, geom.periodicity());
```

5.25.3 Point Mask

The `FabArray` class has a member function `BuildMask` that can be used to set masks indicating the type of points (e.g., valid, outside the domain, etc.). For example,

```
iMultiFab mask(ba, dm, 1, nghost);
int a = 10; // ghost points covered by valid points
int b = 11; // ghost points not covered by valid points
int c = 12; // outside physical domain
int d = 13; // interior points (i.e., valid points)
mask.BuildMask(geom.Domain(), geom.periodicity(), a, b, c, d);
```

5.25.4 Fine Mask

AMReX provides a number of `makeFineMask` functions that can be useful for multi-level AMR calculations. For example, we may want to compute the infinity norm on a coarse AMR level without including data from cells covered by fine level grids.

```
int coarse_value = 1;
int fine_value = 0;
iMultiFab mask = makeFineMask(coarse_mf, fine_boxarray, refine_ratio,
                              coarse_value, fine_value);
Real result = coarse_mf.norminf(mask);
```

5.26 Memory Allocation

Some constructors of `MultiFab`, `FArrayBox`, etc. can take an `Arena` argument for memory allocation. Some constructors of `MultiFab` can take an optional argument `MFInfo`, which can be used to set the arena. This is usually not important for CPU codes, but very important for GPU codes. We will present more details about memory arenas in *Memory Allocation* in Chapter GPU.

Every `FArrayBox` in a `MultiFab` has a contiguous chunk of memory for floating-point data, whereas by default `MultiFab`, as a collection of multiple `FArrayBox`s, does not store all floating-point data in a contiguous chunk of memory. This behavior can be changed for all `MultiFab`s with the `ParmParse` parameter, `amrex.mf.alloc_single_chunk=1`, or for a specific `MultiFab` by passing a `MFInfo` object (e.g., `MFInfo().SetAllocSingleChunk(true)`) to the constructor. One can call `MultiFab::singleChunkPtr()` to obtain a pointer to the single chunk memory. Note that the function returns a null pointer if the `MultiFab` does not use a single contiguous chunk of memory. One can also call `MultiFab::singleChunkSize()` to obtain the size in bytes of the single chunk memory.

AMReX has a Fortran module, `amrex_mempool_module` that can be used to allocate memory for Fortran pointers. The reason that such a module exists in AMReX is that memory allocation is often very slow in multi-threaded OpenMP

parallel regions. AMReX amrex_mempool_module provides a much faster alternative approach, in which each thread has its own memory pool. Here are examples of using the module.

```
use amrex_mempool_module, only : amrex_allocate, amrex_deallocate
real(amrex_real), pointer, contiguous :: a(:,:,:), b(:,:,:,:)
integer :: lo1, hi1, lo2, hi2, lo3, hi3, lo(4), hi(4)
! lo1 = ...
! a(lo1:hi1, lo2:hi2, lo3:hi3)
call amrex_allocate(a, lo1, hi1, lo2, hi2, lo3, hi3)
! b(lo(1):hi(1),lo(2):hi(2),lo(3):hi(3),lo(4):hi(4))
call amrex_allocate(b, lo, hi)
! ......
call amrex_deallocate(a)
call amrex_deallocate(b)
```

The downside of this is we have to use **pointer** instead of **allocatable**. This means we must explicitly free the memory via amrex_deallocate and we need to declare the pointers as **contiguous** for performance reason. Also, we often pass the Fortran pointer to a procedure with explicit array argument to get rid of the pointerness completely.

5.27 Random Numbers

AMReX provides a set of pseudo-random number generators. Most come in two overloads: one that takes no arguments, for use on the host, and one that takes a RandomEngine and can be called from inside a GPU kernel. The engine is supplied by ParallelForRNG, the random-number variant of ParallelFor:

```
amrex::ParallelForRNG(N,
[=] AMREX_GPU_DEVICE (int i, amrex::RandomEngine const& engine) noexcept
{
    p[i] = amrex::Random(engine);
});
```

The generators are thread safe. With OpenMP, each thread draws from its own independent generator. The seed can be set with amrex::ResetRandomSeed(cpu_seed, gpu_seed).

5.27.1 The uniform generators and their intervals

There are two uniform generators on the unit interval, and which one you want depends on what you do with the result:

Function	Interval	Use for
amrex::Random()	[0,1)	positions within a cell, rejection tests, general uniform sampling
amrex::RandomPositive()	(0,1]	anything singular at zero: log(u), 1/u, pow(u,-a)

Both endpoints matter in practice, and each generator excludes the one that is dangerous for its intended use.

amrex::Random excludes the upper endpoint and *includes* zero.

Note that the guarantee is on the value amrex::Random returns, not on whatever is computed from it. Arithmetic downstream can still round onto a boundary: problo + amrex::Random()*dx may evaluate to exactly the upper face of the cell, since the rounded sum can land there even though the draw is strictly below one. Code that needs a value strictly inside a range must therefore guard the result of the computation, not rely on the draw alone.

amrex::RandomPositive excludes the lower endpoint, so that the result can be passed straight to a function that is singular at zero:

```
// exponential distribution with mean tau
amrex::Real const t = -tau * std::log(amrex::RandomPositive(engine));

// power law with exponent -a
amrex::Real const x = std::pow(amrex::RandomPositive(engine), -a);
```

There is no accuracy or performance penalty for picking the interval that fits the use case: whichever one you ask for is obtained from the underlying generator by relocating at most a single endpoint, never by arithmetic. Both guarantees therefore hold in single and double precision, and under every floating-point mode AMReX may be built with, including `-ffast-math` / `--use_fast_math` and flush-to-zero.

Warning

Do **not** write `1 - amrex::Random()` to obtain a non-zero value; draw from `amrex::RandomPositive` instead. The two are not equivalent: the subtraction costs an arithmetic operation and discards resolution, since it is exact only above `0.5` and collapses smaller values onto a coarse grid.

5.27.2 Other distributions

Function	Distribution
`amrex::RandomNormal(mean, stddev)`	normal
`amrex::RandomPoisson(lambda)`	Poisson
`amrex::RandomGamma(alpha, beta)`	Gamma
`amrex::Random_int(n)`	uniform integer on `[0,n-1]`
`amrex::Random_long(n)`	uniform long on `[0,n-1]`, host only

To fill an array rather than draw one value at a time, use `amrex::FillRandom(p, N)` for the uniform distribution and `amrex::FillRandomNormal(p, N, mean, stddev)` for the normal distribution.

Note

`amrex::FillRandom` does not give the same interval on every backend the way `amrex::Random` and `amrex::RandomPositive` do. It is `[0,1)` on the CPU, but the GPU paths hand back whatever the vendor bulk generator produces – nominally `[0,1)` with SYCL and `(0,1]` with CUDA and HIP – so code that depends on either endpoint should guard the values it reads back, or draw with `amrex::Random` / `amrex::RandomPositive` instead.

5.28 Abort, Assertion and Backtrace

`amrex::Abort(const char * message)` is used to terminate a run usually when something goes wrong. This function takes a message and writes it to stderr. Files named like `Backtrace.1` (where 1 means process 1) are produced containing backtrace information of the call stack. In Fortran, we can call `amrex_abort` from the `amrex_error_module`, which takes a Fortran character variable with assumed size (i.e., `len=*`) as a message. A `ParmParse` runtime boolean parameter `amrex.throw_handling` (which is defaulted to 0, i.e., `false`) can be set to 1 (i.e., `true`) so that AMReX will throw an exception instead of aborting.

`AMREX_ASSERT` is a macro that takes a Boolean expression. For a debug build (e.g., `DEBUG=TRUE` using the GNU Make build system), if the expression at runtime is evaluated to false, `amrex::Abort` will be called and the run is thus terminated. For an optimized build (e.g., `DEBUG=FALSE` using the GNU Make build system), the `AMREX_ASSERT`

statement is removed at compile time and thus has no effect at runtime. We often use this as a means of putting debug statements in the code without adding any extra cost for production runs. For example,

```
AMREX_ASSERT(mf.nGrow() > 0 && mf.nComp() == mf2.nComp());
```

Here for debug build we like to assert that `MultiFab mf` has ghost cells and it also has the same number of components as `MultiFab mf2`. If we always want the assertion, we can use `AMREX_ALWAYS_ASSERT`. The assertion macros have a `_WITH_MESSAGE` variant that will print a message when assertion fails. For example,

```
AMREX_ASSERT_WITH_MESSAGE(mf.boxArray() == mf2.boxArray(),
                          "These two mfs must have the same BoxArray");
```

Backtrace files are produced by AMReX signal handler by default when segfault occurs or `Abort` is called. If the application does not want AMReX to handle this, `ParmParse` parameter *amrex.signal_handling=0* can be used to disable it.

See *Assertions and Error Checking* for considerations on using these functions in GPU-enabled code.

CHAPTER

SIX

GRIDDING AND LOAD BALANCING

AMReX provides a great deal of generality when it comes to how to decompose the computational domain into individual logically rectangular grids, and how to distribute those grids to MPI ranks. We use the phrase "load balancing" here to refer to the combined process of grid creation (and re-creation when regridding) and distribution of grids to MPI ranks.

Even for single-level calculations, AMReX provides the flexibility to have different size grids, more than one grid per MPI rank, and different strategies for distributing the grids to MPI ranks.

For multi-level calculations, the same principles for load balancing apply as in single-level calculations, but there is additional complexity in how to tag cells for refinement and how to create the union of grids at levels > 0 where that union most likely does not cover the computational domain.

See *Grid Creation* for how grids are created, i.e. how the `BoxArray` on which `MultiFabs` will be built is defined at each level.

See *Load Balancing* for the strategies AMReX supports for distributing grids to MPI ranks, i.e. defining the `DistributionMapping` with which `MultiFabs` at that level will be built.

We also note that we can create separate grids, and map them in different ways to MPI ranks, for different types of data in a single calculation. We refer to this as the "dual grid approach" and the most common usage is to load balance mesh and particle data separately. See *Dual Grid Approach* for more about this approach.

When running on multicore machines with OpenMP, we can also control the distribution of work by setting the size of grid tiles (by defining `fabarray_mfiter.tile_size`), and if relevant, of particle tiles (by defining `particle.tile_size`). We can also specify the strategy for assigning tiles to OpenMP threads. See *MFIter with Tiling* for more about tiling.

6.1 Grid Creation

To run an AMReX-based application you must specify the domain size by specifying `n_cell` – this is the number of cells spanning the domain in each coordinate direction at level 0.

Users often specify `max_grid_size` as well. The default load balancing algorithm then divides the domain in every direction so that each grid is no longer than `max_grid_size` in that direction. If not specified by the user, `max_grid_size` defaults to 128 in 2D and 32 in 3D (in each coordinate direction).

Another popular input is `blocking_factor`. The value of `blocking_factor` constrains grid creation in that each grid must be divisible by `blocking_factor`. Note that both the domain (at each level) and `max_grid_size` must be divisible by `blocking_factor`, and that `blocking_factor` must be either 1 or a power of 2 (otherwise the gridding algorithm would not in fact create grids divisible by `blocking_factor` because of how `blocking_factor` is used in the gridding algorithm). See *Odd Refinement Ratios and Odd Domain Sizes* for the exceptions that apply when `amr.no_box_split_dir` is set.

If not specified by the user, `blocking_factor` defaults to 8 in each coordinate direction. The typical purpose of `blocking_factor` is to ensure that the grids will be sufficiently coarsenable for good multigrid performance.

The blocking_factor on a level $\ell > 0$ also controls the cost of regridding. Before the tagged cells on level $\ell - 1$ are clustered into grids, they are coarsened by blocking_factor on level ℓ divided by the refinement ratio between the two levels. A blocking_factor of 1 therefore means that the clustering algorithm works on every tagged cell, which is expensive for large domains.

There is one more default behavior to be aware of. There is a boolean refine_grid_layout that defaults to true but can be overridden at run time. If refine_grid_layout is true and the number of grids created is less than the number of processors (Ngrids < Nprocs), then grids will be further subdivided until Ngrids >= Nprocs.

Caveat: if subdividing the grids to achieve Ngrids >= Nprocs would violate the blocking_factor criterion, then additional grids are not created and the number of grids will remain less than the number of processors.

Note that n_cell must be given as three separate integers, one for each coordinate direction.

However, max_grid_size and blocking_factor can be specified as a single value applying to all coordinate directions, or as separate values for each direction.

- If max_grid_size (or blocking_factor) is specified as multiple integers then the first integer applies to level 0, the second to level 1, etc. If you don't specify as many integers as there are levels, the final value will be used for the remaining levels.
- If different values of max_grid_size (or blocking_factor) are wanted for each coordinate direction, then max_grid_size_x, max_grid_size_y and max_grid_size_z (or blocking_factor_x, blocking_factor_y and blocking_factor_z) must be used. If you don't specify as many integers as there are levels, the final value will be used for the remaining levels.

Additional notes:

- To create identical grids of a specific size, e.g. of length m in each direction, then set max_grid_size = m and blocking_factor = m.
- Note that max_grid_size is just an upper bound; with n_cell = 48 and max_grid_size = 32, we will typically have one grid of length 32 and one of length 16.

The grid creation process at level 0 proceeds as follows (if not using the KD-tree approach):

1. The domain is initially defined by a single grid of size n_cell.
2. If n_cell is greater than max_grid_size then the grids are subdivided until each grid is no longer than max_grid_size cells on each side. The blocking_factor criterion (i.e., that the length of each side of each grid is divisible by blocking_factor in that direction) is satisfied during this process.
3. Next, if refine_grid_layout = true and there are more processors than grids at this level, then the grids at this level are further divided until Ngrids >= Nprocs (unless doing so would violate the blocking_factor criterion).

The creation of grids at levels > 0 begins by tagging cells at the coarser level and follows the Berger-Rigoutsos clustering algorithm with the additional constraints of satisfying the blocking_factor and max_grid_size criteria. An additional parameter becomes relevant here: the "grid efficiency", specified as amr.grid_eff in the inputs file. This threshold value, which defaults to 0.7 (or 70%), is used to ensure that grids do not contain too large a fraction of untagged cells. We note that the grid creation process attempts to satisfy the amr.grid_eff constraint but will not do so if it means violating the blocking_factor criterion.

Some applications want the fine levels to cover the entire domain in one coordinate direction, no matter where the cells are tagged. Setting amr.refine_whole_domain_dir to that direction (0 for x, 1 for y, 2 for z; the default of -1 disables this) makes the tagging of a cell behave as if the whole line of cells through it in that direction were tagged. The clustering is then performed in one fewer dimension, so amr.grid_eff refers to the fraction of tagged cells in the plane perpendicular to that direction. The resulting grids may still be chopped in that direction by max_grid_size and refine_grid_layout, but together they always cover the entire domain.

Other applications do not want the grids decomposed in one coordinate direction (for example, atmospheric codes that solve implicitly along vertical columns). Setting amr.no_box_split_dir to that direction (0 for x, 1 for y, 2 for z;

the default of -1 disables this) has the following effects.

- `max_grid_size` and `refine_grid_layout` are ignored in that direction on every level, and the `blocking_factor` does not need to divide `n_cell` in it. (Applications built on `class Amr` rather than `AmrCore` still need an even `n_cell` in every direction.)
- On level 0, if the `blocking_factor` is 1 in the other directions, the domain is split in those directions only, into nearly equal grids that all span the domain in `amr.no_box_split_dir`. Each direction gets the fewest pieces allowed by `max_grid_size`. If there are fewer grids than MPI processes and `refine_grid_layout` permits, the number of pieces is doubled in the direction with the longest grids until there are enough. Any `n_cell` works. With a larger level 0 `blocking_factor` the usual algorithm is used.
- On finer levels, the grids produced by the clustering are merged along that direction so that no two grids share an interior face normal to it, and they are never chopped in it afterwards. A grid still covers only the part of the domain where cells are tagged; two tagged regions at different heights in the same column give two grids that do not touch. Use `amr.refine_whole_domain_dir` as well if every grid must span the entire domain in that direction. Grids at opposite ends of a periodic domain can still touch through the periodic boundary.
- The rules of *Odd Refinement Ratios and Odd Domain Sizes* for `blocking_factor`, `max_grid_size` and domain sizes apply, for odd and even refinement ratios alike.

Users often like to ensure that coarse/fine boundaries are not too close to tagged cells; the way to do this is to set `amr.n_error_buf` to a large integer value (the default is 1). This parameter is used to increase the number of tagged cells before the grids are defined; if cell "*(i,j,k)*" satisfies the tagging criteria, then, for example, if `amr.n_error_buf` is 3, all cells in the 7x7x7 box from lower corner "*(i-3,j-3,k-3)*" to "*(i+3,j+3,k+3)*" will be tagged.

6.2 Odd Refinement Ratios and Odd Domain Sizes

Some applications, for example atmospheric codes that nest a fine domain inside a coarse one, use odd refinement ratios such as 3 together with domains whose sizes are not powers of 2, e.g., `n_cell = 749 679 69`. The usual advice of powers of 2 everywhere does not apply to them. This section explains how to choose `blocking_factor` and `max_grid_size` in that situation. It applies only when `amr.no_box_split_dir` is set; otherwise the usual rules above apply to all refinement ratios.

Blocking factor on the fine levels. On a level with refinement ratio r, choose a `blocking_factor` that is r times a power of 2, for example 24 (that is, 3×8) for $r = 3$, or a plain power of 2 such as 8 in a direction where $r = 1$. The grids on that level are then multiples of the `blocking_factor`, and regridding becomes much cheaper, because the grid generator works with blocks of that many coarse cells in each direction instead of individual cells (with 8, that is 512 times fewer cells in 3D). A `blocking_factor` of 1 is allowed, but it makes regridding expensive on large domains and produces many small grids, so it is best avoided. A power of 2 that is not a multiple of r, such as 8 with $r = 3$, is accepted for backward compatibility, but the grids are then multiples of 6 rather than 8, and a warning says so. The grids are then also no longer than the largest multiple of 6 that fits in `max_grid_size`. In all cases `blocking_factor` divided by r, rounded down, must be a power of 2 (or less than 1), so 16 is rejected for $r = 3$.

Max grid size on the fine levels. `max_grid_size` must be a multiple of the `blocking_factor`, e.g., 96 or 192 for a `blocking_factor` of 24, and at least twice the `blocking_factor` when `n_cell` is not divisible by it. Larger values give fewer, larger grids; smaller values give more grids to distribute across processes.

Level 0. The `blocking_factor` on level 0 must divide `n_cell`, so when `n_cell` has no convenient factors it has to be 1. This costs nothing: the level 0 blocking factor has no effect on regridding, and the level 0 grids do not need to line up with the blocking factor of level 1. `max_grid_size` on level 0 can be anything; it just sets how the domain is split among processes.

Domain sizes. The domain does not need to be divisible by the `blocking_factor`. In a non-periodic direction, the grids that reach the upper domain boundary are simply cut off there, but no grid is ever thinner than the `blocking_factor`. In a periodic direction, choose `n_cell` to be a multiple of the `blocking_factor` divided by r (8 in the example above); if it is not, a smaller power of 2 is used in that direction, the grids there are only multiples of r times that smaller number, and a warning is printed.

For example, with `n_cell = 749 679 69`, `ref_ratio_vect = 3 3 1` and `max_level = 1`, a good choice is

```
amr.no_box_split_dir  = 2
amr.blocking_factor_x = 1 24
amr.blocking_factor_y = 1 24
amr.blocking_factor_z = 1 8
amr.max_grid_size_x   = 188 96
amr.max_grid_size_y   = 188 96
```

- The level 0 blocking factor is 1 because 749, 679 and 69 are odd, so no larger power of 2 divides them.
- On level 1 the blocking factor is $3 \times 8 = 24$ in x and y, where the refinement ratio is 3, and $1 \times 8 = 8$ in z, where it is 1. Level 1 grids are therefore multiples of 24 by 24 by 8 cells, except where they are cut off at the upper domain boundary.
- The level 0 max grid size of 188 splits 749 cells into four grids in x and y. With `amr.no_box_split_dir = 2`, grids are never split in z, so `max_grid_size_z` is not needed.
- The level 1 max grid size is 96, a multiple of 24, in x and y.

6.3 Dual Grid Approach

In AMReX-based applications that have both mesh data and particle data, the mesh work and particle work have very different requirements for load balancing.

Rather than using a combined work estimate to create the same grids for mesh and particle data, we can pursue a "dual grid" approach.

With this approach, the mesh (`MultiFab`) and particle (`ParticleContainer`) data are allocated on different `BoxArrays` with different `DistributionMappings`.

This enables separate load balancing strategies to be used for the mesh and particle work.

The cost of this strategy, of course, is the need to copy mesh data onto temporary `MultiFabs` defined on the particle `BoxArrays` when mesh-particle communication is required.

6.4 Load Balancing

The process of load balancing is typically independent of the process of grid creation; the inputs to load balancing are a given set of grids with a set of weights assigned to each grid. (The exception to this is the KD-tree approach in which the grid creation process is governed by trying to balance the work in each grid.)

Single-level load balancing algorithms are sequentially applied to each AMR level independently, and the resulting distributions are mapped onto the ranks, taking into account the weights already assigned to them (by assigning the heaviest grids to the least loaded rank). Note that the load of each process is measured by how much memory has already been allocated, not how much memory will be allocated. Therefore the following code is not recommended because it tends to generate non-optimal distributions.

```
for (int lev = 0; lev < nlevels; ++lev) {
    // build DistributionMapping for Level lev
}
for (int lev = 0; lev < nlevels; ++lev) {
    // build MultiFabs for Level lev
}
```

Instead, one should do the following:

```
for (int lev = 0; lev < nlevels; ++lev) {
    // build DistributionMapping for Level lev
    // build MultiFabs for Level lev
}
```

Distribution options supported by AMReX include the following; the default is SFC:

- Knapsack: the default weight of a grid in the knapsack algorithm is the number of grid cells, but AMReX supports the option to pass an array of weights – one per grid – or alternatively to pass in a MultiFab of weights per cell which is used to compute the weight per grid.
- SFC: enumerate grids with a space-filling Z-Morton curve, then partition the resulting ordering across ranks in a way that balances the load.
- Round-robin: sort grids and assign them to ranks in round-robin fashion – specifically FAB i is owned by CPU i%N where N is the total number of MPI ranks.

The LoadBalance tutorial measures the cost of each box, builds a new `DistributionMapping` from these costs with `DistributionMapping::makeKnapSack` or `DistributionMapping::makeSFC`, and moves the data to it with `ParallelCopy`.

CHAPTER

SEVEN

AMRCORE SOURCE CODE

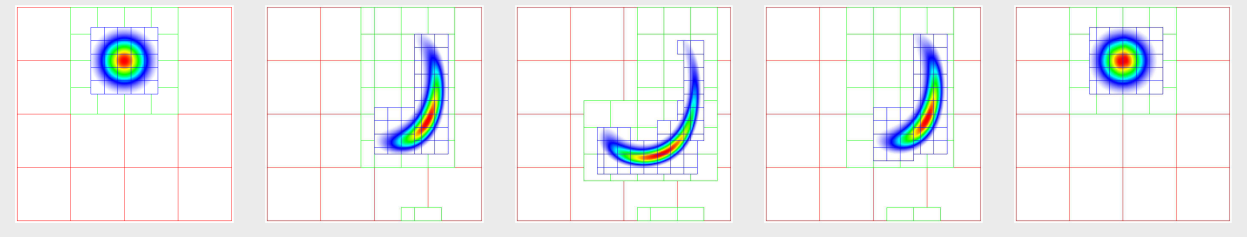
In this chapter, we give an overview of the functionality contained in the `amrex/Src/AmrCore` source code. This directory contains source code for the following:

- Storing information about the grid layout and processor distribution mapping at each level of refinement.
- Functions to create grids at different levels of refinement, including tagging operations.
- Operations on data at different levels of refinement, such as interpolation and restriction operators.
- Flux registers used to store and manipulate fluxes at coarse-fine interfaces.
- Particle support for AMR (see *Particles*).

There is another source directory, `amrex/Src/Amr/`, which contains additional classes used to manage the time-stepping for AMR simulations. However, it is possible to build a fully adaptive, subcycling-in-time simulation code without these additional classes.

In this chapter, we restrict our use to the `amrex/Src/AmrCore` source code and present a tutorial that performs an adaptive, subcycling-in-time simulation of the advection equation for a passively advected scalar. The accompanying tutorial code is available in `amrex-tutorials/AMR/AdvectionAmrCore`. In this example, the velocity field is a specified function of space and time, such that an initial Gaussian profile is displaced but returns to its original configuration at the final time. The boundary conditions are periodic and we use a refinement ratio of $r = 2$ between each AMR level. The results of the two-dimensional simulation are shown in the *AdvectionAmrCore tutorial*.

Table 7.1: Time sequence ($t = 0, 0.5, 1, 1.5, 2$ s) of advection of a Gaussian profile using the AdvectionAmrCore tutorial. The analytic velocity field distorts the profile, and then restores the profile to its original configuration. The red, green, and blue boxes indicate grids at AMR levels $\ell = 0, 1$, and 2.



7.1 AmrCore Source Code: Details

Here we provide more information about the source code in `amrex/Src/AmrCore`.

7.1.1 AmrMesh and AmrCore

For single-level simulations (see e.g., `amrex-tutorials/Base/HeatEquation/main.cpp`) the user needs to build `Geometry`, `DistributionMapping`, and `BoxArray` objects associated with the simulation. For simulations with multiple levels of refinement, the `AmrMesh` class can be thought of as a container to store arrays of these objects (one for each level), and information about the current grid structure.

`amrex/Src/AmrCore/AMReX_AmrMesh.cpp/H` contains the `AmrMesh` class. The protected data members are:

```
protected:
    int            verbose;
    int            max_level;       // Maximum allowed level.
    Vector<IntVect> ref_ratio;       // Refinement ratios [0:finest_level-1]

    int            finest_level;    // Current finest level.

    Vector<IntVect> n_error_buf;     // Buffer cells around each tagged cell.
    Vector<IntVect> blocking_factor; // Blocking factor in grid generation
                                     // (by level).
    Vector<IntVect> max_grid_size;   // Maximum allowable grid size (by level).
    Real           grid_eff;        // Grid efficiency.
    int            n_proper;        // # cells required for proper nesting.

    bool use_fixed_coarse_grids;
    int  use_fixed_upto_level;
    bool refine_grid_layout;        // chop up grids to have the number of
                                    // grids be no less than the number of procs

    Vector<Geometry>            geom;
    Vector<DistributionMapping> dmap;
    Vector<BoxArray>            grids;
```

The following parameters are frequently set via the inputs file or the command line. Their usage is described in the section on *Grid Creation*.

Table 7.2: AmrCore parameters

Variable	Value	Default
amr.verbose	int	0
amr.max_level	int	none
amr.max_grid_size	ints	32 in 3D, 128 in 2D
amr.n_proper	int	1
amr.grid_eff	Real	0.7
amr.n_error_buf	int	1
amr.blocking_factor	int	8
amr.refine_grid_layout	int	true
amr.max_grid_iterations	int	4

AMReX_AmrCore.cpp/H contains the pure virtual class `AmrCore`, which is derived from the `AmrMesh` class. AmrCore does not actually have any data members, just additional member functions, some of which override the base class AmrMesh.

There are no pure virtual functions in `AmrMesh`, but there are five pure virtual functions in the `AmrCore` class. Any applications you create must implement these functions. The tutorial code `amrex-tutorials/AMR/AdvectionAmrCore` provides a sample implementation in the derived class `AmrCoreAdv`.

```
//! Tag cells for refinement.  TagBoxArray tags is built on level lev grids.
virtual void ErrorEst (int lev, TagBoxArray& tags, Real time,
                       int ngrow) override = 0;

//! Make a new level from scratch using provided BoxArray and DistributionMapping.
//! Only used during initialization.
virtual void MakeNewLevelFromScratch (int lev, Real time, const BoxArray& ba,
                                      const DistributionMapping& dm) override = 0;

//! Make a new level using provided BoxArray and DistributionMapping and fill
//  with interpolated coarse level data.
virtual void MakeNewLevelFromCoarse (int lev, Real time, const BoxArray& ba,
                                     const DistributionMapping& dm) = 0;

//! Remake an existing level using provided BoxArray and DistributionMapping
//  and fill with existing fine and coarse data.
virtual void RemakeLevel (int lev, Real time, const BoxArray& ba,
                          const DistributionMapping& dm) = 0;

//! Delete level data
virtual void ClearLevel (int lev) = 0;
```

Refer to the `AmrCoreAdv` class in `amrex-tutorials/AMR/AdvectionAmrCore` for a sample implementation.

7.1.2 TagBox and Cluster

These classes are used in the grid generation process. The `TagBox` class is essentially a data structure that marks which cells are "tagged" for refinement. `Cluster` (and `ClusterList` contained within the same file) are classes that help sort tagged cells and generate a grid structure that contains all the tagged cells. These classes and their member functions are largely hidden from any application codes through simple interfaces such as `regrid` and `ErrorEst` (a routine for tagging cells for refinement).

7.1.3 FillPatchUtil and Interpolater

Many codes, including the AdvectionAmrCore example, contain an array of MultiFabs (one for each level of refinement), and then use FillPatch operations to fill temporary MultiFabs that may include a different number of ghost cells. FillPatch operations fill all cells, valid and ghost, using actual valid data at that level, space-time interpolated data from the next-coarser level, neighboring grids at the same level, and domain boundary conditions (for examples that have non-periodic boundary conditions). Note that at the coarsest level, interior cells and domain boundaries (which can be periodic or prescribed based on physical considerations) need to be filled. At non-coarsest levels, the ghost cells can also be interior or on the domain boundary, but they can also be at coarse-fine interfaces away from the domain boundary. `AMReX_FillPatchUtil.cpp/H` contains two primary functions of interest.

1. `FillPatchSingleLevel()` fills a `MultiFab` and its ghost region at a single level of refinement. The routine is flexible enough to interpolate in time between two MultiFabs associated with different times.
2. `FillPatchTwoLevels()` fills a `MultiFab` and its ghost region at a single level of refinement, assuming there is an underlying coarse level. This routine is flexible enough to interpolate the coarser level in time first using `FillPatchSingleLevel()`.

Note that `FillPatchSingleLevel()` and `FillPatchTwoLevels()` call the single-level routines `MultiFab::FillBoundary` and `FillDomainBoundary()` to fill interior, periodic, and physical boundary ghost cells. In principle, you can write a single-level application that calls `FillPatchSingleLevel()` instead of using `MultiFab::FillBoundary` and `FillDomainBoundary()`.

The FillPatchUtil routines use an Interpolator. This is largely hidden from application codes. AMReX_Interpolater.cpp/H contains the virtual base class Interpolater, which provides an interface for coarse-to-fine spatial interpolation operators. The fillpatch routines described above require an Interpolater for FillPatchTwoLevels(). Within AMReX_Interpolater.cpp/H, the derived classes are:

- NodeBilinear
- CellBilinear
- CellConservativeLinear
- CellConservativeProtected
- CellConservativeQuartic
- CellQuadratic
- PCInterp
- FaceLinear
- FaceDivFree: This is more accurately a divergence-preserving interpolation on face-centered data, i.e., it ensures the divergence of the fine ghost cells matches the value of the divergence of the underlying coarse cell. All fine cells overlying a given coarse cell will have the same divergence, even when the coarse grid divergence is spatially varying. Note that when using this with FillPatch for time sub-cycling, the coarse grid times may not match the fine grid time, in which case FillPatch will create coarse values at the fine time before calling this interpolation and the result of the FillPatch is *not* guaranteed to preserve the original divergence.

These Interpolaters can be executed on CPU or GPU, with certain limitations:

- CellConservativeProtected only works in 2D and 3D.
- CellQuadratic does not support spherical coordinates in 2D.
- CellConservativeQuartic only works with a refinement ratio of 2.
- FaceDivFree only works in 2D and 3D, and the refinement ratio in each direction must be 2 or 4. On GPUs, a refinement ratio other than 2 must be run on the device. In 2D RZ, only a refinement ratio of 2 is supported.

7.1.4 Using FluxRegisters

AMReX_FluxRegister.cpp/H contains the class FluxRegister, which is derived from the class BndryRegister (in amrex/Src/Boundary/AMReX_BndryRegister). In the most general terms, a FluxRegister is a special type of BndryRegister that stores and manipulates data (most often fluxes) at coarse-fine interfaces. A simple usage scenario comes from a conservative discretization of a hyperbolic system:

$$\frac{\partial \phi}{\partial t} = \nabla \cdot \mathbf{F} \rightarrow \frac{\phi_{i,j}^{n+1} - \phi_{i,j}^{n}}{\Delta t} = \frac{F_{i+1/2,j} - F_{i-1/2,j}}{\Delta x} + \frac{F_{i,j+1/2} - F_{i,j-1/2}}{\Delta y}.$$

Consider a two-level, two-dimensional simulation. A standard methodology for advancing the solution in time is to first advance the coarse grid solution ignoring the fine level, and then advance the fine grid solution using the coarse level only to supply boundary conditions. At the coarse-fine interface, the area-weighted fluxes from the fine grid advance do not in general match the underlying flux from the coarse grid face, resulting in a lack of global conservation. Note that for subcycling-in-time algorithms (where for each coarse grid advance, the fine grid is advanced r times using a coarse grid time step reduced by a factor of r, where r is the refinement ratio), the coarse grid flux must be compared to the area *and* time-weighted fine grid fluxes. A FluxRegister accumulates and ultimately stores the net difference in fluxes between the coarse grid and fine grid advance over each face over a given coarse time step. The simplest possible synchronization step is to modify the coarse grid solution in the coarse cells immediately adjacent to the coarse-fine interface to account for the mismatch stored in the FluxRegister. This can be done "simply" by taking the coarse-level divergence of the data in the FluxRegister using the reflux function.

The Fortran routines that perform the actual floating-point work associated with incrementing data in a FluxRegister are contained in the files AMReX_FLUXREG_F.H and AMReX_FLUXREG_xD.F.

7.1.5 AmrParticles and AmrParGDB

The AmrCore/ directory contains derived classes for dealing with particles in a multi-level framework. The description of the base classes are given in the chapter on *Particles*.

AMReX_AmrParticles.cpp/H contains the classes `AmrParticleContainer` and `AmrTracerParticleContainer`, which are derived from the classes `ParticleContainer` (in `amrex/Src/Particle/AMReX_Particles`) and `TracerParticleContainer` (in `amrex/Src/Particle/AMReX_TracerParticles`).

AMReX_AmrParGDB.cpp/H contains the class `AmrParGDB`, which is derived from the class `ParGDBBase` (in `amrex/Src/Particle/AMReX_ParGDB`).

7.2 Example: AdvectionAmrCore

7.2.1 The Advection Equation

We seek to solve the advection equation on a multi-level, adaptive grid structure:

$$\frac{\partial \phi}{\partial t} = -\nabla \cdot (\phi \mathbf{U}).$$

The velocity field is a specified divergence-free (so the flow field is incompressible) function of space and time. The initial scalar field is a Gaussian profile. To integrate these equations on a given level, we use a simple conservative update,

$$\frac{\phi^{n+1}_{i,j} - \phi^{n}_{i,j}}{\Delta t} = -\frac{(\phi u)^{n+1/2}_{i+1/2,j} - (\phi u)^{n+1/2}_{i-1/2,j}}{\Delta x} - \frac{(\phi v)^{n+1/2}_{i,j+1/2} - (\phi v)^{n+1/2}_{i,j-1/2}}{\Delta y},$$

where the velocities on faces are prescribed functions of space and time, and the scalars on faces are computed by upwinding with limited slopes. The fluxes in this case are the face-centered "ϕu" and "ϕv" terms. Each level is advanced with Heun's method (a two-stage Runge-Kutta method), and the fluxes of the two stages are averaged, so that they are time-centered.

We use a subcycling-in-time approach where finer levels are advanced with smaller time steps than coarser levels, and then synchronization is later performed between levels. More specifically, the multi-level procedure can most easily be thought of as a recursive algorithm in which, to advance level ℓ, $0 \le \ell \le \ell_{\max}$, the following steps are taken:

- Advance level ℓ in time by one time step, Δt^{ℓ}, as if it is the only level. If $\ell > 0$, obtain boundary data (i.e. fill the level ℓ ghost cells) using space- and time-interpolated data from the grids at $\ell - 1$ where appropriate.
- If $\ell < \ell_{\max}$
  - Advance level $(\ell + 1)$ for r time steps with $\Delta t^{\ell+1} = \frac{1}{r}\Delta t^{\ell}$.
  - Synchronize the data between levels ℓ and $\ell + 1$.

Specifically, for a 3-level simulation, as depicted in *Schematic of subcycling-in-time algorithm.*:

1. Integrate $\ell = 0$ over Δt.
2. Integrate $\ell = 1$ over $\Delta t/2$.
3. Integrate $\ell = 2$ over $\Delta t/4$.
4. Integrate $\ell = 2$ over $\Delta t/4$.
5. Synchronize levels $\ell = 1, 2$.
6. Integrate $\ell = 1$ over $\Delta t/2$.
7. Integrate $\ell = 2$ over $\Delta t/4$.

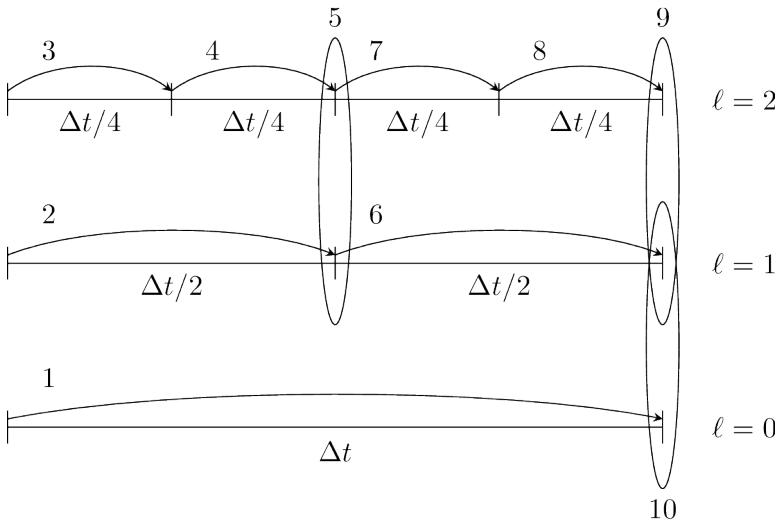


Fig. 7.1: Schematic of subcycling-in-time algorithm.

8. Integrate $\ell = 2$ over $\Delta t/4$.
9. Synchronize levels $\ell = 1, 2$.
10. Synchronize levels $\ell = 0, 1$.

For the scalar field, we keep track of volume and time-weighted fluxes at coarse-fine interfaces. We accumulate area and time-weighted fluxes in `FluxRegister` objects, which can be thought of as special boundary FABsets associated with coarse-fine interfaces. Since the fluxes are area and time-weighted (and sign-weighted, depending on whether they come from the coarse or fine level), the flux registers essentially store the extent by which the solution does not maintain conservation. Conservation only happens if the sum of the (area and time-weighted) fine fluxes equals the coarse flux, which in general is not true.

The idea behind the level $\ell/(\ell+1)$ synchronization step is to correct for sources of mismatch in the composite solution:

1. The data at level ℓ that underlie the level $\ell+1$ data are not synchronized with the level $\ell+1$ data. This is simply corrected by overwriting covered coarse cells to be the average of the overlying fine cells.
2. The area and time-weighted fluxes from the level ℓ faces and the level $\ell+1$ faces do not agree at the $\ell/(\ell+1)$ interface, resulting in a loss of conservation. The remedy is to modify the solution in the coarse cells immediately next to the coarse-fine interface to account for the mismatch stored in the flux register (computed by taking the coarse-level divergence of the flux register data).

7.2.2 Code Structure

The source code of the example is in `amrex-tutorials/AMR/AdvectionAmrCore`.

- `main.cpp` sets defaults for the parameters `AmrCore` requires, runs the simulation, and checks that the total mass is conserved.
- `AmrCoreAdv.H` and `AmrCoreAdv.cpp` contain the `AmrCoreAdv` class, which is derived from `AmrCore`: the level data, the `AmrCore` callbacks, `FillPatch`, and the time stepping.
- `Advance.cpp` advances a single level, computes the fluxes and estimates the time step.
- `IO.cpp` writes plotfiles and checkpoints, and reads a checkpoint for restart.
- `Kernels.H` contains the initial data, the velocity field and the upwind flux.

The code is built with the `GNUmakefile` or `CMakeLists.txt` in that directory and run with the `inputs` file there.

Here is a high-level pseudo-code of the flow of the program:

```
/* AdvectionAmrCore Pseudocode */
main()
  AmrCoreAdv amr_core_adv; // build an AmrCoreAdv object
  amr_core_adv.InitData()  // initialize data on all levels
    AmrCore::InitFromScratch()
      AmrMesh::MakeNewGrids()
        AmrMesh::MakeBaseGrids() // define level 0 grids
        AmrCoreAdv::MakeNewLevelFromScratch()
          /* allocate phi_old, phi_new and flux registers, fill phi */
        if (max_level > 0) {
          do {
            AmrMesh::MakeNewGrids()
              /* construct next finer grid based on tagging criteria */
            AmrCoreAdv::MakeNewLevelFromScratch()
              /* allocate phi_old, phi_new and flux registers, fill phi */
          } while (finest_level < max_level);
        }
  amr_core_adv.Evolve()
    loop over coarse time steps {
      ComputeDt()
      TimeStepWithSubcycling(lev=0) // advance a level
        /* check regrid conditions and regrid if necessary */
        AdvanceLevel()
          /* copy phi into a MultiFab and fill ghost cells */
          /* advance phi */
          /* update flux registers */
        if (lev < finest_level) {
          TimeStepWithSubcycling(lev+1) // recursive call to advance the next-finer␣
↪level "r" times
            /* check regrid conditions and regrid if necessary */
            AdvanceLevel()
              /* copy phi into a MultiFab and fill ghost cells */
              /* advance phi */
              /* update flux registers */
          Reflux()        // synchronize lev and lev+1 using FluxRegister divergence
          AverageDownTo() // set covered coarse cells to be the average of fine
        }
      /* write plotfile and checkpoint */
    }
```

7.2.3 The AmrCoreAdv Class

This example uses the class `AmrCoreAdv`, which is derived from the class `AmrCore` (which is derived from `AmrMesh`). It is declared in `AmrCoreAdv.H`, and its member functions are implemented in the source files listed in the Code Structure section above.

7.2.4 FluxRegisters

The function `AmrCoreAdv::AdvanceLevel()` computes the fluxes on all faces of a level (in `AmrCoreAdv::ComputeFluxes()`) and uses them to update the state. The same fluxes are then used to set or increment the flux registers. Note that the flux register `flux_reg[lev]` is associated with the interface between levels `lev` and `lev-1`.

```
if (do_reflux) {
    // The registers hold flux * area * dt. The coarse level of an
    // interface subtracts its flux, and the fine level adds its fluxes.
    for (int idim = 0; idim < AMREX_SPACEDIM; ++idim) {
        Real area = 1.0_rt;
        for (int d = 0; d < AMREX_SPACEDIM; ++d) {
            if (d != idim) { area *= geom[lev].CellSize(d); }
        }
        if (lev < finest_level) {
            flux_reg[lev+1]->CrseInit(flux[idim], idim, 0, 0, 1, -dt_lev*area);
        }
        if (lev > 0) {
            flux_reg[lev]->FineAdd(flux[idim], idim, 0, 0, 1, dt_lev*area);
        }
    }
}
```

The synchronization is performed at the end of `AmrCoreAdv::TimeStepWithSubcycling`:

```
if (do_reflux) {
    // Replace the coarse flux with the time and area averaged fine flux
    // on the coarse/fine boundary.
    flux_reg[lev+1]->Reflux(phi_new[lev], 1.0_rt, 0, 0, 1, geom[lev]);
}

AverageDownTo(lev); // average lev+1 down to lev
```

7.2.5 Regridding

The regrid function belongs to the `AmrCore` class (it is virtual – in this tutorial we use the implementation in `AmrCore`).

At the beginning of each time step, we check whether we need to regrid. In this example, we use `regrid_int` and keep track of how many times each level has been advanced. When any given level $\ell < \ell_{\max}$ has been advanced a number of times equal to a multiple of `regrid_int`, we call the `regrid` function.

```
void
AmrCoreAdv::TimeStepWithSubcycling (int lev, Real time)
{
    // Regrid levels finer than lev every regrid_int steps of lev, unless a
    // coarser level has just done it.
    if (regrid_int > 0 && lev < max_level && istep[lev] > last_regrid_step[lev]
        && istep[lev] % regrid_int == 0)
    {
        // regrid could add newly refined levels
        // (if finest_level < max_level)
        // so we save the previous finest level index
        int const old_finest = finest_level;
        regrid(lev, time);
        for (int k = lev; k <= finest_level; ++k) {
            last_regrid_step[k] = istep[k];
        }
        // if there are newly created levels, set the time step
        for (int k = old_finest+1; k <= finest_level; ++k) {
```

(continues on next page)

(continued from previous page)

```
            dt[k] = dt[k-1] / nsubsteps[k];
        }
    }
    ...
}
```

Central to the regridding process is the concept of "tagging" cells that need refinement. `ErrorEst` is a pure virtual function of `AmrCore`, so each application code must contain an implementation. In this example, the refinement criterion is an `AMRErrorTag` built from the `adv.phierr` runtime parameter, a list of thresholds starting at level 0; levels beyond the end of the list use its last value. A cell is tagged where ϕ is greater than the threshold of its level. `ErrorEst` applies it to the `TagBoxArray`:

```
void
AmrCoreAdv::ErrorEst (int lev, TagBoxArray& tags, Real time, int /*ngrow*/)
{
    for (auto const& tag : error_tags) {
        tag(tags, &phi_new[lev], TagBox::CLEAR, TagBox::SET, time, lev, Geom(lev));
    }
}
```

An application can also tag cells directly in `ErrorEst`, for example with a `ParallelFor` over the `TagBoxArray`, to implement more elaborate criteria such as gradients or logical expressions.

7.2.6 FillPatch

This example has two functions, `AmrCoreAdv::FillPatch` and `AmrCoreAdv::FillCoarsePatch`, that make use of functions in AmrCore/AMReX_FillPatchUtil.

In `AmrCoreAdv::AdvanceLevel`, we create a temporary `MultiFab` called `Sborder`, which is essentially ϕ but with ghost cells filled in. The valid and ghost cells are filled in from actual valid data at that level, space-time interpolated data from the next-coarser level, neighboring grids at the same level, or domain boundary conditions (for examples that have non-periodic boundary conditions).

```
MultiFab Sborder(grids[lev], dmap[lev], 1, 2);
FillPatch(lev, stage_time, Sborder);
```

Several other calls to fillpatch routines are hidden from the user in the regridding process.

CHAPTER

EIGHT

AMR SOURCE CODE

The source code in `amrex/Src/Amr` contains a number of classes, most notably `Amr`, `AmrLevel`, and `LevelBld`. These classes provide a more well-developed set of tools for writing AMR codes than the classes created for the Advection-AmrCore tutorial.

- The `Amr` class is derived from `AmrCore`, and manages data across the entire AMR hierarchy of grids.
- The `AmrLevel` class is a pure virtual class for managing data at a single level of refinement.
- The `LevelBld` class is a pure virtual class for defining variable types and attributes.

Many of our mature, public application codes contain derived classes that inherit directly from `AmrLevel`. These include:

- The `Castro` class in our compressible astrophysics code, CASTRO, (available in the AMReX-Astro/Castro github repository)
- The `Nyx` class in our computational cosmology code, Nyx (available in the AMReX-Astro/Nyx github repository).
- Our incompressible Navier-Stokes code, IAMR (available in the AMReX-codes/IAMR github repository) has a pure virtual class called `NavierStokesBase` that inherits from `AmrLevel`, and an additional derived class `NavierStokes`.
- Our low Mach number combustion code PeleLM (available in the AMReX-Combustion/PeleLM github repository) contains a derived class `PeleLM` that also inherits from `NavierStokesBase` (but does not use `NavierStokes`).

The tutorial code in `amrex-tutorials/AMR/WaveAmrLevel` gives a simple example of a class derived from `AmrLevel` that solves the wave equation on a subcycling-in-time AMR hierarchy, and `amrex-tutorials/Applications/CNS` is a complete compressible flow code built on these classes. Compare them with the AdvectionAmrCore tutorial and the chapter on *AmrCore Source Code*, which build the same kind of simulation directly on `AmrCore` without the classes in `amrex/Src/Amr`.

The tutorial codes also contain a class derived from `LevelBld` (in `amrex/Src/Amr`). This class is used to define variable types (how many, nodality, interlevel interpolation stencils, etc.).

8.1 Amr Class

The `Amr` class is designed to manage parts of the computation which do not belong on a single level, like establishing and updating the hierarchy of levels, global timestepping, and managing the different `AmrLevels`. Most likely you will not need to derive any classes from `Amr`. Our mature application codes use this base class without any derived classes.

One of the most important data members is an array of `AmrLevels` - the `Amr` class calls many functions from the `AmrLevel` class to do things like advance the solution on a level, compute a time step to be used for a level, etc.

8.2 AmrLevel Class

Pure virtual functions include:

- `computeInitialDt` Compute an array of time steps for each level of refinement. Called at the beginning of the simulation.
- `computeNewDt` Compute an array of time steps for each level of refinement. Called at the end of a coarse level advance.
- `advance` Advance the grids at a level.
- `post_timestep` Work after a time step at a given level. The AMR synchronization is usually done here.
- `post_regrid` Work after regridding, e.g., redistributing particles.
- `post_init` Work after initialization, e.g., AMR synchronization.
- `initData` Initialize the data on a given level at the beginning of the simulation.
- `init` There are two versions of this function used to initialize data on a level during regridding. One version is specifically for the case where the level did not previously exist (a newly created refined level).
- `errorEst` Perform the tagging at a level for refinement.

8.2.1 StateData

The most important data managed by the `AmrLevel` is an array of `StateData`, which holds the scalar fields, etc., in the boxes that together make up the level.

`StateData` is a class that essentially holds a pair of MultiFabs: one at the old time and one at the new time. AMReX knows how to interpolate in time between these states to get data at any intermediate point in time. The main data that we care about in our application codes (such as the fluid state) will be stored as `StateData`. Essentially, data is made `StateData` if we need it to be stored in checkpoints/plotfiles, and/or we want it to be automatically interpolated when we refine. An `AmrLevel` stores an array of `StateData` (in a C++ array called `state`). We index this array using integer keys (defined via an `enum` in, e.g., `AmrLevelWave.H` in the WaveAmrLevel tutorial):

```
enum StateType { State_Type = 0, NUM_STATE_TYPE };
```

In the tutorial code, we use the function `AmrLevelWave::variableSetUp` to tell our simulation about the `StateData` (e.g., how many variables, ghost cells, nodality, etc.). Note that if you have more than one `StateType`, each of the different `StateData` carried in the state array can have different numbers of components, ghost cells, boundary conditions, etc. This is the main reason we separate all this data into separate `StateData` objects collected together in an indexable array.

8.3 LevelBld Class

The `LevelBld` class is a pure virtual class for defining variable types and attributes. To more easily understand its usage, refer to the derived class, `LevelBldWave` in the WaveAmrLevel tutorial. The `variableSetUp` and `variableCleanUp` functions are implemented, and in this tutorial they call routines in the `AmrLevelWave` class, e.g.,

```
void
AmrLevelWave::variableSetUp ()
{
    read_params();

    // One cell-centered StateData with two components and the ghost cells
    // of the stencil. Fine-level ghost cells at the coarse/fine boundary
```

(continues on next page)

(continued from previous page)

```
    // are interpolated from the coarse level with a fourth-order
    // interpolater, to match the fourth-order spatial discretization.
    desc_lst.addDescriptor(State_Type, IndexType::TheCellType(),
                           StateDescriptor::Point, nghost, ncomp,
                           &cell_quartic_interp);

    int lo_bc[AMREX_SPACEDIM] = {AMREX_D_DECL(BCType::ext_dir,    // external Dirichlet
                                              BCType::int_dir,    // periodic
                                              BCType::int_dir)};  // periodic
    int hi_bc[AMREX_SPACEDIM] = {AMREX_D_DECL(BCType::ext_dir,
                                              BCType::int_dir,
                                              BCType::int_dir)};
    Vector<BCRec> bcs(ncomp, BCRec(lo_bc, hi_bc));

    StateDescriptor::BndryFunc bndryfunc(wave_bcfill);
    bndryfunc.setRunOnGPU(true);

    desc_lst.setComponent(State_Type, 0, {"u", "v"}, bcs, bndryfunc);
}
```

We see how to define the `StateType`, including nodality, whether or not we want the variable to represent a point in time or an interval over time (useful for returning the time associated with data), the number of ghost cells, number of components, and the interlevel interpolation (See AMReX_Interpolator for various interpolation types). We also see how to specify physical boundary functions by providing a function (in this case, `wave_bcfill`, defined in `AmrLevelWave.cpp` in the tutorial source code, which fills the external Dirichlet boundaries in the x direction).

8.4 Examples

The WaveAmrLevel tutorial is a small code built on `Amr` and `AmrLevel`. It contains the `AmrLevelWave` class, derived from the base `AmrLevel` class, and the `LevelBldWave` class, derived from the base `LevelBld` class as described above. It advances the state with `AmrLevel::RK`, which provides Runge-Kutta time stepping with subcycling.

The CNS tutorial is a complete compressible flow code built on the same classes, with refluxing, derived variables for plotfiles and GPU support. It is meant to be copied as the starting point of an application.

The pseudocode for the main program of such a code is given below.

```
/* AmrLevel Pseudocode */
main()
  Amr amr(&level_bld);
  amr.init()
  loop {
    amr.coarseTimeStep()
      /* compute dt */
      timeStep()
        amr_level[level]->advance()
        /* call timeStep r times for next-finer level */
        amr_level[level]->post_timestep() // AMR synchronization
      postCoarseTimeStep()
      /* write plotfile and checkpoint */
  }
  /* write final plotfile and checkpoint */
```

8.5 Particles

amrex/Tests/Amr/Advection_AmrLevel is another AmrLevel code, with an option to turn on passively advected particles. Its Exec/SingleVortex/GNUmakefile builds with USE_PARTICLES = TRUE, and the inputs.tracers file there turns the particles on with adv.do_tracers = 1. When you run the code, within each plotfile directory there will be a subdirectory called "Tracer". These can be visualized using either yt or ParaView (refer to the chapter on *Visualization*).

CHAPTER

NINE

FORK-JOIN

An AMReX program consists of a set of MPI ranks cooperating together on distributed data. Typically, all of the ranks in a job compute in a bulk-synchronous, data-parallel fashion, where every rank does the same sequence of operations, each on different parts of the distributed data.

The AMReX Fork-Join functionality described here allows the user to divide the job's MPI ranks into subgroups (i.e. *fork*) and assign each subgroup an independent task to compute in parallel with each other. After all of the forked child tasks complete, they synchronize (i.e. *join*), and the parent task continues execution as before.

The Fork-Join operation can also be invoked in a nested fashion, creating a hierarchy of fork-join operations, where each fork further subdivides the ranks of a task into child tasks. This approach enables heterogeneous computation and reduces the strong scaling penalty for operations with less inherent parallelism or with large communication overheads.

The fork-join operation is accomplished by:

a) redistributing MultiFab data so that **all** of the data in each registered MultiFab is visible to ranks within a subtask, and

b) dividing the root MPI communicator into sub-communicators so that each subgroup of ranks in a task will only synchronize with each other during subtask collectives (e.g. for `MPI_Allreduce`).

When the program starts, all of the ranks in the MPI communicator are in the root task.

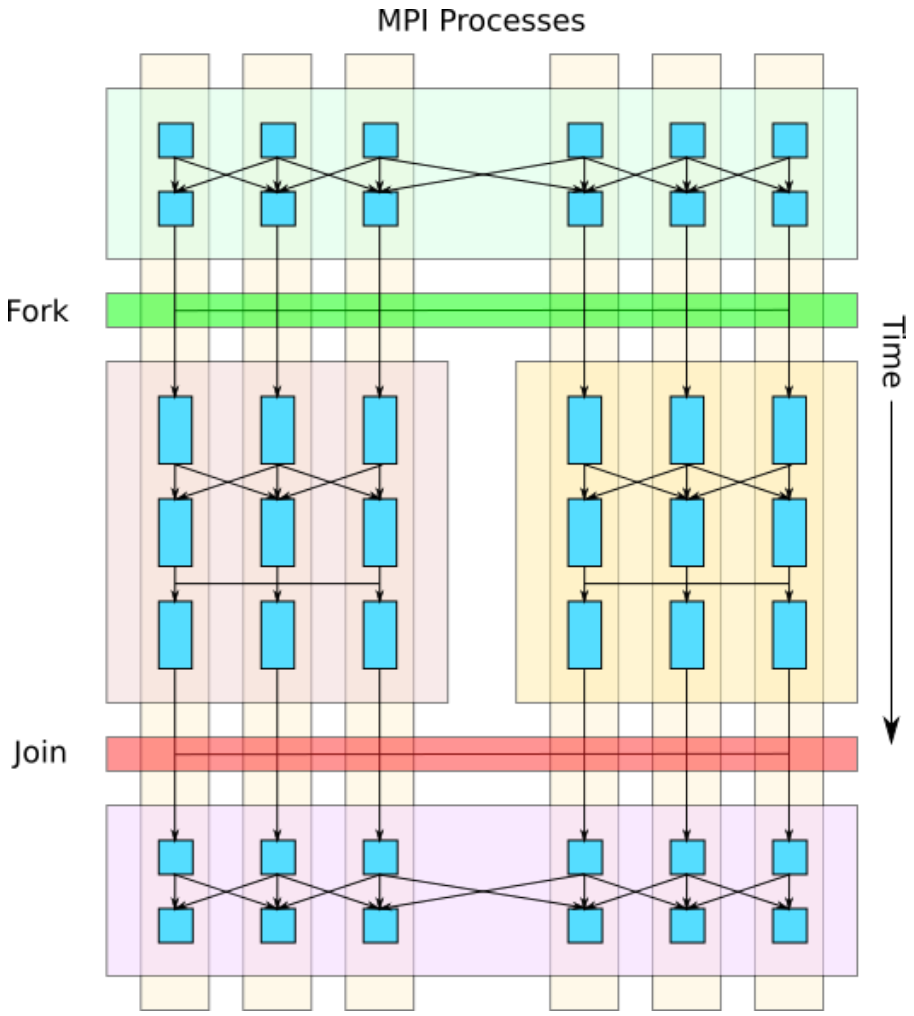


Fig. 9.1: Example of a fork-join operation where the parent task's MPI processes (ranks) are split into two independent child tasks that execute in parallel and then join to resume execution of the parent task.

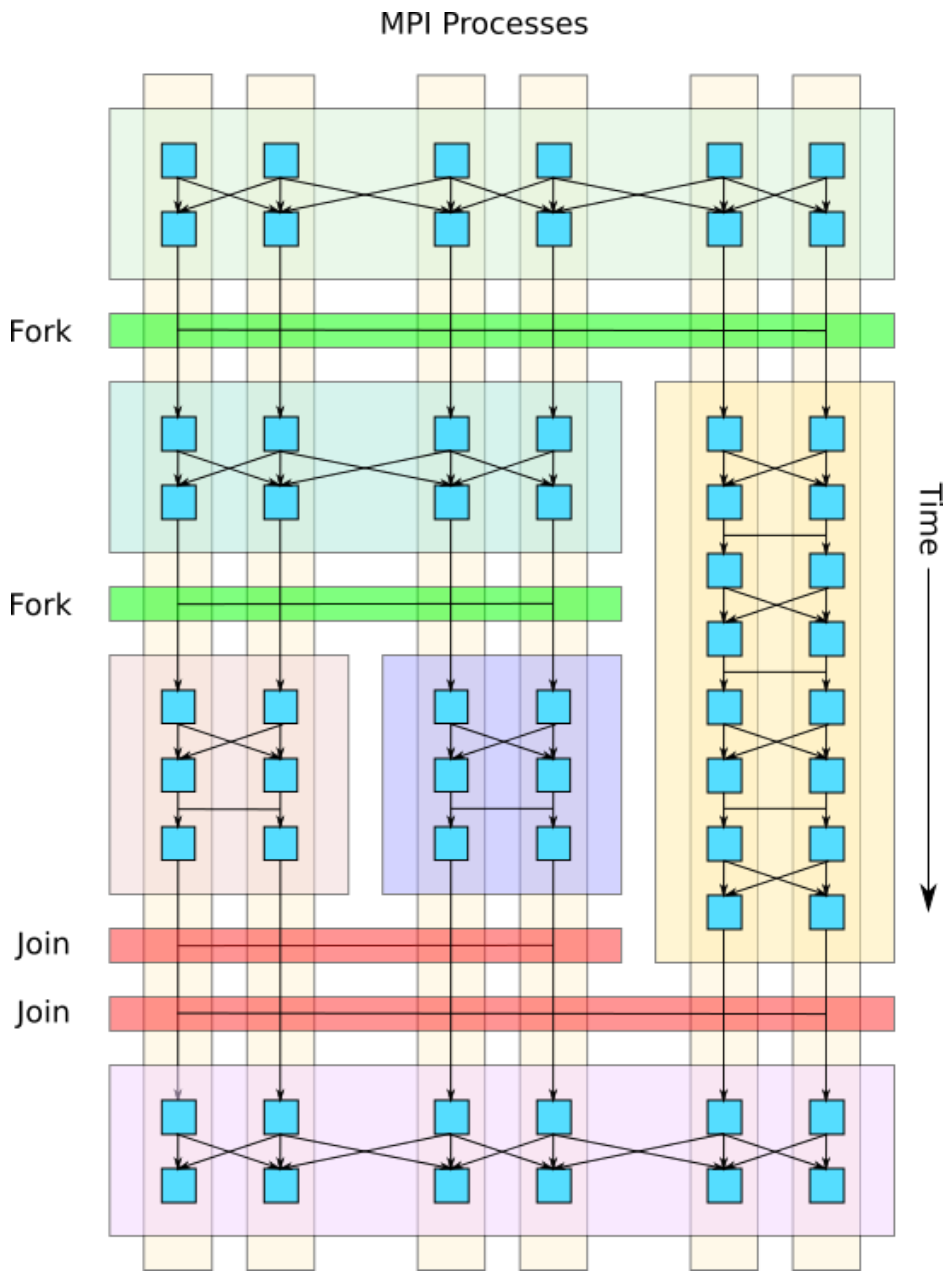


Fig. 9.2: Example of nested fork-join operations where a child task is further split into more subtasks.

CHAPTER

TEN

I/O (PLOTFILE, CHECKPOINT)

In this chapter, we will discuss parallel I/O capabilities for mesh data in AMReX. The section on *Particle I/O* will discuss I/O for particle data.

10.1 Plotfile

AMReX has its own native plotfile format. Many visualization tools are available for AMReX plotfiles (see the chapter on *Visualization*). AMReX provides the following two functions for writing a generic AMReX plotfile. Many AMReX application codes may have their own plotfile routines that store additional information such as compiler options, git hashes of the source codes and `ParmParse` runtime parameters.

```
void WriteSingleLevelPlotfile (const std::string &plotfilename,
                               const MultiFab &mf,
                               const Vector<std::string> &varnames,
                               const Geometry &geom,
                               Real time,
                               int level_step);

void WriteMultiLevelPlotfile (const std::string &plotfilename,
                              int nlevels,
                              const Vector<const MultiFab*> &mf,
                              const Vector<std::string> &varnames,
                              const Vector<Geometry> &geom,
                              Real time,
                              const Vector<int> &level_steps,
                              const Vector<IntVect> &ref_ratio);
```

`WriteSingleLevelPlotfile` is for single level runs and `WriteMultiLevelPlotfile` is for multiple levels. The name of the plotfile is specified by the plotfilename argument. This is the top-level directory name for the plotfile. In AMReX convention, the plotfile name consists of letters followed by numbers (e.g., plt00258). `amrex::Concatenate` is a useful helper function for making such strings.

```
int istep = 258;
const std::string& pfname = amrex::Concatenate("plt",istep); // plt00258

// By default there are 5 digits, but we can change it to say 4.
const std::string& pfname2 = amrex::Concatenate("plt",istep,4); // plt0258

istep =1234567;  // Having more than 5 digits is OK.
const std::string& pfname3 = amrex::Concatenate("plt",istep); // plt1234567
```

The argument mf above (MultiFab for single level and Vector<const MultiFab*> for multi-level) is the data to be written to the disk. Note that many visualization tools expect this to be cell-centered data. So for nodal data, we need to convert them to cell-centered data through some kind of averaging. Also note that if you have data at each AMR level in several MultiFabs, you need to build a new MultiFab at each level to hold all the data on that level. This involves local data copy in memory and is not expected to significantly increase the total wall time for writing plotfiles. For the multi-level version, the function expects Vector<const MultiFab*>, whereas the multi-level data are often stored as Vector<std::unique_ptr<MultiFab>>. AMReX has a helper function for this, and one can use it as follows:

```
WriteMultiLevelPlotfile(......, amrex::GetVecOfConstPtrs(mf), ......);
```

The argument varnames has the names for each component of the MultiFab data. The size of the Vector should be equal to the number of components. The argument geom is for passing Geometry objects that contain the physical domain information. The argument time is for the time associated with the data. The argument level_step is for the current time step associated with the data. For multi-level plotfiles, the argument nlevels is the total number of levels, and we also need to provide the refinement ratio via an Vector of size nlevels-1.

We note that AMReX does not overwrite old plotfiles if the new plotfile has the same name. The old plotfiles will be renamed to new directories named like plt00350.old.46576787980.

10.2 Async Output

AMReX provides the ability to print MultiFabs, plotfiles and particle data asynchronously. Asynchronous output works by creating a copy of the data at the time of the call, which is written to disk by a persistent thread created during AMReX's initialization. This allows the calculation to continue immediately, which can drastically reduce wall time spent writing to disk.

If the number of output files is less than the number of MPI ranks, AMReX's async output requires MPI to be initialized with THREAD_MULTIPLE support. THREAD_MULTIPLE support allows multiple threads to make MPI calls simultaneously. This support is required to allow AMReX applications to perform MPI work while the Async Output concurrently pings ranks to signal that they can safely begin writing to their assigned files. However, THREAD_MULTIPLE can introduce additional overhead as each thread's MPI operations must be scheduled safely around each other. Therefore, AMReX uses a lower level of support, SERIALIZED, by default and applications have to turn on THREAD_MULTIPLE support.

To turn on Async Output, use the input flag amrex.async_out=1. The number of output files can also be set, using amrex.async_out_nfiles. The default number of files is 64. If the number of ranks is larger than the number of files, THREAD_MULTIPLE must be turned on by adding MPI_THREAD_MULTIPLE=TRUE to the GNUMakefile. Otherwise, AMReX will throw an error.

Note that async output always writes FAB data in the machine's native format. The fab.format input parameter is ignored.

Async Output works for a wide range of AMReX calls, including:

- amrex::WriteSingleLevelPlotfile()
- amrex::WriteMultiLevelPlotfile()
- amrex::WriteMLMF()
- VisMF::AsyncWrite()
- ParticleContainer::Checkpoint()
- ParticleContainer::WritePlotFile()
- Amr::writePlotFile()
- Amr::writeSmallPlotFile()

- `Amr::checkpoint()`
- `AmrLevel::writePlotFile()`
- `StateData::checkPoint()`
- `FabSet::write()`

Be aware: when using Async Output, a thread is spawned and exclusively used to perform output throughout the runtime. As such, you may oversubscribe resources if you launch an AMReX application that assigns all available hardware threads in another way, such as OpenMP. If you see any degradation when using Async Output and OpenMP, try using one less thread in `OMP_NUM_THREADS` to prevent oversubscription and get more consistent results.

10.3 HDF5 Plotfile

Besides AMReX's native plotfile, applications can also write plotfiles in the HDF5 format, which is a cross-platform, self-describing file format. The HDF5 plotfiles store the same information as the native format and have the additional compression capability that can reduce the file size. Currently supported compression libraries include SZ and ZFP.

To enable HDF5 output, AMReX must be compiled and linked to an HDF5 library with parallel I/O support, by adding `USE_HDF5=TRUE` and `HDF5_HOME=/path/to/hdf5/install/dir` to the GNUMakefile. Many HPC systems have an HDF5 module available that can be loaded with `module load hdf5` or `module load cray-hdf5-parallel`. To download and compile HDF5 from source code, please go to the HDF5 Download webpage and follow the instructions (latest version is recommended and remember to turn on parallel I/O).

The following are two functions for writing a generic AMReX plotfile in HDF5 format, which are very similar to the AMReX native write functions.

```
void WriteSingleLevelPlotfileHDF5 (const std::string &plotfilename,
                                   const MultiFab &mf,
                                   const Vector<std::string> &varnames,
                                   const Geometry &geom,
                                   Real t,
                                   int level_step,
                                   const std::string &compression);

void WriteMultiLevelPlotfileHDF5 (const std::string &plotfilename,
                                  int nlevels,
                                  const Vector<const MultiFab*> &mf,
                                  const Vector<std::string> &varnames,
                                  const Vector<Geometry> &geom,
                                  Real time,
                                  const Vector<int> &level_steps,
                                  const Vector<IntVect> &ref_ratio,
                                  const std::string &compression);
```

`WriteSingleLevelPlotfileHDF5` is for single level runs and `WriteMultiLevelPlotfileHDF5` is for multiple levels. Their arguments are the same as the native ones except for the last one, which is optional and specifies the compression parameters. These two functions write plotfiles with a Chombo-compatible HDF5 file schema, which can be read by visualization tools such as VisIt and ParaView using their built-in Chombo reader plugin (see the chapter on *Visualization*).

10.3.1 HDF5 Plotfile Compression

To enable SZ or ZFP data compression on the HDF5 datasets, the corresponding compression library and its HDF5 plugin must be available. To compile the SZ or ZFP plugin, please refer to their documentation, H5Z-SZ and H5Z-ZFP, and add `USE_HDF5_SZ=TRUE`, `SZ_HOME=`, or `USE_HDF5_ZFP=TRUE`, `ZFP_HOME=`, `H5Z_HOME=` to the GNUMakefile.

ZLIB compression is available without external libraries or other make flags. Different compression levels (at the cost of read/write time) can be used, just like GZIP.

The string argument `compression` in the above two functions controls whether to enable data compression and its parameters. Currently supported options include:

- **No compression**
  - `None@0`
- **ZLIB compression**
  - `ZLIB@compression_level`
- **SZ compression**
  - `SZ@/path/to/sz.config`
- **ZFP compression**
  - `ZFP_RATE@rate`
  - `ZFP_PRECISION@precision`
  - `ZFP_ACCURACY@accuracy`
  - `ZFP_REVERSIBLE@reversible`

AMReX also provides the `fconvert` tool in `Tools/Plotfile` for converting existing native plotfiles to HDF5 plotfiles after a simulation has completed. It accepts the same compression descriptor string and can therefore be used for post-processing workflows that want HDF5 output with ZLIB or ZFP compression.

Using compression requires data to be stored in a chunked format. The size of these chunks can (and generally should) be configured by changing the `HDF5_CHUNK_SIZE` environment variable, with a default value of 1024 elements provided.

For MPI-enabled builds, two additional file access properties can be tuned via environment variables. `HDF5_ALIGNMENT_SIZE` (in bytes, default 16 MiB) sets both the threshold and alignment passed to `H5Pset_alignment`: object allocations of at least this size are aligned to a multiple of it, while smaller allocations are left unpadded. `HDF5_BLOCK_SIZE` (in bytes, default 4 MiB) sets the minimum metadata block allocation size via `H5Pset_meta_block_size`, which aggregates small metadata writes to reduce I/O overhead on parallel file systems.

10.3.2 HDF5 Asynchronous Output

The HDF5 output also comes with its own asynchronous I/O support, which is different from the native async output mentioned in the previous section. To use the HDF5 asynchronous I/O VOL connector, download and compile by following the instructions at vol-async.

Since the HDF5 asynchronous I/O in AMReX does not use double buffering, vol-async must be compiled with `-DENABLE_WRITE_MEMCPY=1` added to `CFLAGS`. When compiling AMReX, add `USE_HDF5_ASYNC = TRUE`, `ABT_HOME=`, `ASYNC_HOME=`, and `MPI_THREAD_MULTIPLE=TRUE` to the GNUMakefile. Refer to `amrex/Tests/HDF5Benchmark/GNUmakefile` for the example usage.

10.3.3 Alternative HDF5 Plotfile Schema

`WriteSingleLevelPlotfileHDF5` and `WriteMultiLevelPlotfileHDF5` write HDF5 plotfiles that store all the data on an AMR level as one 1D HDF5 dataset. Each AMR box's data is linearized and the data of different variables are concatenated, resulting in an interleaved pattern for each variable. This could be undesirable when compression is used, as it may lead to applying the compression algorithm to multiple variables with different value ranges and characteristics, and reduce the compression ratio. To overcome this issue, two additional functions are provided to write each variable into individual HDF5 datasets: `WriteSingleLevelPlotfileHDF5MultiDset` and `WriteMultiLevelPlotfileHDF5MultiDset`. They use the exact same arguments as `WriteSingleLevelPlotfileHDF5` and `WriteMultiLevelPlotfileHDF5`. However, this alternative schema is not yet supported by the visualization tools.

10.4 Checkpoint File

Checkpoint files are used for restarting simulations from the point at which the checkpoints are written. Each application code has its own set of data needed for restart. AMReX provides I/O functions for basic data structures like `MultiFab` and `BoxArray`. These functions can be used to build codes for reading and writing checkpoint files. Since each application code has its own requirements, there is no standard AMReX checkpoint format. However, we have provided an example restart capability in the tutorials CheckpointRestart (single level) and AdvectionAmrCore (multi-level). Refer to the functions `ReadCheckpointFile()` and `WriteCheckpointFile()` in the latter.

A checkpoint file is actually a directory with a name, e.g., `chk00010` containing a `Header` (text) file, along with subdirectories `Level_0`, `Level_1`, etc. containing the `MultiFab` data at each level of refinement. The `Header` file contains problem-specific data (such as the finest level, simulation time, time step, etc.), along with a printout of the `BoxArray` at each level of refinement.

When starting a simulation from a checkpoint file, a typical sequence in the code could be:

- Read in the `Header` file data (except for the `BoxArray` data).
- For each level of refinement, do the following in order:
  - Read in the `BoxArray`
  - Build a `DistributionMapping`
  - Define any `MultiFab`, `FluxRegister`, etc. objects that are built upon the `BoxArray` and the `DistributionMapping`
  - Read in the `MultiFab` data

We do this one level at a time because when you create a distribution map, it checks how much allocated `MultiFab` data already exists before assigning grids to processors.

Typically a checkpoint file is a directory containing some text files and subdirectories (e.g., `Level_0` and `Level_1`) containing various data. It is a good idea that we first make these directories ready for subsequently writing to disk. For example, to build directories `chk00010`, `chk00010/Level_0`, and `chk00010/Level_1`, you could write:

```
const std::string& checkpointname = amrex::Concatenate("chk",10);

amrex::Print() << "Writing checkpoint " << checkpointname << "\n";

const int nlevels = 2;

bool callBarrier = true;

// ---- prebuild a hierarchy of directories
// ---- dirName is built first.  if dirName exists, it is renamed.  then build
```

(continues on next page)

(continued from previous page)

```
// ---- dirName/subDirPrefix_0 .. dirName/subDirPrefix_nlevels-1
// ---- if callBarrier is true, call ParallelDescriptor::Barrier()
// ---- after all directories are built
// ---- ParallelDescriptor::IOProcessor() creates the directories
amrex::PreBuildDirectorHierarchy(checkpointname, "Level_", nlevels, callBarrier);
```

A checkpoint file of AMReX application codes often has a clear-text Header file that only the I/O process writes using `std::ofstream`. The Header file contains problem-dependent information such as the time, the physical domain size, grids, etc. that are necessary for restarting the simulation. To guarantee that precision is not lost for storing floating-point numbers like time in a clear-text file, the file stream's precision needs to be set properly. A stream buffer can also be used. For example,

```
// write Header file
if (ParallelDescriptor::IOProcessor()) {

    VisMF::IO_Buffer io_buffer(VisMF::IO_Buffer_Size);
    std::ofstream HeaderFile;
    HeaderFile.rdbuf()->pubsetbuf(io_buffer.dataPtr(), io_buffer.size());
    std::string HeaderFileName(checkpointname + "/Header");
    HeaderFile.open(HeaderFileName.c_str(), std::ofstream::out   |
                                            std::ofstream::trunc |
                                            std::ofstream::binary);

    if( ! HeaderFile.good()) {
        amrex::FileOpenFailed(HeaderFileName);
    }

    HeaderFile.precision(17);

    // write out title line
    HeaderFile << "Checkpoint file for AmrCoreAdv\n";

    // write out finest_level
    HeaderFile << finest_level << "\n";

    // write out array of istep
    for (int i = 0; i < istep.size(); ++i) {
        HeaderFile << istep[i] << " ";
    }
    HeaderFile << "\n";

    // write out array of dt
    for (int i = 0; i < dt.size(); ++i) {
        HeaderFile << dt[i] << " ";
    }
    HeaderFile << "\n";

    // write out array of t_new
    for (int i = 0; i < t_new.size(); ++i) {
        HeaderFile << t_new[i] << " ";
    }
    HeaderFile << "\n";
```

(continues on next page)

(continued from previous page)

```
    // write the BoxArray at each level
    for (int lev = 0; lev <= finest_level; ++lev) {
        boxArray(lev).writeOn(HeaderFile);
        HeaderFile << '\n';
    }
}
```

amrex::VisMF is a class that can be used for MultiFab I/O in parallel. How many processes are allowed to perform I/O simultaneously can be set via

```
VisMF::SetNOutFiles(64);  // up to 64 processes, which is also the default.
```

The optimal number is of course system dependent. The following code shows how to write a MultiFab.

```
// write the MultiFab data to, e.g., chk00010/Level_0/
for (int lev = 0; lev <= finest_level; ++lev) {
    VisMF::Write(phi_new[lev],
                 amrex::MultiFabFileFullPrefix(lev, checkpointname, "Level_", "phi"));
}
```

It should also be noted that all the data, including those in ghost cells, are written and read by VisMF::Write/Read.

For reading the Header file, AMReX can have the I/O process read the file from the disk and broadcast it to others as Vector<char>. Then all processes can read the information with std::istringstream. For example,

```
std::string File(restart_chkfile + "/Header");

VisMF::IO_Buffer io_buffer(VisMF::GetIOBufferSize());

Vector<char> fileCharPtr;
ParallelDescriptor::ReadAndBcastFile(File, fileCharPtr);
std::string fileCharPtrString(fileCharPtr.dataPtr());
std::istringstream is(fileCharPtrString, std::istringstream::in);

std::string line, word;

// read in title line
std::getline(is, line);

// read in finest_level
is >> finest_level;
GotoNextLine(is);

// read in array of istep
std::getline(is, line);
{
    std::istringstream lis(line);
    int i = 0;
    while (lis >> word) {
        istep[i++] = std::stoi(word);
    }
}
```

(continues on next page)

(continued from previous page)

```
// read in array of dt
std::getline(is, line);
{
    std::istringstream lis(line);
    int i = 0;
    while (lis >> word) {
        dt[i++] = std::stod(word);
    }
}

// read in array of t_new
std::getline(is, line);
{
    std::istringstream lis(line);
    int i = 0;
    while (lis >> word) {
        t_new[i++] = std::stod(word);
    }
}
```

The following code shows how to read in a `BoxArray`, create a `DistributionMapping`, build `MultiFab` and `FluxRegister` data, and read a `MultiFab` from a checkpoint file on a level-by-level basis:

```
for (int lev = 0; lev <= finest_level; ++lev) {

    // read in level 'lev' BoxArray from Header
    BoxArray ba;
    ba.readFrom(is);
    GotoNextLine(is);

    // create a distribution mapping
    DistributionMapping dm { ba, ParallelDescriptor::NProcs() };

    // set BoxArray grids and DistributionMapping dmap in AMReX_AmrMesh.H class
    SetBoxArray(lev, ba);
    SetDistributionMap(lev, dm);

    // build MultiFab and FluxRegister data
    int ncomp = 1;
    int nghost = 0;
    phi_old[lev].define(grids[lev], dmap[lev], ncomp, nghost);
    phi_new[lev].define(grids[lev], dmap[lev], ncomp, nghost);
    if (lev > 0 && do_reflux) {
        flux_reg[lev] = std::make_unique<FluxRegister>(grids[lev], dmap[lev],␣
↪refRatio(lev-1), lev, ncomp);
    }
}

// read in the MultiFab data
for (int lev = 0; lev <= finest_level; ++lev) {
    VisMF::Read(phi_new[lev],
```

(continues on next page)

(continued from previous page)

```
                amrex::MultiFabFileFullPrefix(lev, restart_chkfile, "Level_", "phi"));
}
```

It should be emphasized that calling `VisMF::Read` with an empty `MultiFab` (i.e., no memory allocated for floating point data) will result in a `MultiFab` with a new `DistributionMapping` that could be different from any other existing `DistributionMapping` objects and is not recommended.

CHAPTER

ELEVEN

LINEAR SOLVERS

AMReX supports both single-level solves and composite solves on multiple AMR levels, with the solution to the linear system defined on cell centers, edges or nodes. AMReX also supports the solution of linear systems with embedded boundaries. (See *Embedded Boundaries* for more details on the embedded boundary representation of complex geometry.) In addition to the iterative solvers discussed in this chapter, AMReX also supports solving Poisson equations using Fast Fourier Transform (FFT) (see chapter *Discrete Fourier Transform* for more information).

The default solution technique is geometric multigrid. AMReX also provides BiCGStab solvers, GMRES, and interfaces to the HYPRE and PETSc libraries.

In this chapter, we give an overview of the linear solvers in AMReX that solve linear systems in the canonical form

$$(A\alpha - B\nabla \cdot \beta\nabla)\phi = f, \tag{11.1}$$

where A and B are scalar constants, α and β are scalar fields, ϕ is the unknown, and f is the right-hand side of the equation. Note that Poisson's equation $\nabla^2\phi = f$ is a special case of the canonical form. The solution ϕ is at either cell centers or nodes.

For the cell-centered solver, α, ϕ and f are represented by cell-centered MultiFabs, and β is represented by `AMREX_SPACEDIM` face type MultiFabs, i.e., there are separate MultiFabs for the β coefficient in each coordinate direction.

For the nodal solver, A and α are assumed to be zero, ϕ and f are nodal, and β (which we later refer to as σ) is cell-centered.

In addition to these solvers, AMReX has support for tensor solves used to calculate the viscous terms that appear in the compressible Navier-Stokes equations. In these solves, all components of the velocity field are solved for simultaneously. The tensor solve functionality is only available for cell-centered velocity. AMReX also supports the curl-curl operator, with solutions defined on cell edges.

The tutorials in Linear Solvers show examples of using the solvers. The ImplicitHeat tutorial shows how to solve the heat equation implicitly using the solver. The tutorials also show how to add linear solvers into the build system.

11.1 MLMG and Linear Operator Classes

Multi-Level Multi-Grid or `MLMG` is a class for solving the linear system using the geometric multigrid method. The constructor of `MLMG` takes the reference to `MLLinOp`, an abstract base class of various linear operator classes, `MLABecLaplacian`, `MLPoisson`, `MLNodeLaplacian`, etc. We choose the type of linear operator class according to the type of linear system to solve. Examples of the linear operators include

- `MLABecLaplacian` for cell-centered canonical form (equation (11.1)).
- `MLPoisson` for cell-centered constant coefficient Poisson's equation $\nabla^2\phi = f$.
- `MLNodeLaplacian` for nodal variable coefficient Poisson's equation $\nabla \cdot (\sigma\nabla\phi) = f$.

- `MLTerrainPoisson` for cell-centered Poisson's equation on a terrain-following mesh (section *Terrain-Following Poisson*).

The constructors of these linear operator classes are in the form like below

```
MLABecLaplacian (const Vector<Geometry>& a_geom,
                 const Vector<BoxArray>& a_grids,
                 const Vector<DistributionMapping>& a_dmap,
                 const LPInfo& a_info = LPInfo(),
                 const Vector<FabFactory<FArrayBox> const*>& a_factory = {},
                 const int a_ncomp = 1);
```

It takes `Vectors` of `Geometry`, `BoxArray` and `DistributionMapping`. The arguments are `Vectors` because MLMG can do multi-level composite solves. If you are using it for single-level, you can do

```
// Given Geometry geom, BoxArray grids, and DistributionMapping dmap on single level
MLABecLaplacian mlabeclaplacian({geom}, {grids}, {dmap});
```

to let the compiler construct `Vectors` for you. Recall that the classes `Vector`, `Geometry`, `BoxArray`, and `DistributionMapping` are defined in chapter *Basics*. There are two new classes that are optional parameters. `LPInfo` is a class for passing parameters. `FabFactory` is used in problems with embedded boundaries (chapter *Embedded Boundaries*).

After the linear operator is built, we need to set up boundary conditions. This will be discussed later in section *Boundary Conditions*.

11.1.1 Coefficients

Next, we consider the coefficients for equation (11.1). For `MLPoisson`, there are no coefficients to set so nothing needs to be done. For `MLABecLaplacian`, we need to call member functions `setScalars`, `setACoeffs`, and `setBCoeffs`. The `setScalars` function sets the scalar constants A and B

```
void setScalars (Real a, Real b) noexcept;
```

For the general case where α and β are scalar fields, we use

```
void setACoeffs (int amrlev, const MultiFab& alpha);
void setBCoeffs (int amrlev, const Array<MultiFab const*,AMREX_SPACEDIM>&␣
↪beta);
```

For the case where α and/or β are scalar constants, there is the option to use

```
void setACoeffs (int amrlev, Real alpha);
void setBCoeffs (int amrlev, Real beta);
void setBCoeffs (int amrlev, Vector<Real> const& beta);
```

Note, however, that the solver behavior is the same regardless of which functions you use to set the coefficients. These functions solely copy the constant value(s) to a MultiFab internal to `MLMG` and so no appreciable efficiency gains can be expected.

For `MLNodeLaplacian`, one can set the variable `sigma` with the member function

```
void setSigma (int amrlev, const MultiFab& a_sigma);
```

or a constant `sigma` during declaration or definition

```
MLNodeLaplacian (const Vector<Geometry>& a_geom,
                 const Vector<BoxArray>& a_grids,
                 const Vector<DistributionMapping>& a_dmap,
                 const LPInfo& a_info = LPInfo(),
                 const Vector<FabFactory<FArrayBox> const*>& a_factory = {},
                 Real  a_const_sigma = Real(0.0));

void define (const Vector<Geometry>& a_geom,
             const Vector<BoxArray>& a_grids,
             const Vector<DistributionMapping>& a_dmap,
             const LPInfo& a_info = LPInfo(),
             const Vector<FabFactory<FArrayBox> const*>& a_factory = {},
             Real  a_const_sigma = Real(0.0));
```

Here, setting a constant `sigma` alters the internal behavior of the solver making it more efficient for this special case.

The `int` `amrlev` parameter should be zero for single-level solves. For multi-level solves, each level needs to be provided with `alpha` and `beta`, or `sigma`. For composite solves, `amrlev` 0 will mean the lowest level for the solver, which is not necessarily the lowest level in the AMR hierarchy. This is so solves can be done on different sections of the AMR hierarchy, e.g. on AMR levels 3 to 5.

After boundary conditions and coefficients are prescribed, the linear operator is ready for an MLMG object like below.

```
MLMG mlmg(mlabeclaplacian);
```

Optional parameters can be set (see section *Parameters*), and then we can use the `MLMG` member function

```
Real solve (const Vector<MultiFab*>& a_sol,
            const Vector<MultiFab const*>& a_rhs,
            Real a_tol_rel, Real a_tol_abs);
```

to solve the problem given an initial guess and a right-hand side. Zero is a perfectly fine initial guess. The two `Reals` in the argument list are the targeted relative and absolute error tolerances. The relative error tolerance is hard-coded to be at least 10^{-16}. Given the linear system $Ax = b$, the solver will terminate when the max-norm of the residual $(b - Ax)$ is less than `std::max(a_tol_abs, a_tol_rel * max_norm)`. By default, `max_norm` is equal to the greater of rhs max-norm and residual max-norm. This behavior can be modified using the `MLMG` member function `setConvergenceNormType (MLMGNormType norm)`, where the available options are

- `MLMGNormType::greater`: The default.
- `MLMGNormType::bnorm`: `max_norm` is set to rhs max-norm.
- `MLMGNormType::resnorm`: `max_norm` is set to residual max-norm.

Set the absolute tolerance to zero if one does not have a good value for it. The return value of `solve` is the max-norm error.

After the solver returns successfully, if needed, we can call

```
void compResidual (const Vector<MultiFab*>& a_res,
                   const Vector<MultiFab*>& a_sol,
                   const Vector<MultiFab const*>& a_rhs);
```

to compute the residual (i.e., $f - L(\phi)$) given the solution and the right-hand side. For cell-centered solvers, we can also call the following functions to compute gradient $\nabla\phi$ and fluxes $-\beta\nabla\phi$.

```
void getGradSolution (const Vector<Array<MultiFab*,AMREX_SPACEDIM> >& a_grad_sol);
void getFluxes       (const Vector<Array<MultiFab*,AMREX_SPACEDIM> >& a_fluxes);
```

11.2 Boundary Conditions

We now discuss how to set up boundary conditions for linear operators. In the following, physical domain boundaries refer to the boundaries of the physical domain, whereas coarse/fine boundaries refer to the boundaries between AMR levels. The following steps must be followed in the exact order.

1) For any type of solver, we first need to set physical domain boundary types via the `MLLinOp` member function

```
void setDomainBC (const Array<LinOpBCType,AMREX_SPACEDIM>& lobc,  // for lower ends
                  const Array<LinOpBCType,AMREX_SPACEDIM>& hibc); // for higher ends
```

The supported BC types at the physical domain boundaries are

- `LinOpBCType::Periodic` for periodic boundary.
- `LinOpBCType::Dirichlet` for Dirichlet boundary condition.
- `LinOpBCType::Neumann` for homogeneous Neumann boundary condition.
- `LinOpBCType::inhomogNeumann` for inhomogeneous Neumann boundary condition.
- `LinOpBCType::Robin` for Robin boundary conditions, $a\phi + b\frac{\partial\phi}{\partial n} = f$.
- `LinOpBCType::reflect_odd` for reflection with sign changed.

2) Cell-centered solvers only: if we want to do a linear solve where the boundary conditions on the coarsest AMR level of the solve come from a coarser level (e.g. the base AMR level of the solve is > 0 and does not cover the entire domain), we must explicitly provide the coarser data. Boundary conditions from a coarser level are Dirichlet by default and can be changed to homogeneous Neumann (i.e., `LinOpBCType::Neumann`).

Note that this step, if needed, must be performed before the step below. The `MLLinOp` member function for this step is

```
void setCoarseFineBC (const MultiFab* crse, int crse_ratio,
                      LinOpBCType bc_type = LinOpBCType::Dirichlet);

void setCoarseFineBC (const MultiFab* crse, IntVect const& crse_ratio,
                      LinOpBCType bc_type = LinOpBCType::Dirichlet);
```

Here `const MultiFab* crse` contains the Dirichlet boundary values at the coarse resolution, and `int crse_ratio` (e.g., 2) is the refinement ratio between the coarsest solver level and the AMR level below it. The MultiFab `crse` does not need to have ghost cells itself. If the coarse-grid BCs for the solve are identically zero, `nullptr` can be passed instead of `crse`.

3) Cell-centered solvers only: before the solve one must always call the `MLLinOp` member function

```
virtual void setLevelBC (int amrlev, const MultiFab* levelbcdata,
                         const MultiFab* robinbc_a = nullptr,
                         const MultiFab* robinbc_b = nullptr,
                         const MultiFab* robinbc_f = nullptr) = 0;
```

If we want to supply an inhomogeneous Dirichlet or inhomogeneous Neumann boundary condition at the domain boundaries, we must supply those values in `MultiFab* levelbcdata`, which must have at least one ghost cell. Note that the argument `amrlev` is relative to the solve, not necessarily the full AMR hierarchy; amrlev = 0 refers to the coarsest level of the solve.

If the boundary condition is Dirichlet the ghost cells outside the domain boundary of `levelbcdata` must hold the value of the solution at the domain boundary; if the boundary condition is Neumann those ghost cells must hold the value of the gradient of the solution normal to the boundary (e.g. it would hold dphi/dx on both the low and high faces in the x-direction).

If the boundary conditions contain no inhomogeneous Dirichlet or Neumann boundaries, we can pass `nullptr` instead of a MultiFab.

We can use the solution array itself to hold these values; the values are copied to internal arrays and will not be overwritten when the solution array itself is being updated by the solver. Note, however, that this call does not provide an initial guess for the solve.

It should be emphasized that the data in `levelbcdata` for Dirichlet or Neumann boundaries are assumed to be exactly on the face of the physical domain; storing these values in the ghost cell of a cell-centered array is a convenience of implementation.

For Robin boundary conditions, the ghost cells in `MultiFab* robinbc_a`, `MultiFab* robinbc_b`, and `MultiFab* robinbc_f` store the numerical values in the condition, $a\phi + b\frac{\partial\phi}{\partial n} = f$.

4) Nodal solver provides the option to use an overset mask:

```
// omask is either 0 or 1. 1 means the node is an unknown. 0 means it's known.
void setOversetMask (int amrlev, const iMultiFab& a_dmask);
```

Note this is an integer (not bool) MultiFab, so the values must be only either 0 or 1.

5) Cell-centered solvers `MLABecLaplacian`, `MLPoisson`, `MLTensorOp` and `MLEBABecLap` also accept an overset mask, passed to the constructor or `define` with one mask per AMR level:

```
// 1 means the cell is an unknown. 0 means it's known.
MLABecLaplacian (const Vector<Geometry>& a_geom,
                 const Vector<BoxArray>& a_grids,
                 const Vector<DistributionMapping>& a_dmap,
                 const Vector<iMultiFab const*>& a_overset_mask,
                 const LPInfo& a_info = LPInfo());
```

The solver does not change the solution in known cells. Their values come from the solution MultiFab passed to `MLMG::solve`, and the right-hand side there is ignored. The mask needs no ghost cells. Multi-level composite solves are supported. Convergence is best when the edges of the masked region on each fine level fall on faces of the next coarser level's cells. With an overset mask, `MLPoisson` does not support metric terms; use `MLABecLaplacian` instead.

11.3 Parameters

There are many parameters that can be set. Here we discuss some commonly used ones.

`MLLinOp::setVerbose(int)`, `MLMG::setVerbose(int)` and `MLMG::setBottomVerbose(int)` control the verbosity of the linear operator, multigrid solver and the bottom solver, respectively.

The multigrid solver is an iterative solver. The maximal number of iterations can be changed with `MLMG::setMaxIter(int)`. We can also do a fixed number of iterations with `MLMG::setFixedIter(int)`. By default, V-cycle is used. We can use `MLMG::setMaxFmgIter(int)` to control how many full multigrid cycles can be done before switching to V-cycle.

`LPInfo::setMaxCoarseningLevel(int)` can be used to control the maximal number of multigrid levels. We usually should not call this function. However, we sometimes build the solver to simply apply the operator (e.g., $L(\phi)$) without needing to solve the system. We can do something as follows to avoid the cost of building coarsened operators for the multigrid.

```
MLABecLaplacian mlabeclap({geom}, {grids}, {dmap}, LPInfo().setMaxCoarseningLevel(0));
// set up BC
// set up coefficients
MLMG mlmg(mlabeclap);
// out = L(in)
mlmg.apply(out, in);  // here both in and out are const Vector<MultiFab*>&
```

`LPInfo::setMixedPrecision(bool)` makes `MLMG` run the multigrid cycles in single precision while the solution, the right-hand side and the residual it converges on stay in double precision. The solver still reaches the requested tolerance. This is available for `MLPoisson`, `MLABecLaplacian` and `MLALaplacian` without embedded boundaries. The bottom solver must be the smoother, CG or BiCGStab, and it runs with the single precision default tolerance unless one is set with `MLMG::setBottomTolerance`. A mixed precision `MLMG` used as a preconditioner needs a flexible Krylov method; `GMRESMLMG` switches to flexible GMRES by itself. If `define` is called again on an operator, build a new `MLMG` for it. A mixed precision `MLMG` aborts if it is used again after its operator is re-defined.

At the bottom of the multigrid cycles, we use a `bottom solver` which may be different than the relaxation used at the other levels. The default bottom solver is the biconjugate gradient stabilized method, but can easily be changed with the `MLMG` member method

```
void setBottomSolver (BottomSolver s);
```

Available choices of the bottom solver are

- `MLMG::BottomSolver::bicgstab`: The default for most operators.
- `MLMG::BottomSolver::cg`: The conjugate gradient method. The matrix must be symmetric.
- `MLMG::BottomSolver::smoother`: Smoother such as Gauss-Seidel.
- `MLMG::BottomSolver::bicgcg`: Start with bicgstab. Switch to cg if bicgstab fails. The matrix must be symmetric.
- `MLMG::BottomSolver::cgbicg`: Start with cg. Switch to bicgstab if cg fails. The matrix must be symmetric.
- `MLMG::BottomSolver::hypre`: One of the solvers available through HYPRE; see the section below on External Solvers
- `MLMG::BottomSolver::petsc`: Currently for cell-centered only.
- `MLMG::BottomSolver::algmg`: AMReX's own algebraic multigrid (see *Algebraic Multigrid*), available in every build for the operators that hypre supports.
- `MLMG::BottomSolver::custom`: A solver provided by the linear operator itself, for operators that ship one. `MLEBNodeFDLaplacian` is currently the only such operator, and it uses this by default. Its custom solver is a BiCGStab that runs entirely on the GPU when the bottom level has a single Box. This is currently available for CUDA and HIP only; other builds fall back to the standard BiCGStab solver.

The `LPInfo` class can be used to control the agglomeration and consolidation strategy for multigrid coarsening.

- `LPInfo::setAgglomeration(bool)` (by default true) can be used to copy the current level of multigrid data to fewer, larger boxes. Two advantages of using this option are that the bottom solver will become smaller, and communication overhead is reduced.
- `LPInfo::setAgglomerationGridSize(int)` controls the grid-length threshold used for agglomeration. By default, the threshold length is set to 32 for GPU builds, and 8, 16 and 32 for CPU builds in 1D, 2D and 3D, respectively. The corresponding volume threshold is L^D, where L is the length threshold and D is `AMREX_SPACEDIM`. When the average box volume falls below this volume threshold, boxes are agglomerated until this is no longer the case. Note that this action is recursive and can happen at several different levels in the multigrid hierarchy.
- `LPInfo::setConsolidation(bool)` (by default true) can be used to continue to transfer a multigrid problem to fewer MPI ranks. There are more setting such as `LPInfo::setConsolidationGridSize(int)`,

LPInfo::setConsolidationRatio(int), and LPInfo::setConsolidationStrategy(int), to give control over how this process works. If agglomeration is used, consolidation is ignored.

- LPInfo::setSemicoarsening(bool) (by default false) allows multigrid to coarsen in only some of the directions when a direction can no longer be coarsened. LPInfo::setMaxSemicoarseningLevel(int) caps how many such levels are built, and LPInfo::setSemicoarseningDirection(int) pins the direction that is left uncoarsened. On semi-coarsened levels the cell-centered solvers smooth with a line solve along the uncoarsened direction. That smoother runs on the CPU only, so cell-centered semi-coarsening is not supported in GPU builds and will abort.

MLMG::setThrowException(bool) controls whether multigrid failure results in aborting (default) or throwing an exception, whereby control will return to the calling application. This includes failures of the hypre IJ and AlgMG bottom solvers. The application code must catch the exception:

```
try {
    mlmg.solve(...);
} catch (const MLMG::error& e) {
    Print()<<e.what()<<std::endl; //Prints description of error

    // Do something else...
}
```

Note that exceptions that are not caught are passed up the calling chain so that application codes using specialized solvers relying on MLMG can still catch the exception. For example, using AMReX-Hydro's NodalProjector

```
try {
    nodal_projector.project(...);
} catch (const MLMG::error& e) {
    // Do something else...
}
```

On GPUs, calling MLMG::setNoGpuSync(true) makes MLMG::solve run in a single-stream region without the implicit stream synchronizations that MFIter and many AMReX functions normally perform (see *Stream and Synchronization*). This is off by default. When it is on, the GPU streams are synchronized once when solve returns, so the solution is complete when control comes back to the application, unless the application itself is inside a Gpu::NoSyncRegion. Whether this is faster depends on the problem. Avoiding the many small synchronizations of a multigrid cycle helps small problems with few boxes per process, but running on a single stream removes the concurrency between the per-box kernels that some solvers (e.g., the nodal and EB solvers) launch, which can make solves with many small boxes slower. Users are encouraged to time their solves with and without setNoGpuSync(true) and use whichever is faster.

11.3.1 Multigrid Type

By default, the coarsest AMR level is solved with geometric multigrid V-cycles down to the bottom solver. For problems where geometric coarsening converges slowly or not at all, such as strongly varying or anisotropic coefficients, MLMG::setMultigridType selects how that level is solved:

- MultigridType::geometric: the default described above.
- MultigridType::algebraic: AlgMG (*Algebraic Multigrid*) solves the whole coarsest AMR level in every MLMG iteration, using the bottom solver's tolerance, iteration limit and verbosity. The geometric levels of that AMR level are then unused, so LPInfo::setMaxCoarseningLevel(0) avoids building them.
- MultigridType::hybrid: geometric multigrid first. If the residual stalls, grows or becomes NaN, MLMG switches to the algebraic solver and restarts from the best iterate it has seen. The switch is reported at verbosity 1. MLMG::setHybridStallCriterion(window, rate) (defaults 4 and 0.8) declares a stall when the residual has not dropped by rate per iteration on average over the last window iterations, and

`MLMG::setHybridDivergenceFactor` (default 10) declares divergence when the residual exceeds that multiple of the best residual. After the switch, the algebraic phase gets its own `setMaxIter` budget.

Finer AMR levels always use the geometric cycles. The algebraic types support the same operators as the hypre bottom solver, for single component `MultiFab` problems. When MLMG serves as a preconditioner (for example in `GMRESMLMG`), the algebraic type applies one AlgMG V-cycle on the coarsest AMR level, so the preconditioner stays a fixed linear operation; the hybrid type needs a convergence test and is not available there or with `setFixedIter`. `MLMG::setAlgMGOptions` takes a callback that receives the `AlgMG` solver so that its multigrid parameters, such as the Krylov acceleration (BiCGStab by default when driven by MLMG), can be changed; the tolerances, iteration limit and verbosity come from the bottom-solver settings. The callback runs once per assembled matrix. Code that calls `AlgMG` members in the callback includes `AMReX_AlgMG.H`. The type can also be read from an inputs file:

```
MultigridType mg_type = MultigridType::geometric;
ParmParse pp("mlmg");
pp.query_enum_case_insensitive("multigrid_type", mg_type); // geometric, algebraic,␣
↪hybrid
mlmg.setMultigridType(mg_type);
```

11.4 Boundary Stencils for Cell-Centered Solvers

We have the option using the `MLMG` member method

```
void setMaxOrder (int maxorder);
```

to set the order of the cell-centered linear operator stencil at physical boundaries with Dirichlet boundary conditions and at coarse-fine boundaries. In both of these cases, the boundary value is not defined at the center of the ghost cell. The order determines the number of interior cells that are used in the extrapolation of the boundary value from the cell face to the center of the ghost cell, where the extrapolated value is then used in the regular stencil. For example, `maxorder = 2` uses the boundary value and the first interior value to extrapolate to the ghost cell center; `maxorder = 3` uses the boundary value and the first two interior values; `maxorder = 4` uses the boundary value and the first three interior values. The hypre, PETSc and AlgMG bottom solvers and the algebraic and hybrid multigrid types assemble the operator into a matrix. AlgMG, PETSc and hypre's IJ interface support orders up to 4, and hypre's structured and semi-structured interfaces up to 3. With embedded boundaries, AlgMG, PETSc and hypre's IJ interface support orders up to 3. MLMG lowers a higher order to what the solver supports, and the operator keeps that order afterwards.

11.5 Curvilinear Coordinates

Some of the linear solvers support curvilinear coordinates including 1D spherical and 2d cylindrical (r, z). In those cases, the divergence operator has extra metric terms. If one does not want the solver to include the metric terms because they have been handled in other ways, one can turn them off with a setter function. For the cell-centered linear solvers *MLABecLaplacian* and *MLPoisson*, one can call `setMetricTerm(bool)` with `false` on the `LPInfo` object passed to the constructor of linear operators. For the node-based *MLNodeLaplacian*, one can call `setRZCorrection (bool)` with `false` on the *MLNodeLaplacian* object.

MLABecLaplacian and *MLPoisson* support both spherical and cylindrical coordinates, while *MLNodeLaplacian* supports only cylindrical at this time. Note that to use cylindrical coordinates with *MLNodeLaplacian*, the application code must scale `sigma` by the radial coordinate before calling `setSigma()`.

11.6 Embedded Boundaries

AMReX supports multi-level solvers for use with embedded boundaries. These include 1) cell-centered solvers with homogeneous Neumann, homogeneous Dirichlet, or inhomogeneous Dirichlet boundary conditions on the EB faces, and 2) nodal solvers with homogeneous Neumann boundary conditions, or inflow velocity conditions on the EB faces.

To use a cell-centered solver with EB, one builds a linear operator MLEBABecLap with EBFArrayBoxFactory (instead of a MLABecLaplacian)

```
MLEBABecLap (const Vector<Geometry>& a_geom,
             const Vector<BoxArray>& a_grids,
             const Vector<DistributionMapping>& a_dmap,
             const LPInfo& a_info,
             const Vector<EBFArrayBoxFactory const*>& a_factory);
```

The usage of this EB-specific class is essentially the same as MLABecLaplacian.

The default boundary condition on EB faces is homogeneous Neumann.

To set homogeneous Dirichlet boundary conditions, call

```
ml_ebabeclap->setEBHomogDirichlet(lev, coeff);
```

where coeff can be a real number (i.e., the value is the same at every cell) or a MultiFab holding the coefficient of the gradient at each cell with an EB face. In other words, coeff is β in the canonical form given in equation (11.1) located at the EB surface centroid.

To set inhomogeneous Dirichlet boundary conditions, call

```
ml_ebabeclap->setEBDirichlet(lev, phi_on_eb, coeff);
```

where phi_on_eb is the MultiFab holding the Dirichlet values in every cut cell, and coeff again is a real number or a MultiFab holding the coefficient of the gradient at each cell with an EB face, i.e., β in equation (11.1) located at the EB surface centroid.

Currently there are options to define the face-based coefficients on face centers vs face centroids, and to interpret the solution variable as being defined on cell centers vs cell centroids.

The default is for the solution variable to be defined at cell centers; to tell the solver to interpret the solution variable as living at cell centroids, you must set

```
ml_ebabeclap->setPhiOnCentroid();
```

The default is for the face-based coefficients to be defined at face centers; to tell the solver that the face-based coefficients should be interpreted as living at face centroids, modify the setBCoeffs command to be

```
ml_ebabeclap->setBCoeffs(lev, beta, MLMG::Location::FaceCentroid);
```

The nodal finite-difference operator MLEBNodeFDLaplacian takes an EBFArrayBoxFactory only on the finest AMR level. It builds the EB information it needs on the multigrid levels from that factory by itself, so multigrid can coarsen as far as the grids allow, regardless of the max_coarsening_level given to EB2::Build. The operator drops coarse multigrid levels on its own when they have too few unknowns, or when a level would no longer see the embedded boundary or part of it. Setting the MLMG verbosity to 2 or higher prints the resulting number of levels.

MLEBNodeFDLaplacian applies semicoarsening automatically on stretched cells. When some directions have cells at least 1.5 times as long as the shortest, those directions are left uncoarsened until the cells are nearly cubic. No setting is needed, and a direction fixed with LPInfo::setSemicoarseningDirection takes precedence. A constant sigma set with setSigma before the first solve is accounted for: a direction with smaller sigma counts as having longer cells. The coarse multigrid levels are built at the first solve, so their number is known only after it.

11.7 External Solvers

AMReX provides interfaces to the HYPRE preconditioners and solvers, including BoomerAMG, GMRES (all variants), PCG, and BICGStab as solvers, and BoomerAMG and Euclid as preconditioners. These can be called as as bottom solvers for both cell-centered and node-based problems.

If it is built with HYPRE support, AMReX initializes HYPRE by default in *amrex::Initialize*. If it is built with CUDA, AMReX will also set up HYPRE to run on device by default. The user can choose to disable the HYPRE initialization by AMReX with `ParmParse` parameter `amrex.init_hypre=[0|1]`.

By default the AMReX linear solver code always tries to geometrically coarsen the problem as much as possible. However, as we have mentioned, we can call `setMaxCoarseningLevel(0)` on the `LPInfo` object passed to the constructor of a linear operator to disable the coarsening completely. In that case the bottom solver is solving the residual correction form of the original problem.

As of March 2025, AMReX supports and is tested with HYPRE version 2.32.0 (check `amrex/.github/workflows/hypre.yml` so see what versions are currently tested). To build HYPRE, follow the next steps:

```
1.- git clone https://github.com/hypre-space/hypre.git
2.- cd hypre/src
3.- git checkout v2.32.0
4.- ./configure
    (if you want to build HYPRE with long long int, do ./configure --enable-bigint )
5.- make install
6.- Create an environment variable with the HYPRE directory --
    HYPRE_DIR=/hypre_path/hypre/src/hypre
```

The `--enable-bigint` option is for CPU builds of HYPRE. It is not currently compatible with HYPRE's CUDA build, so it should not be combined with `--with-cuda`. CUDA users who encounter a HYPRE 32-bit integer overflow do not currently have a bigint HYPRE configuration available as a fallback.

To use HYPRE with CUDA, the *nvcc* compiler is needed along with all other requirements for the CPU build (e.g., *gcc*, *mpicc*). It is very important that the GPU architecture for HYPRE matches that of AMReX. By default, HYPRE assumes its architecture number to be 70, and it is best to build HYPRE for multiple architectures by specifying multiple compute capability numbers (e.g., 80 and 90). If you see a runtime error similar to `terminate called after throwing an instance of 'thrust::system::system_error'`, you likely did not build for the correct architecture.

```
1.- git clone https://github.com/hypre-space/hypre.git
2.- cd hypre/src
3.- git checkout v2.32.0
4.- ./configure --with-cuda --with-gpu-arch='80 90' --enable-unified-memory
    (you can determine the GPU architecture from the command line using
    nvidia-smi --query-gpu=compute_cap --format=csv; if it gives 9.0, gpu-arch is 90)
5.- make install
6.- Create an environment variable with the HYPRE directory --
    HYPRE_DIR=/hypre_path/hypre/src/hypre
```

To use HYPRE, one must include `amrex/Src/Extern/HYPRE` in the build system. For examples of using HYPRE, we refer the reader to `amrex/Tests/LinearSolvers/ABecLaplacian_C` and `amrex/Tests/LinearSolvers/NodeTensorLap`.

The following parameter should be set to true if the problem to be solved has a singular matrix. In this case, the solution is only defined to within a constant. Setting this parameter to true replaces one row in the matrix sent to HYPRE from AMReX by a row that sets the value at one cell to 0.

- `hypre.adjust_singular_matrix`: Default is false.

The following parameters can be set in the inputs file to control the choice of preconditioner and smoother:

- `hypre.hypre_solver`: Default is BoomerAMG.
- `hypre.hypre_preconditioner`: Default is none; otherwise the type must be specified.
- `hypre.recompute_preconditioner`: Option to redo the solver and preconditioner setup on every solve. AMReX's own hypre solvers default to false, because they redo the setup whenever the matrix changes; `HypreIJIface` used directly defaults to true.
- `hypre.write_matrix_files`: Default false. Option to write the matrix to text files.
- `hypre.overwrite_existing_matrix_files`: Default false. Option to overwrite existing matrix files.

The following parameters can be set in the inputs file to control the BoomerAMG solver specifically:

- `hypre.bamg_verbose`: Verbosity of the BoomerAMG preconditioner. Default 0. See *HYPRE_BoomerAMGSetPrintLevel*
- `hypre.bamg_logging`: Default 0. See *HYPRE_BoomerAMGSetLogging*
- `hypre.bamg_coarsen_type`: Default 6 (8 on GPUs). See *HYPRE_BoomerAMGSetCoarsenType*
- `hypre.bamg_cycle_type`: Default 1. See *HYPRE_BoomerAMGSetCycleType*
- `hypre.bamg_relax_type`: Default 6 (18 on GPUs). See *HYPRE_BoomerAMGSetRelaxType*
- `hypre.bamg_relax_order`: Default 1 (0 on GPUs). See *HYPRE_BoomerAMGSetRelaxOrder*
- `hypre.bamg_num_sweeps`: Default 2. See *HYPRE_BoomerAMGSetNumSweeps*
- `hypre.bamg_max_levels`: Default 20. See *HYPRE_BoomerAMGSetMaxLevels*
- `hypre.bamg_strong_threshold`: Default 0.25 for 2D, 0.57 for 3D. See *HYPRE_BoomerAMGSetStrongThreshold*
- `hypre.bamg_interp_type`: Default 0 (6 on GPUs). See *HYPRE_BoomerAMGSetInterpType*
- `hypre.bamg_use_old_default`: Default true (false on GPUs). Only used when BoomerAMG is the solver. See *HYPRE_BoomerAMGSetOldDefault*

With a GPU build of HYPRE, the defaults above switch to HYPRE's recommended GPU options (PMIS coarsening, extended+i interpolation with `hypre.bamg_pmax_elmts` 4, l1-Jacobi relaxation in natural order, and `hypre.bamg_keep_transpose` 1). For symmetric problems, Chebyshev relaxation (`hypre.bamg_relax_type` 16) often needs half as many iterations.

When BoomerAMG is the solver, `hypre.bamg_max_levels` defaults to HYPRE's own default. The defaults of `hypre.bamg_coarsen_type`, `hypre.bamg_interp_type` and `hypre.bamg_pmax_elmts` come from *HYPRE_BoomerAMGSetOldDefault* if `hypre.bamg_use_old_default` is true, and from HYPRE's own defaults otherwise.

Other BoomerAMG parameters: `hypre.bamg_pmax_elmts`, `hypre.bamg_trunc_factor`, `hypre.bamg_agg_num_levels`, `hypre.bamg_agg_interp_type`, `hypre.bamg_agg_pmax_elmts`, `hypre.bamg_agg_trunc_factor`, `hypre.bamg_num_paths`, `hypre.bamg_keep_transpose`, `hypre.bamg_rap2`, `hypre.bamg_mod_rap2`, `hypre.bamg_min_coarse_size`, `hypre.bamg_max_coarse_size`, `hypre.bamg_variant`, `hypre.bamg_set_restriction`, `hypre.bamg_cheby_order`, `hypre.bamg_cheby_fraction`, `hypre.bamg_cheby_eig_est`, `hypre.bamg_cheby_variant` and `hypre.bamg_cheby_scale`. See the corresponding *HYPRE_BoomerAMGSet...* function.

The user is referred to the HYPRE HYPRE Reference Manual for full details on the usage of the parameters described briefly above.

AMReX can also use PETSc as a bottom solver for cell-centered problems. To build PETSc, follow the next steps:

```
1.- git clone https://github.com/petsc/petsc.git
2.- cd petsc
3.- ./configure --prefix=build_dir
4.- Invoke the ``make all'' command given at the end of the previous command output
5.- Invoke the ``make install'' command given at the end of the previous command output
6.- Create an environment variable with the PETSC directory --
    PETSC_DIR=/petsc_path/petsc/build_dir
```

To use PETSc, one must include `amrex/Src/Extern/PETSc` in the build system. For an example of using PETSc, we refer the reader to `amrex/Tests/LinearSolvers/ABecLaplacian_C`.

11.7.1 Composite multi-level solves with HypreMLABecLap

`HypreMLABecLap` solves a cell-centered ABecLaplacian on all AMR levels at once as a single composite system, rather than as an MLMG bottom solver (for example AMReX-Hydro's MAC projection with `mac_proj.use_mlhypre`). By default it assembles a semi-structured hypre matrix and solves it with BoomerAMG. The following parameters, under the ParmParse prefix given to the constructor (default `hypre_mlabeclap`), select and control the IJ mode, in which the composite matrix is assembled on the device into one distributed sparse matrix:

- `hypre_mlabeclap.hypre_solver`: if given, use the IJ mode and solve with `HypreIJIface` configured from the same keys as above (e.g. `hypre_solver = FlexGMRES`, `hypre_preconditioner = BoomerAMG`, `bamg_*`), with the prefix `hypre_mlabeclap`.
- `hypre_mlabeclap.backend`: `hypre` (default) or `algmg`. With `algmg` the assembled IJ system is solved by AMReX's AlgMG (*Algebraic Multigrid*), which implies the IJ mode and does not need `hypre_solver`. AlgMG is configured with `hypre_mlabeclap.algmg.krylov` (`none`, `bicgstab`, `gmres`, `pcg`; default `gmres`, since V-cycles alone can stall on composite systems), `.smoother`, `.interp_type`, `.bottom_solver`, `.strong_threshold`, `.max_levels`, `.max_coarse_size`, `.agg_num_levels`, `.pmax_elmts`, `.trunc_factor`, `.pre_smooth`, `.post_smooth`, `.cheby_degree` and `.verbose`.
- `HypreMLABecLap::setReuseSolverSetup(true)`: a repeated `setup` with new coefficients (on the same grids) updates the matrix values in place and keeps the solver setup (with the AlgMG backend, the hierarchy: `AlgMG::updateMatrix`) until `resetSolverSetup()`. It can be turned on before or after the first `setup`. A solve with a reused setup that does not converge redoes the setup and continues once before failing; `hypre_mlabeclap.reuse_setup_max_iter` (or `setReuseSetupMaxIter`) also redoes it at the next solve when a solve with a reused setup needed more iterations.
- In both modes, `getNumIters()` and `getFinalResidualNorm()` report the last solve; when a solve redid a reused setup and continued, the iterations of both attempts are counted, so it can take up to twice the maximum number of iterations. With `setThrowException(true)` a solve that does not converge, or whose solver fails (a hypre error, a non-finite residual, AlgMG diverging), throws `std::runtime_error` instead of aborting. The absolute tolerance of `solve` must be 0.
- `hypre_mlabeclap.dump_matrix`: default 0. Write the systems of the first N solves with the same dump prefix, counted over all `HypreMLABecLap` objects of the run, in Matrix Market format for analysis outside AMReX: for solve `k`, `<dump_prefix>_<k>_A.mtx` (matrix), `_b.mtx` (right-hand side), `_x.mtx` (solution) and `_rows.txt` (AMR level and cell of each row). `hypre_mlabeclap.dump_prefix` defaults to the ParmParse prefix. Files with more than `hypre_mlabeclap.dump_gather_max` values (default `5e7`) are written one per rank, as `<file>.<rank>` (on one rank, always one file).

The number of rows of the composite system must fit in `HYPRE_Int`, so systems with more than about 2^{31} cells need HYPRE built with `--enable-bigint`. Like the default mode, the IJ mode runs on the communicator of the current `ParallelContext` frame, which must be the same for the construction, `setup` and `solve`.

11.7.2 Composite multi-level nodal solves with HypreMLNodeLap

`HypreMLNodeLap` is the nodal counterpart: it assembles the composite system of a prepared `MLNodeLaplacian` (all AMR levels) into one distributed sparse matrix and solves it with hypre or AlgMG. It is meant for problems where MLMG's geometric coarsening fails, such as strongly stretched mapped meshes or EB geometries with very small cut cells. The solution equals MLMG's composite solution up to the solver tolerance, except at coarse/fine nodes next to cut cells (where MLMG's coarse/fine treatment ignores the cut cells) and, for singular problems whose composite right-hand side is not exactly consistent, by the effect of a different consistency correction. Singular problems are made consistent by subtracting the nodal-volume weighted mean of the right-hand side, and one unknown is pinned.

```
// linop: MLNodeLaplacian with sigma set and masks built
// (after MLMG::prepareForSolve, or a solve)
HypreMLNodeLap hyp(linop, "hypre_mlnodelap", verbose);
hyp.solve(GetVecOfPtrs(phi), GetVecOfConstPtrs(rhs), reltol, abstol, maxiter);
mlmg.getFluxes(GetVecOfPtrs(fluxes), GetVecOfPtrs(phi));
```

As with MLMG, the values of `phi` on Dirichlet domain boundaries are the boundary values. It supports scalar, constant and anisotropic (mapped) sigma, refinement ratios 2 and 4, and in EB builds the cut cells; RZ is not supported. The parameters, under the prefix given to the constructor (default `hypre_mlnodelap`), are the `HypreIJIface` keys above, and `backend`, `algmg.*` (with `algmg.krylov` defaulting to `pcg`, as the assembled system is symmetric), `dump_matrix`, `dump_prefix` and `dump_gather_max` as for `HypreMLABecLap`. The dumped system is the one solved, $S'x' = b'$ with $S' = sSs$ and $x = sx'$, where the scaling s of each unknown is listed in the row map (it is 1 except next to small cut cells, see `eb_scaling`).

- `hypre_mlnodelap.eb_scaling`: default true. In EB builds, scale the unknowns by $1/\sqrt{\text{nodal fluid fraction}}$ so that the rows of nodes next to small cut cells count in the residual norm.

`getNumIters()`, `getFinalResidualNorm()` and `setThrowException()` work as for `HypreMLABecLap`, and so does the use on a sub-communicator (the communicator of the `MLNodeLaplacian`).

11.8 Tensor Solve

Application codes that solve the Navier-Stokes equations need to evaluate the viscous term; solving for this term implicitly requires a multi-component solve with cross terms. Because this is a commonly used motif, we provide a tensor solve for cell-centered velocity components.

Consider a velocity field $U = (u, v, w)$ with all components co-located on cell centers. The viscous term can be written in vector form as

$$\nabla \cdot (\eta \nabla U) + \nabla \cdot (\eta (\nabla U)^T) + \nabla \cdot ((\kappa - \frac{2}{3}\eta)(\nabla \cdot U))$$

and in 3-d Cartesian component form as

$$((\eta u_x)_x + (\eta u_y)_y + (\eta u_z)_z) + ((\eta u_x)_x + (\eta v_x)_y + (\eta w_x)_z) + ((\kappa - \frac{2}{3}\eta)(u_x + v_y + w_z))_x$$
$$((\eta v_x)_x + (\eta v_y)_y + (\eta v_z)_z) + ((\eta u_y)_x + (\eta v_y)_y + (\eta w_y)_z) + ((\kappa - \frac{2}{3}\eta)(u_x + v_y + w_z))_y$$
$$((\eta w_x)_x + (\eta w_y)_y + (\eta w_z)_z) + ((\eta u_z)_x + (\eta v_z)_y + (\eta w_z)_z) + ((\kappa - \frac{2}{3}\eta)(u_x + v_y + w_z))_z$$

Here η is the dynamic viscosity and κ is the bulk viscosity.

We evaluate the following terms from the above using the `MLABecLaplacian` and `MLEBABecLaplacian` operators;

$$((\frac{4}{3}\eta + \kappa)u_x)_x + (\eta u_y)_y + (\eta u_z)_z$$
$$(\eta v_x)_x + ((\frac{4}{3}\eta + \kappa)v_y)_y + (\eta v_z)_z$$
$$(\eta w_x)_x + (\eta w_y)_y + ((\frac{4}{3}\eta + \kappa)w_z)_z$$

the following cross-terms are evaluated separately using the `MLTensorOp` and `MLEBTensorOp` operators.

$$((\kappa - \frac{2}{3}\eta)(v_y + w_z))_x + (\eta v_x)_y + (\eta w_x)_z$$
$$(\eta u_y)_x + ((\kappa - \frac{2}{3}\eta)(u_x + w_z))_y + (\eta w_y)_z$$
$$(\eta u_z)_x + (\eta v_z)_y + ((\kappa - \frac{2}{3}\eta)(u_x + v_y))_z$$

The code below is an example of how to set up the solver to compute the viscous term *divtau* explicitly:

```
Box domain(geom[0].Domain());

// Set BCs for Poisson solver in bc_lo, bc_hi
...

//
// First define the operator "ebtensorop"
// Note we call LPInfo().setMaxCoarseningLevel(0) because we are only applying the␣
↪operator,
//      not doing an implicit solve
//
//       (alpha * a - beta * (del dot b grad)) sol
//
// LPInfo                        info;
MLEBTensorOp ebtensorop(geom, grids, dmap, LPInfo().setMaxCoarseningLevel(0),
                        amrex::GetVecOfConstPtrs(ebfactory));

// It is essential that we set MaxOrder of the solver to 2
// if we want to use the standard sol(i)-sol(i-1) approximation
// for the gradient at Dirichlet boundaries.
// The solver's default order is 3 and this uses three points for the
// gradient at a Dirichlet boundary.
ebtensorop.setMaxOrder(2);

// LinOpBCType Definitions are in amrex/Src/Boundary/AMReX_LO_BCTYPES.H
ebtensorop.setDomainBC ( {(LinOpBCType)bc_lo[0], (LinOpBCType)bc_lo[1], (LinOpBCType)bc_
↪lo[2]},
                         {(LinOpBCType)bc_hi[0], (LinOpBCType)bc_hi[1], (LinOpBCType)bc_
↪hi[2]} );

// Return div (eta grad)) phi
ebtensorop.setScalars(0.0, -1.0);

amrex::Vector<amrex::Array<std::unique_ptr<amrex::MultiFab>, AMREX_SPACEDIM>> b;
b.resize(max_level + 1);
```

(continues on next page)

(continued from previous page)

```
// Compute the coefficients
for (int lev = 0; lev < nlev; lev++)
{
    // We average eta onto faces
    for(int dir = 0; dir < AMREX_SPACEDIM; dir++)
    {
        BoxArray edge_ba = grids[lev];
        edge_ba.surroundingNodes(dir);
        b[lev][dir] = std::make_unique<MultiFab>(edge_ba, dmap[lev], 1, nghost, MFInfo(),
↪ *ebfactory[lev]);
    }

    average_cellcenter_to_face( GetArrOfPtrs(b[lev]), *etan[lev], geom[lev] );

    b[lev][0] -> FillBoundary(geom[lev].periodicity());
    b[lev][1] -> FillBoundary(geom[lev].periodicity());
    b[lev][2] -> FillBoundary(geom[lev].periodicity());

    ebtensorop.setShearViscosity  (lev, GetArrOfConstPtrs(b[lev]));
    ebtensorop.setEBShearViscosity(lev, (*eta[lev]));

    ebtensorop.setLevelBC ( lev, GetVecOfConstPtrs(vel)[lev] );
}

MLMG solver(ebtensorop);

solver.apply(GetVecOfPtrs(divtau), GetVecOfPtrs(vel));
```

A tensor operator and the `MLMG` object built on it may be reused for any number of `solve` and `apply` calls, provided the coefficients are left untouched after the first call. Unlike the other operators, the tensor operators do not support updating coefficients in place: calling `setShearViscosity`, `setBulkViscosity`, `setACoeffs`, or their EB counterparts on an operator that has already been used will abort on the next call. Build a new operator when the viscosities change.

11.9 Multi-Component Operators

This section discusses solving linear systems in which the solution variable ϕ has multiple components. An example (implemented in the CustomLinOp tutorial) might be:

$$D(\phi)_i = \sum_{i=1}^{N} \alpha_{ij} \nabla^2 \phi_j$$

(Note: only operators of the form $D : \mathbb{R}^n \rightarrow \mathbb{R}^n$ are currently allowed.)

- To implement a multi-component *cell-based* operator, inherit from the `MLCellLinOp` class. Override the `getNComp` function to return the number of components (`N`) that the operator will use. The solution and rhs fabs must also have at least one ghost node. `Fapply`, `Fsmooth`, `Fflux` must be implemented such that the solution and rhs fabs all have `N` components.
- Implementing a multi-component *node-based* operator is slightly different. An MC nodal operator must specify that the reflux-free coarse/fine strategy is being used by the solver.

```
solver.setCFStrategy(MLMG::CFStrategy::ghostnodes);
```

The reflux-free method circumvents the need to implement a special `reflux` at the coarse-fine boundary. This is accomplished by using ghost nodes. Each AMR level must have 2 layers of ghost nodes. The second (outermost) layer of nodes is treated as constant by the relaxation, essentially acting as a Dirichlet boundary. The first layer of nodes is evolved using the relaxation, in the same manner as the rest of the solution. When the residual is restricted onto the coarse level (in `reflux`) this allows the residual at the coarse-fine boundary to be interpolated using the first layer of ghost nodes. Fig. 11.1 illustrates how the coarse-fine update takes place.

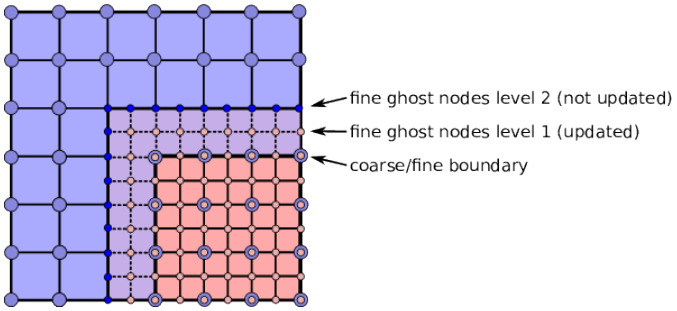


Fig. 11.1: Reflux-free coarse-fine boundary update. Level 2 ghost nodes (small dark blue) are interpolated from coarse boundary. Level 1 ghost nodes are updated during the relaxation along with all the other interior fine nodes. Coarse nodes (large blue) on the coarse/fine boundary are updated by restricting with interior nodes and the first level of ghost nodes. Coarse nodes underneath level 2 ghost nodes are not updated. The remaining coarse nodes are updated by restriction.

The MC nodal operator can inherit from the `MLNodeLinOp` class. `Fapply`, `Fsmooth`, and `Fflux` must update level 1 ghost nodes that are inside the domain. *interpolation* and *restriction* can be implemented as usual. *reflux* is a straightforward restriction from fine to coarse, using level 1 ghost nodes for restriction as described above.

See the CustomLinOp tutorial for a complete working example of a single-level multi-component nodal operator.

11.10 Curl-Curl

The curl-curl solver supports solving the linear system arising from the discretized form of

$$\nabla \times (\alpha \nabla \times \vec{E}) + \beta \vec{E} = \vec{f},$$

where $\vec{E}$ and $\vec{f}$ are defined on cell edges. The coefficient α may be supplied either as a single positive scalar through `MLCurlCurl::setScalars`, or as a nodal `MultiFab` by calling `MLCurlCurl::setAlpha` with one entry per AMR level. The β term can be a non-negative scalar or an edge-centered field set via `setBeta`. An `Array` of three `MultiFabs` is used to store the components of $\vec{E}$ and $\vec{f}$. It is the user's responsibility to ensure that the right-hand-side data are consistent on edges shared by multiple `Boxes`. If needed, you can call `MLCurlCurl::prepareRHS` to perform this synchronization.

The solver supports 1D, 2D and 3D. Note that even in the 1D and 2D cases, $\vec{E}$ still has three components, one for each spatial direction. See the CurlCurl tutorial for an example.

11.11 Terrain-Following Poisson

`MLTerrainPoisson` solves the cell-centered Poisson equation on a 3D terrain-following mesh, i.e., a mesh that is uniform in computational space but whose cell corners are at physical heights $z(x, y, \zeta)$. The operator is

$$L(\phi) = \frac{1}{J} \nabla \cdot (A \nabla \phi),$$

where ∇ is the physical gradient, A holds the face area factors and J is the cell volume factor. MLMG solves the scaled system $\nabla \cdot (A\nabla\phi) = Jf$ for the right-hand side f, so MLMG::apply and the residuals refer to this scaled system and are not divided by J. The data are set with

```
// Physical height of the cell corners, with at least one filled ghost node
void setZPhys (int amrlev, MultiFab const& z_phys_nd);

// Face area factors in x, y and z (required)
void setAreas (int amrlev, Array<MultiFab const*,AMREX_SPACEDIM> const& area);

// Cell volume factor J (optional; 1 if not set)
void setDetJ (int amrlev, MultiFab const& detJ);
```

The supported domain boundary conditions are LinOpBCType::Periodic, LinOpBCType::Neumann and LinOpBCType::Dirichlet, all homogeneous, so the level boundary condition is set with setLevelBC(0, nullptr). The flux through a Neumann face is zero. Only a single AMR level is supported, and its grids must cover the domain. Boxes that span the whole domain in the z-direction are fastest, but boxes split in z are also supported.

The operator is not symmetric, so the bottom solver can be BiCGStab or the smoother. By default, MLMG with this operator uses the smoother when multigrid can coarsen the grids to a few cells in x and y, and BiCGStab otherwise. Multigrid can coarsen that far when the x and y sizes of the domain have no prime factors other than 2, 3 and 5 (e.g., $200 = 2^3 \cdot 5^2$) and the boxes can be coarsened at least once in x and y (e.g., their sizes are even). The problem can be solved with MLMG, as below, or with GMRESMLMG (section *GMRES*), which always uses the smoother as the bottom solver.

```
MLTerrainPoisson linop({geom}, {grids}, {dmap});
linop.setDomainBC(lobc, hibc);
linop.setLevelBC(0, nullptr);
linop.setZPhys(0, z_phys_nd);
linop.setAreas(0, {&ax, &ay, &az});
linop.setDetJ(0, detJ);

MLMG mlmg(linop);
mlmg.solve({&phi}, {&rhs}, reltol, abstol);
```

MLMG::getFluxes returns the fluxes without the area factors A: $-\partial\phi/\partial x$ and $-\partial\phi/\partial y$ on the x- and y-faces, and $-(\partial\phi/\partial z - z_\xi\,\partial\phi/\partial x - z_\eta\,\partial\phi/\partial y)$ on the z-faces, where z_ξ and z_η are the slopes of the coordinate surfaces. MLMG::getGradSolution returns the same with the opposite sign.

11.12 Open Boundary Poisson Solver

To solve Poisson's equation for isolated sources (i.e., no sources outside the boundary), there are several options. One option is to use MLMG and MLPoisson with Dirichlet boundary conditions. The Dirichlet boundary values can be computed using the multipole method, as done in the Castro code (see e.g., https://amrex-astro.github.io/Castro/docs/file/Gravity_8H.html#_CPPv49multipole). Another option is to use the FFT-based solver amrex::FFT::PoissonOpenBC (see chapter *Discrete Fourier Transform*). A third option is amrex::OpenBCSolver, which implements James's method ("The Solution of Poisson's Equation for Isolated Source Distributions", R. A. James, 1977, Journal of Computational Physics, 25, 71). Below are some of the member functions of OpenBCSolver.

```
OpenBCSolver (const Vector<Geometry>& a_geom,
              const Vector<BoxArray>& a_grids,
              const Vector<DistributionMapping>& a_dmap,
              const LPInfo& a_info = LPInfo());
```

(continues on next page)

(continued from previous page)

```
Real solve (const Vector<MultiFab*>& a_sol,
            const Vector<MultiFab const*>& a_rhs,
             Real a_tol_rel, Real a_tol_abs);
```

11.13 Cross-Stencil Projection

amrex::GMG::CrossProj computes the same projection as FFT::CrossProj (see section *Cross-Stencil Projection*), with geometric multigrid instead of FFT. It projects a cell-centered velocity field onto fields that are divergence free with respect to the node-centered divergence computed from the 4 cells around each node. The class is available in 2D only, and it currently only supports periodic boundary conditions. The domain must start at index 0 and have an even number of cells (at least 4) in each direction, and the boxes must have even lengths. The first two components of the MultiFab are the velocity. The input and output must have the same BoxArray and DistributionMapping, and they can be the same MultiFab. If the output has ghost cells, one layer of them is filled. The header is AMReX_CrossProj.H.

```
Geometry geom(...);   // periodic in all directions
MultiFab vel(...);    // cell-centered, AMREX_SPACEDIM components
MultiFab vel_proj(...);

GMG::CrossProj proj(geom);
proj.setRelTol(1.e-10); // optional; see also setAbsTol, setMaxIter, setVerbose
proj.solve(vel_proj, vel); // vel_proj and vel can be the same MultiFab
int niters = proj.getNumIters();
```

The object should be cached for reuse, since it sets up the multigrid hierarchy on the first solve and whenever the grids change.

11.14 GMRES

AMReX provides a template implementation of the generalized minimal residual method (GMRES). The class template parameters are V for a linear algebra vector class and M for a linear operator class. The linear algebra vector class can contain one or several MultiFabs, or your own data container. The linear operator class represents a matrix conceptually. Although it does not need to store the matrix explicitly, it must be able to apply the matrix vector product for a given vector. It also must provide some basic operations such as dot product and linear combination. For the full set of requirements on the operator class, see https://amrex-codes.github.io/amrex/doxygen/classamrex_1_1GMRES.html#details.

By default, GMRES<V,M>::solve starts from a zero initial guess and overwrites the solution vector passed in. Call setInitialGuessNonzero(true) to use the solution vector passed in as the initial guess instead. The relative tolerance is relative to the initial residual norm, which is the norm of the right-hand side only for a zero initial guess. BiCGStab and PCG below behave the same way.

GMRES is right-preconditioned and assumes the preconditioner is the same linear operator in every iteration. If it is not, for example an inner iterative solve stopped by a tolerance, call setFlexible(true) to use flexible GMRES (FGMRES). It stores one more vector per iteration.

An example of using GMRES with a user-defined operator and a multigrid preconditioner can be found in the GMRES tutorial.

AMReX also provides GMRESMLMGT, a class template that solves the linear system in MLMG using GMRES with MLMG itself serving as the preconditioner. Like MLMG, its solve uses the solution passed in as the initial guess. The GMRES tutorial also shows its usage.

11.15 BiCGStab

`BiCGStab<V,M>` in `AMReX_BiCGStab.H` is a right-preconditioned BiCGStab solver with the same template parameters and operator requirements as `GMRES<V,M>`, so an operator class written for GMRES can be used with BiCGStab without changes. Compared with GMRES, it needs a fixed number of vectors and performs a fixed number of dot products per iteration. Use `getStatus` to check for a breakdown.

11.16 PCG

`PCG<V,M>` in `AMReX_PCG.H` is the preconditioned conjugate gradient method with the same operator requirements as `GMRES<V,M>`. It needs a symmetric definite operator and preconditioner of the same sign (positive or negative definite), and is then the cheapest of the three: one operator and one preconditioner application and three global reductions per iteration. `getStatus` reports a loss of definiteness.

11.17 Sparse Linear Algebra

AMReX provides distributed sparse matrices and vectors in `AMReX_Algebra.H`. An `AlgPartition` describes how global row indices are divided among MPI processes: each process owns a contiguous range of rows. `AlgVector<T>` is a vector distributed with such a partition, and `SpMatrix<T>` is a sparse matrix in compressed sparse row (CSR) format whose rows are distributed with a partition. The data live in GPU memory in GPU builds.

A matrix can be built with a fixed number of nonzeros per row and filled with a functor that sets the global column indices and values of each row, or from existing CSR arrays with `define`.

```
// 1D Laplacian with n global rows, 3 nonzeros per row
AlgPartition partition(n);
SpMatrix<Real> A(partition, 3);
A.setVal([=] AMREX_GPU_DEVICE (Long row, Long* col, Real* val)
{
    col[0] = (row+n-1) % n; val[0] = Real(-1);
    col[1] = row;           val[1] = Real(2);
    col[2] = (row+1) % n;   val[2] = Real(-1);
}, CsrSorted{false});
```

The following operations are available.

- `SpMV(y, A, x)` computes $y = Ax$.
- `transpose(A, col_partition)` returns A^T, where `col_partition` is the column partition of `A` and becomes the row partition of the result.
- `SpGEMM(A, B, col_partition)` returns the product AB. Its rows are partitioned like `A`, and `col_partition` is the column partition of `B` and of the product. The rows of `B` needed by other processes are exchanged with MPI, and the local product uses cuSPARSE, rocSPARSE or oneMKL on GPUs and an OpenMP-parallel kernel on CPUs.

Column partitions are set on first use. Since a matrix keeps its column partition, a given matrix must always be used with the same column partition; for example, the same `col_partition` must be passed every time a matrix appears on the right-hand side of `SpGEMM`. The first use in `SpMV`, `SpGEMM`, `transpose` or a smoother also ends the setup of a matrix, in serial builds too: afterwards its entries can no longer be changed with `setVal`, `sortCSR` or the pointers from `data`, `columnIndex` and `rowOffset`.

Utilities in `AMReX_SpMatUtil.H` include `IdentityMatrix`, `RandomMatrix` and `almostEqual` for tests. See `Tests/Algebra` for examples.

The following Krylov solvers work with `SpMatrix` systems. Each of them accepts a preconditioner functor.

- GMRES_MV<T> in AMReX_GMRES_MV.H: GMRES.
- BiCGStab_MV<T> in AMReX_BiCGStab_MV.H: BiCGStab.
- PCG_MV<T> in AMReX_PCG_MV.H: preconditioned conjugate gradient, for symmetric definite systems.

All three are aliases of KrylovMV<S,T> in AMReX_KrylovMV.H. Use getSolver to reach the underlying solver, e.g., to call setInitialGuessNonzero or getStatus.

The smoothers in AMReX_Smoother_MV.H can be used as preconditioners and as multigrid smoothers. All of them work with MPI, and all of them build their scaling on first use, so the first application, and ChebyshevSmoother::lambdaMax, must be called on all processes.

- JacobiSmoother<T>: weighted Jacobi, or l1-Jacobi with the l1 option. Works on CPUs and GPUs.
- ChebyshevSmoother<T>: Chebyshev polynomial smoother. Works on CPUs and GPUs.
- L1GaussSeidelSmoother<T>: hybrid Gauss-Seidel with l1 correction. CPU builds only.

The algebraic multigrid solver AlgMG<T> in AMReX_AlgMG.H is described below.

11.17.1 Algebraic Multigrid

AlgMG<T> solves $Ax = b$ for a square SpMatrix<T> with V-cycles. The setup selects coarse points with PMIS coarsening [DeSterck2006] based on the classical strength of connection [Ruge1987], builds the interpolation P, and forms the coarse operator $A_c = P^T AP$. By default, the interpolation is extended+i [DeSterck2008] [Li2021], the smoother is Chebyshev [Adams2003], and the coarsest level is solved directly. The coarse point selection is deterministic: the same matrix with the same row distribution always gives the same hierarchy.

```
AlgMG<Real> amg(A);   // A must outlive amg
amg.setVerbose(1);
amg.setRelTol(1.e-10);
amg.setBottomSolver(AlgMG<Real>::BottomSolver::bicgstab);
amg.solve(x, b);    // x holds the initial guess
```

The iteration stops when the residual 2-norm is below setRelTol times the norm of the right-hand side; x holds the initial guess. The V-cycle is also available on its own as precond(x, b) for use in other solvers. The setup is done on the first call to solve. Parameters are set per solver object, so several solvers with different settings can coexist. updateMatrix(A2) replaces the matrix by one with the same row partition but keeps the hierarchy of the last setup: only the smoothers are rebuilt, and the V-cycle then approximates the new matrix. That is much cheaper than a new setup when the matrix changes little between solves. The solver runs on CPUs and GPUs with any number of MPI processes. MLMG can use it on the coarsest AMR level or as its bottom solver (see *Parameters*). The AlgMG tutorial uses it on its own and in MLMG. The enumerations below are AMREX_ENUM types, so they can be read from an inputs file with ParmParse::query_enum_case_insensitive. The following can be tuned:

- setInterpType: extended+i (mm_ext_i, the default), extended (mm_ext) or classical direct (direct) [Ruge1987] interpolation.
- setSmoother: Chebyshev (chebyshev, the default), l1-Jacobi (l1_jacobi) [Baker2011], weighted Jacobi (jacobi) or, in CPU builds, l1 hybrid Gauss-Seidel (l1_gauss_seidel) [Baker2011]. In the Poisson tests l1 hybrid Gauss-Seidel needs fewer cycles than the other smoothers, but each sweep is sequential within a process or OpenMP thread.
- setChebyshevDegree and setChebyshevRatio: the Chebyshev smoother is a polynomial of the given degree (2) in the l1-scaled operator that damps its eigenvalues between one divided by the given ratio (6) and one, the upper bound of its spectrum.
- setRelaxWeight: weight of the l1-Jacobi (4/3) and weighted Jacobi (2/3) smoothers.
- setPreSmooth and setPostSmooth: number of smoother sweeps before and after the coarse correction (1 for Chebyshev, 2 for the other smoothers).

- `setBottomSolver`: a direct solve of the coarsest level (`direct`, the default, for up to 1024 rows), smoother sweeps (`jacobi`), or BiCGStab (`bicgstab`) or GMRES (`gmres`) preconditioned by l1-Jacobi (weighted Jacobi when the smoother is `jacobi`).
- `setBottomTol`: relative tolerance of the BiCGStab or GMRES bottom solver (10^{-4}).
- `setKrylovSolver`: use one V-cycle as the preconditioner of an outer BiCGStab (`bicgstab`), GMRES (`gmres`) or conjugate gradient (`pcg`) solver instead of iterating it on its own (`none`, the default). In the tests in `Tests/Algebra/AlgMG` this roughly halved the number of V-cycles. The coarse operators are Galerkin products with $R = P^T$, so the matrix should be symmetric or nearly so (solve row-scaled systems in their unscaled form); PCG also requires definiteness (either sign). With PCG or GMRES, use the direct bottom solver or smoother sweeps (not a Krylov bottom solver), and with PCG also the same number of pre- and post-smoothing sweeps.
- `setSingular(true)`: for singular matrices whose null space is the constant vector, such as the Poisson operator with periodic or Neumann boundaries. As in MLMG, the mean of the right-hand side is removed, but the caller's right-hand side is not modified, and the solution is returned with zero mean.
- `setStrongThreshold`: threshold of the strength of connection, typically between 0.1 and 0.5 (0.25 by default). It trades performance for robustness: a smaller value gives a cheaper hierarchy, with less memory and less work per cycle, but is less robust for anisotropic or strongly varying coefficients, where the solver may need many more cycles. A larger value might be more robust, at the price of a more expensive hierarchy.
- `setPMaxElmts` and `setTruncFactor`: rows of P are truncated to at most four entries by default (0 disables the limit, 32 is the largest allowed, a negative value restores the default), and entries below the given fraction of the row maximum are dropped (0 by default).
- `setMaxCoarseSize` and `setMaxLevels`: coarsening stops when a level has at most this many rows (9) or this many levels (25) have been built.
- `setAggressiveNumLevels(n)`: the first `n` levels use aggressive coarsening [Yang2010] (0 by default). In the tests this roughly halved the operator complexity at the price of more cycles.
- `setAggressiveDirectInterp(true)`: makes the setup of the aggressive levels cheaper but costs further cycles, so it is off by default.

CHAPTER

TWELVE

PARTICLES

In addition to the tools for working with mesh data described in previous chapters, AMReX provides data structures and iterators for performing data-parallel particle simulations. While these tools can be used to implement pure particle methods, they are particularly suited for methods in which particles interact with data defined on a mesh or a hierarchy of meshes. Example applications include Particle-in-Cell (PIC) simulations, Lagrangian tracers, and particles that exert drag forces onto a fluid. The overall goals of AMReX's particle tools are to allow users to express a variety of useful operations on particle data, including particle-mesh and particle-particle operations, in a performance-portable and scalable manner. In the following sections, we give an overview of AMReX's particle classes and how to use them.

12.1 The Particle

The particle classes can be used by including the header AMReX_Particles.H. The most basic particle data structure is the particle itself:

```
Particle<3, 2> p;
```

This is a templated data type, designed to allow flexibility in the number and type of components that the particles carry. The first template parameter is the number of extra `Real` variables this particle will have (either single or double precision[1]), while the second is the number of extra integer variables. It is important to note that this is the number of *extra* real and integer variables; a particle will always have at least `AMREX_SPACEDIM` real components that store the particle's position and one 64-bit integer component that stores a combination of the particle's unique *id* and *cpu* numbers.

The particle struct is stored such that the `Real` components come first and the integer component second. Additionally, the required particle variables are stored before the optional ones, for both the real and the integer components. For example, say we want to define a particle type that stores a mass, three velocity components, and two extra integer flags. Our particle struct would be declared like:

```
Particle<4, 2> p;
```

and the order of the particle components would be (assuming `AMREX_SPACEDIM` is 3): `x y z m vx vy vz idcpu flag1 flag2`.[2]

The *idcpu* variable stores a combination of the MPI process a particle was generated on (the *cpu*) and an identification number that is specific to that process (the *id*). The combination of these numbers is unique across processes. This is done to facilitate the creation of particle initial conditions in parallel. In storing these identifying numbers, 39 bits are devoted to the *id*, allowing approximately 550 billion possible *local id* numbers, and 24 bits are used to store the *cpu*, allowing about 16.8 million unique (MPI) processes.

[1] Particles default to double precision for their real data. To use single precision, compile your code with `USE_SINGLE_PRECISION_PARTICLES=TRUE`.

[2] Note that for the extra particle components, which component refers to which variable is an application-specific convention - the particles have 4 extra real comps, but which one is "mass" is up to the user. We suggest using an `enum` to keep these indices straight; please see `amrex-tutorials/Particles/ElectrostaticPIC/PICParticleContainer.H` for an example of this.

One bit is devoted to marking a particle valid or invalid. This is often used to remove particles from a simulation. During `Redistribute()`, particles with invalid ids are removed from the simulation by default, although this behavior is customizable. Particles with invalid ids are also not written out during plotfile writes or checkpoint / restart operations. The allowed values for `p.id()` are *0* to *2**39 - 1*, and the allowed values for `p.cpu()` are *0* to *2**24 - 1*.

To pack and unpack these numbers, one uses the following syntax:

```
Particle <0, 0> p;
p.id() = 1;
p.cpu() = 0;
amrex::Print() << p.m_idcpu << "\n"; // 9223372036871553024
amrex::Print() << p.id() << " " << p.cpu() << "\n"; // 1 0
```

12.1.1 Setting Particle data

The `Particle` struct provides a number of methods for getting and setting a particle's data. For the required particle components, there are special, named methods. For the "extra" real and integer data, you can use the `rdata` and `idata` methods, respectively.

```
Particle<2, 2> p;

p.pos(0) = 1.0;
p.pos(1) = 2.0;
p.pos(2) = 3.0;
p.id() = 1;
p.cpu()  = 0;

// p.rdata(0) is the first extra real component, not the
// first real component overall
p.rdata(0) = 5.0;
p.rdata(1) = 5.0;

// and likewise for p.idata(0);
p.idata(0) = 17;
p.idata(1) = -64;
```

12.2 The ParticleContainer

One particle by itself is not very useful. To do real calculations, a collection of particles needs to be defined, and the location of the particles within the AMR hierarchy (and the corresponding MPI process) needs to be tracked as the particle positions change. To do this, we provide the `ParticleContainer` class:

```
using MyParticleContainer = ParticleContainer<3, 2, 4, 4>;
MyParticleContainer mypc;
```

Like the `Particle` class itself, the `ParticleContainer` class is templated. The first two template parameters have the same meaning as before: they define the number of each type of variables that the particles in this container will store. Particles added to the container are stored in the Array-of-Structs style. In addition, there are two more optional template parameters that allow the user to specify additional particle variables that will be stored in Struct-of-Arrays form.

12.2.1 Arrays-of-Structs and Structs-of-Arrays

The difference between Array-of-Structs (AoS) and Struct-of-Arrays (SoA) data is in how the data is laid out in memory. For the AoS data, all the variables associated with particle 1 are next to each other in memory, followed by all the variables associated with particle 2, and so on. For variables stored in SoA style, all the particle data for a given component is next to each other in memory, and each component is stored in a separate array. For convenience, we (arbitrarily) refer to the components in the particle struct as particle *data*, and components stored in the Struct-of-Arrays as particle *attributes*. See the figure *below* for an illustration.

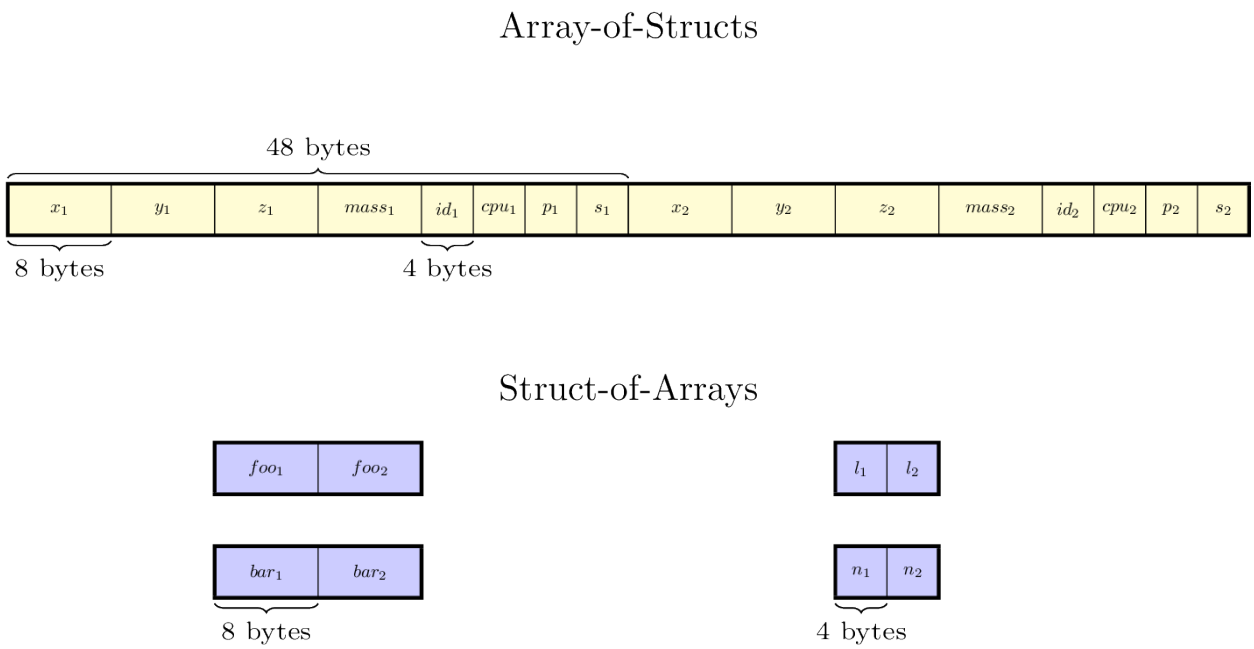


Fig. 12.1: An illustration of how the particle data for a single tile is arranged in memory. This particle container has been defined with `NStructReal = 1`, `NStructInt = 2`, `NArrayReal = 2`, and `NArrayInt = 2`. In this case, each tile in the particle container has five arrays: one with the particle struct data, two additional real arrays, and two additional integer arrays. In the tile shown, there are only 2 particles. We have labeled the extra real data member of the particle struct to be `mass`, while the extra integer members of the particle struct are labeled `p`, and `s`, for "phase" and "state". The variables in the real and integer arrays are labeled `foo`, `bar`, `l`, and `n`, respectively. We have assumed that the particles are double precision.

Attention

The ability to store particle data in AoS form is provided for backward compatibility and convenience; however, for performance reasons, whether targeting CPU or GPU execution, we recommend storing extra particle variables in SoA form. Additionally, starting in AMReX version 23.05, the ability to store *all* particle data, including the particle positions and *idcpu* numbers, is provided via the `amrex::ParticleContainerPureSoA` class. Details on using pure SoA particles are provided in the section on *Pure Struct-of-Array Particles*.

12.2.2 Constructing ParticleContainers

A particle container is always associated with a particular set of AMR grids and a particular set of DistributionMaps that describes which MPI processes those grids live on. For example, if you only have one level, you can define a `ParticleContainer` to store particles on that level using the following constructor:

```
ParticleContainer (const Geometry            & geom,
                   const DistributionMapping & dmap,
                   const BoxArray            & ba);
```

Or, if you have multiple levels, you can use following constructor instead:

```
ParticleContainer (const Vector<Geometry>            & geom,
                   const Vector<DistributionMapping> & dmap,
                   const Vector<BoxArray>            & ba,
                   const Vector<int>                 & rr);
```

The MultiLevel tutorial uses this constructor on a two-level hierarchy. It initializes and redistributes particles across the levels, deposits them on every level with the multi-level `ParticleToMesh` from `AMReX_AmrParticles.H`, and regrids a level.

Note the set of grids used to define the `ParticleContainer` doesn't have to be the same set used to define the simulation's mesh data. However, it is often desirable to have the two hierarchies track each other. If you are using an `AmrCore` class in your simulation (see the Chapter on *AmrCore Source Code*), you can achieve this by using the `AmrParticleContainer` class. The constructor for this class takes a pointer to your AmrCore derived class, instead:

```
AmrTracerParticleContainer (AmrCore* amr_core);
```

In this case, the `Vector<BoxArray>` and `Vector<DistributionMap>` used by your `ParticleContainer` will be updated automatically to match those in your `AmrCore`.

12.2.3 The ParticleTile

The `ParticleContainer` stores the particle data in a manner prescribed by the set of AMR grids used to define it. Local particle data is always stored in a data structure called a `ParticleTile`, which contains a mixture of AoS and SoA components as described above. The tiling behavior of `ParticleTile` is determined by the parameter `particles.do_tiling`:

- If `particles.do_tiling=0`, then there is always exactly one `ParticleTile` per grid. This is equivalent to setting a very large `particles.tile_size` in each direction.
- If `particles.do_tiling=1`, then each grid can have multiple `ParticleTile` objects associated with it based on the `particles.tile_size` parameter.

The AMR grid to which a particle is assigned is determined by examining its position and binning it, using the domain left edge as an offset. By default, a particle is assigned to the finest level that contains its position, although this behavior can be tweaked if desired.

Note

`ParticleTile` data tiling with *MFIter* behaves differently than mesh data. With mesh data, the tiling is strictly logical; the data is laid out in memory the same way whether tiling is turned on or off. With particle data, however, the particles are actually stored in different arrays when tiling is enabled. As with mesh data, the particle tile size can be tuned so that an entire tile's worth of particles will fit into a cache line at once.

12.2.4 Redistribute

Once the particles move, their data may no longer be in the right place in the container. They can be reassigned by calling the `Redistribute()` method of `ParticleContainer`. After calling this method, all the particles will be moved to their proper places in the container, and all invalid particles (particles with id set to `-1`) will be removed. All the MPI communication needed to do this happens automatically.

For short-range particle motion, `Redistribute` can use a local communication algorithm. The overload with a boolean local flag accepts `IntVect` values for both `nGrow` and the maximum number of cells a particle may have moved. Set the local flag to `true` only when every particle has moved no more than the supplied cell count in each direction since the last `Redistribute()` call. Use the global algorithm after initialization, regridding, load balancing, or any update that may send particles arbitrarily far.

On CPU builds, local `Redistribute()` can use a level-0 mask to speed up particle-to-grid lookup for short moves. AMReX bypasses this mask for multi-level redistribution and when the estimated mask storage is too large; in those cases, the local MPI communication algorithm can still be used while particle lookup falls back to a box search. The mask policy can be controlled with `particles.redistribute_use_mask`, `particles.redistribute_mask_max_ratio`, and `particles.redistribute_mask_max_bytes`.

On a single process with a single level, box and tile, no particle can move to another box. `Redistribute()` then works in place: it shifts particles periodically and removes those that left the domain, together with the invalid ones (unless `remove_negative` is `false`), without building communication buffers. The result is the same as that of the full algorithm. This can be turned off with `particles.redistribute_early_out = 0`.

Application codes will likely want to create their own derived ParticleContainer class that specializes the template parameters and adds additional functionality, like setting the initial conditions, moving the particles, etc. See the *particle tutorials* for examples of this. .. particle tutorials: https://amrex-codes.github.io/amrex/tutorials_html/Particles.html

12.3 Initializing Particle Data

In the following code snippet, we demonstrate how to set particle initial conditions for both SoA and AoS data. We loop over all the tiles using `MFIter`, and add as many particles as we want to each one.

```
for (MFIter mfi = MakeMFIter(lev); mfi.isValid(); ++mfi) {

    // ``particles'' starts off empty
    auto& particles = GetParticles(lev)[std::make_pair(mfi.index(),
                                       mfi.LocalTileIndex())];

    ParticleType p;
    p.id()   = ParticleType::NextID();
    p.cpu()  = ParallelDescriptor::MyProc();
    p.pos(0) = ...
    etc...

    // AoS real data
    p.rdata(0) = ...
```

(continues on next page)

(continued from previous page)

```
    p.rdata(1)  = ...

    // AoS int data
    p.idata(0) = ...
    p.idata(1) = ...

    // Particle real attributes (SoA)
    std::array<double, 2> real_attribs;
    real_attribs[0] = ...
    real_attribs[1] = ...

    // Particle int attributes (SoA)
    std::array<int, 2> int_attribs;
    int_attribs[0] = ...
    int_attribs[1]  = ...

    particles.push_back(p);
    particles.push_back_real(real_attribs);
    particles.push_back_int(int_attribs);

    // ... add more particles if desired ...
  }
```

Often, it makes sense to have each process only generate particles that it owns, so that the particles are already in the right place in the container. In general, however, users may need to call `Redistribute()` after adding particles, if the processes generate particles they don't own (for example, if the particle positions are perturbed from the cell centers and thus end up outside their parent grid).

Note

The above code snippet, which successively calls `push_back` on the particle vectors, assumes you have compiled AMReX for CPU execution. For GPU codes, one can either generate particles on the host and copy them to the device, or generate the particles directly on the GPU. For the first approach, please see the sample code here. For an example of generating particles in each cell directly on the GPU, please see the ElectrostaticPIC tutorial.

12.4 Adding particle components at runtime

In addition to the components specified as template parameters, you can also add additional `Real` and `int` components at runtime. These components will be stored in Struct-of-Array style. To add a runtime component, use the `AddRealComp` and `AddIntComp` methods of `ParticleContainer`, like so:

```
const bool communicate_this_comp = true;
for (int i = 0; i < num_runtime_real; ++i)
{
    AddRealComp(communicate_this_comp);
}
for (int i = 0; i < num_runtime_int; ++i)
{
    AddIntComp(communicate_this_comp);
}
```

Runtime-added components can be accessed like regular Struct-of-Array data. The new components will be added at the end of the compile-time defined ones.

When you are using runtime components, it is crucial that when you are adding particles to the container, you call the `DefineAndReturnParticleTile` method for each tile prior to adding any particles. This will make sure the space for the new components has been allocated. For example, in the above section on *initializing particle data*, we accessed the particle tile data using the `GetParticles` method. If runtime components are used, `DefineAndReturnParticleTile` should be used instead:

```
for(MFIter mfi = MakeMFIter(lev); mfi.isValid(); ++mfi)
{
    // instead of this...
    // auto& particles = GetParticles(lev)[std::make_pair(mfi.index(),
    //                                    mfi.LocalTileIndex())];

    // we do this...
    auto& particle_tile = DefineAndReturnParticleTile(lev, mfi);

    // add particles to particle_tile as above...
}
```

12.5 Iterating over Particles

To iterate over the particles on a given level in your container, you can use the `ParIter` class, which comes in both const and non-const flavors. For example, to iterate over all the AoS data:

```
using MyParConstIter = MyParticleContainer::ParConstIterType;
for (MyParConstIter pti(pc, lev); pti.isValid(); ++pti) {
    const auto& particles = pti.GetArrayOfStructs();
    for (const auto& p : particles) {
        // do stuff with p...
    }
}
```

The outer loop will execute once every grid (or tile, if tiling is enabled) *that contains particles*; grids or tiles that don't have any particles will be skipped. You can also access the SoA data using the $ParIter$ as follows:

```
using MyParIter = MyParticleContainer::ParIterType;
for (MyParIter pti(pc, lev); pti.isValid(); ++pti) {
    auto& particle_attributes = pti.GetStructOfArrays();
    RealVector& real_comp0 = particle_attributes.GetRealData(0);
    IntVector&  int_comp1  = particle_attributes.GetIntData(1);
    for (int i = 0; i < pti.numParticles; ++i) {
        // do stuff with your SoA data...
    }
}
```

12.6 Passing particle data into Fortran routines

Because the AMReX particle struct is a Plain-Old-Data type, it is interoperable with Fortran when the `bind(C)` attribute is used. It is therefore possible to pass a grid or tile worth of particles into fortran routines for processing, instead of iterating over them in C++. You can also define a Fortran derived type that is equivalent to C struct used for the particles. For example:

```
use amrex_fort_module, only: amrex_particle_real
use iso_c_binding ,    only: c_int

type, bind(C)  :: particle_t
   real(amrex_particle_real) :: pos(3)
   real(amrex_particle_real) :: vel(3)
   real(amrex_particle_real) :: acc(3)
   integer(c_int64_t) :: idcpu
end type particle_t
```

is equivalent to a particle struct you get with `Particle<6, 0>`. Here, `amrex_particle_real` is either single or doubled precision, depending on whether `USE_SINGLE_PRECISION_PARTICLES` is `TRUE` or not. We recommend always using this type in Fortran routines that work on particle data to avoid hard-to-debug incompatibilities between floating point types.

12.6.1 Pure Struct-of-Array Particles

Beginning with AMReX version 24.05, the requirement that the particle positions and ids be stored in AoS form has been lifted. Users can now use the class `amrex::ParticleContainerPureSoA`, which stores all components in SoA form. When using data layout, it is assumed that the first `AMREX_SPACEDIM` `Real` components store the particle position variables, which are used internally to map particle coordinates to grids and cells.

For the most part, functions that work on the standard `ParticleContainer` will also work on `ParticleContainerPureSoA`. `ParticleTile` can be used to access the underlying `StructOfArrays`, which can be used as before. However, it is particularly convenient to use the `[]` operator of `ParticleTileData`, which allows the same code to work with both AoS and pure SoA particles. For example, within a `ParIter` loop, one can do:

```
// Iterating over SoA Particles
ParticleTileDataType ptd = pti.GetParticleTile().getParticleTileData();

ParallelFor( np, [=] AMREX_GPU_DEVICE (long ip)
{
    ParticleType p = ptd[ip];  // p will be a different type for AoS and pure SoA
    // use p.pos(0), p.id(), etc.
}
```

In this way, code can be written that is agnostic as to the data layout. For more examples of pure SoA particles, please see the SOA tests in `amrex/Tests/Particles/`, or refer to WarpX, Hipace++, or ImpactX, which use this type of particle container.

12.7 Particle Reductions

AMReX provides functions to compute reductions (sums, minima, maxima, and logical and/or) over the particles in a `ParticleContainer`, on CPUs and GPUs alike. The simple entry points `amrex::ReduceSum`, `amrex::ReduceMin`, `amrex::ReduceMax`, `amrex::ReduceLogicalAnd` and `amrex::ReduceLogicalOr` reduce a single quantity computed per particle by a user-provided callable. The general `amrex::ParticleReduce` computes many reductions in a single pass over the particles, combining `amrex::ReduceOps` (e.g., several sums, minima and maxima at once) into one `amrex::ReduceData` result tuple.

The per-particle callable can take the particle in several forms: the most efficient one takes a `ConstParticleTileData` and the particle index, which reads the particle components directly from the underlying (SoA) arrays:

```
using ConstPTDType = typename PC::ConstPTDType;
amrex::ReduceOps<amrex::ReduceOpSum, amrex::ReduceOpMin, amrex::ReduceOpMax> reduce_ops;
auto r = amrex::ParticleReduce<amrex::ReduceData<amrex::Real, amrex::Real, int>> (
             pc, [=] AMREX_GPU_DEVICE (const ConstPTDType& ptd, const int i) noexcept
                         -> amrex::GpuTuple<amrex::Real, amrex::Real, int>
         {
             const amrex::Real a = ptd.rdata(1)[i];
             const amrex::Real b = ptd.rdata(2)[i];
             const int c = ptd.idata(1)[i];
             return {a, b, c};
         }, reduce_ops);
```

Note that no MPI reduction is performed at the end of these operations. Call the reductions in `ParallelDescriptor` manually if you want that behavior.

12.7.1 SIMD-Vectorized Particle Reductions

When AMReX is compiled with SIMD support (CMake option `AMReX_SIMD=ON`, see *Building with CMake*), the entry points `amrex::ParticleReduceSIMD`, `amrex::ReduceSumSIMD`, `amrex::ReduceMinSIMD` and `amrex::ReduceMaxSIMD` evaluate the reduction vectorized over `WIDTH` particles at a time, using SIMD registers as accumulators. The same user code compiles and runs unchanged in every configuration:

- `AMReX_SIMD=ON` (CPU): vectorized main loop plus a scalar remainder loop,
- `AMReX_SIMD=OFF` (CPU): a scalar loop, semantically identical to `amrex::ParticleReduce`,
- GPU builds: the regular device reduction.

The callable is invoked as `f(ptd, si)` where `si` is an `amrex::SIMDindex`. Write it generically over the index type and load particle components with `amrex::simd::load_1d`, which returns a SIMD register in the vectorized main loop and a scalar value otherwise:

```
#include <AMReX_ParticleReduceSIMD.H>

using ConstPTDType = typename PC::ConstPTDType;
amrex::ReduceOps<amrex::ReduceOpSum, amrex::ReduceOpMin> reduce_ops;
auto r = amrex::ParticleReduceSIMD<amrex::ReduceData<amrex::ParticleReal,␣
↪amrex::ParticleReal>> (
             pc, [=] AMREX_GPU_DEVICE (const ConstPTDType& ptd, auto si) noexcept
         {
             auto const a = amrex::simd::load_1d(ptd.rdata(1), si);
             auto const w = amrex::simd::load_1d(ptd.rdata(2), si);
             return amrex::makeTuple(a * w, a);
         }, reduce_ops);
```

For CUDA builds, note that extended device lambdas have nvcc-version dependent restrictions on generic (`auto`) parameters and a small functor with a templated `operator()` is the most portable form of the kernel.

The SIMD width can be chosen explicitly via the `WIDTH` template parameter (e.g., `ParticleReduceSIMD<RD, 8>`). It defaults to the native SIMD register width for `amrex::ParticleReal`, which is a good default in practice, in particular for kernels with many reduction components.

Two differences from `amrex::ParticleReduce` to be aware of:

- Only `ReduceOpSum`, `ReduceOpMin` and `ReduceOpMax` are supported (enforced at compile time).
- SIMD sum reductions reassociate floating-point additions (per-lane partial sums plus a final horizontal fold), so results can differ from the scalar evaluation order by rounding. In particular,

amrex::system::regtest_reduction is not honored by the SIMD path, use the scalar entry points when bitwise reproducible regression data is required.

Under the hood, amrex::ParticleReduceSIMD evaluates each particle tile with amrex::evalReduceSIMD<WIDTH>(n, reduce_data, f, reduce_ops), the SIMD analogue of the element loop in ReduceOps::eval. This engine is not particle-specific: it can be used directly to reduce over any indexed range on the host, with the same single-source callable contract (f(si) with an amrex::SIMDindex, provided by AMReX_ReduceSIMD.H). See Tests/SIMD for a small stand-alone example with a mixed-type tuple.

For a complete example, including a benchmark against the scalar entry points, see Tests/Particles/ParticleReduceSIMD.

12.8 Interacting with Mesh Data

It is common to want to have the mesh communicate information to the particles and vice versa. For example, in Particle-in-Cell calculations, the particles deposit their charges onto the mesh, and later, the electric fields computed on the mesh are interpolated back to the particles.

To help perform these sorts of operations, we provide the ParticleToMesh and MeshToParticles functions. These functions operate on an entire ParticleContainer at once, interpolating data back and forth between an input MultiFab. A user-provided lambda function is passed in that specifies the kind of interpolation to perform. Any needed parallel communication (from particles that contribute some of their weight to guard cells, for example) is performed internally. Additionally, these methods support both a single-grid (the particles and the mesh use the same boxes and distribution mappings) and dual-grid (the particles and mesh have different layouts) formalism. In the latter case, the needed parallel communication is also performed internally.

We show examples of these types of operations below. The first snippet shows how to deposit a particle quantity from the first real component of the particle data to the first component of a MultiFab using linear interpolation.

```
const auto plo = geom.ProbLoArray();
const auto dxi = geom.InvCellSizeArray();
amrex::ParticleToMesh(myPC, partMF, 0,
                      [=] AMREX_GPU_DEVICE (const␣
↪MyParticleContainer::ParticleTileType::ConstParticleTileDataType& ptd, int i,
                                            amrex::Array4<amrex::Real> const& rho)
    {
        auto p = ptd[i];
        ParticleInterpolator::Linear interp(p, plo, dxi);

        interp.ParticleToMesh(p, rho, 0, 0, 1,
                    [=] AMREX_GPU_DEVICE (const MyParticleContainer::ParticleType& part,␣
↪int comp)
                    {
                        return part.rdata(comp);
                    });
    });
```

The inverse operation, in which the particles communicate data *to* the mesh, is quite similar:

```
amrex::MeshToParticle(myPC, acceleration, 0,
    [=] AMREX_GPU_DEVICE (MyParticleContainer::ParticleType& p,
                          amrex::Array4<const amrex::Real> const& acc)
    {
        ParticleInterpolator::Linear interp(p, plo, dxi);
```

(continues on next page)

(continued from previous page)

```
        interp.MeshToParticle(p, acc, 0, 1+AMREX_SPACEDIM, AMREX_SPACEDIM,
                [=] AMREX_GPU_DEVICE (amrex::Array4<const amrex::Real> const& arr,
                                      int i, int j, int k, int comp)
                {
                    return arr(i, j, k, comp);  // no weighting
                },
                [=] AMREX_GPU_DEVICE (MyParticleContainer::ParticleType& part,
                                      int comp, amrex::Real val)
                {
                    part.rdata(comp) += ParticleReal(val);
                });
    });
```

In this case, we linearly interpolate *AMREX_SPACEDIM* values starting from the 0th component of the input `MultiFab` to the particles, writing them starting at particle component 1. Note that `ParticleInterpolator::MeshToParticle` takes *two* lambda functions, one that generates the particle quantity to interpolate and another that shows how to update the mesh value.

Finally, the snippet below shows how to use this function to simply count the number of particles in each cell (i.e. to deposit using "nearest neighbor" interpolation)

```
amrex::ParticleToMesh(myPC, partiMF, 0,
 [=] AMREX_GPU_DEVICE (const MyParticleContainer::SuperParticleType& p,
                       amrex::Array4<int> const& count)
 {
     ParticleInterpolator::Nearest interp(p, plo, dxi);

     interp.ParticleToMesh(p, count, 0, 0, 1,
         [=] AMREX_GPU_DEVICE (const MyParticleContainer::ParticleType& /*p*/, int /
↪*comp*/) -> int
         {
             return 1;  // just count the particles per cell
         });
 });
```

For more examples of interacting with mesh data, we refer readers to the ParticleMesh tutorial, or, for a complete example of an electrostatic PIC calculation, to the ElectrostaticPIC tutorial.

12.9 Short Range Forces

In a PIC calculation, the particles don't interact with each other directly; they only see each other through the mesh. An alternative use case is particles that exert short-range forces on each other. In this case, beyond some cut-off distance, the particles don't interact with each other and therefore don't need to be included in the force calculation. Our approach to these kind of particles is to fill "neighbor buffers" on each tile that contain copies of the particles on neighboring tiles that are within some number of cells N_g of the tile boundaries. See Fig. 12.2, below for an illustration. By choosing the number of ghost cells to match the interaction radius of the particles, you can capture all of the neighbors that can possibly influence the particles in the valid region of the tile. The forces on the particles on different tiles can then be computed independently of each other using a variety of methods.

For a `ParticleContainer` that does this neighbor finding, please see `NeighborParticleContainer` in `amrex/Src/Particle/AMReX_NeighborParticles.H`. The `NeighborParticleContainer` has additional methods called `fillNeighbors()` and `clearNeighbors()` that fill the `neighbors` data structure with copies of the

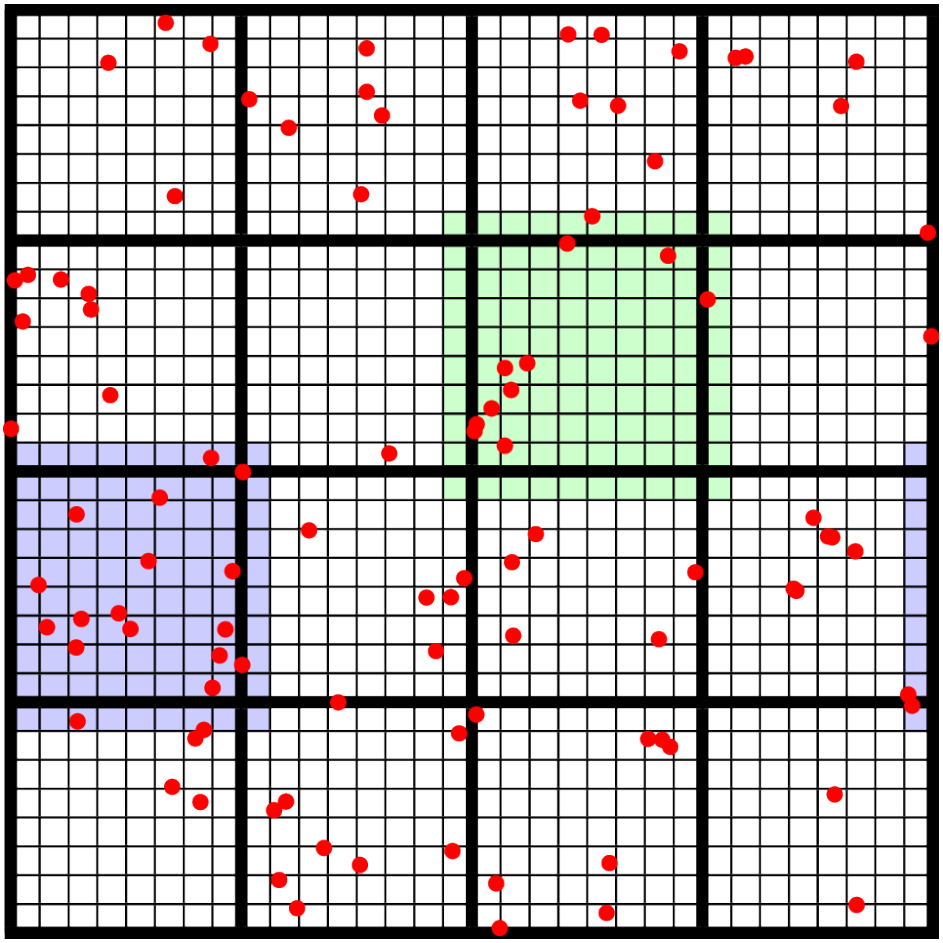


Fig. 12.2: An illustration of filling neighbor particles for short-range force calculations. Here, we have a domain consisting of one 32×32 grid, broken up into 8×8 tiles. The number of ghost cells is taken to be 1. For the tile in green, particles on other tiles in the entire shaded region will be copied and packed into the green tile's neighbor buffer. These particles can then be included in the force calculation. If the domain is periodic, particles in the grown region for the blue tile that lie on the other side of the domain will also be copied, and their positions will be modified so that a naive distance calculation between valid particles and neighbors will be correct.

proper particles. A tutorial that uses these features is available at NeighborList. For example, a function **void** `MDParticleContainer::computeForces()` can compute the forces on a given tile via direct summation over the real and neighbor particles, as follows:

```
void MDParticleContainer::computeForces()
{
    BL_PROFILE("MDParticleContainer::computeForces");

    const int lev = 0;
    const Geometry& geom = Geom(lev);
    auto& plev  = GetParticles(lev);

    for(MFIter mfi = MakeMFIter(lev); mfi.isValid(); ++mfi)
    {
        int gid = mfi.index();
        int tid = mfi.LocalTileIndex();
        auto index = std::make_pair(gid, tid);

        auto& ptile = plev[index];
        auto& aos   = ptile.GetArrayOfStructs();
        const size_t np = aos.numParticles();

        auto nbor_data = m_neighbor_list[lev][index].data();
        ParticleType* pstruct = aos().dataPtr();

       // now we loop over the neighbor list and compute the forces
        AMREX_FOR_1D ( np, i,
        {
            ParticleType& p1 = pstruct[i];
            p1.rdata(PIdx::ax) = 0.0;
            p1.rdata(PIdx::ay) = 0.0;
            p1.rdata(PIdx::az) = 0.0;

            for (const auto& p2 : nbor_data.getNeighbors(i))
            {
                Real dx = p1.pos(0) - p2.pos(0);
                Real dy = p1.pos(1) - p2.pos(1);
                Real dz = p1.pos(2) - p2.pos(2);

                Real r2 = dx*dx + dy*dy + dz*dz;
                r2 = amrex::max(r2, Params::min_r*Params::min_r);

                if (r2 > Params::cutoff*Params::cutoff) return;

                Real r = sqrt(r2);

                Real coef = (1.0 - Params::cutoff / r) / r2;
                p1.rdata(PIdx::ax) += coef * dx;
                p1.rdata(PIdx::ay) += coef * dy;
                p1.rdata(PIdx::az) += coef * dz;
            }
        });
    }
}
```

Doing a direct N^2 summation over the particles on a tile is avoided by binning the particles by cell and building a neighbor list. The data structure used to represent the neighbor lists is illustrated in Fig. 12.3.

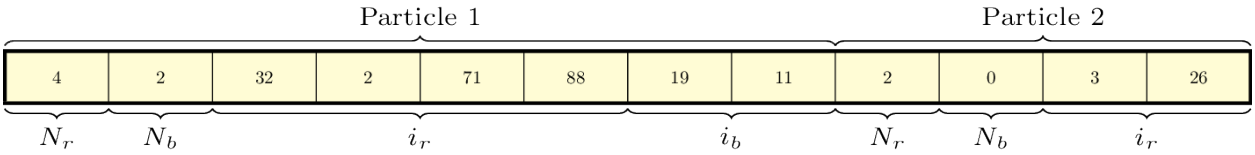


Fig. 12.3: : An illustration of the neighbor list data structure used by AMReX. The list for each tile is represented by an array of integers. The first number in the array is the number of real (i.e., not in the neighbor buffers) collision partners for the first particle on this tile, while the second is the number of collision partners from nearby tiles in the neighbor buffer. Based on the number of collision partners, the next several entries are the indices of the collision partners in the real and neighbor particle arrays, respectively. This pattern continues for all the particles on this tile.

This array can then be used to compute the forces on all the particles in one scan. Users can define their own `NeighborParticleContainer` subclasses that have their own collision criteria by overloading the virtual `check_pair` function.

12.10 Particle I/O

AMReX provides routines for writing particle data to disk for analysis, visualization, and for checkpoint / restart. The most important methods are the `WritePlotFile`, `Checkpoint`, and `Restart` methods of `ParticleContainer`, which all use a parallel-aware binary file format for reading and writing particle data on a grid-by-grid basis. These methods are designed to complement the functions in AMReX_PlotFileUtil.H for performing mesh data I/O. For example:

```
WriteMultiLevelPlotfile("plt00000", output_levs, GetVecOfConstPtrs(output),
                        varnames, geom, 0.0, level_steps, outputRR);
pc.Checkpoint("plt00000", "particle0");
```

will create a plotfile called "plt00000" and write the mesh data in `output` to it, and then write the particle data in a subdirectory called "particle0". There is also the `WriteAsciiFile` method, which writes the particles in a human-readable text format. This is mainly useful for testing and debugging.

The binary file format is readable by either `yt` or `ParaView`. See the chapter on *Visualization* for more information on visualizing AMReX datasets, including those with particles.

12.11 Input parameters

There are several runtime parameters users can set in their `inputs` files that control the behavior of the AMReX particle classes. These are summarized below. They should be preceded by "particles" in your inputs deck.

The first set of parameters concerns the tiling capability of the ParticleContainer. If you are seeing poor performance with OpenMP, the first thing to look at is whether there are enough tiles available for each thread to work on.

	Description	Type	Default
do_tiling	Whether to use tiling for particles. Should be on when using OpenMP, and off when running on GPUs.	Bool	false
tile_size	If tiling is on, the maximum tile_size in each direction	Ints	1024000,8,8

The next set concerns runtime parameters that control particle I/O. Parallel file systems tend not to like it when too many MPI tasks touch the disk at once. Additionally, performance can degrade if all MPI tasks try writing to the same file, or if too many small files are created. In general, the "correct" values of these parameters will depend on the size of your problem (i.e., number of boxes, number of MPI tasks), as well as the system you are using. If you are experiencing problems with particle I/O, you could try varying some or all of these parameters.

	Description	Type	Default
particles_nfiles	How many files to use when writing particle data to plt directories	Int	1024
nreaders	How many MPI tasks to use as readers when initializing particles from binary files.	Ints	64
nparts_per	How many particles each task should read from said files before calling Redistribute	Ints	100000
datadigits_read	This is for backward compatibility; do not use it unless you need to read an old (pre-mid-2017) AMReX dataset.	Int	5
use_prepo	This is an optimization for large particle datasets that groups MPI calls needed during the I/O together. Try it if you are seeing poor I/O speeds on large problems.	Bool	false

The following runtime parameters affect the behavior of virtual particles in Nyx.

	Description	Type	Default
aggregation_type	How to create virtual particles from finer levels. The options are: "None" - don't do any aggregation. "Cell" - when creating virtuals, combine all particles that are in the same cell.	String	"None"
aggregation_buffer	If aggregation is on, the number of cells around the coarse/fine boundary in which no aggregation should be performed.	Int	2

CHAPTER

THIRTEEN

FORTRAN INTERFACE

The core of AMReX is written in C++. For Fortran users who want to write all of their programs in Fortran, AMReX provides Fortran interfaces around most functionality except for the `AmrLevel` class (see the chapter on *Amr Source Code*) and particles (see the chapter on *Particles*). We should not confuse the Fortran interface in this chapter with the Fortran kernel functions called inside `MFIter` loops in codes (see the section on *Fortran and C++ Kernels*). For the latter, Fortran is used in some sense as a domain-specific language with native multi-dimensional arrays, whereas here Fortran is used to drive the whole application code. To better understand AMReX, Fortran interface users should read the rest of the documentation except for the Chapters on *Amr Source Code* & *Particles*.

13.1 Getting Started

We have discussed AMReX's build systems in the chapter on *Building AMReX*. To build with GNU Make, we need to include the Fortran interface source tree into the make system. The source code for the Fortran interface are in `amrex/Src/F_Interfaces` and there are several subdirectories. The "Base" directory includes sources for the basic functionality, the "AmrCore" directory wraps around the `AmrCore` class (see the chapter on *AmrCore Source Code*), and the "Octree" directory adds support for octree-type AMR grids. Each directory has a "Make.package" file that can be included in make files (see the HeatEquation tutorial and `amrex/Tests/FortranInterface/Advection_F` for examples). The libamrex approach includes the Fortran interface by default.

A minimal program is shown below in its entirety. The HeatEquation tutorial is a complete example that solves the heat equation with the Fortran interfaces.

```
program main
  use amrex_base_module
  implicit none
  call amrex_init()
  if (amrex_parallel_ioprocessor()) then
     print *, "Hello world!"
  end if
  call amrex_finalize()
end program main
```

To access the AMReX Fortran interfaces, we can use these three modules, `amrex_base_module` for the basic functionality (Section 2 *The Basics*), `amrex_amrcore_module` for AMR support (Section 3 *Amr Core Infrastructure*) and `amrex_octree_module` for octree style AMR (Section 4 *Octree*).

13.2 The Basics

Module `amrex_base_module` is a collection of various Fortran modules providing interfaces to most of the basics of AMReX C++ library (see the chapter on *Basics*). These modules shown in this section can be used without being explicitly included because they are included by `amrex_base_module`.

The spatial dimension is an integer parameter `amrex_spacedim`. We can also use the `AMREX_SPACEDIM` macro in preprocessed Fortran codes (e.g., .F90 files) just like in the C++ codes. Unlike in C++, the convention for AMReX Fortran interface is that coordinate direction index starts at 1.

There is an integer parameter `amrex_real`, a Fortran kind parameter for **`real`**. Fortran **`real`**`(amrex_real)` corresponds to `amrex::Real` in C++, which is either double or single precision depending on the setting of precision.

The module `amrex_parallel_module` (`amrex/Src/F_Interfaces/Base/AMReX_parallel_mod.F90`) includes wrappers to the `ParallelDescriptor` namespace, which is in turn a wrapper to the parallel communication library used by AMReX (e.g. MPI).

The module `amrex_parmparse_module` (`amrex/Src/Base/AMReX_parmparse_mod.F90`) provides an interface to `ParmParse` (see the section on *ParmParse*). Here are some examples.

```
type(amrex_parmparse) :: pp
integer :: n_cell, max_grid_size
call amrex_parmparse_build(pp)
call pp%get("n_cell", n_cell)
max_grid_size = 32 ! default size
call pp%query("max_grid_size", max_grid_size)
call amrex_parmparse_destroy(pp) ! optional if compiler supports finalization
```

Finalization is a Fortran 2003 feature that some compilers may not support. For those compilers, we must explicitly destroy the objects, otherwise there will be memory leaks. This applies to many other derived types.

`amrex_box` is a derived type in `amrex_box_module` `amrex/Src/F_Interfaces/Base/AMReX_box_mod.F90`. It has three members, `lo` (lower corner), `hi` (upper corner) and `nodal` (logical flag for index type).

`amrex_geometry` is a wrapper for the `Geometry` class containing information for the physical domain. Below is an example of building it.

```
integer :: n_cell
type(amrex_box) :: domain
type(amrex_geometry) :: geom
! n_cell = ...
! Define a single box covering the domain
domain = amrex_box((/0,0,0/), (/n_cell-1, n_cell-1, n_cell-1/))
! This defines an amrex_geometry object.
call amrex_geometry_build(geom, domain)
!
! ...
!
call amrex_geometry_destroy(geom)
```

`amrex_boxarray` (`amrex/Src/F_Interfaces/Base/AMReX_boxarray_mod.F90`) is a wrapper for the `BoxArray` class, and `amrex_distromap` (`amrex/Src/F_Interfaces/Base/AMReX_distromap_mod.F90`) is a wrapper for the `DistributionMapping` class. Here is an example of building a `BoxArray` and a `DistributionMapping`.

```
integer :: n_cell
type(amrex_box) :: domain
type(amrex_boxarray) :: ba
type(amrex_distromap) :: dm
! n_cell = ...
! Define a single box covering the domain
domain = amrex_box((/0,0,0/), (/n_cell-1, n_cell-1, n_cell-1/))
! Initialize the boxarray "ba" from the single box "bx"
```

(continues on next page)

(continued from previous page)

```
call amrex_boxarray_build(ba, domain)
! Break up boxarray "ba" into chunks no larger than "max_grid_size"
call ba%maxSize(max_grid_size)
! Build a DistributionMapping for the boxarray
call amrex_distromap_build(dm, ba)
!
! ...
!
call amrex_distromap_destroy(dm)
call amrex_boxarray_destroy(ba)
```

Given amrex_boxarray and amrex_distromap, we can build amrex_multifab, a wrapper for the MultiFab class, as follows.

```
integer :: ncomp, nghost
type(amrex_boxarray) :: ba
type(amrex_distromap) :: dm
type(amrex_multifab) :: mf, ndmf
! Build amrex_boxarray and amrex_distromap
! ncomp = ...
! nghost = ...
! ...
! Build amrex_multifab with ncomp components and nghost ghost cells
call amrex_multifab_build(mf, ba, dm, ncomp, nghost)
! Build a nodal multifab
call amrex_multifab_build(ndmf,ba,dm,ncomp,nghost,(/.true.,.true.,.true./))
!
! ...
!
call amrex_multifab_destroy(mf)
call amrex_multifab_destroy(ndmf)
```

There are many type-bound procedures for amrex_multifab. For example:

```
ncomp   ! Return the number of components
nghost  ! Return the number of ghost cells
setval  ! Set the data to the given value
copy    ! Copy data from given amrex_multifab to this amrex_multifab
```

Note that the copy function here only works on copying data from another amrex_multifab built with the same amrex_distromap, like the MultiFab::Copy function in C++. amrex_multifab also has two parallel communication procedures, fill_boundary and parallel_copy. Their interface and usage are very similar to functions FillBoundary and ParallelCopy for MultiFab in C++.

```
type(amrex_geometry) :: geom
type(amrex_multifab) :: mf, mfsrc
! ...
call mf%fill_boundary(geom)        ! Fill all components
call mf%fill_boundary(geom, 1, 3) ! Fill 3 components starting with component 1

call mf%parallel_copy(mfsrc, geom) ! Parallel copy from another multifab
```

It should be emphasized that the component index for amrex_multifab starts with 1, following Fortran convention.

This is different from the C++ side of AMReX.

AMReX provides a Fortran interface to `MFIter` for iterating over the data in `amrex_multifab`. The Fortran type for this is `amrex_mfiter`. Here is an example of using `amrex_mfiter` to loop over `amrex_multifab` with tiling and launch a kernel function.

```
integer :: plo(4), phi(4)
type(amrex_box) :: bx
real(amrex_real), contiguous, dimension(:,:,:,:), pointer :: po, pn
type(amrex_multifab) :: old_phi, new_phi
type(amrex_mfiter) :: mfi
! Define old_phi and new_phi ...
! In this example they are built with the same boxarray and distromap.
! And they have the same number of ghost cells and 1 component.
call amrex_mfiter_build(mfi, old_phi, tiling=.true.)
do while (mfi%next())
  bx = mfi%tilebox()
  po => old_phi%dataptr(mfi)
  pn => new_phi%dataptr(mfi)
  plo = lbound(po)
  phi = ubound(po)
  call update_phi(bx%lo, bx&hi, po, pn, plo, phi)
end do
call amrex_mfiter_destroy(mfi)
```

Here procedure `update_phi` is

```
subroutine update_phi (lo, hi, pold, pnew, plo, phi)
 integer, intent(in) :: lo(3), hi(3), plo(3), phi(3)
  real(amrex_real),intent(in   ) pold(plo(1):phi(1),plo(2):phi(2),plo(3):phi(3))
  real(amrex_real),intent(inout) pnew(plo(1):phi(1),plo(2):phi(2),plo(3):phi(3))
  ! ...
end subroutine update_phi
```

Note that amrex_multifab's procedure `dataptr` takes `amrex_mfiter` and returns a 4-dimensional Fortran pointer. For performance, we should declare the pointer as **`contiguous`**. In C++, the similar operation returns a reference to `FArrayBox`. However, `FArrayBox` and Fortran pointer have a similar capability of containing array bound information. We can call `lbound` and `ubound` on the pointer to return its lower and upper bounds. The first three dimensions of the bounds are spatial and the fourth is for the number of component.

Many of the derived Fortran types in (e.g., `amrex_multifab`, `amrex_boxarray`, `amrex_distromap`, `amrex_mfiter`, and `amrex_geometry`) contain a **`type`**(**`c_ptr`**) that points a C++ object. They also contain a **`logical`** type indicating whether or not this object owns the underlying object (i.e., responsible for deleting the object). Due to the semantics of Fortran, one should not return these types with functions. Instead we should pass them as arguments to procedures (preferably with **`intent`** specified). These five types all have assignment(=) operator that performs a shallow copy. After the assignment, the original objects still owns the data and the copy is just an alias. For example,

```
type(amrex_multifab) :: mf1, mf2
call amrex_multifab_build(mf1, ...)
call amrex_multifab_build(mf2, ...)
! At this point, both mf1 and mf2 are data owners
mf2 = mf1   ! This will destroy the original data in mf2.
            ! Then mf2 becomes a shallow copy of mf1.
            ! mf1 is still the owner of the data.
```

(continues on next page)

(continued from previous page)

```
call amrex_multifab_destroy(mf1)
! mf2 no longer contains a valid pointer because mf1 has been destroyed.
call amrex_multifab_destroy(mf2)  ! But we still need to destroy it.
```

If we need to transfer the ownership, `amrex_multifab`, `amrex_boxarray` and `amrex_distromap` provide type-bound `move` procedure. We can use it as follows

```
type(amrex_multifab) :: mf1, mf2
call amrex_multifab_build(mf1, ...)
call mf2%move(mf1)   ! mf2 is now the data owner and mf1 is not.
call amrex_multifab_destroy(mf1)
call amrex_multifab_destroy(mf2)
```

`amrex_multifab` also has a type-bound `swap` procedure for exchanging the data.

AMReX also provides `amrex_plotfile_module` for writing plotfiles. The interface is similar to the C++ versions.

13.3 Amr Core Infrastructure

The module `amrex_amr_module` provides interfaces to AMR core infrastructure. With AMR, the main program might look like below,

```
program main
  use amrex_amr_module
  implicit none
  call amrex_init()
  call amrex_amrcore_init()
  call my_amr_init()       ! user's own code, not part of AMReX
  ! ...
  call my_amr_finalize()   ! user's own code, not part of AMReX
  call amrex_amrcore_finalize()
  call amrex_finalize()
end program main
```

Here we need to call `amrex_amrcore_init` and `amrex_amrcore_finalize`. And usually we need to call application code specific procedures to provide some "hooks" needed by AMReX. In C++, this is achieved by using virtual functions. In Fortran, we need to call

```
subroutine amrex_init_virtual_functions (mk_lev_scrtch, mk_lev_crse, &
                                         mk_lev_re, clr_lev, err_est)

  ! Make a new level from scratch using provided boxarray and distromap
  ! Only used during initialization.
  procedure(amrex_make_level_proc)  :: mk_lev_scrtch
  ! Make a new level using provided boxarray and distromap, and fill
  ! with interpolated coarse level data.
  procedure(amrex_make_level_proc)  :: mk_lev_crse
  ! Remake an existing level using provided boxarray and distromap,
  ! and fill with existing fine and coarse data.
  procedure(amrex_make_level_proc)  :: mk_lev_re
  ! Delete level data
  procedure(amrex_clear_level_proc) :: clr_lev
```

(continues on next page)

(continued from previous page)

```
  ! Tag cells for refinement
  procedure(amrex_error_est_proc)   :: err_est
end subroutine amrex_init_virtual_functions
```

We need to provide five functions and these functions have three types of interfaces:

```
subroutine amrex_make_level_proc (lev, time, ba, dm) bind(c)
  import
  implicit none
  integer, intent(in), value :: lev
  real(amrex_real), intent(in), value :: time
  type(c_ptr), intent(in), value :: ba, dm
end subroutine amrex_make_level_proc

subroutine amrex_clear_level_proc (lev) bind(c)
  import
  implicit none
  integer, intent(in) , value :: lev
end subroutine amrex_clear_level_proc

subroutine amrex_error_est_proc (lev, tags, time, tagval, clearval) bind(c)
  import
  implicit none
  integer, intent(in), value :: lev
  type(c_ptr), intent(in), value :: tags
  real(amrex_real), intent(in), value :: time
  character(c_char), intent(in), value :: tagval, clearval
end subroutine amrex_error_est_proc
```

`amrex/Tests/FortranInterface/Advection_F/Source/my_amr_mod.F90` shows an example of the setup process. The user provided `procedure(amrex_error_est_proc)` has a tags argument that is of type `c_ptr` and its value is a pointer to a `TagBoxArray` object. We need to convert this into a Fortran `amrex_tagboxarray` object.

```
type(amrex_tagboxarray) :: tag
tag = tags
```

The module `amrex_fillpatch_module` provides an interface to C++ functions `FillPatchSinglelevel` and `FillPatchTwoLevels`. To use it, the application code needs to provide procedures for interpolation and filling physical boundaries. See `amrex/Tests/FortranInterface/Advection_F/Source/fillpatch_mod.F90` for an example.

Module `amrex_fluxregister_module` provides an interface to `FluxRegister` (see the section on *Using FluxRegisters*). Its usage is demonstrated in `amrex/Tests/FortranInterface/Advection_F`.

13.4 Octree

In AMReX, the union of fine level grids is properly contained within the union of coarse level grids. There are no required direct parent-child connections between levels. Therefore, grids in AMReX in general cannot be represented by trees. Nevertheless, octree-type grids are supported via the Fortran interface, because grids are more general than octree grids. A tutorial example using amrex_octree_module (`amrex/Src/F_Interfaces/Octree/AMReX_octree_mod.f90`) is available at `amrex/Tests/FortranInterface/Advection_octree_F/`. Procedures `amrex_octree_init` and `amrex_octree_finalize` must be called as follows,

```
program main
  use amrex_amrcore_module
  use amrex_octree_module
  implicit none
  call amrex_init()
  call amrex_octree_init()  ! This should be called before amrex_amrcore_init.
  call amrex_amrcore_init()
  call my_amr_init()       ! user's own code, not part of AMReX
  ! ...
  call my_amr_finalize()   ! user's own code, not part of AMReX
  call amrex_amrcore_finalize()
  call amrex_octree_finalize()
  call amrex_finalize()
end program main
```

By default, the grid size is 8^3, and this can be changed via `ParmParse` parameter `amr.max_grid_size`. The module `amrex_octree_module` provides `amrex_octree_iter` that can be used to iterate over leaves of octree. For example,

```
type(amrex_octree_iter) :: oti
type(multifab) :: phi_new(*)   ! one multifab for each level
integer :: ilev, igrd
type(amrex_box) :: bx
real(amrex_real), contiguous, pointer, dimension(:,:,:,:) :: pout
call amrex_octree_iter_build(oti)
do while(oti%next())
   ilev = oti%level()
   igrd = oti%grid_index()
   bx   = oti%box()
   pout => phi_new(ilev)%dataptr(igrd)
   ! ...
end do
call amrex_octree_iter_destroy(oti)
```

CHAPTER

FOURTEEN

PYTHON INTERFACE

The core of AMReX is written in C++. For users who want to write all of their programs in Python, or for C++ application developers who would like to add Python interfaces to their applications for scripting, rapid prototyping, code coupling, and/or AI/ML workflows, many AMReX classes, functions, and data containers are also available.

Please see pyAMReX (manual) for further details.

CHAPTER

FIFTEEN

EMBEDDED BOUNDARIES

For computations with complex geometries, AMReX provides data structures and algorithms to employ an embedded boundary (EB) approach to PDE discretizations. In this approach, the underlying computational mesh is uniform and block-structured, but the boundary of the irregular-shaped computational domain conceptually cuts through this mesh. Each cell in the mesh becomes labeled as regular, cut or covered, and the finite-volume based discretization methods traditionally used in AMReX applications can be modified to incorporate these cell shapes. See Fig. 15.1 for an illustration.

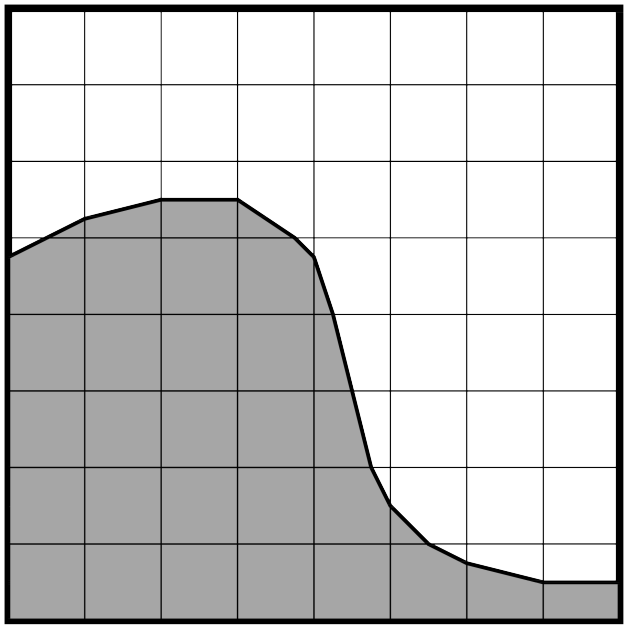


Fig. 15.1: In the embedded boundary approach to discretizing PDEs, the (uniform) rectangular mesh is cut by the irregular shape of the computational domain. The cells in the mesh are labeled as regular, cut, or covered.

Note that in a completely general implementation of the EB approach, there would be no restrictions on the shape or complexity of the EB surface. With this generality comes the possibility that the process of "cutting" the cells results in a single (i, j, k) cell being broken into multiple cell fragments. The current release of AMReX does not support multi-valued cells, thus there is a practical restriction on the complexity of domains (and numerical algorithms) supported.

AMReX's relatively simple grid generation technique allows computational meshes for rather complex geometries to be generated quickly and robustly. However, the technique can produce arbitrarily small cut cells in the domain. In practice, such small cells can have significant impact on the robustness and stability of traditional finite volume methods. The redistribution section in AMReX-Hydro's documentation provides an overview of the finite volume discretization

in an embedded boundary cell and a class of approaches to deal with this "small cell" problem in a robust and efficient way.

This chapter discusses the EB tools, data structures and algorithms currently supported by AMReX to enable the construction of discretizations of conservation law systems. The discussion will focus on general requirements associated with building fluxes and taking divergences of them to advance such systems. We also give examples of how to initialize the geometry data structures and access them to build the numerical difference operators. Finally, we present EB support for linear solvers.

15.1 Initializing the Geometric Database

In AMReX, geometric information is stored in a distributed database class that must be initialized at the start of the calculation. The procedure is as follows:

- Define an implicit function of position which describes the surface of the embedded object. Specifically, the function class must have a public member function that takes a position and returns a negative value if that position is inside the fluid, a positive value in the body, and identically zero at the embedded boundary.

```
Real operator() (const Array<Real,AMREX_SPACEDIM>& p) const;
```

- Make a `EB2::GeometryShop` object using the implicit function.
- Build an `EB2::IndexSpace` with the `EB2::GeometryShop` object and a `Geometry` object that contains the information about the domain and the mesh.

Here is a simple example of initializing the database for an embedded sphere.

```
Real radius = 0.5;
Array<Real,AMREX_SPACEDIM> center{0., 0., 0.}; //Center of the sphere
bool inside = false;  // Is the fluid inside the sphere?
EB2::SphereIF sphere(radius, center, inside);

auto shop = EB2::makeShop(sphere);

Geometry geom(...);
EB2::Build(shop, geom, 0, 0);
```

Alternatively, the EB information can be initialized from an STL file specified by a `ParmParse` parameter `eb2.stl_file`. (This also requires setting `eb2.geom_type = stl`.) The initialization is done by calling

```
EB2::Build (const Geometry& geom,
            int required_coarsening_level,
            int max_coarsening_level,
            int ngrow = 4,
            bool build_coarse_level_by_coarsening = true);
```

Additionally, one can use `eb2.stl_scale`, `eb2.stl_center` and `eb2.stl_reverse_normal` to scale, translate and reverse the object, respectively.

15.1.1 Implicit Function

In `amrex/Src/EB/`, there are a number of predefined implicit function classes for basic shapes. One can use these directly or as templates for their own classes.

- `AllRegularIF`: No embedded boundaries at all.
- `BoxIF`: Box.

- `CylinderIF`: Cylinder.
- `EllipsoidIF`: Ellipsoid.
- `PlaneIF`: Half-space plane.
- `SphereIF`: Sphere.

AMReX also provides a number of transformation operations to apply to an object.

- `makeComplement`: Complement of an object. E.g., a sphere with fluid on the outside becomes a sphere with fluid inside.
- `makeIntersection`: Intersection of two or more objects.
- `makeUnion`: Union of two or more objects.
- `Translate`: Translates an object.
- `scale`: Scales an object.
- `rotate`: Rotates an object.
- `lathe`: Creates a surface of revolution by rotating a 2D object around an axis.

Here are some examples of using these functions.

```
EB2::SphereIF sphere1(...);
EB2::SphereIF sphere2(...);
EB2::BoxIF box(...);
EB2::CylinderIF cylinder(...);
EB2::PlaneIF plane(...);

// union of two spheres
auto twospheres = EB2::makeUnion(sphere1, sphere2);

// intersection of a rotated box, a plane and the union of two spheres
auto box_plane = EB2::makeIntersection(amrex::rotate(box,...),
                                       plane,
                                       twospheres);

// scale a cylinder by a factor of 2 in x and y directions, and 3 in z-direction.
auto scylinder = EB2::scale(cylinder, {2., 2., 3.});
```

15.1.2 EB2::GeometryShop

Given an implicit function object, say `f`, we can make a `GeometryShop` object with

```
auto shop = EB2::makeShop(f);
```

15.1.3 EB2::IndexSpace

We build `EB2::IndexSpace` with one of several functions depending on the application needs.

Standard Build with Automatic Coarsening

```
template <typename G>
void EB2::Build (const G& gshop, const Geometry& geom,
                 int required_coarsening_level,
                 int max_coarsening_level,
```

(continues on next page)

(continued from previous page)

```
                int ngrow = 4,
                bool build_coarse_level_by_coarsening = true,
                bool extend_domain_face = ExtendDomainFace(),
                int num_coarsen_opt = NumCoarsenOpt());
```

Here the template parameter is a `EB2::GeometryShop`. `Geometry` (see section *RealBox and Geometry*) describes the rectangular problem domain and the mesh on the finest AMR level. Coarse level EB data is generated from coarsening the original fine data. The **`int`** `required_coarsening_level` parameter specifies the number of coarsening levels required. This is usually set to $N-1$, where N is the total number of AMR levels. The **`int`** `max_coarsening_levels` parameter specifies the number of coarsening levels AMReX should try to have. This is usually set to a big number, say 20 if multigrid solvers are used. This essentially tells the build to coarsen as much as it can. If there are no multigrid solvers, the parameter should be set to the same as `required_coarsening_level`. The nodal solver `MLEBNodeFDLaplacian` does not use coarsened EB levels, so it does not need this parameter to be large. It should be noted that coarsening could create multi-valued cells even if the fine level does not have any multi-valued cells. This occurs when the embedded boundary cuts a cell in such a way that there is fluid on multiple sides of the boundary within that cell. Because multi-valued cells are not supported, it will cause a runtime error if the required coarsening level generates multi-valued cells. The optional **`int`** `ngrow` parameter specifies the number of ghost cells outside the domain on required levels. For levels coarser than the required level, no EB data are generated for ghost cells outside the domain.

The optional **`bool`** `extend_domain_face` parameter controls the embedded boundary outside the domain. If it is true (the default), the geometry outside each non-periodic domain face is an extrusion of the geometry on that face, that is, the embedded boundary is extended perpendicularly from the face. This holds on every level, including the coarse levels built by coarsening. If it is false, the geometry outside the domain comes from the implicit function itself. Fig. 15.2 illustrates the two choices for a body that crosses the lower domain face. The parameter has the same effect for geometries built from an STL file (see below).

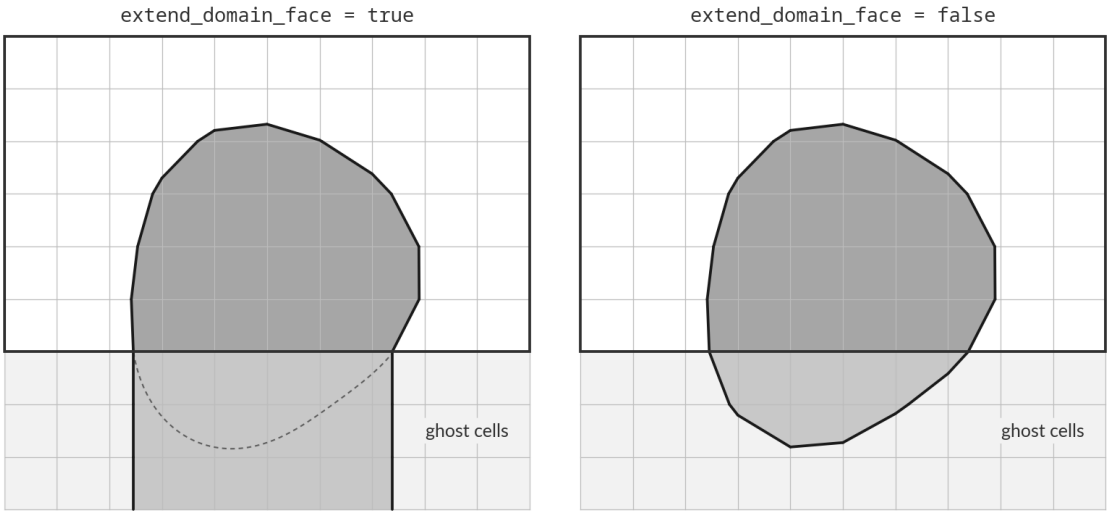


Fig. 15.2: Embedded boundary in the cut cells for a body crossing the lower domain face. The lighter shading marks the ghost cells outside the domain and the body in them. Left: with `extend_domain_face = true`, the body outside the domain is extended perpendicularly from the domain face, and the actual surface there (dashed) is not used. Right: with `extend_domain_face = false`, the actual surface is used outside the domain.

Build with Explicit Multi-Level Geometry

For applications requiring explicit control over the geometry at each AMR level:

```
template <typename G>
void EB2::Build (const G& gshop, Vector<Geometry> geom,
                 int ngrow = 4,
                 bool extend_domain_face = ExtendDomainFace(),
                 int num_coarsen_opt = NumCoarsenOpt());
```

This version takes a `Vector<Geometry>` where each element corresponds to the geometry of a specific AMR level. The vector can be unordered, as it will be sorted based on `numPts`. Unlike the standard `Build` function, coarse level EB data is generated directly from the provided geometries rather than through automatic coarsening. This is useful when coarse level domains are not simple coarsenings of the fine level, or when you need precise control over the domain and mesh spacing at each level.

Build from STL File

As mentioned earlier, the EB information can alternatively be initialized from an STL file using:

```
void EB2::Build (const Geometry& geom,
                 int required_coarsening_level,
                 int max_coarsening_level,
                 int ngrow = 4,
                 bool build_coarse_level_by_coarsening = true,
                 bool extend_domain_face = ExtendDomainFace(),
                 int num_coarsen_opt = NumCoarsenOpt());
```

This requires setting `ParmParse` parameters `eb2.geom_type = stl` and `eb2.stl_file` to specify the STL file path. The embedded boundary outside the domain follows `extend_domain_face` as for a `EB2::GeometryShop`: if it is true, the boundary is extended perpendicularly from the domain face; if it is false, the boundary is taken from the STL mesh itself. A grid box that no triangle touches is classified as entirely regular or entirely covered from a single point, which is correct only for a water-tight mesh.

Managing IndexSpace Objects

Regardless of which `Build` variant is used, the newly built `EB2::IndexSpace` is pushed onto a stack. Static function `EB2::IndexSpace::top()` returns a `const &` to the new `EB2::IndexSpace` object. We usually only need to build one `EB2::IndexSpace` object. However, if your application needs multiple `EB2::IndexSpace` objects, you can save the pointers for later use. For simplicity, we assume there is only one `EB2::IndexSpace` object for the rest of this chapter.

15.2 EBFArrayBoxFactory

After the EB database is initialized, the next object we build is `EBFArrayBoxFactory`. This object provides access to the EB database in the format of basic AMReX objects such as `BaseFab`, `FArrayBox`, `FabArray`, and `MultiFab`. We can construct it with

```
EBFArrayBoxFactory (const Geometry& a_geom,
                    const BoxArray& a_ba,
                    const DistributionMapping& a_dm,
                    const Vector<int>& a_ngrow,
                    EBSupport a_support);
```

or

```
std::unique_ptr<EBFArrayBoxFactory>
makeEBFabFactory (const Geometry& a_geom,
                  const BoxArray& a_ba,
```

(continues on next page)

(continued from previous page)

```
                const DistributionMapping& a_dm,
                const Vector<int>& a_ngrow,
                EBSupport a_support);
```

Argument `Vector<int> const& a_ngrow` specifies the number of ghost cells we need for EB data at various `EBSupport` levels, and argument `EBSupport a_support` specifies the level of support needed.

- `EBSupport::basic`: basic flags for cell types
- `EBSupport::volume`: basic plus volume fraction and centroid
- `EBSupport::full`: volume plus area fraction, boundary centroid and face centroid

`EBFArrayBoxFactory` is derived from `FabFactory<FArrayBox>`. `MultiFab` constructors have an optional argument `const FabFactory<FArrayBox>&`. We can use `EBFArrayBoxFactory` to build `MultiFabs` that carry EB data. The member function of `FabArray`

```
const FabFactory<FAB>& Factory () const;
```

can then be used to return a reference to the `EBFArrayBoxFactory` used for building the `MultiFab`. Using `dynamic_cast`, we can test whether a `MultiFab` is built with an `EBFArrayBoxFactory`.

```
auto factory = dynamic_cast<EBFArrayBoxFactory const*>(&(mf.Factory()));
if (factory) {
    // this is EBFArrayBoxFactory
} else {
    // regular FabFactory<FArrayBox>
}
```

15.3 Embedded Boundary Data

Through member functions of `EBFArrayBoxFactory`, we have access to the following data:

```
// see section on EBCellFlagFab
const FabArray<EBCellFlagFab>& getMultiEBCellFlagFab () const;

// volume fraction
const MultiFab& getVolFrac () const;

// volume centroid
const MultiCutFab& getCentroid () const;

// embedded boundary centroid
const MultiCutFab& getBndryCent () const;

// embedded boundary normal direction
const MultiCutFab& getBndryNormal () const;

// embedded boundary surface area
const MultiCutFab& getBndryArea () const;

// area fractions
Array<const MultiCutFab*,AMREX_SPACEDIM> getAreaFrac () const;
```

(continues on next page)

(continued from previous page)

```
// face centroid
Array<const MultiCutFab*,AMREX_SPACEDIM> getFaceCent () const;
```

- **Volume fraction** is stored in a single-component `MultiFab`. Data are in the range of $[0, 1]$ with zero representing covered cells and one for regular cells.
- **Volume centroid** (also called cell centroid) is in a `MultiCutFab` with `AMREX_SPACEDIM` components. Each component of the data is in the range $[-0.5, 0.5]$, based on each cell's local coordinates with respect to the regular cell's center.
- **Boundary centroid** is also in a `MultiCutFab` with `AMREX_SPACEDIM` components. Each component of the data is in the range $[-0.5, 0.5]$, based on each cell's local coordinates with respect to the regular cell's center.
- **Boundary normal** is in a `MultiCutFab` with `AMREX_SPACEDIM` components representing the unit vector in physical space pointing toward the covered part.
- **Boundary area** is in a `MultiCutFab` with a single component representing the dimensionless boundary area. When the cell is isotropic (i.e., $\Delta x = \Delta y = \Delta z$), it's trivial to convert it to physical units. If the cell size is anisotropic, the conversion requires multiplying by a factor of $\sqrt{(n_x \Delta y \Delta z)^2 + (n_y \Delta x \Delta z)^2 + (n_z \Delta x \Delta y)^2}$, where n is the boundary normal vector.
- **Area fractions** are returned in an `Array` of `MultiCutFab` pointers. For each direction, the area fraction is for the face in that direction. Data are in the range of $[0, 1]$ with zero representing a covered face and one an un-cut face.
- **Face centroids** are returned in an `Array` of `MultiCutFab` pointers. There are two components for each direction, and the ordering is always the same as the original ordering of the coordinates. For example, for y face, the component 0 is for x coordinate and 1 for z. The coordinates are in each face's local frame normalized to the range of $[-0.5, 0.5]$.

15.4 Embedded Boundary Data Structures

A `MultiCutFab` is very similar to a `MultiFab`. Its data can be accessed with subscript operator

```
const CutFab& operator[] (const MFIter& mfi) const;
```

Here `CutFab` is derived from `FArrayBox` and can be passed to Fortran just like `FArrayBox`. The difference between `MultiCutFab` and `MultiFab` is that to save memory `MultiCutFab` only has data on boxes that contain cut cells. It is an error to call `operator[]` if that box does not have cut cells. Thus the call must be in a `if` test block (see section *EBCellFlagFab*).

15.4.1 EBCellFlagFab

`EBCellFlagFab` contains information on cell types. We can use it to determine if a box contains cut cells.

```
auto const& flags = factory->getMultiEBCellFlagFab();
MultiCutFab const& centroid = factory->getCentroid();

for (MFIter mfi ...) {
    const Box& bx = mfi.tilebox();
    FabType t = flags[mfi].getType(bx);
    if (FabType::regular == t) {
        // This box is regular
    } else if (FabType::covered == t) {
```

(continues on next page)

(continued from previous page)

```
        // This box is covered
    } else if (FabType::singlevalued == t) {
        // This box has cut cells
        // Getting cutfab is safe
        const auto& centroid_fab = centroid[mfi];
    }
}
```

`EBCellFlagFab` is derived from `BaseFab`. Its data are stored in an array of 32-bit integers, and can be used in C++ or passed to Fortran just like an `IArrayBox` (section *BaseFab, FArrayBox, IArrayBox, and Array4*). AMReX provides a Fortran module called `amrex_ebcellflag_module`. This module contains procedures for testing cell types and getting neighbor information. For example

```
use amrex_ebcellflag_module, only : is_regular_cell, is_single_valued_cell, is_covered_
↪cell

integer, intent(in) :: flags(...)

integer :: i,j,k

do k = ...
    do j = ...
        do i = ...
            if (is_covered_cell(flags(i,j,k))) then
                ! this is a completely covered cell
            else if (is_regular_cell(flags(i,j,k))) then
                ! this is a regular cell
            else if (is_single_valued_cell(flags(i,j,k))) then
                ! this is a cut cell
            end if
        end do
    end do
end do
```

15.5 Small Cell Problem and Redistribution

First, we review finite volume discretizations with embedded boundaries as used by AMReX-based applications. Then we illustrate the small cell problem.

15.5.1 Finite Volume Discretizations

Consider a system of PDEs to advance a conserved quantity U with fluxes F:

$$\frac{\partial U}{\partial t} + \nabla \cdot F = 0. \tag{15.1}$$

A conservative, finite volume discretization starts with the divergence theorem

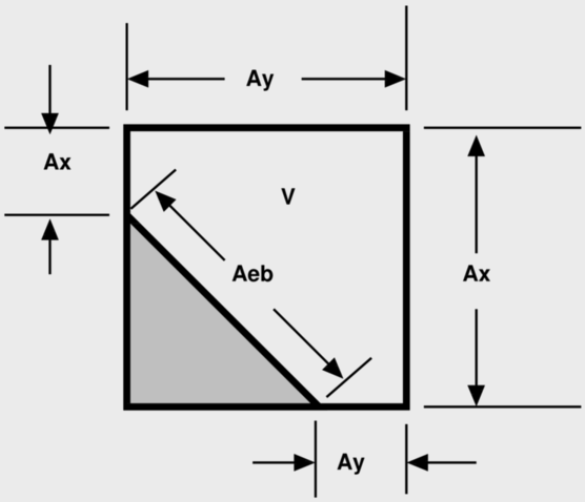
$$\int_V \nabla \cdot F dV = \int_{\partial V} F \cdot n dA.$$

In an embedded boundary cell, the "conservative divergence" is discretized (as $D^c(F)$) as follows

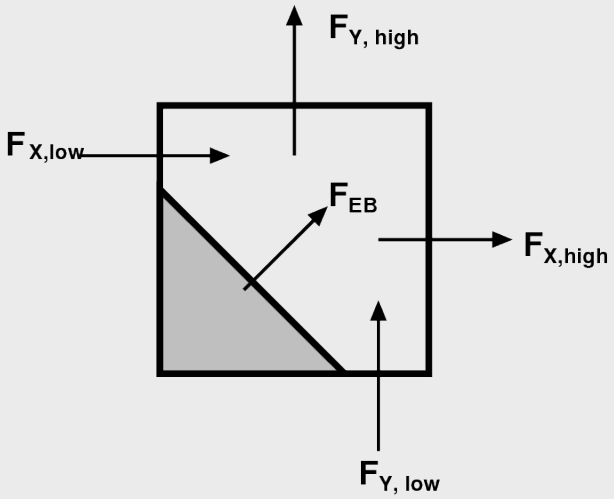
$$D^c(F) = \frac{1}{\kappa h}\left(\sum_{d=1}^{D}(F_{d,\mathrm{hi}}\,\alpha_{d,\mathrm{hi}} - F_{d,\mathrm{lo}}\,\alpha_{d,\mathrm{lo}}) + F^{EB}\alpha^{EB}\right). \tag{15.2}$$

Geometry is discretely represented by volumes ($V = \kappa h^d$) and apertures ($A = \alpha h^{d-1}$), where h is the (uniform) mesh spacing at that AMR level, κ is the volume fraction and α are the area fractions. Without multivalued cells the volume fractions, area fractions and cell and face centroids (see Table 15.1) are the only geometric information needed to compute second-order fluxes centered at the face centroids, and to infer the connectivity of the cells. Cells are connected if adjacent on the Cartesian mesh, and only via coordinate-aligned faces on the mesh. If an aperture, $\alpha = 0$, between two cells, they are not directly connected to each other.

Table 15.1: Illustration of embedded boundary cutting a two-dimensional cell.



A typical two-dimensional uniform cell that is cut by the embedded boundary. The grey area represents the region excluded from the calculation. The portion of the cell faces faces (labelled with A) through which fluxes flow are the "uncovered" regions of the full cell faces. The volume (labelled V) is the uncovered region of the interior.



Fluxes in a cut cell.

15.5.2 Small Cells and Stability

In the context of time-explicit advance methods for, say hyperbolic conservation laws, a naive discretization in time of (15.1) using (15.2),

$$U^{n+1} = U^n - \delta t D^c(F)$$

would have a time step constraint $\delta t \sim h\kappa^{1/D}/V_m$, which goes to zero as the size of the smallest volume fraction κ in the calculation. Since EB volume fractions can be arbitrarily small, this presents an unacceptable constraint. This is the so-called "small cell problem," and AMReX-based applications address it with redistribution methods.

15.5.3 Flux Redistribution

Consider a conservative update in the form:

$$(\rho\phi)_t + \nabla \cdot (\rho\phi u) = RHS$$

For each valid cell in the domain, compute the conservative divergence, $(\nabla \cdot F)^c$, of the convective fluxes, F

$$(\nabla \cdot F)_i^c = \frac{1}{\mathcal{V}_i} \sum_{f=1}^{N_f} (F_f \cdot n_f) A_f$$

Here N_f is the number of faces of cell i, $\vec{n}_f$ and A_f are the unit normal and area of the f -th face respectively, and $\mathcal{V}_i$ is the volume of cell i given by

$$\mathcal{V}_i = (\Delta x \Delta y \Delta z) \cdot \mathcal{K}_i$$

where $\mathcal{K}_i$ is the volume fraction of cell i .

Now, a conservative update can be written as

$$\frac{\rho^{n+1}\phi^{n+1} - \rho^n \phi^n}{\Delta t} = -\nabla \cdot F^c$$

For each cell cut by the EB geometry, compute the non-conservative update, $\nabla \cdot F^{nc}$,

$$\nabla \cdot F_i^{nc} = \frac{\sum\limits_{j \in N(i)} \mathcal{K}_j \nabla \cdot F_j^c}{\sum\limits_{j \in N(i)} \mathcal{K}_j}$$

where $N(i)$ is the index set of cell i and its neighbors.

For each cell cut by the EB geometry, compute the convective update $\nabla \cdot F^{EB}$ as follows:

$$\nabla \cdot F_i^{EB} = \mathcal{K}_i \nabla \cdot F_i^c + (1 - \mathcal{K}_i) \nabla \cdot \mathcal{F}_i^{nc}$$

For each cell cut by the EB geometry, redistribute its mass loss, δM_i , to its neighbors:

$$\nabla \cdot F_j^{EB} := \nabla \cdot F_j^{EB} + w_{ij} \delta M_i \qquad \forall j \in N(i) \setminus i$$

where the mass loss in cell i, δM_i, is given by

$$\delta M_i = \mathcal{K}_i (1 - \mathcal{K}_i)[\nabla \cdot F_i^c - \nabla \cdot F_i^{nc}]$$

and the weights, w_{ij} , are

$$w_{ij} = \frac{1}{\sum\limits_{j \in N(i) \setminus i} \mathcal{K}_j}$$

Note that $\nabla \cdot F_i^{EB}$ gives an update for $\rho\phi$; i.e.,

$$\frac{(\rho\phi_i)^{n+1} - (\rho\phi_i)^n}{\Delta t} = -\nabla \cdot F_i^{EB}$$

Typically, the redistribution neighborhood for each cell is one that can be reached via a monotonic path in each coordinate direction of unit length (see, e.g., Fig. 15.3)

15.5.4 State Redistribution

For state redistribution, we implement the weighted state redistribution algorithm as described in Guiliani et al (2021), which is available on arXiv. This is an extension of the original state redistribution algorithm of Berger and Guiliani (2020).

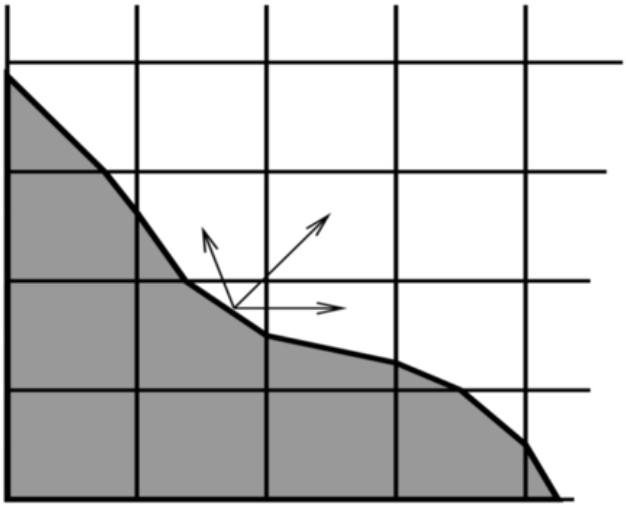


Fig. 15.3: Redistribution illustration. Excess update distributed to neighbor cells.

Each small cell is merged with enough of its neighbors that the merged neighborhood has a volume of at least `target_volfrac` (0.5 by default). A neighborhood never reaches across a domain boundary unless that direction is periodic, because the state outside a non-periodic boundary is not part of the solution and including it would break conservation. As a consequence, a small cell sitting against a non-periodic domain boundary can end up with a neighborhood whose volume is less than `target_volfrac` – it is merged with everything the domain makes available, which is the best that can be done there. Away from domain boundaries, failing to reach `target_volfrac` still indicates a problem with the geometry and aborts.

15.6 Linear Solvers

Linear solvers for the canonical form (equation (11.1)) have been discussed in Chapter *Linear Solvers*.

AMReX supports multi-level 1) cell-centered solvers with homogeneous Neumann, homogeneous Dirichlet, or inhomogeneous Dirichlet boundary conditions on the EB faces, and 2) nodal solvers with homogeneous Neumann boundary conditions, or inflow velocity conditions on the EB faces.

To use a cell-centered solver with EB, one builds a linear operator `MLEBABecLap` with `EBFArrayBoxFactory` (instead of a `MLABecLaplacian`)

```
MLEBABecLap (const Vector<Geometry>& a_geom,
             const Vector<BoxArray>& a_grids,
             const Vector<DistributionMapping>& a_dmap,
             const LPInfo& a_info,
             const Vector<EBFArrayBoxFactory const*>& a_factory);
```

The usage of this EB-specific class is essentially the same as `MLABecLaplacian`.

The default boundary condition on EB faces is homogeneous Neumann.

To set homogeneous Dirichlet boundary conditions, call

```
ml_ebabeclap->setEBHomogDirichlet(lev, coeff);
```

where coeff can be a real number (i.e. the value is the same at every cell) or is the MultiFab holding the coefficient of the gradient at each cell with an EB face.

To set inhomogeneous Dirichlet boundary conditions, call

```
ml_ebabeclap->setEBDirichlet(lev, phi_on_eb, coeff);
```

where phi_on_eb is the MultiFab holding the Dirichlet values in every cut cell, and coeff again is a real number (i.e. the value is the same at every cell) or a MultiFab holding the coefficient of the gradient at each cell with an EB face.

Currently there are options to define the face-based coefficients on face centers vs face centroids, and to interpret the solution variable as being defined on cell centers vs cell centroids.

The default is for the solution variable to be defined at cell centers; to tell the solver to interpret the solution variable as living at cell centroids, you must set

```
ml_ebabeclap->setPhiOnCentroid();
```

The default is for the face-based coefficients to be defined at face centers; to tell the solver that the face-based coefficients should be interpreted as living at face centroids, modify the setBCoeffs command to be

```
ml_ebabeclap->setBCoeffs(lev, beta, MLMG::Location::FaceCentroid);
```

15.7 Tutorials

EB/Geometry builds embedded boundary geometries from implicit functions, constructive solid geometry, the parser and STL files, and queries the cut-cell data.

EB/Poisson is a single-level code that solves the Poisson equation outside a spherical obstacle with the cell-centered solver `MLEBABecLap` and Dirichlet conditions on the embedded boundary.

EB/Advection is a single-level code that advects a scalar around an embedded boundary with a conservative finite-volume method and redistribution for the small cut cells.

`amrex/Tests/EB/CNS` is an AMR code for the compressible Navier-Stokes equations with embedded boundaries. It uses `EBFluxRegister` for refluxing.

CHAPTER

SIXTEEN

DISCRETE FOURIER TRANSFORM

AMReX provides support for parallel discrete Fourier transforms. The implementation utilizes cuFFT, rocFFT, oneMKL and FFTW, for CUDA, HIP, SYCL and CPU builds, respectively. It also provides FFT-based Poisson solvers.

16.1 FFT::R2C Class

Class template *FFT::R2C* supports discrete Fourier transforms between real and complex data across MPI processes. The name R2C indicates that the forward transform converts real data to complex data, while the backward transform converts complex data to real data. It should be noted that both directions of transformation are supported, not just from real to complex.

The implementation utilizes cuFFT, rocFFT, oneMKL and FFTW, for CUDA, HIP, SYCL and CPU builds, respectively. Because the parallel communication is handled by AMReX, it does not need the parallel version of FFTW. Furthermore, there is no constraint on the domain decomposition such as one Box per process. This class performs parallel FFT on AMReX's parallel data containers (e.g., `MultiFab` and `FabArray<BaseFab<ComplexData<Real>>>`.

Other than using column-major order, AMReX follows the convention of FFTW. Applying the forward transform followed by the backward transform scales the original data by the size of the input array. The layout of the complex data also follows the FFTW convention, where the complex Hermitian output array has *(nx/2+1,ny,nz)* elements. Here *nx*, *ny* and *nz* are the sizes of the real array and the division is rounded down.

Below are examples of using `FFT::R2C`.

```
Geometry geom(...);
MultiFab mfin(...);
MultiFab mfout(...);

auto scaling = 1. / geom.Domain().d_numPts();

FFT::R2C r2c(geom.Domain());
r2c.forwardThenBackward(mfin, mfout,
    [=] AMREX_GPU_DEVICE (int, int, int, auto& sp)
    {
        sp *= scaling;
    });

// Use R2C provided spectral data layout.
auto const& [cba, cdm] = r2c.getSpectralDataLayout();
cMultiFab cmf(cba, cdm, 1, 0);
FFT::R2C<Real,FFT::Direction::forward> r2c_forward(geom.Domain());
r2c_forward(mfin, cmf);
```

(continues on next page)

(continued from previous page)

```
FFT::R2C<Real,FFT::Direction::backward> r2c_backward(geom.Domain());
r2c_backward(cmf, mfout);
```

Note that using `forwardThenBackward` is expected to be more efficient than separate calls to `forward` and `backward` because some parallel communication can be avoided. For the spectral data, the example above builds `cMultiFab` using `FFT::R2C` provided layout. You can also use your own `BoxArray` and `DistributionMapping`, but it might result in extra communication. It should also be noted that a lot of preparatory work is done in the construction of an `FFT::R2C` object. Therefore, one should cache it for reuse if possible. Although `FFT::R2C` does not have a default constructor, one could always use `std::unique_ptr<FFT::R2C<Real>>` to store an object in one's class.

16.1.1 Choosing FFT lengths

`FFT::nextFastLen(target, nfactors)` returns the smallest FFT length greater than or equal to `target` whose prime factors are limited to the first `nfactors` values from {2, 3, 5, 7, 11, 13}. If `nfactors` is omitted, `FFT::FastNumPrimeFactors()` provides the platform-dependent default. It currently returns 5 for CUDA and 6 for other backends. This default is a performance tuning policy and may change in the future. This helper can be used to choose a padded FFT domain size that is expected to perform well with common FFT backends.

Class template *FFT::R2C* also supports batched FFTs. The batch size is set in an `FFT::Info` object passed to the constructor of `FFT::R2C`. Below is an example.

```
int batch_size = 10;
Geometry geom(...);
MultiFab mf(ba, dm, batch_size, 0);

FFT::Info info{};
info.setBatchSize(batch_size));
FFT::R2C<Real,FFT::Direction::both> r2c(geom.Domain(), info);

auto const& [cba, cdm] = r2c.getSpectralDataLayout();
cMultiFab cmf(cba, cdm, batch_size, 0);

r2c.forward(mf, cmf);

// Do work on cmf.

r2c.backward(cmf, mf);
```

Function `forwardThenBackward` is also supported for a batched FFT. A callable taking a `GpuComplex<Real>&` is called once for each component of the batch at each spectral point. When the components are coupled, such as when projecting a vector field in spectral space, the callable can take a `CellData` argument instead, so that it receives all components of the batch at a given spectral point at once.

```
auto scaling = 1. / geom.Domain().d_numPts();

r2c.forwardThenBackward(mf, mf2,
    [=] AMREX_GPU_DEVICE (int, int, int, CellData<GpuComplex<Real>> sp)
    {
        for (int n = 0; n < sp.nComp(); ++n) {
            sp[n] *= scaling;
        }
    });
```

The distributed FFT moves data between processes when it switches directions. `FFT::Info::setCommStrategy` selects how:

- `FFT::CommStrategy::automatic` (default): AMReX chooses.
- `FFT::CommStrategy::pointtopoint`: individual messages between pairs of processes.
- `FFT::CommStrategy::alltoallv`: one `MPI_Alltoallv` per exchange.
- `FFT::CommStrategy::alltoall`: one `MPI_Alltoall` per exchange.

AMReX may use a different method when the requested one does not apply, for example on a single process.

16.2 FFT::C2C Class

`FFT::C2C` is a class template that supports complex to complex Fourier transforms. It has a similar interface as `FFT::R2C`.

16.3 FFT::LocalR2C Class

Class template *FFT::LocalR2C* supports local discrete Fourier transforms between real and complex data. The name R2C indicates that the forward transform converts real data to complex data, while the backward transform converts complex data to real data. It should be noted that both directions of transformation are supported, not just from real to complex.

Below is an example of using `FFT::LocalR2C`.

```
MultiFab mf(...);
BaseFab<GpuComplex<T>> cfab;
for (MFIter mfi(mf); mfi.isValid(); ++mfi) {
    FFT::LocalR2C fft(mfi.fabbox().length());
    cfab.resize(IntVect(0), fft.spectralSize()-1);
    fft.forward(mf[mfi].dataPtr(), cfab.dataPtr());
}
```

16.4 Poisson Solver

AMReX provides FFT-based Poisson solvers. Here, Poisson's equation is

$$\nabla^2 \phi = \rho.$$

`FFT::Poisson` supports periodic (`FFT::Boundary::periodic`), homogeneous Neumann (`FFT::Boundary::even`), and homogeneous Dirichlet (`FFT::Boundary::odd`) boundaries using FFT. Below is an example of using the solver.

```
Geometry geom(...);
MultiFab soln(...);
MultiFab rhs(...);

Array<std::pair<FFT::Boundary,FFT::Boundary>,AMREX_SPACEDIM>
        fft_bc{...};

bool has_dirichlet = false;
for (int idim = 0; idim < AMREX_SPACEDIM; ++idim) {
```

(continues on next page)

(continued from previous page)

```
    has_dirichlet = has_dirichlet ||
        fft_bc[idim].first == FFT::Boundary::odd ||
        fft_bc[idim].second == FFT::Boundary::odd;
}
if (! has_dirichlet) {
    // Shift rhs so that its sum is zero.
    auto rhosum = rhs.sum(0);
    rhs.plus(-rhosum/geom.Domain().d_numPts(), 0, 1);
}

FFT::Poisson fft_poisson(geom, fft_bc);
fft_poisson.solve(soln, rhs);
```

`FFT::PoissonOpenBC` is a 3D only solver that supports open boundaries. Its implementation utilizes `FFT::OpenBCSolver`, which can be used for implementing convolution based solvers with a user provided Green's function. If users want to extend the open BC solver to 2D or other types of Green's function, they could use `FFT::PoissonOpenBC` as an example. Below is an example of solving Poisson's equation with open boundaries.

`FFT::OpenBCSolver` currently supports one right-hand-side component per solve. It does not support `FFT::Info::setBatchSize` values greater than one. The solver uses an internal doubled convolution domain in each transformed direction. By default, the one-sided length is rounded up with `FFT::nextFastLen(n)` before doubling, using the platform-dependent `FFT::FastNumPrimeFactors()` default described above. This extra padding changes only the internal FFT work arrays, not the user-provided `MultiFab` domains. Users can disable it with `FFT::Info::setOpenBCPadding(false)` or tune the factor count with `FFT::Info::setOpenBCPaddingNumPrimeFactors(nfactors)`.

```
Geometry geom(...);
MultiFab soln(...); // soln can be either nodal or cell-centered.
MultiFab rhs(...);  // rhs must have the same index type as soln.

int ng = ...; // ng can be non-zero, if we want to compute potential
              // outside the domain.
FFT::PoissonOpenBC openbc_solver(geom, soln.ixType(), IntVect(ng));
openbc_solver.solve(soln, rhs);
```

`FFT::PoissonHybrid` is a 3D only solver that supports Dirichlet and Neumann boundary in the last dimension. The last dimension is solved with a tridiagonal solver that can support non-uniform cell size in the z-direction. For most applications, `FFT::Poisson` should be used.

The Poisson solvers take an optional `FFT::Info` argument, of which only the communication strategy set by `FFT::Info::setCommStrategy` is used.

Similar to `FFT::R2C`, the Poisson solvers should be cached for reuse, and one might need to use `std::unique_ptr<FFT::Poisson<MultiFab>>` because there is no default constructor.

16.5 Stokes Solver

AMReX provides an FFT-based Stokes solver for periodic domains in 2D and 3D. The generalized Stokes problem is

$$(\alpha - \eta\nabla^2)\mathbf{u} + \nabla p = \mathbf{f},$$

with the divergence-free constraint enforced in Fourier space. Here, $\mathbf{u}$ is the velocity, p is the pressure, $\mathbf{f}$ is the right-hand side, and α and η are scalar coefficients.

The solver uses a MAC staggered grid. The velocity components and their corresponding right-hand side components must be at x-face, y-face, and z-face centers, while the pressure is cell-centered. It currently only supports periodic boundary conditions.

Below is an example of using the solver.

```
Geometry geom(...);
MultiFab pres(...);        // cell-centered
MultiFab u(...);           // at x-face centers
MultiFab v(...);           // at y-face centers
MultiFab w(...);           // at z-face centers
MultiFab rhsx(...);        // at x-face centers
MultiFab rhsy(...);        // at y-face centers
MultiFab rhsz(...);        // at z-face centers

FFT::Stokes stokes(geom);
stokes.solve(u, v, w, pres,                 // outputs
             rhsx, rhsy, rhsz, alpha, eta); // inputs
```

Similar to the other FFT classes, the solver should be cached for reuse.

16.6 Cross-Stencil Projection

`FFT::CrossProj` projects a cell-centered velocity field onto fields that are divergence free with respect to a node-centered divergence. In 2D, the divergence at the node shared by cells (i, j), $(i + 1, j)$, $(i, j + 1)$ and $(i + 1, j + 1)$ is

$$\frac{u_{i+1,j} + u_{i+1,j+1} - u_{i,j} - u_{i,j+1}}{2\Delta x} + \frac{v_{i,j+1} + v_{i+1,j+1} - v_{i,j} - v_{i+1,j}}{2\Delta y}.$$

In 3D, it is computed similarly from the 8 cells surrounding the node. The projected velocity has zero divergence at every node, and projecting it again leaves it unchanged. The class is available in 2D and 3D, and it currently only supports periodic boundary conditions. The domain must have at least 2 cells in each direction. The first `AMREX_SPACEDIM` components of the `MultiFab` are the velocity. If the output has ghost cells, one layer of them is filled. Below is an example of using it.

```
Geometry geom(...);   // periodic in all directions
MultiFab vel(...);    // cell-centered, AMREX_SPACEDIM components
MultiFab vel_proj(...);

FFT::CrossProj proj(geom);
proj.solve(vel_proj, vel); // vel_proj and vel can be the same MultiFab
```

Similar to the other FFT classes, the object should be cached for reuse. In 2D, a multigrid solver with the same interface, `GMG::CrossProj`, is described in section *Cross-Stencil Projection*.

16.7 Raw Pointer Interface

If you only want to use AMReX as a parallel FFT library without using other functionality or data containers, you could use the raw pointer interface. Below is an example.

```
MPI_Init(&argc, &argv);

// We don't need to call the full amrex::Initialize
```

(continues on next page)

(continued from previous page)

```
amrex::Init_FFT(MPI_COMM_WORLD);

int nprocs, myproc;
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
MPI_Comm_rank(MPI_COMM_WORLD, &myproc);

using RT = double;
using CT = std::complex<RT>; // or cufftDoubleComplex, etc.

std::array<int,3> domain_size{128,128,128};

// FFT between real and complex.
// Domain decomposition is flexible. The only constraint for the raw
// pointer interface is that there can be only zero or one local box
// per process, whereas the MultiFab interface can take any number of
// boxes. In this case, we choose to do manual domain decomposition for
// the real (i.e., forward) domain, and use the domain decomposition
// provided by amrex for the complex (i.e., backward) domain.
{
    amrex::FFT::R2C<RT,amrex::FFT::Direction::both> r2c(domain_size);

    // amrex supports 1d, 2d and 3d domain decomposition. For simplicity,
    // let's do 1d decomposition in the z-direction.
    int nz = (domain_size[2] + nprocs - 1) / nprocs;
    int zlo = nz * myproc;
    nz = std::max(std::min(nz,domain_size[2]-zlo), 0);
    std::array<int,3> local_start{0,0,zlo};
    std::array<int,3> local_size{domain_size[0],domain_size[1],nz};

    // Let amrex know the domain decomposition in the forward domain.
    r2c.setLocalDomain(local_start,local_size);

    // Use amrex's domain decomposition in the backward domain.
    auto const& [local_start_sp, local_size_sp] = r2c.getLocalSpectralDomain();

    auto nr = std::size_t(local_size[0])
            * std::size_t(local_size[1])
            * std::size_t(local_size[2]);
    auto* pr = (RT*)std::malloc(sizeof(RT)*nr); // or use cudaMalloc
    // Initialize data ...

    auto nc = std::size_t(local_size_sp[0])
            * std::size_t(local_size_sp[1])
            * std::size_t(local_size_sp[2]);
    auto* pc = (CT*)std::malloc(sizeof(CT)*nc); // or use cudaMalloc

    r2c.forward(pr, pc); // forward transform from real to complex

    // work on the complex data pointed by pc ...

    r2c.backward(pc, pr); // backward transform from complex to real
```

(continues on next page)

(continued from previous page)

```
    std::free(pr);
    std::free(pc);
}

// Batched FFT between complex and complex.
// In this case, we choose to use the domain decomposition provided
// by amrex for the forward domain, and do manual domain decomposition
// for the backward domain.
int nbatch = 3; // batch size
{
    amrex::FFT::Info info{};
    info.setBatchSize(nbatch);
    amrex::FFT::C2C<RT,amrex::FFT::Direction::both> c2c(domain_size,info);

    // Use amrex's domain decomposition in the forward domain.
    auto const& [local_start, local_size] = c2c.getLocalDomain();

    int nx = (domain_size[0] + nprocs - 1) / nprocs;
    int xlo = nx * myproc;
    nx = std::max(std::min(nx,domain_size[0]-xlo), 0);
    std::array<int,3> local_start_sp{xlo,0,0};
    std::array<int,3> local_size_sp{nx,domain_size[1],domain_size[2]};

    // Let amrex know the domain decomposition in the backward domain.
    c2c.setLocalSpectralDomain(local_start_sp, local_size_sp);

    auto nf = std::size_t(local_size[0])
            * std::size_t(local_size[1])
            * std::size_t(local_size[2]);
    auto* pf = (CT*)std::malloc(sizeof(CT)*nf*nbatch); // or use cudaMalloc
    // Initialize data ...

    auto nb = std::size_t(local_size_sp[0])
            * std::size_t(local_size_sp[1])
            * std::size_t(local_size_sp[2]);
    auto* pb = (CT*)std::malloc(sizeof(CT)*nb*nbatch);

    c2c.forward(pf, pb); // forward transform

    // work on the data pointed by pb

    c2c.backward(pb, pf); // backward transform

    std::free(pf);
    std::free(pb);
}

amrex::Finalize_FFT();

MPI_Finalize();
```

CHAPTER

SEVENTEEN

TIME INTEGRATION

AMReX provides a basic explicit time integrator capable of Forward Euler or both predefined and custom Runge-Kutta schemes designed to advance data on a particular AMR level by a time step. This integrator is designed to be flexible, requiring the user to supply a right-hand side function taking a `MultiFab` of state data and filling a `MultiFab` of the corresponding right-hand side data. The user simply needs to supply a C++ lambda function to implement whatever right-hand-side operations they need.

17.1 A Simple Time Integrator Setup

This is best shown with some sample code that sets up a time integrator and asks it to step forward by some interval `dt`. The user needs to supply the right-hand side function using the `TimeIntegrator::set_rhs()` function.

```
#include <AMReX_TimeIntegrator.H>

MultiFab Sold; // MultiFab containing old-time state data
MultiFab Snew; // MultiFab where we want new-time state data

// [Fill Sold here]

// Create a time integrator that will work with
// MultiFabs with the same BoxArray, DistributionMapping,
// and number of components as the state_data MultiFab.
TimeIntegrator<MultiFab> integrator(Sold);

// Create a function that fills the state BCs and computes the RHS
auto rhs_fun = [&](MultiFab& rhs, MultiFab& state, const Real time){
    // [Calculate the rhs MultiFab given the state MultiFab]
};

// Attach the right-hand-side function to the integrator
integrator.set_rhs(rhs_fun);

// integrate forward one step from `time` by `dt` to fill Snew
integrator.advance(Sold, Snew, time, dt);
```

17.2 Picking a Time Integration Method

The user can customize which integration method they wish to use with a set of runtime parameters that allow choosing between a simple Forward Euler method or a generic explicit Runge-Kutta method. If Runge-Kutta is selected, then the user can choose which of a set of predefined Butcher tables to use, or can choose to use a custom table and supply it manually.

An example of integrator options is given below. For a complete list of options see the section on *Time Integration Runtime Parameters*.

```
# INTEGRATION

## *** Selecting the integrator backend ***
## integration.type can take on the following string or int values:
## (without the quotation marks)
## "ForwardEuler" or "0" = Native Forward Euler Integrator
## "RungeKutta" or "1"   = Native Explicit Runge Kutta
## "SUNDIALS" or "2"     = SUNDIALS Integrator
## for example:
integration.type = RungeKutta

## *** Parameters Needed For Native Explicit Runge-Kutta ***
#
## integration.rk.type can take the following values:
### 0 = User-specified Butcher Tableau
### 1 = Forward Euler
### 2 = Trapezoid Method
### 3 = SSPRK3 Method
### 4 = RK4 Method
integration.rk.type = 3

## If using a user-specified Butcher Tableau, then
## set nodes, weights, and table entries here:
#
## The Butcher Tableau is read as a flattened,
## lower triangular matrix (but including the diagonal)
## in row major format.
integration.rk.weights = 1
integration.rk.nodes = 0
integration.rk.tableau = 0.0
```

17.3 Using SUNDIALS

The AMReX Time Integration interface also supports a SUNDIALS backend that provides explicit, implicit, and implicit-explicit (ImEx) Runge-Kutta methods as well as multirate (MRI) methods from the ARKODE package in SUNDIALS. Presently, SUNDIALS v6.0 or later is required, but v7.4.0 has been successfully tested. To install SUNDIALS, the full documentation is available at https://sundials.readthedocs.io/en/latest/sundials/Install_link.html.

Here is a summary of steps that you need to take using CMake. First obtain the source code from either https://computing.llnl.gov/projects/sundials/sundials-software or the GitHub page at https://github.com/LLNL/sundials. Once you have unpacked or cloned the source code, run the following (altering the ENABLE_MPI and ENABLE_CUDA lines as appropriate):

```
cmake -S /path_to_sundials_source_code -B /path_to_sundials_build_dir -D CMAKE_INSTALL_
↪PREFIX=/path_to_sundials_install_dir -D ENABLE_MPI=ON -D ENABLE_CUDA=ON -DCMAKE_CUDA_
↪ARCHITECTURES=XX
cd /path_to_sundials_build_dir
make
make install
```

To determine the `XX` in the `-DCMAKE_CUDA_ARCHITECTURES=XX` argument, run the following command:

```
nvidia-smi --query-gpu=compute_cap --format=csv,noheader
```

If the result is `7.0`, use `70`, etc.

To use SUNDIALS integrators, the user needs to compile their AMReX application with `USE_SUNDIALS=TRUE` and `SUNDIALS_HOME=/path_to_sundials_install_dir`.

The SUNDIALS interface supports `MultiFab` or `Vector<MultiFab>` data types. Using a `Vector<MultiFab>` permits integrating state data with different spatial centering (e.g., cell-centered, face-centered, nodal) concurrently.

The same code as above can be used with SUNDIALS explicit or implicit Runge-Kutta methods without any modification. To select a SUNDIALS explicit Runge-Kutta integrator, one needs only to add the following two input parameters at runtime:

```
integration.type = SUNDIALS
integration.sundials.type = ERK
```

One can select a different method type by changing the value of *integration.sundials.type* to one of the following values:

Input Option	SUNDIALS Method Type
ERK	Explicit Runge-Kutta method
DIRK	Diagonally Implicit Runge-Kutta method
IMEX-RK	Implicit-Explicit Additive Runge-Kutta method
EX-MRI	Explicit Multirate Infinitesimal method
IM-MRI	Implicit Multirate Infinitesimal method
IMEX-MRI	Implicit-Explicit Multirate Infinitesimal method

For ImEx methods, the user needs to supply two right-hand side functions, an implicit and an explicit function, using the function `TimeIntegrator::set_imex_rhs()`. Similarly for multirate methods, the user needs to supply slow and fast right-hand side functions using `TimeIntegrator::set_rhs()` to set the slow function and `TimeIntegrator::set_fast_rhs()` to set the fast function. With multirate methods, one also needs to select the fast time scale method type using the input option *integration.sundials.fast_type*, which may be set to `ERK` or `DIRK`.

To select a specific SUNDIALS method, use the input option *integration.sundials.method* for ERK and DIRK methods as well as the slow time scale method with an MRI integrator. Use *integration.sundials.method_i* and *integration.sundials.method_e* to set the implicit and explicit method in an ImEx method, and *integration.sundials.fast_method* to set the ERK or DIRK method used at the fast time scale with an MRI integrator. These options may be set to any valid SUNDIALS method name. See the following sections in the SUNDIALS documentation for more details:

- ERK methods
- DIRK methods
- ImEx methods
- MRI methods

An example of integrator options is given below. For a complete list of options see the section on *Time Integration Runtime Parameters*.

```
# INTEGRATION WITH SUNDIALS

# *** Select the SUNDIALS integrator backend ***
```

(continues on next page)

(continued from previous page)

```
integration.type = SUNDIALS

# *** Select the SUNDIALS method type ***
# ERK      = Explicit Runge-Kutta method
# DIRK     = Diagonally Implicit Runge-Kutta method
# IMEX-RK  = Implicit-Explicit Additive Runge-Kutta method
# EX-MRI   = Explicit Multirate Infinitesimal method
# IM-MRI   = Implicit Multirate Infinitesimal method
# IMEX-MRI = Implicit-Explicit Multirate Infinitesimal method
integration.sundials.type = ERK

# *** Select a specific SUNDIALS ERK method ***
integration.sundials.method = ARKODE_BOGACKI_SHAMPINE_4_2_3

# *** Select a specific SUNDIALS ImEx method ***
integration.sundials.method_i = ARKODE_ARK324L2SA_DIRK_4_2_3
integration.sundials.method_e = ARKODE_ARK324L2SA_ERK_4_2_3

# *** Select a specific SUNDIALS MRI method ***
integration.sundials.method = ARKODE_MIS_KW3
integration.sundials.fast_method = ARKODE_KNOTH_WOLKE_3_3
```

The features of this interface evolve with the needs of our codes, so they may not yet support all SUNDIALS configurations available. If you find you need SUNDIALS options we have not implemented, please let us know.

CHAPTER

EIGHTEEN

GPU

In this chapter, we will present GPU support in AMReX. AMReX targets NVIDIA, AMD, and Intel GPUs using their native vendor languages and therefore requires CUDA, HIP/ROCm, and SYCL for NVIDIA, AMD, and Intel GPUs, respectively. Users can also use OpenMP and/or OpenACC in their applications if desired.

AMReX supports NVIDIA GPUs with compute capability >= 6 and CUDA >= 12.2, and AMD GPUs with ROCm >= 6. While SYCL compilers are still under development in preparation for Aurora, AMReX officially supports only the latest publicly released version of the oneAPI compiler.

For complete details of CUDA, HIP, SYCL, OpenMP, and OpenACC languages, see their respective documentation.

Note that this documentation is currently focused on CUDA and HIP. SYCL documentation is forthcoming.

Most of the AMReX Tutorials run on GPUs; the GpuContainers tutorial covers GPU memory, containers and asynchronous copies.

18.1 Overview of AMReX GPU Support

AMReX's GPU support focuses on providing performance portability across a range of important architectures with minimal code changes required at the application level. This allows application teams to use a single, maintainable codebase that works on a variety of platforms while allowing for the performance tuning of specific, high-impact kernels if desired.

Internally, AMReX uses the native programming languages for GPUs: CUDA for NVIDIA, HIP for AMD and SYCL for Intel. This will be designated with `CUDA/HIP/SYCL` throughout the documentation. However, application teams can also use OpenACC or OpenMP in their individual codes.

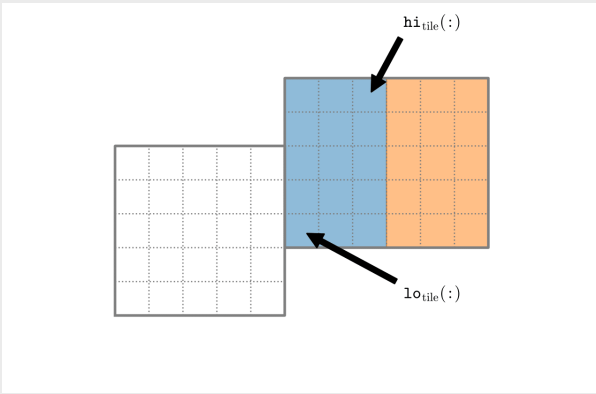
At this time, AMReX does not support cross-native language compilation (HIP for non-AMD systems and SYCL for non Intel systems). It may work with a given version, but AMReX does not track or guarantee such functionality.

AMReX uses an `MPI+X` approach to hierarchical parallelism. When running on CPUs, `X` is `OpenMP`, and threads are used to process tiles assigned to the same MPI rank concurrently, as detailed in *MFIter with Tiling*. On GPUs, `X` is one of `CUDA/HIP/SYCL`, and tiling is disabled by default to mitigate the overhead associated with kernel launching. Instead, kernels are usually launched at the `Box` level, and one or more cells in a given `Box` are mapped to each GPU thread, as detailed in Table 18.1 below.

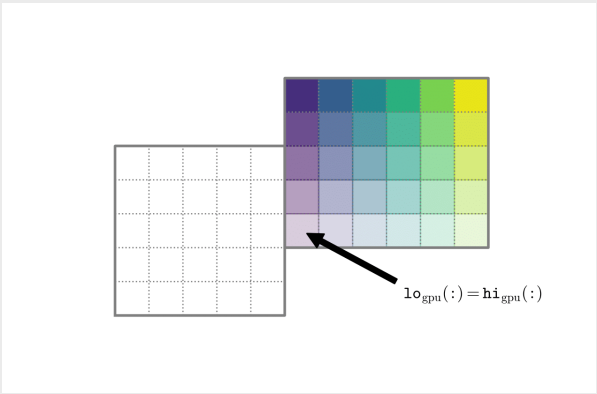
Presented here is an overview of important features of AMReX's GPU strategy. Additional information that is required for creating GPU applications is detailed throughout the rest of this chapter:

- Each MPI rank offloads its work to a single GPU. Multiple ranks can share the same device, but for best performance we usually recommend `(MPI ranks == Number of GPUs)`.
- To provide performance portability, GPU kernels are usually launched through `ParallelFor` looping constructs that use GPU extended lambdas to specify the work to be performed on each loop element. When compiled with GPU support, these constructs launch kernels with a large number of GPU threads that only work on a few cells each. This work distribution is illustrated in Table 18.1.

Table 18.1: Comparison of OpenMP and GPU work distribution. Pictures provided by Mike Zingale and the CASTRO team.



OpenMP tiled box. OpenMP threads break down the valid box into two large boxes (blue and orange). The lo and hi of one tiled box are marked.



GPU threaded box. Each GPU thread works on a few cells of the valid box. This example uses one cell per thread, each thread using a box with lo = hi.

- These kernels are usually launched inside AMReX's `MFIter` and `ParIter` loops, since in AMReX's approach to parallelism it is assumed that separate `Box` objects can be processed independently. However, AMReX also provides a `MultiFab` version of `ParallelFor` that can process an entire level worth of `Box` objects in a single kernel launch when it is safe to do so. `ParallelForStrided` and `ParallelForRedBlack` do the same for a strided subset of points, e.g., one color of a multi-color or red-black Gauss-Seidel sweep.
- AMReX can utilize GPU managed memory to automatically handle memory movement for mesh and particle data. Simple data structures, such as `IntVect`s can be passed by value and complex data structures, such as `FArrayBox`es, have specialized AMReX classes to handle the data movement for the user. This is particularly useful for the early stages of porting an application to GPUs. However, for best performance on a variety of platforms, we recommend disabling managed memory and handling host/device data migration explicitly. managed memory is not used by `FArrayBox` and `MultiFab` by default.
- Best performance is usually achieved when keeping mesh and particle data structures on the GPU for as long as possible, minimizing movement back to the CPU. In many AMReX applications, the mesh and particle data can stay on the GPU for most subroutines except for I/O operations.
- AMReX further parallelizes GPU applications by utilizing streams. Streams guarantee execution order of kernels within the same stream, while allowing different streams to run simultaneously. AMReX places each iteration of `MFIter` loops on separate streams, allowing each independent iteration to be run simultaneously and sequentially, while maximizing GPU usage.

  The AMReX implementation of streams is illustrated in Fig. 18.1. The CPU runs the first iteration of the MFIter loop (blue), which contains three GPU kernels. The kernels begin immediately in GPU Stream 1 and run in the same order they were added. The second (red) and third (green) iterations are similarly launched in Streams 2 and 3. The fourth (orange) and fifth (purple) iterations require more GPU resources than remain, so they have to wait until resources are freed before beginning. Meanwhile, after all the loop iterations are launched, the CPU reaches a synchronize in the MFIter's destructor and waits for all GPU launches to complete before continuing.
- The Fortran interface of AMReX does not currently have GPU support. AMReX recommends porting Fortran code to C++ when coding for GPUs.

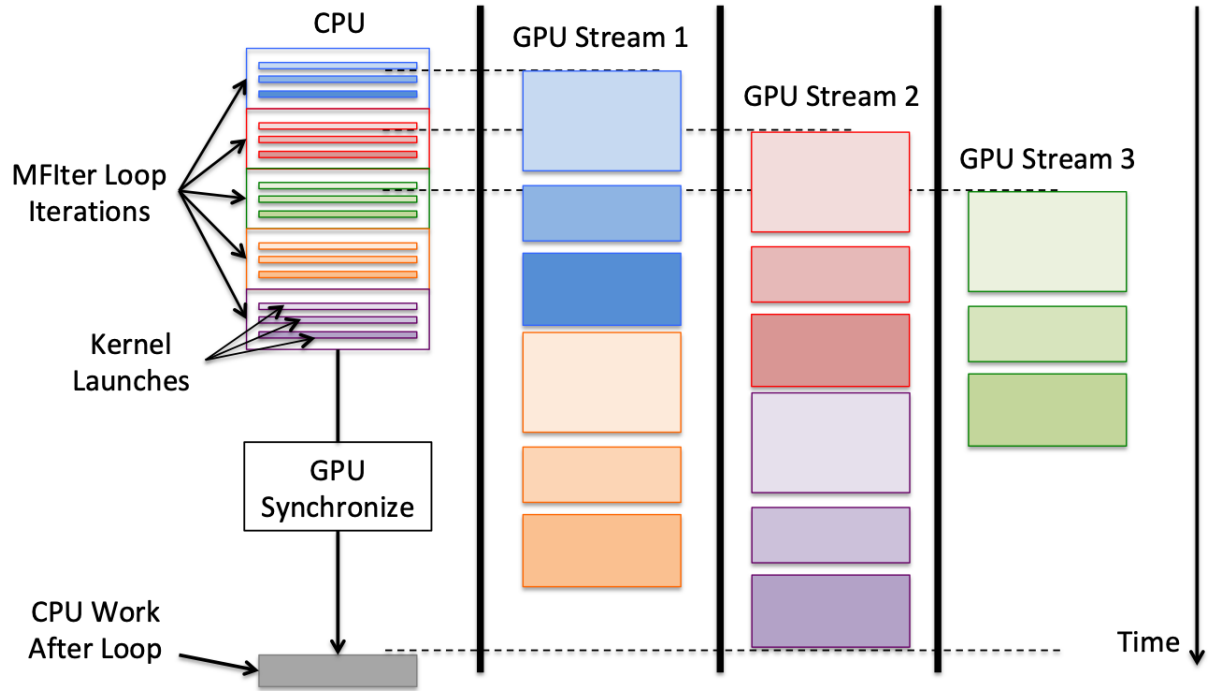


Fig. 18.1: Timeline illustration of GPU streams. Illustrates the case of an MFIter loop of five iterations with three GPU kernels each being run with three GPU streams.

18.2 Building GPU Support

18.2.1 Building with GNU Make

To build AMReX with GPU support, add `USE_CUDA=TRUE`, `USE_HIP=TRUE` or `USE_SYCL=TRUE` to the `GNUmakefile` or as a command line argument.

AMReX does not require OpenACC, but application codes can use them if they are supported by the compiler. For OpenACC support, add `USE_ACC=TRUE`. PGI, Cray and GNU compilers support OpenACC. Thus, for OpenACC, you must use `COMP=pgi`, `COMP=cray` or `COMP=gnu`.

Currently, only IBM is supported with OpenMP offloading. To use OpenMP offloading, make with `USE_OMP_OFFLOAD=TRUE`.

Compiling AMReX with CUDA requires compiling the code through NVIDIA's CUDA compiler driver in addition to the standard compiler. This driver is called `nvcc` and it requires a host compiler to work through. The default host compiler for NVCC is GCC even if `COMP` is set to a different compiler. One can change this by setting `NVCC_HOST_COMP`. For example, `COMP=pgi` alone will compile C/C++ codes with NVCC/GCC and Fortran codes with PGI, and link with PGI. Using `COMP=pgi` and `NVCC_HOST_COMP=pgi` will compile C/C++ codes with PGI and NVCC/PGI.

With CUDA, `GPU_MIN_BLOCKS=B` (`-DAMReX_GPU_MIN_BLOCKS=B` in CMake) limits the registers per thread in GPU code so that `B` blocks of `GPU_MAX_THREADS` threads fit on a streaming multiprocessor. A larger `B` gives higher occupancy but can cause register spilling. Keep `B` times `GPU_MAX_THREADS` within the GPU's limit of resident threads per multiprocessor.

You can use `amrex-tutorials/Base/HelloWorld/` to test your programming environment. For example, building with:

```
make COMP=gnu USE_CUDA=TRUE USE_MPI=FALSE
```

should produce an executable named `main3d.gnu.CUDA.ex`. You can run it and that will generate results like:

```
$ ./main3d.gnu.CUDA.ex
Initializing AMReX (26.11-6-gb33ab585fb22)...
Initializing CUDA...
CUDA initialized with 1 device.
AMReX (26.11-6-gb33ab585fb22) initialized
Hello world from AMReX version 26.11-6-gb33ab585fb22
  MPI ranks:      1
  OpenMP threads: 1
  GPU backend:    CUDA
  Rank 0 says hello
AMReX (26.11-6-gb33ab585fb22) finalized
```

SYCL configuration variables

When building with `USE_SYCL=TRUE`, one can set the following makefile variables to configure the build

Table 18.2: AMReX SYCL-specific GNU Make build options

Variable Name	Description	Default	Possible values
SYCL_AOT	Enable SYCL ahead-of-time compilation	FALSE	TRUE, FALSE
SYCL_AOT_GRF_MODE	Specify AOT register file mode	Default	Default, Large, Auto-Large
AMREX_INTEL_ARCH	Specify target if AOT is enabled	None	pvc, etc.
SYCL_SPLIT_KERNEL	Enable SYCL kernel splitting	FALSE	TRUE, FALSE
USE_ONEDPL	Enable SYCL's oneDPL algorithms	NO	YES, NO
SYCL_SUB_GROUP_SIZE	Specify subgroup size	32	64, 32, 16
SYCL_PARALLEL_LINK_JOBS	Number of parallel jobs in device link	1	1, 2, 3, etc.

18.2.2 Building with CMake

To build AMReX with GPU support in CMake, add `-DAMReX_GPU_BACKEND=CUDA|HIP|SYCL` to the `cmake` invocation, for CUDA, HIP and SYCL, respectively. By default, AMReX uses 256 threads per GPU block/group in most situations. This can be changed with `-DAMReX_GPU_MAX_THREADS=N`, where `N` is 128 for example.

Enabling CUDA support

To build AMReX with CUDA support in CMake, add `-DAMReX_GPU_BACKEND=CUDA` to the `cmake` invocation. For a full list of CUDA-specific configuration options, check the *table* below.

Table 18.3: AMReX CUDA-specific build options

Variable Name	Description	Default	Possible values
CMAKE_CUDA_ARCHITECTURES	Target CUDA architecture(s) (standard CMake)	native	See below
AMReX_CUDA_FASTMATH	Enable CUDA fastmath library	YES	YES, NO
AMReX_CUDA_BACKTRACE	Host function symbol names (e.g. cuda-memcheck)	Auto	YES, NO
AMReX_CUDA_COMPILATION_TIME	CSV table with time for each compilation phase	NO	YES, NO
AMReX_CUDA_DEBUG	Device debug information (optimizations: off)	YES: Debug	YES, NO
AMReX_CUDA_ERROR_CAPTURE_TI	Error if a CUDA lambda captures a class' this	NO	YES, NO
AMReX_CUDA_ERROR_CROSS _EXECUTION_SPACE_CALL	Error if a host function is called from a host device function	NO	YES, NO
AMReX_CUDA_KEEP_FILES	Keep intermediate files (folder: nvcc_tmp)	NO	YES, NO
AMReX_CUDA_OBJDIR_AS_TEMPDII	Place intermediate files in object file folder	NO	YES, NO
AMReX_CUDA_LTO	Enable CUDA device link-time optimization (requires AMReX_GPU_RDC)	NO	YES, NO
AMReX_CUDA_MAXREGCOUNT	Limits the number of CUDA registers available	255	User-defined
AMReX_GPU_MIN_BLOCKS	Min resident blocks per SM (see above)	None	User-defined
AMReX_CUDA_PTX_VERBOSE	Verbose code generation statistics in ptxas	NO	YES, NO
AMReX_CUDA_SHOW_CODELINES	Source information in PTX (optimizations: on)	NO	YES, NO
AMReX_CUDA_SHOW_LINENUMBER	Line-number information (optimizations: on)	YES	YES, NO
AMReX_CUDA_WARN_CAPTURE_TH	Warn if a CUDA lambda captures a class' this	YES	YES, NO

The target architecture(s) to build for are selected via the standard CMake option `-DCMAKE_CUDA_ARCHITECTURES=<arch>` (or the `CUDAARCHS` environment variable), where `<arch>` is one or more CUDA compute capabilities given as integers without the dot, e.g., `80` for compute capability 8.0. Multiple architectures can be set as a semicolon-separated list, e.g., `-DCMAKE_CUDA_ARCHITECTURES="80;90"`. Building for multiple architectures generally results in a larger library and longer build times. The special values `native` (the default), `all` and `all-major`, as well as the `-real`/`-virtual` and `<NN>a` suffixes documented for CMake's CUDA_ARCHITECTURES target property, are supported, as is `OFF` for packagers who pass the architecture flags themselves. Architectures below compute capability 6.0 are not supported by AMReX; they are dropped with a message - including when they are part of what `all` or `all-major` stands for - and configuration stops with an error if nothing is left.

```
cmake [options] -DAMReX_GPU_BACKEND=CUDA -DCMAKE_CUDA_ARCHITECTURES=80 /path/to/amrex/
↪source
```

If no architecture is specified, AMReX defaults to `native`, which builds for the GPU(s) installed in the machine running CMake. If a GPU is visible at configuration time, CMake resolves `native` right away and the resulting architecture is printed, used for the build and exported to downstream projects. If no GPU is visible, configuration stops with an error: `native` is resolved when CMake runs, not when the code is compiled. On machines without a visible GPU (e.g., HPC login nodes, CI runners or container builds) an explicit architecture must therefore be provided, such as `-DCMAKE_CUDA_ARCHITECTURES=80`. This includes configuring on a login node and building on a GPU node.

Note

The legacy `-DAMReX_CUDA_ARCH` option and the `AMREX_CUDA_ARCH` environment variable are deprecated but still honored (with a warning): they map to `CMAKE_CUDA_ARCHITECTURES` and the standard `CUDAARCHS` environment variable, respectively, and keep their historical precedence over them. The exception is `-DCMAKE_CUDA_ARCHITECTURES=...` passed on the `cmake` command line, which wins over the deprecated hints. Please migrate to the standard CMake variables. Note that `AMREX_CUDA_ARCH` is unaffected in GNU Make builds, where it remains the way to select architectures.

The architectures are selected again on every configure step, so a hint that is still in place - an exported `AMREX_CUDA_ARCH`, for instance - can change what an earlier step chose. AMReX warns when that happens; pass `-DCMAKE_CUDA_ARCHITECTURES=...` to pin the choice for a build directory.

Their historical values are translated for convenience: `Auto` becomes `native`, `All` and `Common` become `all` and `all-major`, generation names such as `Volta`, `Turing`, `Ampere` or `Hopper` become the corresponding compute capability (`Blackwell` becomes both `100` and `120`, the two families it spans), dots are dropped (`8.0` becomes `80`) and a trailing `+PTX` is removed (the plain integer form of `CMAKE_CUDA_ARCHITECTURES` already embeds PTX). A whitespace-separated list is accepted as well, as in GNU Make builds (`"70 80"`), and a hint that is set but empty or blank counts as not set at all. None of these legacy spellings are accepted by `CMAKE_CUDA_ARCHITECTURES` itself, so use the values documented above when you migrate.

Setting `-DAMReX_CUDA_LTO=ON` enables CUDA device link-time optimization. This requires relocatable device code, so it is an error to combine it with `-DAMReX_GPU_RDC=OFF`. Device LTO is currently available only with the NVIDIA `nvcc` compiler; with a Clang `-x cuda` compiler it is gracefully disabled with a warning. It only applies to device code: host code generation is unchanged. It also applies to the AMReX-dependent targets of the AMReX build itself (tests, tutorials and tools), and downstream targets that are set up with `setup_target_for_cuda_compilation` take part in it as well: most AMReX device code is instantiated in the translation units of the calling code, so a target compiled straight to SASS would only get nvlink's *partial* LTO. Because it is applied per architecture, device LTO also needs the architectures to be known at configure time: `all` and `all-major` are always expanded when it is on, and `CMAKE_CUDA_ARCHITECTURES=OFF` is an error rather than a build that quietly comes out without device LTO. (An unresolved `native` is already an error regardless of this option, see above.)

Note that AMReX supports NVIDIA GPU architectures with compute capability 6.0 or higher and CUDA Toolkit version 12.2 or higher.

Environment hints. Compiler and architecture selection can also be driven through the standard CMake and compiler environment variables. These are especially handy on HPC systems and in CI, where they are typically exported in a job or module script before invoking `cmake`. Example on a Cray/HPE machine:

```
# For Cray/HPE machines: necessary to use CUDA-Aware MPI and run a job
# Note: also sometimes already set by a loaded module.
export CRAY_ACCEL_TARGET=nvidia80

# overwrite the "native" default to optimize CUDA compilation for A100
export CUDAARCHS=80

# optimize CPU (host) microarchitecture for AMD EPYC 3rd Gen (Milan/Zen3)
export CXXFLAGS="-march=znver3"
export CFLAGS="-march=znver3"

# CMake compiler selection
export CC=$(which gcc)       # or $(which cc) for Cray/HPE
export CXX=$(which g++)      # or $(which CC) for Cray/HPE
export FC=$(which gfortran)  # or $(which ftn) for Cray/HPE
```

(continues on next page)

(continued from previous page)

```
export CUDACXX=$(which nvcc)
export CUDAHOSTCXX=${CXX}
```

In order to import the CUDA-enabled AMReX library into your CMake project, you need to include the following code into the appropriate CMakeLists.txt file:

```
# Find CUDA-enabled AMReX installation
find_package(AMReX REQUIRED CUDA)
```

If instead of using an external installation of AMReX you prefer to include AMReX as a subproject in your CMake setup, simply enable the CUDA language before adding the AMReX source directory:

```
# Select the target architecture(s) before enabling the CUDA language,
# otherwise CMake picks the compiler default for your own targets
set(CMAKE_CUDA_ARCHITECTURES 80)

# Enable CUDA in your CMake project
enable_language(CUDA)

# Include AMReX source directory ONLY AFTER enabling the CUDA language
add_subdirectory(/path/to/amrex/source/dir)
```

Note

Select the architectures yourself before `enable_language(CUDA)`, as shown above. If you do not, your own CUDA targets are built for the CUDA compiler's default architecture when they are created before `add_subdirectory`, and for AMReX's selection when they are created after it.

To ensure consistency between CUDA-enabled AMReX and any CMake target that links against it, we provide the helper function `setup_target_for_cuda_compilation()`:

```
# Set all sources for my_target
target_sources(my_target source1 source2 source3 ...)

# Setup my_target to be compiled with CUDA and be linked against CUDA-enabled AMReX
# MUST be done AFTER all sources have been assigned to my_target
setup_target_for_cuda_compilation(my_target)

# Link against amrex
target_link_libraries(my_target PUBLIC AMReX::amrex)
```

Enabling HIP Support

To build AMReX with HIP support in CMake, add `-DAMReX_GPU_BACKEND=HIP -DAMReX_AMD_ARCH=<target-arch> -DCMAKE_CXX_COMPILER=<your-hip-compiler>` to the `cmake` invocation. If you don't need Fortran features (`AMReX_FORTRAN=OFF`), it is recommended to use AMD's `clang++` as the HIP compiler. (Please see these issues for reference in rocm/HIP <= 4.2.0 [1] [2].)

In AMReX CMake, the HIP compiler is treated as a special C++ compiler and therefore the standard CMake variables used to customize the compilation process for C++, for example `CMAKE_CXX_FLAGS`, can be used for HIP as well.

Since CMake does not support autodetection of HIP compilers/target architectures yet, `CMAKE_CXX_COMPILER` must be set to a valid HIP compiler, i.e. `clang++` or `hipcc`, and `AMReX_AMD_ARCH` to the target architecture you are

building for. Thus **AMReX_AMD_ARCH and CMAKE_CXX_COMPILER are required user inputs when AMReX_GPU_BACKEND=HIP**. We also read the *environment variable* AMREX_AMD_ARCH (note: all caps), and the C++ compiler can be specified as usual, e.g., with `export CXX=$(which clang++)`. Below is an example configuration for HIP on Tulip:

```
cmake -S . -B build -DAMReX_GPU_BACKEND=HIP -DCMAKE_CXX_COMPILER=$(which clang++) -
↪DAMReX_AMD_ARCH="gfx906;gfx908"  # [other options]
cmake --build build -j 6
```

Enabling SYCL Support

To build AMReX with SYCL support in CMake, add `-DAMReX_GPU_BACKEND=SYCL -DCMAKE_CXX_COMPILER=<your-sycl-compiler>` to the `cmake` invocation. For a full list of SYCL-specific configuration options, check the *table* below.

In AMReX CMake, the SYCL compiler is treated as a special C++ compiler and therefore the standard CMake variables used to customize the compilation process for C++, for example `CMAKE_CXX_FLAGS`, can be used for SYCL as well.

Since CMake does not support autodetection of SYCL compilers yet, `CMAKE_CXX_COMPILER` must be set to a valid SYCL compiler, i.e., `icpx`. Thus **CMAKE_CXX_COMPILER is a required user-input when AMReX_GPU_BACKEND=SYCL**. At this time, **the only supported SYCL compiler is icpx**. Below is an example configuration for SYCL:

```
cmake -DAMReX_GPU_BACKEND=SYCL -DCMAKE_CXX_COMPILER=$(which icpx)  [other options] /path/
↪to/amrex/source
```

Table 18.4: AMReX SYCL-specific build options

Variable Name	Description	Default	Possible values
AMReX_SYCL_AOT	Enable SYCL ahead-of-time compilation	NO	YES, NO
AMReX_SYCL_AOT_GRF_MODE	Specify AOT register file mode	Default	Default, Large, AutoLarge
AMREX_INTEL_ARCH	Specify target if AOT is enabled	None	pvc, etc.
AMReX_SYCL_SPLIT_KERNEL	Enable SYCL kernel splitting	YES	YES, NO
AMReX_SYCL_ONEDPL	Enable SYCL's oneDPL algorithms	NO	YES, NO
AMReX_SYCL_SUB_GROUP_SIZE	Specify subgroup size	32	64, 32, 16
AMReX_PARALLEL_LINK_JOBS	Specify number of parallel link jobs	1	positive integer

18.3 Gpu Namespace and Macros

Most GPU related classes and functions are in `namespace Gpu`, which is inside `namespace amrex`. For example, the GPU configuration class `Device` can be referenced to at `amrex::Gpu::Device`.

For portability, AMReX defines some macros for CUDA/HIP function qualifiers and they should be preferred to allow execution when `USE_CUDA=FALSE` and `USE_HIP=FALSE`. These include:

```
#define AMREX_GPU_HOST        __host__
#define AMREX_GPU_DEVICE      __device__
#define AMREX_GPU_GLOBAL      __global__
#define AMREX_GPU_HOST_DEVICE __host__ __device__
```

Note that when AMReX is not built with CUDA/HIP/SYCL, these macros expand to empty space.

When AMReX is compiled with USE_CUDA=TRUE, USE_HIP=TRUE, USE_SYCL=TRUE, or USE_ACC=TRUE the preprocessor macros AMREX_USE_CUDA, AMREX_USE_HIP, AMREX_USE_SYCL, or AMREX_USE_ACC respectively are defined for conditional programming, as well as AMREX_USE_GPU. This AMREX_USE_GPU definition can be used in application code if different functionality should be used when AMReX is built with GPU support. When AMReX is compiled with USE_OMP_OFFLOAD=TRUE, AMREX_USE_OMP_OFFLOAD is defined.

The macros AMREX_IF_ON_DEVICE((code_for_device)) and AMREX_IF_ON_HOST((code_for_host)) should be used when a __host__ __device__ function requires separate code for the CPU and GPU implementations.

18.4 Memory Allocation

To provide portability and improve memory allocation performance, AMReX provides a number of memory pools. When compiled without GPU support, all Arenas use standard **new** and **delete** operators. With GPU support, the Arenas each allocate with a specific type of GPU memory:

Table 18.5: Memory Arenas

Arena	Memory Type
The_Arena()	managed or device memory
The_Device_Arena()	device memory, could be an alias to The_Arena()
The_Managed_Arena()	managed memory, could be an alias to The_Arena()
The_Pinned_Arena()	pinned memory

The Arena object returned by these calls provides access to two functions:

```
void* alloc (std::size_t sz);
void free (void* p);
```

The_Arena() is used for memory allocation of data in BaseFab. By default, it allocates device memory. This can be changed with a boolean runtime parameter amrex.the_arena_is_managed=1. When managed memory is enabled, the data in a MultiFab is placed in device memory by default and is accessible from both CPU host and GPU device. This allows application codes to develop their GPU capability gradually. The behavior of The_Managed_Arena() likewise depends on the amrex.the_arena_is_managed parameter. If amrex.the_arena_is_managed=0, The_Managed_Arena() is a separate pool of managed memory. If amrex.the_arena_is_managed=1, The_Managed_Arena() is simply aliased to The_Arena() to reduce memory fragmentation.

In amrex::Initialize, a large amount of GPU device memory is allocated and is kept in The_Arena(). The default is 3/4 of the total device memory divided by the number of MPI processes sharing the device. On integrated GPUs (e.g., AMD MI300A) that share memory with the host, the default is 3/8 instead, leaving memory for the host. SYCL devices are not detected as integrated. The size can be changed with a ParmParse parameter, amrex.the_arena_init_size, in the unit of bytes. Note that only the processes in the communicator passed to amrex::Initialize are counted. In MPMD runs, or when an application splits the communicator and initializes AMReX on each part, processes from different communicators may share a device. In that case, the default oversubscribes the device, and the size should be set explicitly. The default can also be changed with an environment variable AMREX_THE_ARENA_INIT_SIZE=X, where X is the number of bytes. When both the ParmParse parameter and the environment variable are present, the former will override the latter. In both cases, the number string could have optional single quotes ' as separators (e.g., 10'000'000'000). It may also use floating-point notation (2.5e10), as long as converting it does not introduce any loss of precision.

The default initial size for other arenas is 8388608 (i.e., 8 MB). For The_Managed_Arena() and The_Device_Arena(), it can be changed with amrex.the_managed_arena_init_size and amrex.the_device_arena_init_size, respectively, if they are not an alias to The_Arena(). For The_Pinned_Arena(),

it can be changed with amrex.the_pinned_arena_init_size. The user can also specify a release threshold for these arenas. If the memory usage in an arena is below the threshold, the arena will keep the memory for later reuse, otherwise it will try to release memory back to the system if it is not being used. By default, the release threshold for The_Arena() is set to be a huge number that prevents the memory being released automatically, and it can be changed with a parameter, amrex.the_arena_release_threshold. For The_Pinned_Arena(), the default release threshold is the size of the total device memory, and the runtime parameter is amrex.the_pinned_arena_release_threshold. If it is a separate arena, the behavior of The_Device_Area() or The_Managed_Arena() can be changed with amrex.the_device_arena_release_threshold or amrex.the_managed_arena_release_threshold. Note that the units for all the parameter discussed above are bytes. All these arenas also have a member function freeUnused() that can be used to manually release unused memory back to the system.

If an arena runs out of memory, it throws amrex::OutOfMemoryError, which is derived from std::bad_alloc, and its what() describes the request and the current memory usage. An uncaught exception terminates the run.

If you want to print out the current memory usage of the Arenas, you can call amrex::Arena::PrintUsage(). When AMReX is built with SUNDIALS turned on, amrex::sundials::The_SUNMemory_Helper() can be provided to SUNDIALS data structures so that they use the appropriate Arena object when allocating memory. For example, it can be provided to the SUNDIALS CUDA vector:

```
N_Vector x = N_VNewWithMemHelp_Cuda(size, use_managed_memory, *The_SUNMemory_Helper());
```

18.5 GPU Safe Classes and Functions

AMReX GPU work takes place inside of MFIter and ParIter loops. Therefore, there are two ways classes and functions have been modified to interact with the GPU:

1. A number of functions used within these loops are labeled using AMREX_GPU_HOST_DEVICE and can be called on the device. This includes member functions, such as IntVect::type(), as well as non-member functions, such as amrex::min and amrex::max. In specialized cases, classes are labeled such that objects can be constructed and destroyed on the device, and their functions can be called there, including IntVect.

2. Functions that contain MFIter or ParIter loops have been rewritten to contain device launches. For example, the FillBoundary function cannot be called from device code, but calling it from CPU will launch GPU kernels if AMReX is compiled with GPU support.

Necessary and convenient AMReX functions and objects have been given a device version and/or device access.

In this section, we discuss some examples of AMReX device classes and functions that are important for programming GPUs.

18.5.1 GpuArray, Array1D, Array2D, and Array3D

GpuArray, Array1D, Array2D, and Array3D are trivial types that work on both host and device. They can be used whenever a fixed size array needs to be passed to the GPU or created on GPU. A variety of functions in AMReX return GpuArray and they can be lambda-captured to GPU code. For example, GeometryData::CellSizeArray(), GeometryData::InvCellSizeArray() and Box::length3d() all return GpuArrays.

18.5.2 AsyncArray

Where the GpuArray is a statically-sized array designed to be passed by value onto the device, AsyncArray is a dynamically-sized array container designed to work between the CPU and GPU. AsyncArray stores a CPU pointer and a GPU pointer and coordinates the movement of an array of objects between the two. It can take initial values from the host and move them to the device. It can copy the data from device back to host. It can also be used as scratch space on device.

The destructor of AsyncArray does not release the memory immediately. The memory is held until the next time the GPU stream is synchronized, so it continues to exist after the AsyncArray object goes out of scope while GPU kernels

already launched in the stream may still be using it. In case too much memory is being held, the stream is synchronized automatically. The resulting `AsyncArray` class is "async-safe", meaning it can be safely used in asynchronous code regions that contain both CPU work and GPU launches, including `MFIter` loops.

`AsyncArray` is also portable. When AMReX is compiled without GPU support, the object only stores and handles the CPU version of the data.

An example using `AsyncArray` is given below,

```
Real h_s = 0.0;
AsyncArray<Real> aa_s(&h_s, 1);  // Build AsyncArray of size 1
Real* d_s = aa_s.data();         // Get associated device pointer

for (MFIter mfi(mf); mfi.isValid(); ++mfi)
{
    Vector<Real> h_v = a_cpu_function();
    AsyncArray<Real> aa_v1(h_v.data(), h_v.size());
    Real* d_v1 = aa_v1.data();  // A device copy of the data

    std::size_t n = ...;
    AsyncArray<Real> aa_v2(n);  // Allocate temporary space on device
    Real* d_v2 = aa_v2.data();  // A device pointer to uninitialized data

    ... // gpu kernels using the data pointed by d_v1 and atomically
        // updating the data pointed by d_s.
        // d_v2 can be used as scratch space and for pass data
        // between kernels.

    // If needed, we can copy the data back to host using
    // AsyncArray::copyToHost(host_pointer, number_of_elements);

    // At the end of each loop the compiler inserts a call to the
    // destructor of aa_v* on cpu.  Objects aa_v* are deleted, but
    // their associated memory pointed by d_v* is not deleted
    // immediately until the gpu kernels in this loop finish.
}

aa_s.copyToHost(&h_s, 1); // Copy the value back to host
```

18.5.3 Gpu Vectors

AMReX also provides a number of dynamic vectors for use with GPU kernels. These are configured to use the different AMReX memory Arenas, as summarized below. By using the memory Arenas, we can avoid expensive allocations and deallocations when (for example) resizing vectors.

Table 18.6: Memory Arenas Associated with each Gpu Vector

Vector	Arena
DeviceVector	The_Arena()
HostVector	The_Pinned_Arena()
ManagedVector	The_Managed_Arena()

These classes behave almost identically to an `amrex::Vector`, (see *Vector, Array, GpuArray, Array1D, Array2D, and Array3D*), except that they can only hold "plain-old-data" objects (e.g. Reals, integers, amrex Particles, etc…). If you

want a resizable vector that doesn't use a memory Arena, simply use `amrex::Vector`.

Note that, even if the data in the vector is managed and available on GPUs, the member functions of e.g. `Gpu::ManagedVector` are not. To use the data on the GPU, it is necessary to pass the underlying data pointer in to the GPU kernels. The managed data pointer can be accessed using the `data()` member function.

Be aware: resizing of dynamically allocated memory on the GPU is unsupported. All resizing of the vector should be done on the CPU, in a manner that avoids race conditions with concurrent GPU kernels.

Also note: `Gpu::ManagedVector` is not async-safe. It cannot be safely constructed inside of an MFIter loop with GPU kernels and great care should be used when accessing `Gpu::ManagedVector` data on GPUs to avoid race conditions.

18.5.4 Gpu::Buffer, Gpu::ManagedVector, and Gpu::TrackedVector

`Gpu::Buffer` (`AMReX_GpuBuffer.H`) and `Gpu::TrackedVector` (`AMReX_TrackedVector.H`) pair a host allocation with a device mirror.

`Gpu::Buffer` uses `Gpu::PinnedVector` on the host and `copyToDeviceAsync()` / `copyToHost()` for transfers. Use it for **frequent, performance-oriented** async copies during a normal GPU run.

`Gpu::ManagedVector` is the arena-backed unified-memory vector introduced under **Gpu Vectors** above (`The_Managed_Arena()`). Like `Gpu::Buffer` it can only be used while AMReX is initialized / a GPU device context exists.

`Gpu::TrackedVector` exposes a host `std::vector` via `host()` / `host_const()` and a device `Gpu::NonManagedDeviceVector` via `device()` / `device_const()` (GPU builds only). Every accessor **automatically synchronizes** the other side when needed: if the host was modified and you call `device()` or `device_const()`, a synchronous host-to-device copy runs first (and vice versa). Writable accessors (`host()`, `device()`) additionally mark the returned side as dirty so that the next access to the opposite side triggers a copy back. Read-only accessors (`host_const()`, `device_const()`) leave the status as `up_to_date` after syncing. You may populate the host **before** `amrex::Initialize()`. Device memory is only valid while AMReX is initialized. On `amrex::Finalize()`, AMReX clears device storage via `release_gpu()` and leaves the host copy for reuse, which supports **Python / pyAMReX** and other workflows that cross multiple AMReX initialize/finalize cycles. Use read-only `host_const()` / `device_const()` when you are not writing, so the object does not flip to a dirty state unnecessarily.

	`Gpu::Buffer`	`Gpu::ManagedVector`	`Gpu::TrackedVector`
Lifetime	Only between `amrex::Initialize/Finalize()`	Only between `amrex::Initialize/Finalize()`	Anytime and cross-session, GPU part only between `amrex::Initialize/Finalize()`
Usage	`operator[]` etc., explicit `copyToDeviceAsync` / `copyToHost`	Single `data()` like `amrex::Vector`	Separate `host()` / `device()` (and `*_const`)
Synchronization	explicit	implicit	automatic on access, tracks status
Performance	Best: pinned host enables asynchronous transfers	Implicit memory migration can add latency	Synchronous copy adds latency
Best for	hot copy loops inside a run	maximum simplicity	interactive and cross-AMReX session usage, e.g., in pyAMReX for user inputs that do not change often

A minimal `Gpu::Buffer` pattern (host fill, async upload, kernel pointer):

```
amrex::Initialize(argc, argv);

Gpu::Buffer<int> buf(n);
for (int i = 0; i < n; ++i) { buf[i] = i; }

int* dp = buf.copyToDeviceAsync();
// launch kernels using dp, then optionally:
buf.copyToHost();
```

`Gpu::ManagedVector` example (unified memory, accessible from both host and device):

```
amrex::Initialize(argc, argv);

Gpu::ManagedVector<int> mv(n);
for (int i = 0; i < n; ++i) { mv[i] = i; }

int* dp = mv.data();
amrex::ParallelFor(n, [=] AMREX_GPU_DEVICE (int i) {
    dp[i] *= 2;  // access on device
});

Gpu::streamSynchronize();
// mv[i] now accessible on host with updated values
```

`Gpu::TrackedVector` example: On GPU builds, you can create this type at any time, even before `amrex::Initialize()`. `amrex::Finalize()` releases device storage for the vector but keeps the host `std::vector`, so a later `Initialize()` session can access `device_const()` or `device()` and the host data is automatically copied to the device.

```
// Host data before AMReX init; GPU available after Initialize().
amrex::Gpu::TrackedVector<int> cross_session;
cross_session.host() = {7, 8, 9};

// ... a lot of other interactive user code, e.g., to set up
// complex input data, optimization libraries or ML frameworks
// in multi-simulation workflows ...

amrex::Initialize(argc, argv);
{
    // device access auto-syncs host data to device.
    int const* dp = cross_session.device_const().data();
    // use dp in kernels ...
}
amrex::Finalize();

// cross_session.device() is not available now and will throw,
// but you can keep using cross_session.host() / .host_const()

amrex::Initialize(argc, argv);
{
    // Device buffer is re-created automatically on access.
    int const* dp = cross_session.device_const().data();
    // kernels may read via device_const().data()
```

(continues on next page)

(continued from previous page)

```
    // or write via device().data()
}
amrex::Finalize();
```

Optional: Call `release_gpu()` when you need to free device memory while keeping the host `std::vector` for later (unless already released, `amrex::Finalize()` clears device storage registered for the object).

Generally, after device kernels, call `Gpu::streamSynchronize()` (or equivalent ordering) before relying on host data, as for any other device work.

18.5.5 MultiFab Reductions

AMReX provides functions for performing standard reduction operations on `MultiFabs`, including `MultiFab::sum` and `MultiFab::max`. When AMReX is built with GPU support, these functions automatically implement the corresponding reductions on GPUs in an efficient manner.

Function template `ParReduce` can be used to implement user-defined reduction functions over `MultiFabs`. For example, the following function computes the sum of total kinetic energy using the data in a `MultiFab` storing the mass and momentum density.

```
Real compute_ek (MultiFab const& mf)
{
    auto const& ma = mf.const_arrays();
    return ParReduce(TypeList<ReduceOpSum>{}, TypeList<Real>{},
                     mf, IntVect(0), // zero ghost cells
            [=] AMREX_GPU_DEVICE (int box_no, int i, int j, int k)
                noexcept -> GpuTuple<Real>
            {
                Array4<Real const> const& a = ma[box_no];
                Real rho = a(i,j,k,0);
                Real rhovx = a(i,j,k,1);
                Real rhovy = a(i,j,k,2);
                Real rhovz = a(i,j,k,3);
                Real ek = (rhovx*rhovx+rhovy*rhovy+rhovz*rhovz)/(2.*rho);
                return { ek };
            });
}
```

As another example, the following function computes the max- and 1-norm of a `MultiFab` in the masked region specified by an `iMultiFab`.

```
GpuTuple<Real,Real> compute_norms (MultiFab const& mf,
                                   iMultiFab const& mask)
{
    auto const& data_ma = mf.const_arrays();
    auto const& mask_ma = mask.const_arrays();
    return ParReduce(TypeList<ReduceOpMax,ReduceOpSum>{},
                     TypeList<Real,Real>{},
                     mf, IntVect(0), // zero ghost cells
            [=] AMREX_GPU_DEVICE (int box_no, int i, int j, int k)
                noexcept -> GpuTuple<Real,Real>
            {
                if (mask_ma[box_no](i,j,k)) {
                    Real a = std::abs(data_ma[box_no](i,j,k));
```

(continues on next page)

(continued from previous page)

```
                return { a, a };
            } else {
                return { 0., 0. };
            }
        });
}
```

It should be noted that the reduction result of `ParReduce` is local and it is the user's responsibility if MPI communication is needed.

`ParReduce` with a number of components reduces over the components as well. To get a separate result for each component, use `ParReducePerComp`, which takes the same arguments and returns a `Vector` with one result per component. `ParReducePerBoxComp` returns one result per local box and component, stored at index `box_no*ncomp+n`. For example, the following function computes the minimum and maximum of every component of a `MultiFab`.

```
Vector<GpuTuple<Real,Real>> compute_minmax (MultiFab const& mf)
{
    auto const& ma = mf.const_arrays();
    return ParReducePerComp(TypeList<ReduceOpMin,ReduceOpMax>{},
                            TypeList<Real,Real>{},
                            mf, IntVect(0), mf.nComp(),
        [=] AMREX_GPU_DEVICE (int box_no, int i, int j, int k, int n)
            noexcept -> GpuTuple<Real,Real>
        {
            Real a = ma[box_no](i,j,k,n);
            return { a, a };
        });
}
```

Like `ParReduce`, these functions return local results.

18.5.6 Box, IntVect and IndexType

In AMReX, `Box`, `IntVect` and `IndexType` are classes for representing indices. These classes and most of their member functions, including constructors and destructors, have both host and device versions. They can be used freely in device code.

18.5.7 Geometry

AMReX's `Geometry` class is not a GPU safe class. However, we often need to use geometric information such as cell size and physical coordinates in GPU kernels. We can use the following member functions and pass the returned values to GPU kernels:

```
GpuArray<Real,AMREX_SPACEDIM> ProbLoArray () const noexcept;
GpuArray<Real,AMREX_SPACEDIM> ProbHiArray () const noexcept;
GpuArray<int,AMREX_SPACEDIM> isPeriodicArray () const noexcept;
GpuArray<Real,AMREX_SPACEDIM> CellSizeArray () const noexcept;
GpuArray<Real,AMREX_SPACEDIM> InvCellSizeArray () const noexcept;
```

Alternatively, we can copy the data into a GPU safe class that can be passed by value to GPU kernels. This class is called `GeometryData`, which is created by calling `Geometry::data()`. The accessor functions of `GeometryData` are identical to `Geometry`.

18.5.8 BaseFab, FArrayBox, IArrayBox

BaseFab<T>, IArrayBox and FArrayBox have some GPU support. They cannot be constructed in device code unless they are constructed as an alias to Array4. Many of their member functions can be used in device code as long as they have been constructed in device memory. Some of the device member functions include array, dataPtr, box, nComp, and setVal.

All BaseFab<T> objects in FabArray<FAB> are allocated in CPU memory, including IArrayBox and FArrayBox, which are derived from BaseFab, and the array data contained are allocated in either device or managed memory. We cannot pass a BaseFab object by value because they do not have copy constructor. However, we can make an Array4 using member function BaseFab::array(), and pass it by value to GPU kernels. In GPU device code, we can use Array4 or, if necessary, we can make an alias BaseFab from an Array4. For example,

```
AMREX_GPU_HOST_DEVICE void g (FArrayBox& fab) { ... }

AMREX_GPU_HOST_DEVICE void f (Box const& bx, Array4<Real> const& a)
{
  FArrayBox fab(a,bx.ixType());
  g(fab);
}
```

18.5.9 Elixir

We often have temporary FArrayBoxes in MFIter loops. These objects go out of scope at the end of each iteration. Because of the asynchronous nature of GPU kernel execution, their destructors might get called before their data are used on GPU. Elixir can be used to extend the life of the data. For example,

```
for (MFIter mfi(mf); mfi.isValid(); ++mfi) {
  const Box& bx = mfi.tilebox();
  FArrayBox tmp_fab(bx, numcomps);
  Elixir tmp_eli = tmp_fab.elixir();
  Array4<Real> const& tmp_arr = tmp_fab.array();

  // GPU kernels using the temporary
}
```

Without Elixir, the code above will likely cause memory errors because the temporary FArrayBox is deleted on cpu before the gpu kernels use its memory. With Elixir, the ownership of the memory is transferred to Elixir that is guaranteed to be async-safe.

18.5.10 Async Arena

Using a stream-ordered allocator, the temporary memory allocation and deallocation issue discussed above can be solved. Instead of using Elixir, we can write code like below,

```
for (MFIter mfi(mf); mfi.isValid(); ++mfi) {
  const Box& bx = mfi.tilebox();
  FArrayBox tmp_fab(bx, numcomps, The_Async_Arena());
  Array4<Real> const& tmp_arr = tmp_fab.array();

  // GPU kernels using the temporary
}
```

The freed memory is held in a buffer until the next time the GPU stream is synchronized. In case too much memory is in this buffer, the stream is synchronized automatically. This is now the recommended way because it's usually more efficient than Elixir. Note that the code above works on all platforms.

18.6 Kernel Launch

In this section, how to offload work to the GPU will be demonstrated. AMReX supports offloading work with CUDA, HIP, SYCL, OpenACC, or OpenMP.

When using CUDA, HIP, or SYCL, AMReX provides users with portable C++ function calls or C++ macros that launch a user-defined lambda function. When compiled without CUDA/HIP/SYCL, the lambda function is ran on the CPU. When compiled with CUDA/HIP/SYCL, the launch function prepares and launches the lambda function on the GPU. The preparation includes calculating the appropriate number of blocks and threads, selecting the CUDA stream or HIP stream or SYCL queue, and defining the appropriate work chunk for each GPU thread.

When using OpenACC or OpenMP offloading pragmas, the users add the appropriate pragmas to their work loops and functions to offload to the GPU. These work in conjunction with AMReX's internal CUDA-based memory management, described earlier, to ensure the required data is available on the GPU when the offloaded function is executed.

The available launch schema are presented here in three categories: launching nested loops over Boxes or 1D arrays, launching generic work and launching using OpenACC or OpenMP pragmas. Examples of launching nested loops with `ParallelFor` can be found in the Base tutorials, e.g., MultiFab and GpuContainers. Users should also refer to Chapter *Basics* as needed for information about basic AMReX classes.

AMReX also recommends writing primary floating point operation kernels in C++ using AMReX's `Array4` object syntax. It provides a multi-dimensional array syntax, similar in appearance to Fortran, while maintaining performance. The details can be found in *Array4* and *C++ Kernel*.

18.6.1 Launching C++ nested loops

The most common AMReX work construct is a set of nested loops over the cells in a box. AMReX provides C++ functions and macro equivalents to port nested loops efficiently onto the GPU. There are 3 different nested loop GPU launches: a 4D launch for work over a box and a number of components, a 3D launch for work over a box and a 1D launch for work over a number of arbitrary elements. Each of these launches provides a performance portable set of nested loops for both CPU and GPU applications.

These loop launches should only be used when each iteration of the nested loop is independent of other iterations. Therefore, these launches have been marked with `AMREX_PRAGMA_SIMD` when using the CPU and they should only be used for `simd`-capable nested loops. Calculations that cannot vectorize should be rewritten wherever possible to allow efficient utilization of GPU hardware.

For loops whose iterations are *not* independent, AMReX provides `amrex::For`. It supports the same set of signatures as `amrex::ParallelFor` (including the variants taking a `TypeList` of `CompileTimeOptions`) and generates identical code on GPU, but omits `AMREX_PRAGMA_SIMD` on CPU. A common case requiring `amrex::For` is scatter-style accumulation with `Gpu::Atomic` operations: on the host these compile to plain (non-atomic) updates, so different iterations may update the same memory location. Using `amrex::ParallelFor` for such a loop licenses the compiler to vectorize it anyway, which can silently drop updates from vector lanes that touch the same address.

However, it is important for applications to use these launches whenever appropriate because they contain optimizations for both CPU and GPU variations of nested loops. For example, on the GPU the spatial coordinate loops are reduced to a single loop and the component loop is moved to these innermost loops. AMReX's launch functions apply the appropriate optimizations for compiling both with and without GPU support in a compact and readable format.

Compile-time options for ParallelFor

When a kernel has a small set of runtime choices, `amrex::ParallelFor` can generate a specialized kernel for each choice using `amrex::CompileTimeOptions`. The runtime value selects which specialization to launch. Inside the lambda, the selected option is a compile-time constant, so `if constexpr` can discard branches that do not apply to that specialization. This interface works on both CPUs and GPUs.

For example, the following code either assigns or adds to component zero of a `MultiFab` named `mf`:

```
#include <AMReX_GpuLaunch.H>
#include <AMReX_MFIter.H>
#include <AMReX_MultiFab.H>

enum Update : int { Assign = 0, Add = 1 };
int update = Add; // May instead be selected from runtime input.

for (amrex::MFIter mfi(mf, amrex::TilingIfNotGPU()); mfi.isValid(); ++mfi)
{
    auto const& box = mfi.tilebox();
    auto const arr = mf.array(mfi);

    amrex::ParallelFor(
        amrex::TypeList<amrex::CompileTimeOptions<Assign, Add>>{},
        {update}, box,
        [=] AMREX_GPU_DEVICE (int i, int j, int k, auto option)
        {
            // Reference the captured array before the constexpr branches.
            auto const& a = arr;
            if constexpr (option.value == Assign) {
                a(i,j,k,0) = 1.0;
            } else {
                a(i,j,k,0) += 1.0;
            }
        });
}
```

The `TypeList` contains one `CompileTimeOptions` set for each runtime parameter. Each set lists the allowed `int` values; every runtime value must belong to its corresponding set. The next argument supplies the runtime values in the same order. After the usual loop indices, the lambda receives one `auto` argument per option set, also in that order. Each argument is a `std::integral_constant<int, value>` whose `.value` can be used in a compile-time condition.

For multiple parameters, use, for example, `amrex::TypeList<amrex::CompileTimeOptions<0, 1>, amrex::CompileTimeOptions<0, 1, 2>>{}` with runtime values `{first_option, second_option}` and trailing lambda arguments `auto first, auto second`. All combinations are instantiated: this example produces six specializations, but each call launches only the matching one. Keep the option sets small, since their Cartesian product can increase compilation time and executable size.

Compile-time options are supported for the 1D `N`, `Box`, and `Box` plus component-count overloads. For the component-count overload, the option arguments follow the component index `n`. The same interface is available for `amrex::For`, which omits the CPU SIMD pragma as described above.

Note

CUDA extended device lambdas have a restriction on first capturing a variable inside an `if constexpr` branch. If NVCC reports a capture error, reference the captured variable before the branches, as shown with `arr` above. Alternatively, use a regular `if`; its condition is still known at compile time and can be optimized by the compiler.

Loop launch examples

AMReX also provides a variation of the launch function that is implemented as a C++ macro. It behaves identically to the function, but hides the lambda function from the user. There are some subtle differences between the two implementations, that will be discussed. It is up to the user to select which version they would like to use. For simplicity,

the function variation will be discussed throughout the rest of this documentation, however all code snippets will also include the macro variation for reference.

A 4D example of the launch function, `amrex::ParallelFor`, is given here:

```
int ncomp = mf.nComp();
for (MFIter mfi(mf,TilingIfNotGPU()); mfi.isValid(); ++mfi)
{
    const Box& bx = mfi.tilebox();
    Array4<Real> const& fab = mf.array(mfi);

    amrex::ParallelFor(bx, ncomp,
    [=] AMREX_GPU_DEVICE (int i, int j, int k, int n)
    {
        fab(i,j,k,n) += 1.;
    });

    /* MACRO VARIATION:
    /
    /   AMREX_PARALLEL_FOR_4D ( bx, ncomp, i, j, k, n,
    /   {
    /       fab(i,j,k,n) += 1.;
    /   });
    */
}
```

This code works whether it is compiled for GPUs or CPUs. `TilingIfNotGPU()` returns `false` in the GPU case to turn off tiling and maximize the amount of work given to the GPU in each launch. When tiling is off, `tilebox()` returns the `validbox()`. The `BaseFab::array()` function returns a lightweight `Array4` object that defines access to the underlying `FArrayBox` data. The `Array4`s are then captured by the C++ lambda functions defined in the launch function.

`amrex::ParallelFor()` expands into different variations of a quadruply-nested **for** loop depending on dimensionality and whether it is being implemented on CPU or GPU. The best way to understand this function is to take a look at the 4D `amrex::ParallelFor` that is implemented when AMReX is compiled without GPU support, such as `USE_CUDA=FALSE`. A simplified version is reproduced here:

```
void ParallelFor (Box const& box, int ncomp, /* LAMBDA FUNCTION */)
{
    const Dim3 lo = amrex::lbound(box);
    const Dim3 hi = amrex::ubound(box);

    for (int n = 0; n < ncomp; ++n) {
        for (int z = lo.z; z <= hi.z; ++z) {
        for (int y = lo.y; y <= hi.y; ++y) {
        AMREX_PRAGMA_SIMD
        for (int x = lo.x; x <= hi.x; ++x) {
            /* LAUNCH LAMBDA FUNCTION (x,y,z,n) */
        }}}
    }
}
```

`amrex::ParallelFor` takes a `Box` and a number of components, which define the bounds of the quadruply-nested **for** loop, and a lambda function to run on each iteration of the nested loop. The lambda function takes the loop iterators as parameters, allowing the current cell to be indexed in the lambda. In addition to the loop indices, the lambda function

captures any necessary objects defined in the local scope.

CUDA lambda functions can only capture by value, as the information must be able to be copied onto the device. In this example, the lambda function captures a `Array4` object, `fab`, that defines how to access the `FArrayBox`. The macro uses `fab` to increment the value of each cell within the `Box bx`. If AMReX is compiled with GPU support, this incrementation is performed on the GPU, with GPU optimized loops.

This 4D launch can also be used to work over any sequential set of components, by passing the number of consecutive components and adding the iterator to the starting component: `fab(i,j,k,n_start+n)`.

The 3D variation of the loop launch does not include a component loop and has the syntax shown here:

```
for (MFIter mfi(mf,TilingIfNotGPU()); mfi.isValid(); ++mfi)
{
    const Box& bx = mfi.tilebox();
    Array4<Real> const& fab = mf.array(mfi);
    amrex::ParallelFor(bx,
    [=] AMREX_GPU_DEVICE (int i, int j, int k)
    {
        fab(i,j,k) += 1.;
    });

    /* MACRO VARIATION:
    /
    /   AMREX_PARALLEL_FOR_3D ( bx, i, j, k,
    /   {
    /       fab(i,j,k) += 1.;
    /   });
    */
}
```

Finally, a 1D version is available for looping over a number of elements, such as particles. An example of a 1D function launch is given here:

```
for (MFIter mfi(mf); mfi.isValid(); ++mfi)
{
    FArrayBox& fab = mf[mfi];
    Real* AMREX_RESTRICT p = fab.dataPtr();
    const long nitems = fab.box().numPts() * fab.nComp();

    amrex::ParallelFor(nitems,
    [=] AMREX_GPU_DEVICE (long idx)
    {
        p[idx] += 1.;
    });

    /* MACRO VARIATION:
    /
    /   AMREX_PARALLEL_FOR_1D ( nitems, idx,
    /   {
    /       p[idx] += 1.;
    /   });
    */
}
```

Instead of passing an `Array4`, `FArrayBox::dataPtr()` is called to obtain a pointer to the `FArrayBox` data. This

is an alternative way to access the FArrayBox data on the GPU. Instead of passing a Box to define the loop bounds, a long or int number of elements is passed to bound the single for loop. This construct can be used to work on any contiguous set of memory by passing the number of elements to work on and indexing the pointer to the starting element: p[idx + 15].

18.6.2 GPU block size

By default, ParallelFor launches AMREX_GPU_MAX_THREADS threads per GPU block, where AMREX_GPU_MAX_THREADS is a compile-time constant with a default value of 256. The users can also explicitly specify the number of threads per block by ParallelFor<MY_BLOCK_SIZE>(...), where MY_BLOCK_SIZE is a multiple of the warp size (e.g., 128). This allows the users to do performance tuning for individual kernels.

18.6.3 Launching general kernels

To launch more general work on the GPU, AMReX provides a standard launch function: amrex::launch. Instead of creating nested loops, this function prepares the device launch based on a Box, launches with an appropriate sized GPU kernel and constructs a thread Box that defines the work for each thread. On the CPU, the thread Box is set equal to the total launch Box, so tiling works as expected. On the GPU, the thread Box usually contains a single cell to allow all GPU threads to be utilized effectively.

An example of a generic function launch is shown here:

```
for (MFIter mfi(mf,TilingIfNotGPU()); mfi.isValid(); ++mfi)
{
    const Box& bx = mfi.tilebox();
    Array4<Real> const& arr = mf.array(mfi);

    amrex::launch(bx,
    [=] AMREX_GPU_DEVICE (Box const& tbx)
    {
        pluseone_array4(tbx, arr);
        FArrayBox fab(arr, tbx.ixType());
        plusone_fab(tbx, fab); // this version takes FArrayBox
    });

    /* MACRO VARIATION
    /
    /   AMREX_LAUNCH_DEVICE_LAMBDA ( bx, tbx,
    /   {
    /       plusone_array4(tbx, arr);
    /       plusone_fab(tbx, FArrayBox(arr,tbx.ixType()));
    /   });
    */
}
```

It also shows how to make a FArrayBox from Array4 when needed. Note that FarrayBoxes cannot be passed to GPU kernels directly. TilingIfNotGPU() returns false in the GPU case to turn off tiling and maximize the amount of work given to the GPU in each launch, which substantially improves performance. When tiling is off, tilebox() returns the validbox() of the FArrayBox for that iteration.

18.6.4 Launching over many small boxes

Some operations touch many small boxes of different sizes, for example the ghost cells outside the domain boundary of every box, and none of them has enough cells to fill the GPU. `ParallelFor` can take a `Vector` of tags instead of a single `Box` and run all of them in one kernel launch. A tag is a small struct with a `box()` member (or `size()` for 1D work) and whatever else the kernel needs, such as an `Array4`; AMReX provides `Array4BoxTag` and others, and user-defined tags work too. The kernel receives the cell index and the tag,

```
Vector<Array4BoxTag<Real>> tags;
for (MFIter mfi(mf); mfi.isValid(); ++mfi) {
    tags.push_back({mf.array(mfi), some_box(mfi)});
}
ParallelFor(tags, ncomp,
[=] AMREX_GPU_DEVICE (int i, int j, int k, int n, Array4BoxTag<Real> const& tag)
{
    tag.dfab(i,j,k,n) = ...;
});
```

This copies the tags to the device and synchronizes the stream on every call. When the same tags are used repeatedly, build a `TagVector` from them once and pass it to `ParallelFor` instead. See the TagParallelFor tutorial.

18.6.5 Launching with shared memory

`amrex::launch<nthreads>(nblocks, stream, f)` runs `f` on a given number of thread blocks. The kernel body is written in the native programming model, so it differs between CUDA/HIP and SYCL, and there is no CPU version; a separate CPU path is needed. It is for performance hot spots that benefit from shared memory or block-wide synchronization among the threads of a block. The SharedMemory tutorial uses it for a binned sum with block-private partial sums, with a CPU implementation alongside.

18.6.6 Offloading work using OpenACC or OpenMP pragmas

When using OpenACC or OpenMP with AMReX, the GPU offloading work is done with pragmas placed on the nested loops. This leaves the `MFIter` loop largely unchanged. An example GPU pragma based `MFIter` loop that calls a Fortran function is given here:

```
for (MFIter mfi(mf,TilingIfNotGPU()); mfi.isValid(); ++mfi)
{
    const Box& bx = mfi.tilebox();
    FArrayBox& fab = mf[mfi];
    plusone_acc(BL_TO_FORTRAN_BOX(tbx),
                BL_TO_FORTRAN_ANYD(fab));
}
```

The function `plusone_acc` is a CPU host function. The `FArrayBox` reference from `operator[]` is a reference to a `FArrayBox` in host memory with data that has been placed in GPU memory. `BL_TO_FORTRAN_BOX` and `BL_TO_FORTRAN_ANYD` behave identically to implementations used on the CPU. These macros return the individual components of the AMReX C++ objects to allow passing to the Fortran function.

The corresponding OpenACC labeled loop in `plusone_acc` is:

```
!dat = pointer to fab's GPU data

!$acc kernels deviceptr(dat)
do       k = lo(3), hi(3)
   do    j = lo(2), hi(2)
```

(continues on next page)

(continued from previous page)

```
      do i = lo(1), hi(1)
         dat(i,j,k) = dat(i,j,k) + 1.0_amrex_real
      end do
   end do
end do
!$acc end kernels
```

Since the data pointer passed to `plusone_acc` points to device memory, OpenACC can be told the data is available on the device using the `deviceptr` construct. For further details about OpenACC programming, consult the OpenACC user's guide.

The OpenMP implementation of this loop is similar, only requiring changing the pragmas utilized to obtain the proper offloading. The OpenMP labeled version of this loop is:

```
!dat = pointer to fab's GPU data

!$omp target teams distribute parallel do collapse(3) schedule(static,1) is_device_
↪ptr(dat)
do       k = lo(3), hi(3)
   do    j = lo(2), hi(2)
      do i = lo(1), hi(1)
         dat(i,j,k) = dat(i,j,k) + 1.0_amrex_real
      end do
   end do
end do
```

In this case, `is_device_ptr` is used to indicate that `dat` is available in device memory. For further details about programming with OpenMP for GPU offloading, consult the OpenMP user's guide.

18.6.7 Kernel launch details

CUDA (and HIP) kernel calls are asynchronous and they return before the kernel is finished on the GPU. So the `MFIter` loop finishes iterating on the CPU and is ready to move on to the next work before the actual work completes on the GPU. To guarantee consistency, there is an implicit device synchronization (a GPU barrier) in the destructor of `MFIter`. This ensures that all GPU work inside of an `MFIter` loop will complete before code outside of the loop is executed. Any kernel launches made outside of an `MFIter` loop must ensure appropriate device synchronization occurs. This can be done by calling `Gpu::streamSynchronize()`.

CUDA and HIP supports multiple streams and kernels. Kernels launched in the same stream are executed sequentially, but different streams of kernel launches may be run in parallel. For each iteration of `MFIter`, AMReX uses a different GPU stream (up to 4 streams in total). This allows each iteration of an `MFIter` loop to run independently, but in the expected sequence, and maximize the use of GPU parallelism. However, AMReX uses the default GPU stream outside of `MFIter` loops.

Launching kernels with AMReX's launch macros or functions implement a C++ lambda function. Lambda functions used for launches on the GPU have some restrictions the user must understand. First, the function enclosing the extended lambda must not have private or protected access within its parent class, otherwise the code will not compile. This can be fixed by changing the access of the enclosing function to public.

Another pitfall that must be considered: if the lambda function accesses a member of the enclosing class, the lambda function actually captures `this` pointer by value and accesses variables and functions via `this->`. If the object is not accessible on GPU, the code will not work as intended. For example,

```
class MyClass {
public:
```

(continues on next page)

(continued from previous page)

```
    Box bx;
    int m;                          // integer created on the host.
    void f () {
        amrex::ParallelFor(bx,
        [=] AMREX_GPU_DEVICE (int i, int j, int k)
        {
            printf("m = %d\n", m);   // Failed attempt to use m on the GPU.
        });
    }
};
```

The function `f` in the code above will not work unless the `MyClass` object is in unified memory. If it is undesirable to put the object into unified memory, a local copy of the information can be created for the lambda to capture. For example:

```
class MyClass {
public:
    Box bx;
    int m;
    void f () {
        int local_m = m;                // Local temporary copy of m.
        amrex::ParallelFor(bx,
        [=] AMREX_GPU_DEVICE (int i, int j, int k)
        {
            printf("m = %d\n", local_m);  // Lambda captures local_m by value.
        });
    }
};
```

C++ macros have some important limitations. For example, commas outside of a set of parentheses are interpreted by the macro, leading to errors such as:

```
AMREX_PARALLEL_FOR_3D (bx, tbx,
{
    Real a, b;   <---- Error. Macro reads "{ Real a" as a parameter
                                          and "b; }" as
                                          another.
    Real a;      <---- OK
    Real b;
});
```

One should also avoid using **continue** and **return** inside the macros because it is not an actual **for** loop. Users that choose to implement the macro launches should be aware of the limitations of C++ preprocessing macros to ensure GPU offloading is done properly.

Finally, AMReX's most common CPU threading strategy for GPU/CPU systems is to utilize OpenMP threads to maintain multi-threaded parallelism on work chosen to run on the host. This means OpenMP pragmas should be maintained where CPU work is performed and usually turned off where work is offloaded onto the GPU. OpenMP pragmas can be turned off using the conditional pragma and `Gpu::notInLaunchRegion()`, as shown below:

```
#ifdef AMREX_USE_OMP
#pragma omp parallel if (Gpu::notInLaunchRegion())
#endif
```

It is generally expected that simply using OpenMP threads to launch GPU work quicker will show little improvement or even perform worse. So, this conditional statement should be added to MFIter loops that contain GPU work, unless users specifically test the performance or are designing more complex workflows that require OpenMP.

18.7 Stream and Synchronization

As mentioned in Section *Overview of AMReX GPU Support*, AMReX uses a number of GPU streams that are either CUDA streams or HIP streams or SYCL queues. Many GPU functions (e.g., `ParallelFor` and `Gpu::copyAsync`) are asynchronous with respect to the host. To facilitate synchronization that is sometimes necessary, AMReX provides `Gpu::streamSynchronize()` and `Gpu::streamSynchronizeAll()` to synchronize the current stream and all AMReX streams, respectively. For performance reasons, one should try to minimize the number of synchronization calls. For example,

```
// The synchronous version is NOT recommended
Gpu::copy(Gpu::deviceToHost, ....);
Gpu::copy(Gpu::deviceToHost, ....);
Gpu::copy(Gpu::deviceToHost, ....);

// NOT recommended because of unnecessary synchronization
Gpu::copyAsync(Gpu::deviceToHost, ....);
Gpu::streamSynchronize();
Gpu::copyAsync(Gpu::deviceToHost, ....);
Gpu::streamSynchronize();
Gpu::copyAsync(Gpu::deviceToHost, ....);
Gpu::streamSynchronize();

// recommended
Gpu::copyAsync(Gpu::deviceToHost, ....);
Gpu::copyAsync(Gpu::deviceToHost, ....);
Gpu::copyAsync(Gpu::deviceToHost, ....);
Gpu::streamSynchronize();
```

In addition to stream synchronization, there is also `Gpu::synchronize()` that will perform a device wide synchronization. However, a device wide synchronization is usually too excessive and it might interfere with other libraries (e.g., MPI).

AMReX functions can exhibit different synchronization behaviors. Functions that launch user-provided GPU kernels are typically asynchronous. For example, GPU kernels launched by the `ParallelFor` family of functions (including variants for 1D arrays, `Box`es, and an entire `MultiFab` in a single kernel) are asynchronous with respect to the host.

The only exception in the `ParallelFor` family is `ParallelForRNG`, which is used when random number generation is required inside the GPU kernel. It contains an implicit stream synchronization to avoid potential race conditions. Many member functions of `BaseFab` and `FArrayBox` are also asynchronous. However, implicit synchronizations occur in a number of situations. Some are unavoidable. For example, `Reduce::Sum` returns a value on the host, and therefore must perform an implicit synchronization. Another example is the communication functions such as `FillBoundary`. Since MPI has no notion of GPU streams, the data must be synchronized before being passed to MPI calls.

Other synchronizations are introduced for safety. A notable example is `MFIter`, which performs implicit synchronizations at both the beginning and end of the loop (as a whole, not on each iteration). These synchronizations can be disabled using `MFItInfo`. For example,

```
// There is no implicit synchronization in the following loop.
for (MFIter mfi(mf, MFItInfo{}.DisableDeviceSync()); mfi.isValid(); ++mfi)
{ ... }
```

One could also use Gpu::NoSyncRegion, as shown below:

```
{
  Gpu::NoSyncRegion no_sync{}; // No implicit GPU synchronization in the
                               // current scope

  for (MFIter mfi(mf); mfi.isValid(); ++mfi) { ... }

  // Gpu::streamSynchronize(); Explicit synchronization is allowed.

  for (MFIter mfi(mf2); mfi.isValid(); ++mfi) { .... }
}
```

This approach suppresses implicit synchronization for all operations within the scoped region and restores the previous synchronization setting upon exiting. A variant, Gpu::SyncAtExitOnly, additionally synchronizes the GPU streams when the scope is exited, unless the enclosing code is already inside a Gpu::NoSyncRegion. This is convenient for functions that want to skip the implicit synchronizations internally but return with all their GPU work completed. For example, MLMG::solve uses it together with Gpu::SingleStreamRegion when MLMG::setNoGpuSync(true) has been called.

Inside a no-sync region, all GPU work should be launched on a single stream (e.g., by also using Gpu::SingleStreamRegion) unless the ordering between streams is managed explicitly. Temporary MultiFabs and FArrayBoxes that are created and destroyed inside the region should be allocated from The_Async_Arena() (e.g., with MFInfo().SetArena(The_Async_Arena())), which releases their device memory in a stream-ordered fashion. Memory freed to an ordinary arena, such as The_Arena(), may be returned to the system (e.g., when the arena is under pressure or amrex.the_arena_release_threshold is small) while kernels using it are still queued on the stream. Whether that is safe depends on the backend (CUDA and HIP synchronize the whole device before releasing memory, SYCL does not), so it must not be relied upon. The auxiliary pinned host buffers AMReX allocates for a FabArray are always released in a stream-ordered fashion. Communication functions such as FillBoundary and ParallelCopy may return with their unpack kernels still queued on the stream. Host code that needs to read the results of GPU work must still synchronize explicitly (reductions such as MultiFab::sum do this internally), and so must code that hands GPU data to a library that runs on its own stream (e.g., HYPRE).

18.8 External GPU Streams

AMReX normally launches GPU kernels from an internal pool of GPU streams and round-robins across them inside MFIter loops. When an application needs to integrate with an externally managed stream, AMReX provides an override mechanism. External-stream overrides form a stack and all calls must occur outside OpenMP parallel regions. If you install a new external stream while another is already active, the new handle becomes current until it is reset, at which point AMReX restores the previous external stream.

- Call amrex::Gpu::setExternalGpuStream(stream) (or use the RAII helper amrex::Gpu::ExternalGpuStreamRegion(stream,sync_on_exit)) on the host. Every subsequent call to amrex::Gpu::gpuStream() returns the supplied stream. Calls must be paired with amrex::Gpu::resetExternalGpuStream(sync_stream) (RAII helpers pass the same flag via sync_on_exit); popping the current stream restores the previous external stream, if any. Passing amrex::Gpu::ExternalStreamSync::No to sync_stream/sync_on_exit skips the final Gpu::streamSynchronize unless deferred frees recorded by The_Async_Arena are still pending, in which case AMReX forces a synchronization to keep the arena safe. The external stream or queue must belong to the active AMReX device. For SYCL, the queue must also use the same SYCL context as AMReX and must be an in-order queue. AMReX selects an active GPU during amrex::Initialize from the device platform's default context, whose overloads accept an optional trailing int device_id argument; pass the desired GPU there if an external runtime needs AMReX to adopt a specific device before the stream is created. Conversely, if AMReX should drive the selection, query amrex::Gpu::Device::deviceId() and configure the external

runtime (for example, by calling `cudaSetDevice` or `hipSetDevice` before constructing the stream) so that the stream is associated with the device AMReX is already using.

- All asynchronous frees recorded through `amrex::Gpu::freeAsync` and The_Async_Arena continue to work because AMReX tracks the external stream with an internal `StreamManager`.

Limitations and best practices:

- Entering or exiting the override inside an OpenMP parallel region is illegal and will trigger an assertion. Configure the stream on the main thread before launching GPU work, if OpenMP threading is used.
- Backend compatibility rules are checked when the override is installed. CUDA and HIP external streams must be associated with the current `amrex::Gpu::Device::deviceId()`. SYCL external queues must use AMReX's SYCL device and SYCL context, and must be in-order.
- While an override is active, `Gpu::numGpuStreams()` reports 1. This keeps `MFIter` from trying to pipeline across multiple AMReX-managed streams, but it also means you cannot currently provide a *set* of external streams for MFIter to round-robin across.
- Objects that use stream-ordered memory (for example, a `BaseFab` allocated from `The_Async_Arena`) should generally be created and destroyed within the same external-stream region. In particular, avoid creating or resizing such an object inside an external-stream region and then destroying it after that region has exited, or creating it outside and then resizing or destroying it inside the region. Stream-ordered frees remember the stream associated with the allocation, and AMReX requires that stream to still be managed when the free occurs. If that stream was external and has already been popped, AMReX aborts instead of guessing where to enqueue the deferred free.
- Likewise, reuse a given `Reducer` or `ReduceData` object only on a single external stream. Reusing the same object across different external stream regions, or across an external stream and AMReX stream 0, is not supported. If a reduction is launched inside an external-stream region, read its final value before leaving that region.
- Make sure the external stream remains valid for the duration of the override. Destroying it before the RAII guard goes out of scope is undefined behavior.
- Using `ExternalStreamSync::No` transfers synchronization responsibility to the caller. After `resetExternalGpuStream(ExternalStreamSync::No)`, AMReX no longer tracks that external stream, and later AMReX global synchronization helpers are not guaranteed to wait for work already queued on it. It is the user's responsibility to synchronize that external stream before any dependent teardown or finalization.

Example: wrap AMReX work inside an externally created CUDA stream:

```
#ifdef AMREX_USE_CUDA
  cudaStream_t external{};
  AMREX_CUDA_SAFE_CALL(cudaStreamCreateWithFlags(&external, cudaStreamNonBlocking));
#endif
  {
    amrex::Gpu::ExternalGpuStreamRegion stream_scope(external);
    MultiFab mf(...);
    MFItInfo info;
    info.DisableDeviceSync();
    for (MFIter mfi(mf,info); mfi.isValid(); ++mfi) {
      auto const& box = mfi.tilebox();
      auto const arr = mf.array(mfi);
      amrex::ParallelFor(box, [=] AMREX_GPU_DEVICE (int i, int j, int k) {
        arr(i,j,k) = 0.0_rt;
      });
    }
    // Optional: launch non-AMReX CUDA kernels on 'external' here.
```

(continues on next page)

(continued from previous page)

```
  }
#ifdef AMREX_USE_CUDA
  AMREX_CUDA_SAFE_CALL(cudaStreamDestroy(external));
#endif
```

The same pattern works for HIP streams and, in SYCL builds, with `sycl::queue` objects.

18.9 An Example of Migrating to GPU

The nature of GPU programming poses difficulties for a number of common AMReX patterns, such as the one below:

```
// Given MultiFab uin and uout
#ifdef AMREX_USE_OMP
#pragma omp parallel
#endif
{
  FArrayBox q;
  for (MFIter mfi(uin,true); mfi.isValid(); ++mfi)
  {
    const Box& tbx = mfi.tilebox();
    const Box& gbx = amrex::grow(tbx,1);
    q.resize(gbx);

    // Do some work with uin[mfi] as input and q as output.
    // The output region is gbx;
    f1(gbx, q, uin[mfi]);

    // Then do more work with q as input and uout[mfi] as output.
    // The output region is tbx.
    f2(tbx, uout[mfi], q);
  }
}
```

There are several issues in migrating this code to GPUs that need to be addressed. First, functions `f1` and `f2` have different work regions (`tbx` and `gbx`, respectively) and there are data dependencies between the two (`q`). This makes it difficult to put them into a single GPU kernel, so two separate kernels will be launched, one for each function.

As we have discussed, AMReX uses multiple CUDA streams or HIP streams or SYCL queues for launching kernels. Because `q` is used inside `MFIter` loops, multiple GPU kernels on different streams are accessing its data. This creates a race condition. One way to fix this is to move `FArrayBox q` inside the loop to make it local to each loop and use `Elixir` to make it async-safe (see Section *Elixir*). This strategy works well for GPU. However it is not optimal for OpenMP CPU threads when the GPU is not used, because of the memory allocation inside OpenMP parallel region. It turns out it is actually unnecessary to make `FArrayBox q` local to each iteration when `Elixir` is used to extend the life of its floating point data. The code below shows an example of how to rewrite the example in a performance portable way.

```
// Given MultiFab uin and uout
#ifdef AMREX_USE_OMP
#pragma omp parallel if (Gpu::notInLaunchRegion())
#endif
{
  FArrayBox q;
```

(continues on next page)

(continued from previous page)

```
  for (MFIter mfi(uin,TilingIfNotGPU()); mfi.isValid(); ++mfi)
  {
    const Box& tbx = mfi.tilebox();
    const Box& gbx = amrex::grow(tbx,1);
    q.resize(gbx);
    Elixir eli = q.elixir();
    Array4<Real> const& qarr = q.array();

    Array4<Real const> const& uinarr = uin.const_array(mfi);
    Array4<Real> const& uoutarr = uout.array(mfi);

    amrex::launch(gbx,
    [=] AMREX_GPU_DEVICE (Box const& b)
    {
      f1(b, qarr, uinarr);
    });

    amrex::launch(tbx,
    [=] AMREX_GPU_DEVICE (Box const& b)
    {
      f2(b, uoutarr, qarr);
    });
  }
}
```

18.10 Assertions and Error Checking

To help debugging, we often use `amrex::Assert` and `amrex::Abort`. These functions are GPU safe and can be used in GPU kernels. However, implementing these functions requires additional GPU registers, which will reduce overall performance. Therefore, by default these functions and the macro `AMREX_ALWAYS_ASSERT` are no-ops for optimized builds (e.g., `DEBUG=FALSE` using the GNU Make build system) when called from kernels run on GPU. Calls to these functions from GPU kernels are active for debug builds and can optionally be activated at compile time for optimized builds (e.g., `DEBUG=FALSE` and `USE_ASSERTION=TRUE` using the GNU Make build system).

In CPU code, `AMREX_GPU_ERROR_CHECK()` can be called to check the health of previous GPU launches. This call looks up the return message from the most recently completed GPU launch and aborts if it was not successful. Many kernel launch macros as well as the `MFIter` destructor include a call to `AMREX_GPU_ERROR_CHECK()`. This prevents additional launches from being called if a previous launch caused an error and ensures all GPU launches within an `MFIter` loop completed successfully before continuing work.

However, due to asynchronicity, determining the source of the error can be difficult. Even if GPU kernels launched earlier in the code result in a CUDA error or HIP error, the error may not be output at a nearby call to `AMREX_GPU_ERROR_CHECK()` by the CPU. When tracking down a CUDA launch error, `Gpu::synchronize()`, `Gpu::streamSynchronize()`, or `Gpu::streamSynchronizeAll()` can be used to synchronize the device, the current GPU stream, or all GPU streams, respectively, and track down the specific launch that causes the error. This error-checking macro will not return any information for SYCL.

18.11 Particle Support

As with `MultiFab`, particle data stored in AMReX `ParticleContainer` classes can be stored in GPU-accessible memory when AMReX is compiled with GPU support. The type of memory used by a given `ParticleContainer` can be controlled by the `Allocator` template parameter. By default, when compiled with GPU support

ParticleContainer uses The_Arena(). This means that the dataPtr associated with particle data can be passed into GPU kernels. These kernels can be launched with a variety of approaches, including AMReX's native kernel launching mechanisms as well OpenMP and OpenACC. Using AMReX's C++ syntax, a kernel launch involving particle data might look like:

```
for(MyParIter pti(pc, lev); pti.isValid(); ++pti)
{
    auto& ptile = pti.GetParticleTile();
    auto ptd = tile.getParticleTileData();
    const auto np = tile.numParticles();
    amrex::ParallelFor( np,
    [=] AMREX_GPU_DEVICE (const int ip) noexcept
    {
        ptd.id(i).make_invalid();
    });
}
```

The above code simply invalidates all particle on all particle tiles. The ParticleTileData object is analogous to Array4 in that it stores pointers to particle data and can be used on either the host or the device. This is a convenient way to pass particle data into GPU kernels because the same object can be used regardless of whether the data layout is AoS or SoA.

An example Fortran particle subroutine offloaded via OpenACC might look like the following:

```
subroutine push_position_boris(np, structs, uxp, uyp, uzp, gaminv, dt)

use em_particle_module, only : particle_t
use amrex_fort_module, only : amrex_real
implicit none

integer,          intent(in), value  :: np
type(particle_t), intent(inout)      :: structs(np)
real(amrex_real), intent(in)         :: uxp(np), uyp(np), uzp(np), gaminv(np)
real(amrex_real), intent(in), value  :: dt

integer                              :: ip

!$acc parallel deviceptr(structs, uxp, uyp, uzp, gaminv)
!$acc loop gang vector
do ip = 1, np
    structs(ip)%pos(1) = structs(ip)%pos(1) + uxp(ip)*gaminv(ip)*dt
    structs(ip)%pos(2) = structs(ip)%pos(2) + uyp(ip)*gaminv(ip)*dt
    structs(ip)%pos(3) = structs(ip)%pos(3) + uzp(ip)*gaminv(ip)*dt
end do
!$acc end loop
!$acc end parallel

end subroutine push_position_boris
```

Note the use of the *!$acc parallel deviceptr* clause to specify which data has been placed in device memory. This instructs OpenACC to treat those variables as if they already live on the device, bypassing the usual copies. For a complete example of a particle code that runs on GPUs, please see the ElectrostaticPIC tutorial.

GPU-aware implementations of many common particle operations are provided with AMReX, including neighbor list construction and traversal, particle-mesh deposition and interpolation, parallel reductions of particle data, and a set

of transformation and filtering operations that are useful when operating on sets of particles. For examples of these features in use, please see `Tests/Particles/`.

Finally, the parallel communication of particle data has been ported and optimized for performance on GPU platforms. This includes `Redistribute()`, which moves particles back to the proper grids after their positions have changed, as well as `fillNeighbors()` and `updateNeighbors()`, which are used to exchange halo particles. As with `MultiFab` data, these have been designed to minimize host / device traffic as much as possible, and can take advantage of the GPU-aware MPI implementations available on platforms such as ORNL's Frontier.

18.12 Profiling with GPUs

When profiling for GPUs, AMReX recommends `nvprof`, NVIDIA's visual profiler. `nvprof` returns data on how long each kernel launch lasted on the GPU, the number of threads and registers used, the occupancy of the GPU and recommendations for improving the code. For more information on how to use `nvprof`, see NVIDIA's User's Guide as well as the help web pages of your favorite supercomputing facility that uses NVIDIA GPUs.

AMReX's internal profilers do not hook into backend profiling information, but TinyProfiler can measure selected GPU scopes by synchronizing the current AMReX GPU stream at their timer boundaries.

Due to asynchronous GPU kernel launches, an ordinary host timer might finish before the work launched inside it. Use `BL_PROFILE_GPU_SYNC` (or its variable and region variants) when a TinyProfiler timer must include work launched on the current stream. With the full profiler (`PROFILE=TRUE`), these macros are ordinary, unsynchronized timers. For example:

```
BL_PROFILE_VAR_GPU_SYNC("A_NAME", blp); // Synchronize, then start timing
for (MFIter mfi(mf); mfi.isValid(); ++mfi)
{
    // gpu works
}
BL_PROFILE_VAR_STOP(blp);           // Synchronize, then stop timing
```

The synchronized macros add synchronization points that are unnecessary for correctness and may affect application performance. Once one executes, the TinyProfiler report focuses on synchronized timers, synchronized regions, and the outermost timer; see *GPU-synchronized TinyProfiler timers* for the complete behavior. The runtime parameter `tiny_profiler.device_synchronize_around_region` instead synchronizes every TinyProfiler timer boundary and keeps the complete report. For detailed kernel information, use a backend profiling tool.

18.13 Assigning GPUs to MPI Ranks

If each MPI rank sees only one GPU, AMReX uses it. If multiple GPUs are visible, AMReX picks one for each rank based on the rank's position within the node and, on some machines such as Perlmutter and Frontier, the machine's CPU/GPU affinity.

Some GPU-aware MPI libraries, such as Cray MPICH and Open MPI, set up GPU communication on the current GPU during MPI initialization. A process that switches to another GPU afterwards may get much slower GPU-aware MPI communication. Recent Cray MPICH versions report this as "This process is not using the same device it did during gtl_init". To avoid it, the GPU must be selected before MPI is initialized.

- If AMReX initializes MPI, it selects the GPU first, using the node-local rank provided by the job launcher (e.g., `SLURM_LOCALID` for `srun` and `OMPI_COMM_WORLD_LOCAL_RANK` for Open MPI's `mpirun`). Nothing needs to be done.
- If the application initializes MPI itself, it should call `amrex::Gpu::SelectDeviceBeforeMPIInit()` before `MPI_Init`. `amrex::Initialize` then uses the selected GPU.

```
int main (int argc, char* argv[])
{
    amrex::Gpu::SelectDeviceBeforeMPIInit();
    MPI_Init(&argc, &argv);
    amrex::Initialize(argc, argv, true, MPI_COMM_WORLD);
    ...
}
```

- Alternatively, make only one GPU visible to each rank before the program starts. Slurm's GPU binding options (e.g., `--gpus-per-task=1` and `--gpu-bind=closest`) do this and give each rank the GPU closest to its CPU cores. This is the recommended way to launch jobs on Frontier, for example. However, on systems where Slurm restricts device access with cgroups, these options can prevent GPU-aware MPI from transferring data directly between GPUs on the same node. On Perlmutter, this makes GPU-aware MPI hang, or run very slowly if the direct transfers are disabled. On such systems, use `--gpu-bind=none` and set `CUDA_VISIBLE_DEVICES` (CUDA) or `ROCR_VISIBLE_DEVICES` (HIP) in a wrapper script instead. On Perlmutter, with four ranks per node, the following wrapper script gives each rank the GPU closest to its CPU cores.

```
#!/bin/bash
# Usage: srun ... --cpu-bind=cores --gpu-bind=none ./wrapper.sh ./main3d.ex inputs
export CUDA_VISIBLE_DEVICES=$((3-SLURM_LOCALID))
exec "$@"
```

For HIP, set `ROCR_VISIBLE_DEVICES` instead. The order of the GPUs is machine dependent; check the system's documentation. For example, with eight ranks per node:

```
#!/bin/bash
# Usage: srun ... --gpu-bind=none ./wrapper.sh ./main3d.ex inputs
gpu_order=(4 5 2 3 6 7 0 1)  # machine dependent
export ROCR_VISIBLE_DEVICES=${gpu_order[$SLURM_LOCALID]}
exec "$@"
```

GPU-aware MPI communication is slower if a rank can see more than one AMD GPU, even when the GPU is selected before MPI initialization. On AMD systems, making only one GPU visible to each rank is therefore the preferred setup, whether or not AMReX initializes MPI.

18.14 Performance Tips

Here are some helpful performance tips to keep in mind when working with AMReX for GPUs:

- To obtain the best performance when using CUDA kernel launches, all device functions called within the launch region should be inlined. Inlined functions use substantially fewer registers, freeing up GPU resources to perform other tasks. This increases parallel performance and greatly reduces runtime. Functions are written inline by putting their definitions in the `.H` file and using the `AMREX_FORCE_INLINE` AMReX macro. For example:

```
AMREX_GPU_DEVICE
AMREX_FORCE_INLINE
void plusone_cudacpp (amrex::Box const& bx, amrex::FArrayBox& fab)
{
    ...
}
```

- Pay attention to what GPUs your job scheduler is assigning to each MPI rank. In most cases you'll achieve the best performance when a single MPI rank is assigned to each GPU, and has boxes large enough to saturate that GPU's compute capacity. While there are some cases where multiple MPI ranks per GPU can make sense

(typically this would be when you have some portion of your code that is not GPU accelerated and want to have many MPI ranks to make that part faster), this is probably the minority of cases. For example, on OLCF Summit you would want to ensure that your resource sets contain one MPI rank and GPU each, using *jsrun -n N -a 1 -c 7 -g 1*, where *N* is the total number of MPI ranks/GPUs you want to use. (See the OLCF [job step viewer](https://jobstepviewer.olcf.ornl.gov/) for more information.)

Conversely, if you choose to have multiple GPUs visible to each MPI rank, AMReX will attempt to do the best job it can assigning MPI ranks to GPUs by doing round robin assignment. This may be suboptimal because this assignment scheme would not be aware of locality benefits that come from having an MPI rank be on the same socket as the GPU it is managing. See *Assigning GPUs to MPI Ranks* for how this interacts with GPU-aware MPI.

18.15 Inputs Parameters

The following inputs parameters control the behavior of amrex when running on GPUs. They should be prefaced by "amrex" in your `inputs` file.

	Description	Type	Default
use_gpu_a	Whether to use GPU memory for communication buffers during MPI calls. If true, the buffers will use device memory. If false (i.e., 0), they will use pinned memory. It will be activated if AMReX detects that GPU-aware MPI is supported by the MPI library (MPICH, OpenMPI, and derivative implementations).	Bool	MPI-dependent
fb.on_arri	With GPU-aware MPI, `FillBoundary` and `SumBoundary` unpack each message as soon as it arrives when a rank receives at least this many bytes in one call, overlapping the unpacking with the remaining communication. With fewer bytes, the messages are unpacked together after all of them have arrived.	Long	65536
abort_on_	If the size of free memory on the GPU is less than the size of a requested allocation, AMReX will throw amrex::OutOfMemoryError with an error describing how much free memory there is and what was requested.	Bool	0
the_arena_	Whether `The_Arena()` allocates managed memory.	Bool	0

CHAPTER

NINETEEN

VISUALIZATION

There are several visualization tools that can be used for AMReX plotfiles. The standard tool used within the AMReX community is Amrvis, a package developed and supported by CCSE that is designed specifically for highly efficient visualization of block-structured hierarchical AMR data. Plotfiles can also be viewed using VisIt, ParaView, and yt. Particle data can be viewed using ParaView.

19.1 Amrvis

Our favorite visualization tool is Amrvis. We heartily encourage you to build the `amrvis1d`, `amrvis2d`, and `amrvis3d` executables, and use them to visualize your data. A useful feature is `View/Dataset`, which allows you to view data in a nested spreadsheet that reflects the AMR hierarchy – this can be handy for debugging. Other display options include: the ability to select the number of levels of data to show, whether to display grid boxes, and to specify the color palette. Below are instructions and tips for using Amrvis. Additional information is contained in the document `Amrvis/Docs/Amrvis.tex` (which can be built into a `pdf` using `pdflatex`).

1. **Download and Build**:

   Amrvis is available for download from the `AMReX-Codes/Amrvis` GitHub repository. To download, use

   ```
   git clone https://github.com/AMReX-Codes/Amrvis
   ```

   To build, `cd` into `Amrvis/`, and edit `GNUmakefile` by setting the variable `COMP` to your compiler suite.

   Type `make DIM=1`, `make DIM=2`, or `make DIM=3` to build. The result is an executable that looks like `amrvis2d.<ver>.ex`.

   3D Data Visualization with Volpack

   If you want to build Amrvis with `DIM=3` for display of 3-dimensional data, you must first download and build `volpack`. This can be done by cloning the repository or via package manager. To install by cloning the repository:

   ```
   git clone https://ccse.lbl.gov/pub/Downloads/volpack.git
   ```

   After downloading, `cd` into `volpack/` and type `make`.

   To install via package manager, it is necessary to install the package, `libvolpack1-dev`. This package is available for Debian Linux and can be installed with the command:

   ```
   sudo apt install libvolpack1-dev
   ```

Note

Amrvis requires the OSF/Motif libraries and headers. If you don't have these you will need to install the development version of motif through your package manager. `lesstif` gives some functionality and will allow you to build the Amrvis executable, but Amrvis may exhibit subtle anomalies.

On most Linux distributions, the motif library is provided by the `openmotif` package, and its header files (like `Xm.h`) are provided by `openmotif-devel`. If those packages are not installed, then use the OS-specific package management tool to install them.

Note

These instructions assume that the install directories for Amrvis and volpack share the same parent directory. To install volpack in a different location specify the location of volpack in Amrvis's `GNUmakefile` by changing the variable `VOLPACKDIR` to the desired location.

After building you may want to create an alias for convenience. To do this type,

```
alias amrvis2d /tmp/Amrvis/amrvis2d.<ver>.ex
```

2. **Configure**:

   The settings for Amrvis are saved in the configuration file `.amrvis.defaults` in your home directory. A default version of this file is available in the parent directory of the Amrvis repo. Run the command `cp Amrvis/amrvis.defaults ~/.amrvis.defaults` to copy it to your home directory. A color palette is also available in the Amrvis directory as a file named `Palette`. To configure Amrvis to use this palette you can open the `.amrvis.defaults` file in your home directory and edit the line containing `palette` to point to the location of this file. For example,

   ```
   palette     ~/Amrvis/Palette
   ```

   Other lines in `.amrvis.defaults` control options such as the initial field to display, the number format, window size, etc. If there are multiple instances of the same option, the last option takes precedence.

3. **Run**:

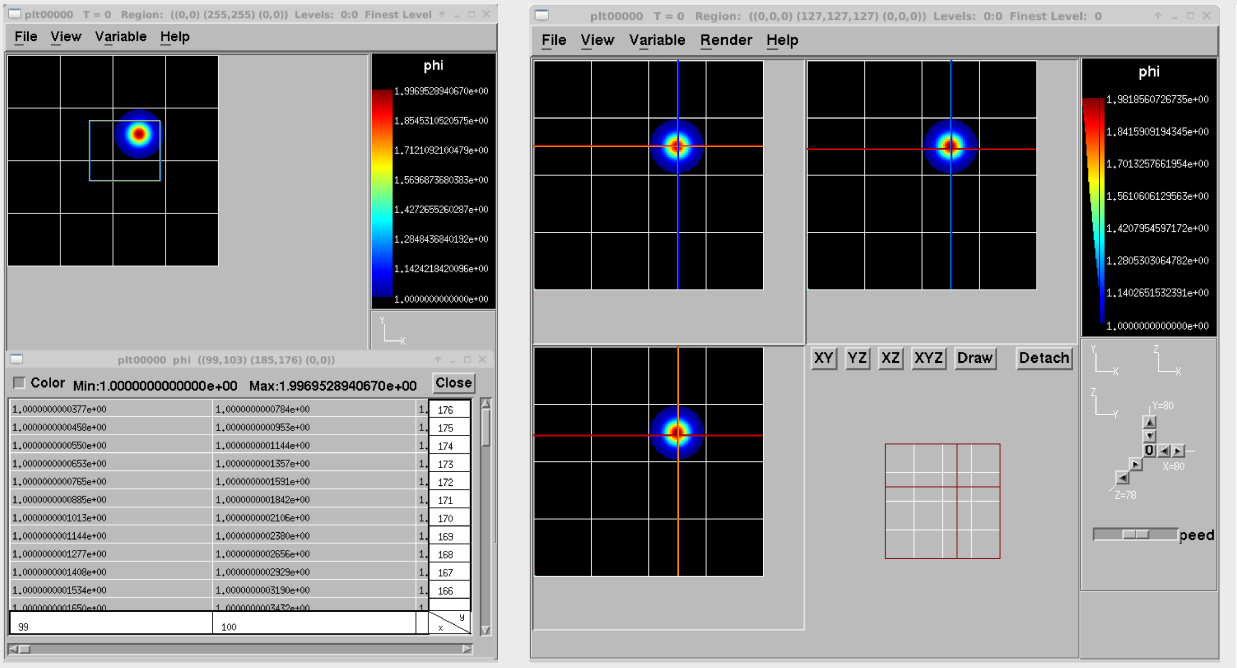
   By default, the plotfiles are directories that have the form pltXXXXX, where XXXXX is a number corresponding to the timestep that the file was created. Use `amrvis2d <filename>` or `amrvis3d <filename>` to see a single

plotfile, or for 2D data sets, `amrvis2d -a plt*`, which will animate the sequence of plotfiles. FArrayBoxes and MultiFabs can also be viewed with the `-fab` and `-mf` options. When opening MultiFabs, use the name of the MultiFab's header file `amrvis2d -mf MyMultiFab_H`.

You can use the "Variable" menu to change the variable. You can left-click drag a box around a region and click "View" → "Dataset" in order to look at the actual numerical values (see Table 19.1). Or you can simply left click on a point to obtain the numerical value. You can also export the pictures in several different formats under `File/Export`. In 2D you can right or center click to get line-out plots. In 3D you can right or center click to change the planes, and hold shift+(right or center) click to get line-out plots.

We have created a number of routines to convert AMReX plotfile data to other formats (such as Matlab), but in order to properly interpret the hierarchical AMR data, each tends to require its own idiosyncrasies. If you would like to display the data in another format, please leave a message on AMReX's GitHub Discussions page.

Table 19.1: . 2D and 3D images generated using Amrvis.



19.1.1 Building Amrvis on macOS

As previously outlined at the end of section *Building with GNU Make*, it is recommended to build using the homebrew package manager to install gcc. Furthermore, you will also need x11 and openmotif. These can be installed using homebrew also:

1. `brew cask install xquartz`
2. `brew install openmotif`

Note that when the `GNUmakefile` detects a macOS install, it assumes that dependencies are installed in the locations that Homebrew uses. Namely the `/usr/local/` tree for regular dependencies and the `/opt/` tree for X11.

19.2 VisIt

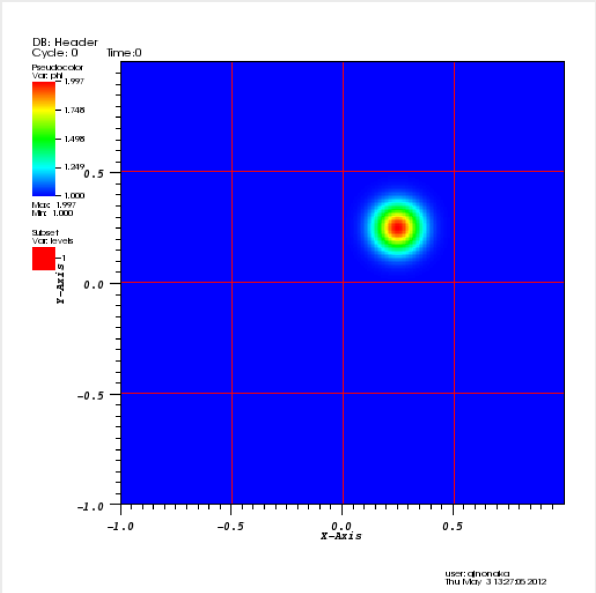
AMReX data can also be visualized by VisIt, an open source visualization and analysis software. To follow along with this example, first build and run the heat equation example (see the section on *Example: Heat Equation Solver*).

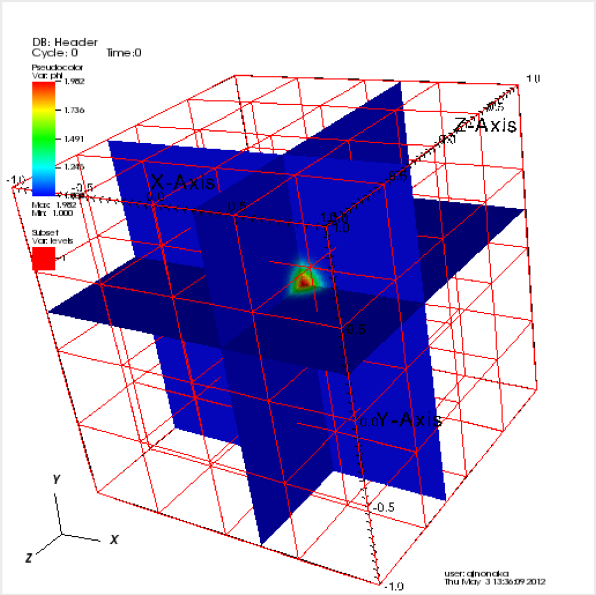
Next, download and install VisIt from https://wci.llnl.gov/simulation/computer-codes/visit. To open a single plotfile, run VisIt, then select "File" → "Open file …", then select the Header file associated with the plotfile of interest (e.g., `plt00000/Header`). Assuming you ran the simulation in 2D, here are instructions for making a simple plot:

- To view the data, select "Add" → "Pseudocolor" → "phi", and then select "Draw".
- To view the grid structure (not particularly interesting yet, but when we add AMR it will be), select "Add" → "Subset" → "levels". Then double-click the text "Subset - levels", enable the "Wireframe" option, select "Apply", select "Dismiss", and then select "Draw".
- To save the image, select "File" → "Set save options", then customize the image format to your liking, then click "Save".

Table 19.2 shows images made this way from an older heat equation example, so your images will look different.

Table 19.2: : 2D (left) and 3D (right) images generated using VisIt.





In 3D, you must apply the "Operators" → "Slicing" → "ThreeSlice", with the "ThreeSlice operator attribute" set to the point where the slices intersect, e.g., `x=0.25`, `y=0.25`, and `z=0.25`. You can left-click and drag over the image to rotate it, as in the right side of Table 19.2.

To make a movie, you must first create a text file named `movie.visit` with a list of the Header files for the individual frames. This can most easily be done using the command:

```
~/amrex-tutorials/Base/HeatEquation> ls -1v plt*/Header | tee movie.visit
plt00000/Header
plt00010/Header
plt00020/Header
plt00030/Header
plt00040/Header
```

The next step is to run VisIt, select "File" → "Open file…", then select movie.visit. Create an image to your liking and press the "play" button on the VCR-like control panel to preview all the frames. To save the movie, choose "File" → "Save movie …", and follow the on-screen instructions.

Warning

The VisIt reader determines the value of `Cycle` from the name of the plotfile (directory), specifically from the integer that follows the string "plt" in the plotfile name. So if you call it `plt00100`, `myplt00100` or `this_is_my_plt00100` then it will correctly recognize and print `Cycle: 100`. If you call it `plt00100_old` it will also correctly recognize and print `Cycle: 100`.

However, if you do not have `plt` followed immediately by the number, e.g. you name it `pltx00100`, then VisIt will not be able to correctly recognize and print the value for `Cycle`. (It will still read and display the data itself.)

19.2.1 VisIt HDF5 Format

The plotfiles generated with the HDF5 format can be visualized by VisIt as well. To open a single plotfile, run VisIt, then select "File" → "Open file …", then select the HDF5 plotfile of interest (e.g.,``plt00000.h5``), and select "Chombo" in the "Open file as type" dropdown menu. VisIt can also recognize the time steps automatically based on the numbers in the HDF5 plotfile names in a directory.

19.3 ParaView

The open source visualization package ParaView v5.7 and later can be used to view 2D and 3D plotfiles, as well as particle data. Download the package at https://www.paraview.org/.

To open a plotfile (for example, you could run the `HeatEquation` tutorial in 3D with `plot_int=100 stop_time=0.15`):

1. Run ParaView v5.7, then select "File" → "Open".
2. Navigate to your run directory, and select the fluid or particle plotfile. Note that you can either open single or multiple plotfiles at once by selecting them one by one or select a file ensemble, labeled as `plt..` and indicated as a Group in the "Type" column of the file explorer (see Fig. 19.3). In the latter case, ParaView will load the plotfiles as a time series. ParaView will ask you about the file type – choose "AMReX/BoxLib Grid Reader" or "AMReX/BoxLib Particles Reader". Note that if your plotfile prefix is not `plt` or any other type supported by default, then in `Files of type` you need to first select `All files (*)`.
3. Under the "Cell Arrays" field, select a variable (e.g., "phi") and click "Apply". Note that the default number of refinement levels loaded and visualized is 1. Change to the required number of AMR levels before clicking "Apply".
4. Under "Representation" select "Surface".
5. Under "Coloring" select the variable you chose above.
6. To add planes, near the top left you will see a cube icon with a green plane slicing through it. If you hover your mouse over it, it will say "Slice". Click that button.
7. You can play with the Plane Parameters to define a plane of data to view, as shown in Fig. 19.1.

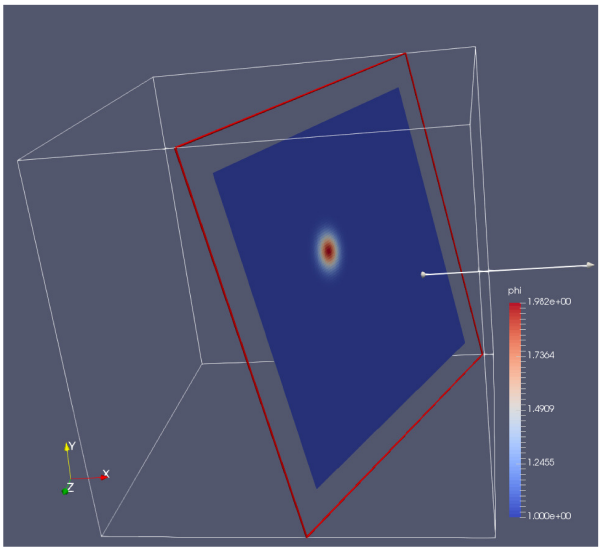


Fig. 19.1: : Plotfile image generated with ParaView

19.3.1 Creating and Loading `.series` Files

Another useful feature in ParaView for loading and reloading a group of plotfiles is using a `.series` file (similar to the `.visit` file in VisIt). It is a text file (say `plot_files.series`) which lists the plotfiles in JSON format, as shown below.

```
{ "file-series-version": "1.0", "files": [
{ "name": "plt00000", "time": 0},
{ "name": "plt00100", "time": 1},
{ "name": "plt00200", "time": 2},
{ "name": "plt00300", "time": 3},
{ "name": "plt00400", "time": 4},
{ "name": "plt00500", "time": 5},
{ "name": "plt00600", "time": 6},
{ "name": "plt00700", "time": 7},
{ "name": "plt00800", "time": 8},
{ "name": "plt00900", "time": 9},
{ "name": "plt01000", "time": 10},] }
```

`write_series_file.sh` is a bash script that can generate such a `.series` file. Navigate to the directory with the plotfiles and save this script. Then run the bash script by executing the following command in the terminal.

```
bash write_series_file.sh
```

This will generate a file `plot_files.series` that indexes the time variable based on the order of the plotfile numbers. Note that if your plotfile prefix is not `plt`, you can manually edit `write_series_file.sh` accordingly.

To make a `.series` file which reads the time out of the plotfile header, use `write_series_file_timestamp.sh`.

Open ParaView, and then select "File" → "Open". In the "Files of Type" dropdown menu (see Fig. 19.3) choose the option `All Files (*)`. Then choose `plot_files.series` and click "OK". Now the plotfiles have been loaded as a Group as in Step 2 of section *ParaView*. Now, you can follow the steps 2 to 7 in the section *ParaView* to plot. As new plotfiles are generated, just re-run the bash script to re-generate the `plot_files.series` file, right-click on `plot_files.series` in the ParaView menu, and click on "Reload Files" (see Fig. 19.2).

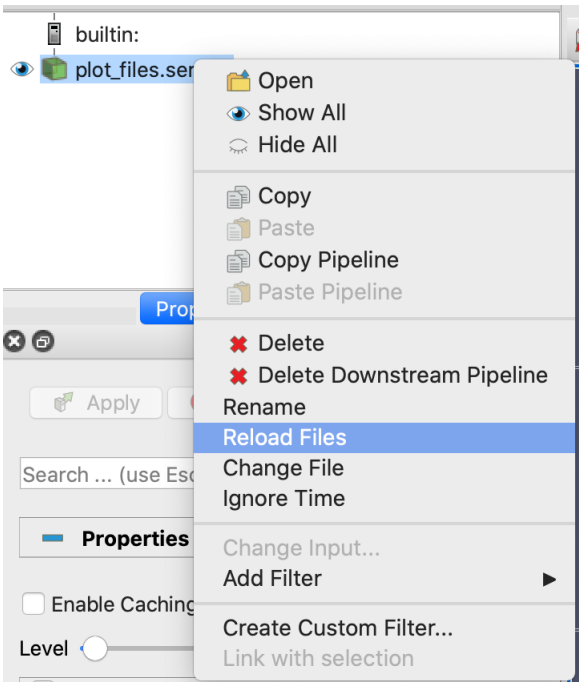


Fig. 19.2: : File dialog in ParaView showing how to reload a series file

19.3.2 Building an Iso-surface

Note that ParaView is not able to generate iso-surfaces from cell-centered data. To build an iso-surface (or iso-line in 2D):

1. Perform a cell to node interpolation: "Filters" → "Alphabetical" → "Cell Data to Point Data".
2. Use the "Contour" icon (next to the calculator) to select the data from which to build the contour ("Contour by"), enters the iso-surfaces values and click "Apply".

19.3.3 Visualizing Particle Data

To visualize particle data within plotfile directories (for example, you could run the NeighborList example in the Particles tutorials):

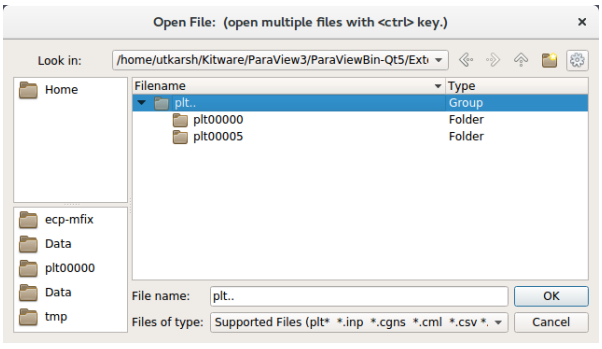


Fig. 19.3: : File dialog in ParaView showing a group of plotfile directories selected

1. Run ParaView v5.7, and select "File" → "Open". You will see a combined "plt.." group. Click on "+" to expand the group, if you want to inspect the files in the group. You can select an individual plotfile directory or select a group of directories to read them as a time series, as shown in Fig. 19.3, and click "OK". ParaView will ask you about the file type – choose "AMReX/BoxLib Particles Reader".
2. The "Properties" panel in ParaView allows you to specify the "Particle Type", which defaults to "particles". Using the "Properties" panel, you can also choose which point arrays to read.
3. Click "Apply" and under "Representation" select "Point Gaussian".
4. Change the Gaussian Radius if you like. You can scroll through the frames with the VCR-like controls at the top, as shown in Fig. 19.4.

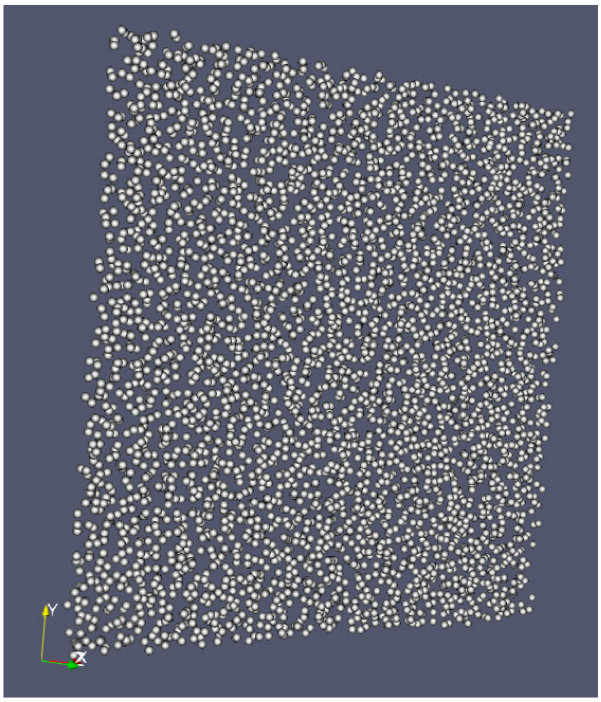


Fig. 19.4: : Particle image generated with ParaView

Following these instructions, you can open fluid and/or particle plotfiles and visualize them together on the same panel view.

Once you have loaded an AMReX plotfile time series (fluid and/or particles), you can generate a movie following these instructions:

1. "File" → "Save Animation…".
2. Enter a file name, select ".avi" as the Type of File and click "OK".
3. Adjust the resolution, compression and framerate, and click "OK"

19.3.4 Plot a Vector Field

ParaView can be used to plot a vector field from AMR plotfile data. In this example we will assume a single vector has been stored as three separate variables, `V_x`, `V_y` and `V_z`.

The easiest way to create a vector `V` from the components is to first download the Python file `makevector.py`.

If you are running ParaView remotely, put `makevector.py` in the location of the plotfiles you will open under the heading `Remote Plugins`.

1. Now open a plotfile or plotfile group, using `File` → `Open`. A pop-up will appear; select "AMReX/BoxLib Grid Reader".
2. Select the plotfile or group in the Pipeline Browser. The Cell Array Status window of the Properties should populate with the values `V_x`, `V_y` and `V_z`. Select these values and click apply.
3. Select `Tools` → `Manage Plugins...` then choose `Load New...`. Select `makevector.py`; after you load it you will see `makevector` in the list of plugins.
4. Select the MakeVector filter from `Filters` → `Alphabetical` → `MakeVector` and apply. You will now have the vector `V` listed with your other variables.

Alternatively, if you prefer not to use the python plugin, you can follow the steps below:

1. Select the Cell Centers filter from `Filters` → `Alphabetical` → `Cell Centers` and apply.
2. Next we'll define a vector variable using the Calculator filter. Select `Filters` → `Alphabetical` → `Calculator`. Under the Properties heading, set the Attribute Type to Point Data. The Result Array Name is the name of the vector value we will create. In the line below that we define a new vector value with the equation: `V_x*iHat + V_y*jHat + V_z*kHat` Note that, the values `V_x`, `V_y` and `V_z`, should be selectable from the dropdown Scalars menu. Apply the filter.

To plot the arrows corresponding to this vector field

1. Select the Glyph filter, `Filters` → `Alphabetical` → `Glyph`. Under the heading, Glyph Source, select `Arrow`. Under Orientation, select the name of the vector value created in the last step. The default name is `Result`. Apply the filter to display the vector field.

   One may want to adjust the appearance of the vector field by scaling each vector by its magnitude. To do this, look under the Scale heading, select the vector value as the Scale Array and select Scale by Magnitude.

   To adjust the number and location of vectors displayed, one may alter the settings under the Masking heading.

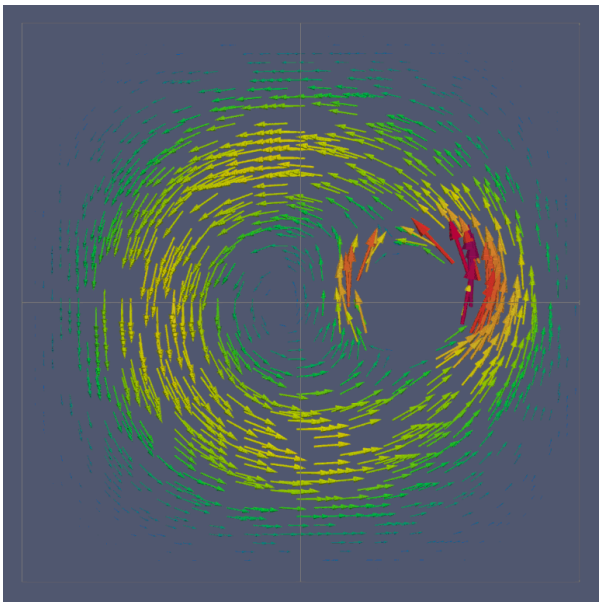


Fig. 19.5: Vector Field generated with ParaView

19.3.5 Saving and Loading State Files

ParaView allows users to save the *state* of their visualization, viz. variable mappings, filters, color maps, etc. The same display style can then be applied to different datasets. See also the ParaView documentation at section 10 and section 8.4.

There are two file formats available for saving and loading state: `.pvsm` (XML) and `.py` (Python). Examples of both methods are given below. For these examples, we will be working with the data stored in the folder `plt..` that results from running the `HeatEquation` tutorial in 3D, as in the example at the beginning of this section.

To save the state of this example, go to `File > Save State...`, and select the format, location, and name of the state file you want to save. In order to later reload the state you've saved, see below.

Note that the `Save State` option will write state files containing the absolute path to the data files you have loaded. This can cause issues if you're using state files saved on another machine, have moved your data files, or if ParaView doesn't know which directory to look in. When loading state from a `pvsm` file, there are menu options available to navigate to the data source directory. When running a Python script to load data, this can cause an error or crash. We outline the way to avoid this below.

For demonstration purposes, we have also included state files in both formats: `slice_state1.pvsm` and `slice_state1.py`

Loading from a ParaView state file (.pvsm)

1. Go to `File > Load State`, navigate to `slice_state1.pvsm` in the file browser and click `OK`. A new window labeled `Load State Options` appears with a drop-down menu.
2. Select `Search files under specified directory`, and click the `...` button to the right of the `Data Directory` line.
3. Navigate to `amrex-tutorials/Base/HeatEquation` and click `OK`. This is where the plotfiles will have been saved if you've built and run the `HeatEquation` tutorial there.

What you see displayed should be a 2D slice of the solution to the 3D equation.

In general, you can use the load state menu options to navigate to whichever data files you wish to load using your saved `pvsm` state.

Loading from a Python state file (.py)

In order to load state from a `.py` file, you must execute the file as a script from the Python shell within ParaView.

1. If the Python Shell is not displayed, click the checkbox in `View > Python Shell`.
2. Click the `Run Script` button at the bottom right of the Python shell, navigate to the file `amrex/Docs/sphinx_documentation/source/Visualization/slice_state1.py` in the file navigator, and click `OK`.

You should see a 2D slice of the solution to the 3D heat equation. If ParaView reports an error or crashes, see below.

Aside: Working directory in the ParaView Python shell

When you load a state from a Python script in ParaView, it will look in the current working directory of the Python shell to resolve the paths to the data files provided in the Python script.

By default, the current working directory of the ParaView Python shell will be the directory from which you launched ParaView.

Warning

If your Python script makes direct reference to a set of files that can't be found from your current working directory, then running that script will result in an error, and potentially cause ParaView to crash. This can be addressed by changing the cwd of your Python shell to the proper location.

To check the cwd of the ParaView Python shell, run

```
>>> import os
>>> os.getcwd()
```

and make sure that your cwd contains the folder you want to search in so that Python can resolve the path. To change the cwd, run

```
>>> os.chdir('path/to/folder/containing/your/plotfiles')
```

Modifying a State File to Work with Different Data

As noted above, a Python state file exported from ParaView will save the path to the data files you used, if any, from your working directory. In order to use such a file with different data, you can modify your Python state file to use the glob package to create a list of potential data file names to search in your current path. The steps are outlined below, including the line numbers in the file `slice_state1.py`.

1. Import the glob package

   ```
   4: import glob
   ```

2. Create a variable that will hold the list of names to search for. Following the AMReX convention for naming plotfiles, use

   ```
   62: PlotFiles = sorted(glob.glob("plt" + "[0-9]" * 5))
   ```

3. In the code section that builds an `AMReX/BoxLib Grid Reader`, replace the list of data paths with the variable `PlotFiles`

   ```
   65: plt00000 = AMReXBoxLibGridReader(registrationName="plt00000*",␣
   ↪FileNames=PlotFiles)
   ```

4. Change the cwd of your Python shell, as outlined above, to a directory containing the other plotfiles you wish to view.

5. Click `Run Script` and navigate to the Python state file you wish to run in the pop-up file navigator.

You should now be able to view a new data set with the same filters, color mappings, etc. that you saved to your state file.

19.3.6 ParaView HDF5 Format

The plotfiles generated with the HDF5 format can be visualized by ParaView. To open a single plotfile, run ParaView, select "File" → "Open", then select the HDF5 plotfile (e.g., `plt00000.h5`). You can select an individual plotfile or select a group of files to read as a time series, then click "OK". ParaView will ask you about the file type – choose "VisItChomboReader".

19.4 yt

yt, an open source Python package available at http://yt-project.org/, can be used for analyzing and visualizing mesh and particle data generated by AMReX codes. Some of the AMReX developers are also yt project members. Below we describe how to use yt on both a local workstation and at the NERSC HPC facility for high-throughput visualization of large data sets.

Note - AMReX datasets require yt version 3.4 or greater.

We also note that there is active development of an xarray-like interface for AMReX simulation data via yt; see the xamr docs for more details.

19.4.1 Using on a local workstation

Running yt on a local system generally provides good interactivity, but limited performance. Consequently, this configuration is best when doing exploratory visualization (e.g., experimenting with camera angles, lighting, and color schemes) of small data sets.

To use yt on an AMReX plot file, first start a Jupyter notebook or an IPython kernel, and import the `yt` module:

```
In [1]: import yt

In [2]: print(yt.__version__)
3.4-dev
```

Next, load a plot file; in this example we use a plot file from the Nyx cosmology application:

```
In [3]: ds = yt.load("plt00401")
yt : [INFO     ] 2017-05-23 10:03:56,182 Parameters: current_time              = 0.
↪00605694344696544
yt : [INFO     ] 2017-05-23 10:03:56,182 Parameters: domain_dimensions         = [128␣
↪128 128]
yt : [INFO     ] 2017-05-23 10:03:56,182 Parameters: domain_left_edge          = [ 0.  0.
↪  0.]
yt : [INFO     ] 2017-05-23 10:03:56,183 Parameters: domain_right_edge         = [ 14.
↪24501  14.24501  14.24501]

In [4]: ds.field_list
Out[4]:
[('DM', 'particle_mass'),
 ('DM', 'particle_position_x'),
 ('DM', 'particle_position_y'),
 ('DM', 'particle_position_z'),
 ('DM', 'particle_velocity_x'),
 ('DM', 'particle_velocity_y'),
 ('DM', 'particle_velocity_z'),
 ('all', 'particle_mass'),
 ('all', 'particle_position_x'),
 ('all', 'particle_position_y'),
 ('all', 'particle_position_z'),
 ('all', 'particle_velocity_x'),
 ('all', 'particle_velocity_y'),
 ('all', 'particle_velocity_z'),
 ('boxlib', 'density'),
 ('boxlib', 'particle_mass_density')]
```

From here one can make slice plots, 3-D volume renderings, etc. An example of the slice plot feature is shown below:

```
In [9]: slc = yt.SlicePlot(ds, "z", "density")
yt : [INFO     ] 2017-05-23 10:08:25,358 xlim = 0.000000 14.245010
yt : [INFO     ] 2017-05-23 10:08:25,358 ylim = 0.000000 14.245010
yt : [INFO     ] 2017-05-23 10:08:25,359 xlim = 0.000000 14.245010
yt : [INFO     ] 2017-05-23 10:08:25,359 ylim = 0.000000 14.245010

In [10]: slc.show()

In [11]: slc.save()
yt : [INFO     ] 2017-05-23 10:08:34,021 Saving plot plt00401_Slice_z_density.png
Out[11]: ['plt00401_Slice_z_density.png']
```

The resulting image is Fig. 19.6. One can also make volume renderings with ; an example is shown below:

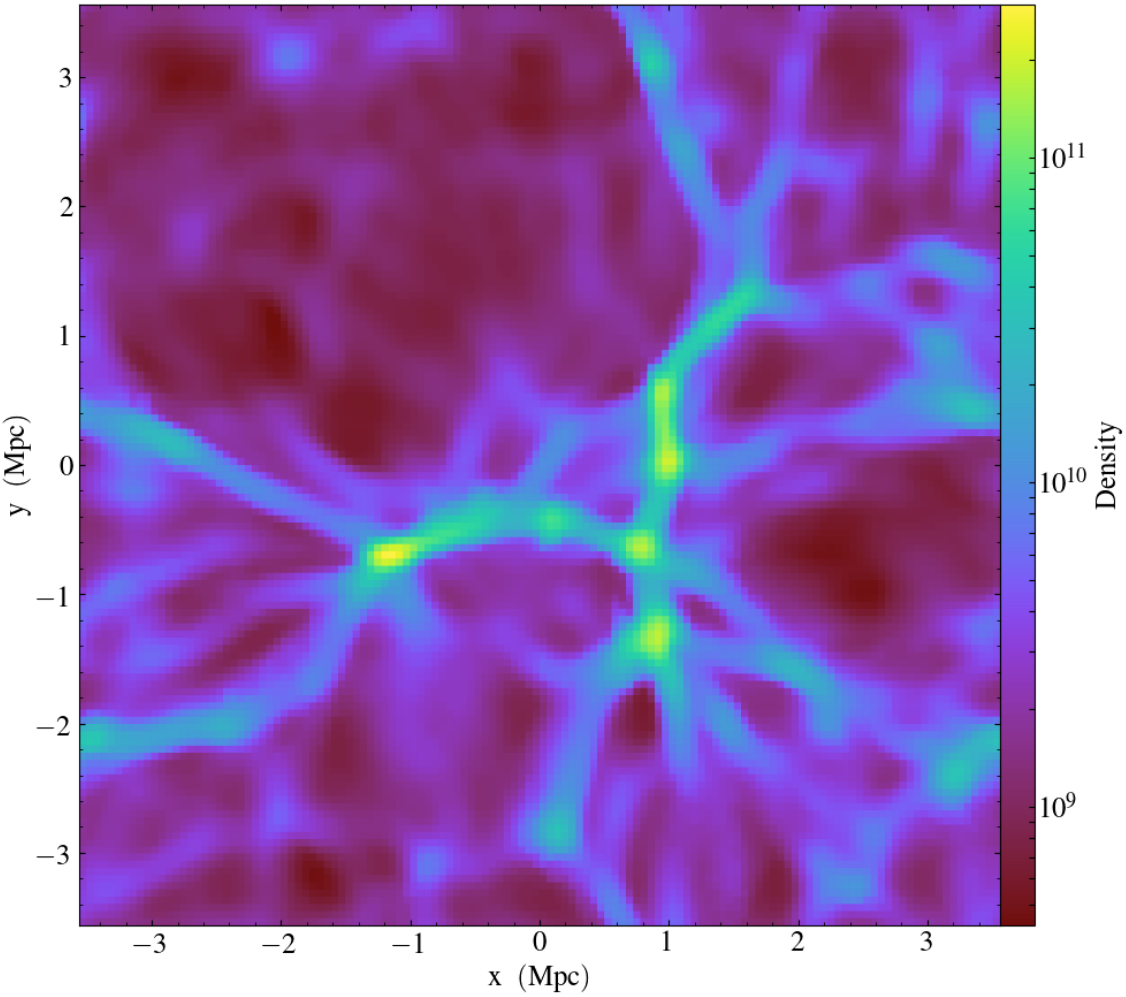


Fig. 19.6: : Slice plot of 128^3 Nyx simulation using yt.

```
In [12]: sc = yt.create_scene(ds, field="density", lens_type="perspective")
```

(continues on next page)

(continued from previous page)

```
In [13]: source = sc[0]

In [14]: source.tfh.set_bounds((1e8, 1e15))

In [15]: source.tfh.set_log(True)

In [16]: source.tfh.grey_opacity = True

In [17]: sc.show()
<Scene Object>:
Sources:
    source_00: <Volume Source>:YTRegion (plt00401): , center=[  1.09888770e+25   1.
↪09888770e+25   1.09888770e+25] cm, left_edge=[ 0.  0.  0.] cm, right_edge=[  2.
↪19777540e+25   2.19777540e+25   2.19777540e+25] cm transfer_function:None
Camera:
    <Camera Object>:
    position:[ 14.24501  14.24501  14.24501] code_length
    focus:[ 7.122505  7.122505  7.122505] code_length
    north_vector:[ 0.81649658 -0.40824829 -0.40824829]
    width:[ 21.367515  21.367515  21.367515] code_length
    light:None
    resolution:(512, 512)
Lens: <Lens Object>:
    lens_type:perspective
    viewpoint:[ 0.95423473  0.95423473  0.95423473] code_length

In [19]: sc.save()
yt : [INFO     ] 2017-05-23 10:15:07,825 Rendering scene (Can take a while).
yt : [INFO     ] 2017-05-23 10:15:07,825 Creating volume
yt : [INFO     ] 2017-05-23 10:15:07,996 Creating transfer function
yt : [INFO     ] 2017-05-23 10:15:07,997 Calculating data bounds. This may take a while.
Set the TransferFunctionHelper.bounds to avoid this.
yt : [INFO     ] 2017-05-23 10:15:16,471 Saving render plt00401_Render_density.png
```

The output of this is Fig. 19.7.

19.4.2 Using yt at NERSC (*under development*)

Because yt is Python-based, it is portable and can be used in many software environments. Here we focus on yt's capabilities at NERSC, which provides resources for performing both interactive and batch queue-based visualization and analysis of AMReX data. Coupled with yt's MPI and OpenMP parallelization capabilities, this can enable high-throughput visualization and analysis workflows.

Interactive yt with Jupyter notebooks

Unlike VisIt (see the section on *VisIt*), yt has no client-server interface. Such an interface is often crucial when one has large data sets generated on a remote system, but wishes to visualize the data on a local workstation. Both copying the data between the two systems, as well as visualizing the data itself on a workstation, can be prohibitively slow.

Fortunately, NERSC has implemented several resources which allow one to interact with yt remotely, emulating a client-server model. In particular, NERSC now hosts Jupyter notebooks which run IPython kernels on the Cori system; this provides users access to the `$HOME`, `/project`, and `$SCRATCH` file systems from a web browser-based Jupyter

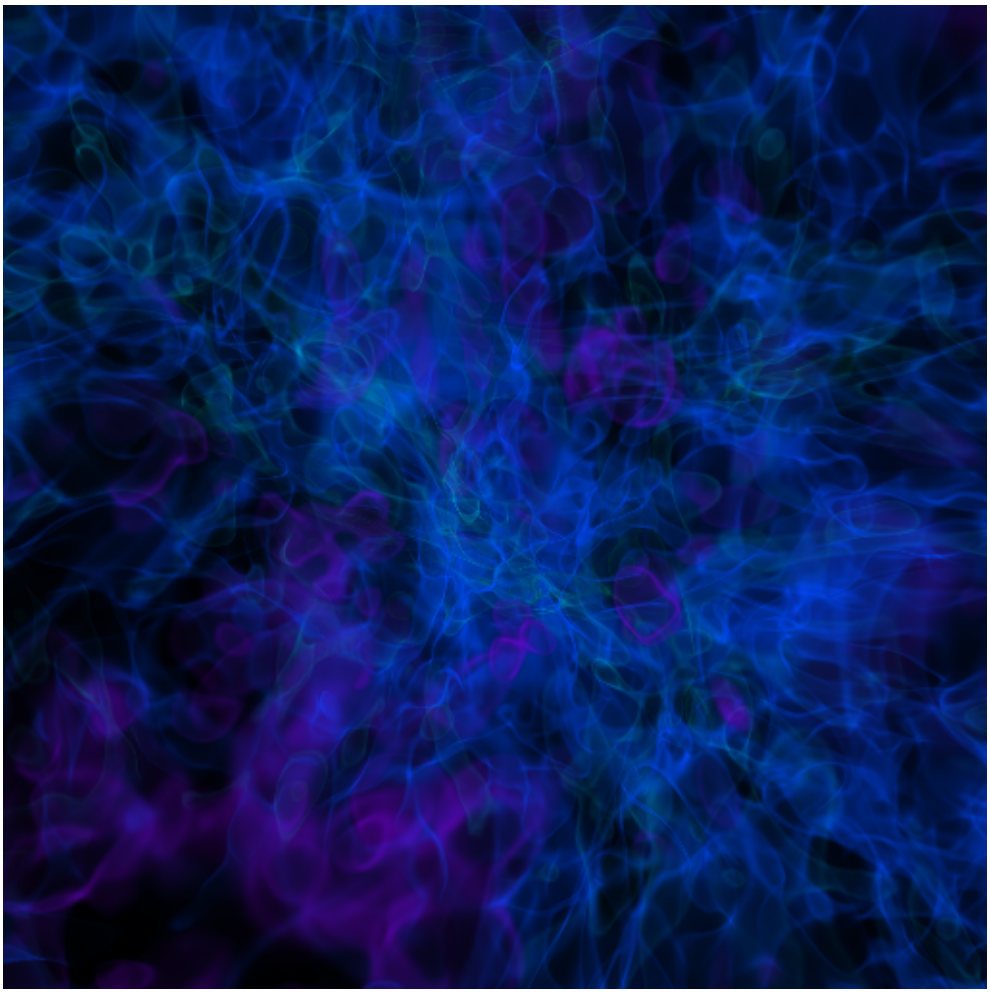


Fig. 19.7: Volume rendering of 128^3 Nyx simulation using yt. This corresponds to the same plot file used to generate the slice plot in Fig. 19.6.

notebook. ***Please note that Jupyter hosting at NERSC is still under development, and the environment may change without notice.***

NERSC also provides Anaconda Python, which allows users to create their own customizable Python environments. It is recommended to install yt in such an environment. One can do so with the following example:

```
user@cori10:~> module load python/3.5-anaconda
user@cori10:~> conda create -p $HOME/yt-conda numpy
user@cori10:~> source activate $HOME/yt-conda
(/global/homes/u/user/yt-conda/) user@cori10:~> pip install yt
```

More information about Anaconda Python at NERSC is here: http://www.nersc.gov/users/data-analytics/data-analytics/python/anaconda-python/.

One can then configure this Anaconda environment to run in a Jupyter notebook hosted on the Cori system. Currently this is available in two places: on https://ipython.nersc.gov, and on https://jupyter-dev.nersc.gov. The latter likely reflects what the stable, production environment for Jupyter notebooks will look like at NERSC, but it is still under development and subject to change. To load this custom Python kernel in a Jupyter notebook, follow the instructions at this URL under the "Custom Kernels" heading: http://www.nersc.gov/users/data-analytics/data-analytics/web-applications-for-data-analytics. After writing the appropriate `kernel.json` file, the custom kernel will appear as an available Jupyter notebook. Then one can interactively visualize AMReX plot files in the web browser.[1]

Parallel

Besides the benefit of no longer needing to move data back and forth between NERSC and one's local workstation to do visualization and analysis, an additional feature of yt which takes advantage of the computational resources at NERSC is its parallelization capabilities. yt supports both MPI- and OpenMP-based parallelization of various tasks, which are discussed here: http://yt-project.org/doc/analyzing/parallel_computation.html.

Configuring yt for MPI parallelization at NERSC is a more complex task than discussed in the official yt documentation; the command `pip install mpi4py` is not sufficient. Rather, one must compile `mpi4py` from source using the Cray compiler wrappers `cc`, `CC`, and `ftn` on Cori. Instructions for compiling `mpi4py` at NERSC are provided here: http://www.nersc.gov/users/data-analytics/data-analytics/python/anaconda-python/#toc-anchor-3. After `mpi4py` has been compiled, one can use the regular Python interpreter in the Anaconda environment as normal; when executing yt operations which support MPI parallelization, the multiple MPI processes will spawn automatically.

Although several components of yt support MPI parallelization, a few are particularly useful:

- **Time series analysis.** Often one runs a simulation for many time steps and periodically writes plot files to disk for visualization and post-processing. yt supports parallelization over time series data via the `DatasetSeries` object. yt can iterate over a `DatasetSeries` in parallel, with different MPI processes operating on different elements of the series. This page provides more documentation: http://yt-project.org/doc/analyzing/time_series_analysis.html#time-series-analysis.
- **Volume rendering**. yt implements spatial decomposition among MPI processes for volume rendering procedures, which can be computationally expensive. Note that yt also implements OpenMP parallelization in volume rendering, and so one can execute volume rendering with a hybrid MPI+OpenMP approach. See this URL for more detail: http://yt-project.org/doc/visualizing/volume_rendering.html?highlight=openmp#openmp-parallelization.
- **Generic parallelization over multiple objects.** Sometimes one wishes to loop over a series which is not a `DatasetSeries`, e.g., performing translational or rotational operations on a camera to make a volume rendering in which the field of view moves through the simulation. In this case, one is applying a set of operations on a single object (a single plot file), rather than over a time series of data. For this workflow, yt provides the `parallel_objects()` function. See this URL for more details: http://yt-project.org/doc/analyzing/parallel_computation.html#parallelizing-over-multiple-objects.

[1] It is convenient to use the magic command `%matplotlib inline` in order to render matplotlib figures in the same browser window as the notebook, as opposed to displaying it as a new window.

An example of MPI parallelization in yt is shown below, where one animates a time series of plot files from an IAMR simulation while revolving the camera such that it completes two full revolutions over the span of the animation:

```
import yt
import glob
import numpy as np

yt.enable_parallelism()

base_dir1 = '/global/cscratch1/sd/user/Nyx_run_p1'
base_dir2 = '/global/cscratch1/sd/user/Nyx_run_p2'
base_dir3 = '/global/cscratch1/sd/user/Nyx_run_p3'

glob1 = glob.glob(base_dir1 + '/plt*')
glob2 = glob.glob(base_dir2 + '/plt*')
glob3 = glob.glob(base_dir3 + '/plt*')

files = sorted(glob1 + glob2 + glob3)

ts = yt.DatasetSeries(files, parallel=True)

frame = 0
num_frames = len(ts)
num_revol = 2

slices = np.arange(len(ts))

for i in yt.parallel_objects(slices):
    sc = yt.create_scene(ts[i], lens_type='perspective', field='z_velocity')

    source = sc[0]
    source.tfh.set_bounds((1e-2, 9e+0))
    source.tfh.set_log(False)
    source.tfh.grey_opacity = False

    cam = sc.camera

    cam.rotate(num_revol*(2.0*np.pi)*(i/num_frames),
               rot_center=np.array([0.0, 0.0, 0.0]))

    sc.save(sigma_clip=5.0)
```

When executed on 4 CPUs on a Haswell node of Cori, the output looks like the following:

```
user@nid00009:~/yt_vis/> srun -n 4 -c 2 --cpu_bind=cores python make_yt_
↪movie.py
yt : [INFO     ] 2017-05-23 16:51:33,565 Global parallel computation␣
↪enabled: 0 / 4
yt : [INFO     ] 2017-05-23 16:51:33,565 Global parallel computation␣
↪enabled: 2 / 4
yt : [INFO     ] 2017-05-23 16:51:33,566 Global parallel computation␣
↪enabled: 1 / 4
yt : [INFO     ] 2017-05-23 16:51:33,566 Global parallel computation␣
```

(continues on next page)

(continued from previous page)

```
↪enabled: 3 / 4
P003 yt : [INFO     ] 2017-05-23 16:51:33,957 Parameters: current_time     ␣
↪         = 0.103169376949795
P003 yt : [INFO     ] 2017-05-23 16:51:33,957 Parameters: domain_dimensions␣
↪         = [128 128 128]
P003 yt : [INFO     ] 2017-05-23 16:51:33,957 Parameters: domain_left_edge ␣
↪         = [ 0.  0.  0.]
P003 yt : [INFO     ] 2017-05-23 16:51:33,958 Parameters: domain_right_edge␣
↪         = [ 6.28318531  6.28318531  6.28318531]
P000 yt : [INFO     ] 2017-05-23 16:51:33,969 Parameters: current_time     ␣
↪         = 0.0
P000 yt : [INFO     ] 2017-05-23 16:51:33,969 Parameters: domain_dimensions␣
↪         = [128 128 128]
P002 yt : [INFO     ] 2017-05-23 16:51:33,969 Parameters: current_time     ␣
↪         = 0.0687808060674485
P000 yt : [INFO     ] 2017-05-23 16:51:33,969 Parameters: domain_left_edge ␣
↪         = [ 0.  0.  0.]
P002 yt : [INFO     ] 2017-05-23 16:51:33,969 Parameters: domain_dimensions␣
↪         = [128 128 128]
P000 yt : [INFO     ] 2017-05-23 16:51:33,970 Parameters: domain_right_edge␣
↪         = [ 6.28318531  6.28318531  6.28318531]
P002 yt : [INFO     ] 2017-05-23 16:51:33,970 Parameters: domain_left_edge ␣
↪         = [ 0.  0.  0.]
P002 yt : [INFO     ] 2017-05-23 16:51:33,970 Parameters: domain_right_edge␣
↪         = [ 6.28318531  6.28318531  6.28318531]
P001 yt : [INFO     ] 2017-05-23 16:51:33,973 Parameters: current_time     ␣
↪         = 0.0343922351851018
P001 yt : [INFO     ] 2017-05-23 16:51:33,973 Parameters: domain_dimensions␣
↪         = [128 128 128]
P001 yt : [INFO     ] 2017-05-23 16:51:33,974 Parameters: domain_left_edge ␣
↪         = [ 0.  0.  0.]
P001 yt : [INFO     ] 2017-05-23 16:51:33,974 Parameters: domain_right_edge␣
↪         = [ 6.28318531  6.28318531  6.28318531]
P000 yt : [INFO     ] 2017-05-23 16:51:34,589 Rendering scene (Can take a␣
↪while).
P000 yt : [INFO     ] 2017-05-23 16:51:34,590 Creating volume
P003 yt : [INFO     ] 2017-05-23 16:51:34,592 Rendering scene (Can take a␣
↪while).
P002 yt : [INFO     ] 2017-05-23 16:51:34,592 Rendering scene (Can take a␣
↪while).
P003 yt : [INFO     ] 2017-05-23 16:51:34,593 Creating volume
P002 yt : [INFO     ] 2017-05-23 16:51:34,593 Creating volume
P001 yt : [INFO     ] 2017-05-23 16:51:34,606 Rendering scene (Can take a␣
↪while).
P001 yt : [INFO     ] 2017-05-23 16:51:34,607 Creating volume
```

Because the `parallel_objects()` function transforms the loop into a data-parallel problem, this procedure strong scales nearly perfectly to an arbitrarily large number of MPI processes, allowing for rapid rendering of large time series of data.

CHAPTER

TWENTY

POST-PROCESSING

There are utilities you can build that can read plotfiles into a `MultiFab` and perform post-processing. Since the data is read into a `MultiFab`, you can perform standard `MFIter` loops to iterate over the data to perform calculations.

20.1 Post-Processing

The following is a list of tools you may find useful for processing plotfile data generated by AMReX codes.

20.1.1 WritePlotfileToASCII

This basic routine reads in a single-level plotfile and writes the entire contents to the standard output, one line at a time for each data value. After reading in the plotfile to a `MultiFab`, the program copies the data into a separate `MultiFab` with one large grid to make writing the data out sequentially an easier task.

In `amrex/Tools/Postprocessing/C_Src`, edit `GNUMakefile` to read `EBASE = WritePlotfileToASCII` and `NEEDS_f90_SRC = FALSE` and then `make` to generate an executable. To run the executable, `<executable> infile=<plotfilename>`. You can modify the C++ file to write out certain components, coordinates, row/column formatting, etc.

20.1.2 fextract

This basic routine reads in a single-level plotfile and extracts selected contents along a 1-D axis to an ASCII file.

How to build and run

In `amrex/Tools/Plotfile`, just type `make` to generate an executable. To run the executable, execute `./fextract.gnu.ex` to see the full command line and all the available options. It is possible to select the axis (`-d` flag) where the data are collected. By default the axis is taken at the center of the domain. A generic ASCII file is generated by default, which contains many details of the simulation. However, data can be exported in a raw CSV file with the command `-csv`.

Example

```
user@machine:~/AMReX/amrex/Tools/Plotfile(postproc_docs)$ ./fextract.gnu.ex \
> ~/AMReX/FHDeX/exec/multispec/Reg_Equil_2d_Bench/plt0000003

 slicing along x-direction at coarse grid (j,k)=(16,0) and output to /home/user/AMReX/
↪FHDeX/exec/multispec/Reg_Equil_2d_Bench/plt0000003.slice
```

This produces an ASCII file of the form:

```
user@machine:~/AMReX/FHDeX/exec/multispec/Reg_Equil_2d_Bench(main)$ cat plt0000003.slice
# 1-d slice in x-direction, file: /home/user/AMReX/FHDeX/exec/multispec/Reg_Equil_2d_
↪Bench/plt0000003
# time =  0.30000000000000004
#                          x                       rho                      rho1                 ␣
↪       rho2
                       0.5       2.9993686498953114      0.60059557892152249       1.
↪0502705977511799
                       1.5       3.0003554204928884      0.59935306004478783       1.
↪0508550827449006
                       2.5       3.0008794559257246       0.5990345897671786       1.
↪0500559828760208
                       3.5       2.9997442287698322      0.60001913923213179       1.
↪0508294996618532
                       4.5       3.0001395958111967      0.60021852440041579       1.
↪0487977074444519
                       5.5       3.0000989976613459      0.60022830117083248       1.
↪0489080268816791
```

20.1.3 fcompare

Compares two plotfiles, zone by zone, to machine precision and reports the maximum absolute and relative errors for each variable.

How to build and run

In `amrex/Tools/Plotfile`, type `make` and then `./fcompare.gnu.ex` to run. Typing `./fcompare.gnu.ex` without inputs will bring up usage and options. Use `--allow_diff_dx` to compare plotfile data without requiring identical cell sizes in the ASCII headers.

Example

```
user@machine:~/AMReX/amrex/Tools/Plotfile$   ./fcompare.gnu.ex \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000000 \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000003

           variable name           absolute error            relative error
                                      (||A - B||)         (||A - B||/||A||)
 ----------------------------------------------------------------------------
 level = 0
 rho                                  0.020039805             0.00845645443
 rho1                                 0.01703166127           0.01450634203
 rho2                                 0.01737072831           0.01479513491
 rho3                                 0.01436258458           0.01436258458
 c1                                  0.003022939351           0.00610148453
 c2                                  0.003167240107          0.006392740399
 c3                                  0.006190179458          0.006190179458
 averaged_velx                      0.0001120979347           0.02141254606
 averaged_vely                      0.0001120979347           0.02141254606
 shifted_velx                       0.0001151524563           0.02145887678
```

(continues on next page)

(continued from previous page)

```
shifted_vely                          0.0001151524563             0.02145887678
pres                                    0.05687549245          1.797693135e+308
```

20.1.4 fboxinfo

Displays information about AMR levels and boxes. Works with 1-, 2- or 3-dimensional datasets.

How to build and run

In `amrex/Tools/Plotfile`, type `make` and then `./fboxinfo.gnu.ex` to run. Typing `./fboxinfo.gnu.ex` without inputs will bring up usage and options.

Example

```
user@machine:~/AMReX/amrex/Tools/Plotfile$ ./fboxinfo.gnu.ex \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000000

 plotfile: /home/user/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000000
 level   0: number of boxes =      4, volume = 100.00%
            maximum zones =        64 x      64
```

20.1.5 fvarnames

Takes a single plotfile and displays a list of the variables present.

How to build and run

In `amrex/Tools/Plotfile`, type `make` and then `./fvarnames.gnu.ex` to run. Typing `./fvarnames.gnu.ex` without inputs will bring up usage and description.

Example

```
user@machine:~/AMReX/amrex/Tools/Plotfile$ ./fvarnames.gnu.ex \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000000
    0   rho
    1   rho1
    2   rho2
    3   rho3
    4   c1
    5   c2
    6   c3
    7   averaged_velx
    8   averaged_vely
    9   shifted_velx
   10   shifted_vely
   11   pres
```

20.1.6 ftime

Takes a whitespace-separated list of plotfiles and returns the time for each plotfile.

How to build and run

In `amrex/Tools/Plotfile`, type `make` and then `./ftime.gnu.ex` to run. Typing `./ftime.gnu.ex` without inputs will bring up usage and description.

Example

```
user@machine :~/AMReX/amrex/Tools/Plotfile$ ./ftime.gnu.ex \
> ~/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000000 \
> ~/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000002 \
> ~/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000004 \
> ~/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000006

 /home/user/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000000    0
 /home/user/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000002    4.
↪0000000000000001e-13
 /home/user/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000004    8.
↪0000000000000002e-13
 /home/user/AMReX/FHDeX/exec/multispec/Spinodal_Charges_2d_Bench/plt0000006    1.
↪1999999999999999e-12
```

20.1.7 fsnapshot

Produces an image of a 2-D plotfile, or a slice of a 3-D plotfile.

How to build and run

In `amrex/Tools/Plotfile`, type `make` and then `./fsnapshot.gnu.ex` to run. Typing `./fsnapshot.gnu.ex` without inputs will bring up usage and options.

Example

In this example, an image of the data from the 2-D plotfile `plt0000003` is created.

```
user@silentm:~/AMReX/amrex/Tools/Plotfile$ ./fsnapshot.gnu.ex \
> -v rho -p Palette ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000003
 plotfile variable maximum = 2.349724636
 plotfile variable minimum = 1
```

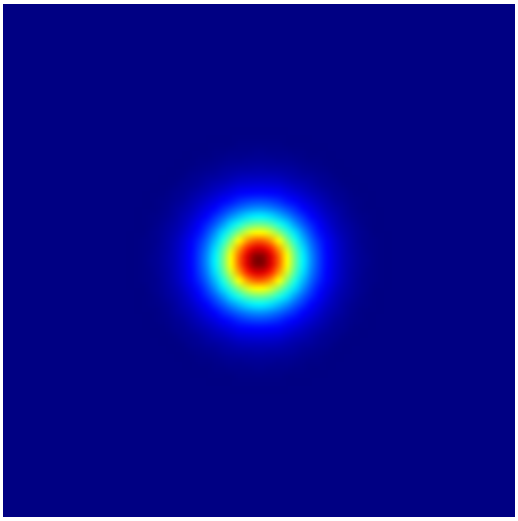
This command tells `fsnapshot` to plot the variable `rho` using the palette `Palette` which is available in the current directory, `amrex/Tools/Plotfile`. The image is created in the same directory as the plotfile folder.

```
user@machine:~/AMReX/amrex/Tools/Plotfile$ ls ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_
↪2d_Bench/
 plt0000000  plt0000003  plt0000003.rho.ppm
```

The image is produced in the portable pixmap format (.ppm). It can be displayed using the command `display` from ImageMagick as seen below.

```
user@machine:~/AMReX/amrex/Tools/Plotfile$ display \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000003.rho.ppm
```

This should produce a window to view the image. (The example here is enlarged for clarity.):



20.1.8 fnan

Takes a single plot file and reports whether each variable contains NaN values.

How to build and run

In `amrex/Tools/Plotfile`, type `make` and then `./fnan.gnu.ex` to run. Typing `./fnan.gnu.ex` without inputs will bring up usage and description.

Example

```
user@machine:~/AMReX/amrex/Tools/Plotfile$ ./fnan.gnu.ex \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000003
 rho           : clean
 rho1          : clean
 rho2          : clean
 rho3          : clean
 c1            : clean
 c2            : clean
 c3            : clean
 averaged_velx : clean
 averaged_vely : clean
 shifted_velx  : clean
 shifted_vely  : clean
 pres          : clean
```

In this example, there were no NaN values found in the variable data.

20.1.9 fextrema

Reports the extrema (min/max) for each variable in a plotfile.

How to build and run

In `amrex/Tools/Plotfile`, type `make` and then `./fextrema.gnu.ex` to run. Typing `./fextrema.gnu.ex` without inputs will bring up usage and options.

Example

```
user@:~/AMReX/amrex/Tools/Plotfile(postproc_docs)$ ./fextrema.gnu.ex \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000000 \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000003
#                    time |rho                                          |rho1             ␣
↪                         |rho2                                            |rho3          ␣
↪                            |c1                                             |c2          ␣
↪                               |c3                                              |averaged_
↪velx                             |averaged_vely                                    ␣
↪|shifted_velx                                  |shifted_vely                          ␣
↪    |pres                                        |
#                          |          min                    max           |          min     ␣
↪              max           |          min                    max           |          min  ␣
↪                 max           |          min                    max           |        ␣
↪min                    max           |          min                    max           |     ␣
↪    min                    max           |          min                    max           |  ␣
↪       min                    max           |          min                    max          ␣
↪|          min                    max           |
                     0                          1           2.369764441          8.
↪277319027e-17           1.174083806          8.277319027e-17           1.174083806       ␣
↪    0.02159682815                     1          8.277319027e-17          0.4954432542     ␣
↪     8.277319027e-17          0.4954432542           0.009113491527                     1 ␣
↪        -0.005235152063        0.005235152063          -0.005235152063        0.
↪005235152063          -0.005366192156        0.005366192156          -0.005366192156     ␣
↪ 0.005366192156                          0                    0
                    0.03                          1           2.349724636          8.
↪277319027e-17           1.157052145          8.277319027e-17           1.156713078       ␣
↪    0.03595941273                     1          8.277319027e-17          0.4924203149     ␣
↪     8.277319027e-17          0.4922760141           0.01530367099                      1 ␣
↪        -0.005172583789        0.005172583789          -0.005172583789        0.
↪005172583789          -0.005287367803        0.005287367803          -0.005287367803     ␣
↪ 0.005287367803          -0.004924487345         0.05687549245
```

20.1.10 fconvert

Converts one or more native AMReX plotfiles to HDF5 plotfiles. The tool passes through AMReX HDF5 compression descriptors such as `None@0`, `ZLIB@5`, and `ZFP_ACCURACY@0.001`.

How to build and run

In `amrex/Tools/Plotfile`, type `make programs=fconvert USE_HDF5=TRUE HDF5_HOME=/path/to/hdf5` and then `./fconvert.gnu.ex` to run. Typing `./fconvert.gnu.ex` without inputs will bring up usage and options.

Example

```
user@machine:~/AMReX/amrex/Tools/Plotfile$ ./fconvert.gnu.ex \
> --compression ZFP_ACCURACY@0.001 \
> --output converted \
> ~/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/plt0000000
fconvert: converting /home/user/AMReX/FHDeX/exec/multispec/Reg_DetBubble_2d_Bench/
↪plt0000000 -> converted.h5 with compression ZFP_ACCURACY@0.001
fconvert: finished converted.h5 in 0.123456 seconds
```

20.1.11 faverage

Compute the lateral average of a variable in a plotfile, with optional density weighting. For 2-D, a profile $f(y)$ is returned where the average was done over x. For 3-D, a profile $f(z)$ is returned where the average was done over x and y.

How to build and run

In `amrex/Tools/Plotfile`, type `make programs=faverage` and then `./faverage.gnu.ex` to run. Typing `./faverage.gnu.ex` without inputs will bring up usage and options.

Example

```
user@:~/AMReX/amrex/Tools/Plotfile$ ./faverage.gnu.ex -v density plt0000000
```

will compute the average density as a function of height, outputting a data file `plt0000000.slice`.

20.1.12 fgradient

Compute the gradient of variables in a plotfile.

How to build and run

In `amrex/Tools/Plotfile`, type `make programs=fgradient` and then `./fgradient.gnu.ex` to run. Typing `./fgradient.gnu.ex` without inputs will bring up usage and options.

Example

```
user@:~/AMReX/amrex/Tools/Plotfile$ ./fgradient.gnu.ex -v density plt0000000
```

will compute the gradient of density as a function of height, outputting a new plotfile `grad.plt0000000`.

CHAPTER

TWENTYONE

DEBUGGING

Debugging is an art. Everyone has their own favorite method. Here we offer a few tips we have found to be useful.

To aid debugging, AMReX handles various signals in the C standard library that are raised during runs. This gives us a chance to print more information using Linux/Unix backtrace capability. The signals include segmentation fault (or "segfault"), interruption by the user (control-c), assertion errors, and floating-point exceptions (NaNs, division by zero, and overflow). The handling of segfault, assertion errors and interruption by control-C are enabled by default. Note that `AMREX_ASSERT()` is only on when compiled with `DEBUG=TRUE` or `USE_ASSERTION=TRUE` in GNU make, or with `-DCMAKE_BUILD_TYPE=Debug` or `-DAMReX_ASSERTIONS=YES` in CMake. The trapping of floating point exceptions is not enabled by default unless the code is compiled with `DEBUG=TRUE` in GNU make, or with `-DCMAKE_BUILD_TYPE=Debug` or `-DAMReX_FPE=YES` in CMake to turn on compiler flags if supported. Alternatively, one can always use runtime parameters to control the handling of floating-point exceptions: `amrex.fpe_trap_invalid` for NaNs, `amrex.fpe_trap_zero` for division by zero and `amrex.fpe_trap_overflow` for overflow. Note that by default clang assumes floating-point exceptions are not trapped, so an optimized build may raise exceptions the source code does not (e.g., in unused SIMD lanes). To trap floating-point exceptions in an optimized clang build, also compile with `-ffp-exception-behavior=maytrap`. The CMake options `AMReX_FPE=ON` and `AMReX_TESTING=ON` and the GNU Make options `DEBUG=TRUE` and `TEST=TRUE` add it for clang. Because the flag limits optimizations, these builds are meant for debugging, not production runs. GCC does not need this. To trap the use of uninitialized values, AMReX also initializes `FArrayBox`s in `MultiFab`s and arrays allocated by `bl_allocate` to signaling NaNs when it is compiled with `TEST=TRUE` or `DEBUG=TRUE` in GNU make, or with `-DCMAKE_BUILD_TYPE=Debug` in CMake. One can also control this setting for `FArrayBox` using the runtime parameter `fab.init_snan`. Note for Macs: Apple silicon chips using the Arm64 architecture are not able to trap division by zero.

One can get more information than the backtrace of the call stack by instrumenting the code. Here is an example. You know the line `Real rho = state(cell,0);` is causing a segfault. You could add a print statement before that. But it might print thousands (or even millions) of lines before it hits the segfault. What you could do is the following,

```
#include <AMReX_BLBackTrace.H>

std::ostringstream ss;
ss << "state.box() = " << state.box() << " cell = " << cell;
BL_BACKTRACE_PUSH(ss.str()); // PUSH takes std::string

Real rho = state(cell,0);  // state is a Fab, and cell is an IntVect.

BL_BACKTRACE_POP(); // One can omit this line.  In that case,
                    // there is an implicit POP when "PUSH" is
                    // out of scope.
```

When it hits the segfault, you will only see the last printout.

Writing a `MultiFab` to disk with

```
VisMF::Write(const FabArray<FArrayBox>& mf, const std::string& name)
```

in `AMReX_VisMF.H` and examining it with `Amrvis` (section *Amrvis*) can be helpful as well. In `AMReX_MultiFabUtil.H`, function

```
void print_state(const MultiFab& mf, const IntVect& cell, const int n=-1,
                 const IntVect& ng = IntVect::TheZeroVector());
```

can output the data for a single cell. `n` is the component, with the default being to print all components. `ng` is the number of ghost cells to include.

Valgrind is one of our favorite debugging tools. For MPI runs, one can tell Valgrind to output to different files for different processes. For example,

```
mpiexec -n 4 valgrind --leak-check=yes --track-origins=yes --log-file=vallog.%p ./foo.
↪exe ...
```

21.1 Breaking into Debuggers

In order to break into debuggers and use modern IDEs, the backtrace signal handling described above needs to be disabled.

The following runtime options need to be set in order to prevent AMReX from catching the break signals before a debugger can attach to a crashing process:

```
amrex.throw_exception = 1
amrex.signal_handling = 0
```

This default behavior can also be modified by applications, see for example this custom application initializer.

21.2 Basic Gpu Debugging

The asynchronous nature of GPU execution can make tracking down bugs complex. The relative timing of improperly coded functions can cause variations in output, and the timing of error messages may not relate linearly to a place in the code. One strategy to isolate specific kernel failures is to add `amrex::Gpu::synchronize()` or `amrex::Gpu::streamSynchronize()` after every `ParallelFor` or similar `amrex::launch`-type call. These synchronization commands will halt execution of the code until the GPU or GPU stream, respectively, has finished processing all previously requested tasks, thereby making it easier to locate and identify sources of error.

21.2.1 Debuggers and Related Tools

Users may also find debuggers useful. Architecture-agnostic tools include `gdb`, `hpctoolkit`, and `Valgrind`. Note that there are architecture-specific implementations of `gdb` such as `cuda-gdb`, `rocgdb`, `gdb-amd`, and Intel `gdb`. Several of these variants are described in the following sections.

For advanced debugging topics and tools, refer to system-specific documentation (e.g., https://docs.olcf.ornl.gov/systems/summit_user_guide.html#debugging).

21.2.2 CUDA-Specific Tests

- To test if your kernels have launched, run:

```
nvprof ./main3d.xxx
```

If using NVIDIA Nsight Compute instead, access `nvprof` functionality with:

```
nsys nvprof ./main3d.xxx
```

- Run `nvprof -o profile%p.nvvp ./main3d.xxxx` or `nsys profile -o nsys_out.%q{SLURM_PROCID}.%q{SLURM_JOBID} ./main3d.xxx` for a small problem and examine page faults using `nvvp` or `nsight-sys $(pwd)/nsys_out.#.######.qdrep`.
- Run under `cuda-memcheck` or the newer version `compute-sanitizer` to identify memory errors.
- Run under `cuda-gdb` to identify kernel errors.
- To help identify race conditions, globally disable asynchronicity of kernel launches for all CUDA applications by setting `CUDA_LAUNCH_BLOCKING=1` in your environment. This will ensure that only one CUDA kernel will run at a time.

21.2.3 AMD ROCm-Specific Tests

- To test if your kernels have launched, run:

```
rocprof ./main3d.xxx
```

- Run `rocprof --hsa-trace --stats --timestamp on --roctx-trace ./main3d.xxxx` for a small problem and examine tracing using `chrome://tracing`.
- Run under `rocgdb` for source-level debugging.
- To help identify race conditions, globally disable asynchronicity of kernel launches by setting `CUDA_LAUNCH_BLOCKING=1` or `HIP_LAUNCH_BLOCKING=1` in your environment. This will ensure that only one kernel runs at a time. See the AMD ROCm docs' chicken bits section for more debugging environment variables.

21.2.4 Intel GPU-Specific Tests

- To test if your kernels have launched, run:

```
./ze_tracer ./main3d.xxx
```

- Run Intel Advisor, `advisor --collect=survey ./main3d.xxx` for a small problem with one MPI process, and examine the metrics.
- Run under `gdb` with the Intel Distribution for GDB.
- To report backend information, set `ZE_DEBUG=1` in your environment.

CHAPTER

TWENTYTWO

RUNTIME PARAMETERS

This chapter contains a list of AMReX ParmParse runtime parameters and their **default** values. They can be set by including them in an inputs file, specifying them at the command line, or passing a function to amrex::Initialize; that function adds parameters to AMReX's ParmParse's parameter database. For more information on ParmParse, see *ParmParse*.

Important

AMReX reserves the following prefixes in ParmParse parameters: amr, amrex, blprofiler, device, DistributionMapping, eb2, fab, fabarray, geometry, integration, particles, tiny_profiler, and vismf.

22.1 AMR

AMReX applications with AMR use either class AmrCore or the more specialized class Amr. Since class Amr is derived from class AmrCore, the parameters for the AmrCore class also apply to the Amr class. Additionally, class AmrCore is derived from class AmrMesh, so AmrMesh member functions are also available to AmrCore and Amr.

22.1.1 AmrCore Class

Below is a list of important ParmParse parameters. However, AMReX applications can choose to avoid them entirely by using this AmrCore constructor AmrCore(Geometry const& level_0_geom, AmrInfo const& amr_info), where struct AmrInfo contains all the information that can be set via ParmParse.

amr.**verbose: int = 0**

This controls the verbosity level of AmrCore functions.

amr.**n_cell: int array = [none]**

This parameter is used only when n_cell is not provided as an argument to AmrCore constructors. It specifies the number of cells in each dimension on Level 0.

amr.**max_level: int = [none]**

This parameter is used only when max_level is not provided as an argument to AmrCore constructors. It specifies the maximum level of refinement allowed. Note that the total number of levels, including the base level 0, is max_level+1.

amr.**ref_ratio: int array = 2 2 2 ... 2**

If the refinement ratio is not provided as an argument to AmrCore constructors and *amr.ref_ratio_vect* is not found in the ParmParse database, this parameter will be used to set the refinement ratios between AMR levels. If there are more AMR levels than the size of the integer parameter array, the last integer will be used as the refinement ratio for the unspecified levels. For example, if max_level is 4 and the provided amr.ref_ratio parameter is 2 4, the refinement ratios are 2, 4, 4 and 4, for levels 0/1, 1/2, 2/3 and 3/4, respectively.

amr.**ref_ratio_vect: int array = [none]**

If the refinement ratio is not provided as an argument to AmrCore constructors and *amr.ref_ratio_vect* is found in the ParmParse database, it will be used to set the refinement ratios between AMR levels. It's an error if the size of the integer array, if found, is less than max_level*AMREX_SPACEDIM. The first AMREX_SPACEDIM numbers specify the refinement ratios in the AMREX_SPACEDIM dimensions between levels 0 and 1, the next AMREX_SPACEDIM numbers specify the ratios for levels 1 and 2, and so on.

amr.**max_grid_size: int array = [build dependent]**

This controls the maximum grid size on AMR levels, one value for each level. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels. The default value is 128 for 1D and 2D runs. For 3D runs, the default value is 64 and 32, for GPU and CPU runs, respectively. Note that the user can also call AmrMesh::SetMaxGridSize to set the maximum grid sizes. Additionally, the values set by this parameter can be overridden by *amr.max_grid_size_x*, *amr.max_grid_size_y* and *amr.max_grid_size_z*.

amr.**max_grid_size_x: int array = [none]**

If provided, this will override the maximum grid size in the x-direction set by *amr.max_grid_size*. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels.

amr.**max_grid_size_y: int array = [none]**

If provided, this will override the maximum grid size in the y-direction set by *amr.max_grid_size*. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels.

amr.**max_grid_size_z: int array = [none]**

If provided, this will override the maximum grid size in the z-direction set by *amr.max_grid_size*. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels.

amr.**blocking_factor: int array = [build dependent]**

This controls the blocking factor on AMR levels, one value for each level. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels. The default value is 8. The values must be powers of 2, except that with *amr.no_box_split_dir* set, on a level with an odd refinement ratio the blocking factor may also be the refinement ratio times a power of 2 (e.g., 24 for ref_ratio 3), and the blocking factor divided by the ratio must be a power of 2. See *Odd Refinement Ratios and Odd Domain Sizes* for details. Note that the user can also call AmrMesh::SetBlockingFactor to set the blocking factors. Additionally, the values set by this parameter can be overridden by *amr.blocking_factor_x*, *amr.blocking_factor_y* and *amr.blocking_factor_z*.

amr.**blocking_factor_x: int array = [none]**

If provided, this will override the blocking factor in the x-direction set by *amr.blocking_factor*. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels.

amr.**blocking_factor_y: int array = [none]**

If provided, this will override the blocking factor in the y-direction set by *amr.blocking_factor*. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels.

amr.**blocking_factor_z: int array = [none]**

If provided, this will override the blocking factor in the z-direction set by *amr.blocking_factor*. If the size of the integer array is less than the total number of levels, the last integer will be used for the unspecified levels.

amr.**n_proper: int = 1**

This parameter controls the proper nesting of grids on AMR levels. For example, if we have blocking_factor = 8, ref_ratio = 2 and n_proper = 1, there will be at least 8/2*1 = 4 coarse level cells outside the fine level grids except at the physical boundaries. Note that the user can also call AmrMesh::SetNProper(int) to set the proper nesting parameter.

amr.**grid_eff: amrex::Real = 0.7**

This parameter controls the grid efficiency threshold during grid creation. While a higher value can enhance efficiency, it may negatively impact overall performance, especially for GPU runs, because it tends to create smaller grids. Note that the user can also call `AmrMesh::SetGridEff(Real)` to set the grid efficiency threshold.

amr.**n_error_buf: int array = 1 1 1 ... 1**

This parameter controls how many extra cells will be tagged around every tagged cell. For example, if `n_error_buf = 2`, tagging cell `(i,j,k)` will result in the tagging of the region of from lower corner `(i-2, j-2,k-2)` to upper corner `(i+2,j+2,k+2)`. If the size of the integer array is less than the number of levels, the last integer will be used for the unspecified levels. Note that the values set by this parameter can be overridden by *amr.n_error_buf_x*, *amr.n_error_buf_y* and *amr.n_error_buf_z*.

amr.**n_error_buf_x: int array = [none]**

This parameter controls the error buffer size in the x-direction. If the size of the integer array is less than the number of levels, the last integer will be used for the unspecified levels.

amr.**n_error_buf_y: int array = [none]**

This parameter controls the error buffer size in the y-direction. If the size of the integer array is less than the number of levels, the last integer will be used for the unspecified levels.

amr.**n_error_buf_z**

This parameter controls the error buffer size in the z-direction. If the size of the integer array is less than the number of levels, the last integer will be used for the unspecified levels.

amr.**refine_grid_layout: bool = true**

If it's true, AMReX will attempt to chop new grids into smaller chunks ensuring at least one grid per MPI process, provided this does not violate the blocking factor constraint.

amr.**refine_grid_layout_x: bool = [none]**

This parameter, if found, will override the `amrex.refine_grid_layout` parameter in the x-direction.

amr.**refine_grid_layout_y: bool = [none]**

This parameter, if found, will override the `amrex.refine_grid_layout` parameter in the y-direction.

amr.**refine_grid_layout_z: bool = [none]**

This parameter, if found, will override the `amrex.refine_grid_layout` parameter in the z-direction.

amr.**refine_whole_domain_dir: int = -1**

If this is 0, 1 or 2, the fine levels will cover the entire domain in that coordinate direction, no matter where the cells are tagged. Tagging a cell then behaves as if the whole line of cells through it in that direction were tagged, and *amr.grid_eff* refers to the fraction of tagged cells in the plane perpendicular to that direction. A negative value, the default, disables this.

amr.**no_box_split_dir: int = -1**

If this is 0, 1 or 2, the grids are never decomposed in that coordinate direction: *amr.max_grid_size* and *amr.refine_grid_layout* are ignored in it, level 0 is split in the other directions only (when the level 0 blocking factor is 1 in those directions), and on finer levels no two grids share an interior face normal to it. This does not exclude contact through a periodic boundary. See *Grid Creation* for details. A negative value, the default, disables this.

amr.**check_input: bool = true**

If this is true, AMReX will check if the various parameters in `AmrMesh` are reasonable.

22.1.2 Amr Class

Warning

These parameters are specific to `class Amr` based applications. If your application uses `class AmrCore` directly, they do not apply unless you have provided implementations for them.

Subcycling

amr.**subcycling_mode: string = Auto**

This controls the subcycling mode of `class Amr`. Possible value are `None` for no subcycling, or `Auto` for subcycling.

Regrid

amr.**regrid_int: int array = 1 1 1 ... 1**

This controls how often we perform the regrid operation on AMR levels 0 to `max_level-1`. If the parameter is a single value, it will be used on all levels. If the parameter is an array of more than one values, the size must be at least `max_level` and values after the first `max_level` elements are ignored.

amr.**regrid_on_restart: bool = false**

This controls whether regridding is performed immediately after restart.

amr.**force_regrid_level_zero: bool = false**

This controls whether regridding is performed on level 0.

amr.**compute_new_dt_on_regrid: bool = false**

This controls whether we re-compute `dt` after regrid.

amr.**initial_grid_file: string = [none]**

If this is set, the initial grids will be read from the specified file.

amr.**regrid_file: string = [none]**

If this is set, regrid will use the grids in the specified file.

I/O

amr.**restart: string = [none]**

If this is set, the simulation will restart from the specified checkpoint file.

amr.**plotfile_on_restart: bool = false**

If this is set to true, a plotfile will be written after restart.

amr.**file_name_digits: int = 5**

This parameter specifies the minimum number of digits in checkpoint and plotfile names.

amr.**checkpoint_files_output: bool = true**

This controls whether we write checkpoint files.

amr.**check_file: string = chk**

This sets the "root" of checkpoint file names. For example, the checkpoint files are named `chk00000`, `chk001000`, etc. by default.

amr.**check_int: int = -1**

This controls the interval of writing checkpoint files, defined as the number of level 0 steps between each checkpoint. A value less than 1 indicates no checkpoint files will be written.

amr.**check_per: amrex::Real = -1**

This controls the interval of writing checkpoint files, defined as the time (not the wall time) elapsed between each checkpoint. A value less or equal to 0 indicates no checkpoint files will be written.

amr.**checkpoint_nfiles: int = 64**

This is the maximum number of binary files per `MultiFab` when writing checkpoint files.

amr.**plot_files_output: bool = true**

This controls whether we write plot files.

amr.**plot_file: string = plt**

This sets the "root" of plot file names. For example, the plot files are named `plt00000`, `plt001000`, etc. by default.

amr.**plot_int: int = -1**

This controls the interval of writing plot files, defined as the number of level 0 steps between each plot file. A value less than 1 indicates no plot files will be written.

amr.**plot_per: amrex::Real = -1**

This controls the interval of writing plot files, defined as the time (not the wall time) elapsed between each plot file. A value less or equal to 0 indicates no plot files will be written.

amr.**plot_log_per: amrex::Real = -1**

This controls the interval of writing plot files, defined as the `log10` time (not the wall time) elapsed between each plot file. A value less or equal to 0 indicates no plot files will be written.

amr.**plot_max_level: int = amr.max_level**

This controls the finest level in a plot file. For example, if the finest level in a run is 3, but this parameter is set to 1, only levels 0 and 1 will be saved in a plot file.

amr.**plot_nfiles: int = 64**

This is the maximum number of binary files per `MultiFab` when writing plot files.

amr.**plot_vars: string array = [none]**

If this parameter is set, the variables specified in the string array will be the state variables saved in the plot files. The special values `ALL` and `NONE` mean that all or none of the state variables will be saved. If this parameter is not set, all state variables will be saved.

amr.**derive_plot_vars: string array = [none]**

If this parameter is set, the variables specified in the string array will be the derive variables saved in the plot files. The special values `ALL` and `NONE` mean that all or none of the derive variables will be saved. If this parameter is not set, none of the derive variables will be saved. For multi-component derived quantities, specify the registered derived quantity name. Component names are output as plotfile field names, but they do not independently select individual components.

amr.**small_plot_file: string = smallplt**

This sets the "root" of small plot file names. For example, the small plot files are named `smallplt00000`, `smallplt001000`, etc. by default.

amr.**small_plot_int: int = -1**

This controls the interval of writing small plot files, defined as the number of level 0 steps between each small plot file. A value less than 1 indicates no small plot files will be written.

amr.**small_plot_per: amrex::Real = -1**

This controls the interval of writing small plot files, defined as the time (not the wall time) elapsed between each small plot file. A value less or equal to 0 indicates no small plot files will be written.

amr.**small_plot_log_per: amrex::Real = -1**

This controls the interval of writing small plot files, defined as the log10 time (not the wall time) elapsed between each small plot file. A value less or equal to 0 indicates no small plot files will be written.

amr.**small_plot_vars: string array = [none]**

If this parameter is set, the variables specified in the string array will be the state variables saved in the small plot files. The special values ALL and NONE mean that all or none of the state variables will be saved. If this parameter is not set, none of the state variables will be saved.

amr.**derive_small_plot_vars: string array = [none]**

If this parameter is set, the variables specified in the string array will be the derive variables saved in the small plot files. The special values ALL and NONE mean that all or none of the derive variables will be saved. If this parameter is not set, none of the derive variables will be saved. For multi-component derived quantities, specify the registered derived quantity name. Component names are output as small-plot field names, but they do not independently select individual components.

amr.**message_int: int = 10**

This controls the interval of checking messages during a run, defined as the number of level 0 steps between checks. A value less than 1 indicates no checking will be performed. A message refers to a file created by the user on the disk, where only the file name is checked, not its content. If the file name matches one of the following predefined names, appropriate actions will be taken.

dump_and_continue
Make a checkpoint file and continue running the simulation.

stop_run
Stop the simulation.

dump_and_stop
Make a checkpoint file and stop the simulation.

plot_and_continue
Make a plot file and continue running the simulation.

small_plot_and_continue
Make a small plot file and continue running the simulation.

amr.**write_plotfile_with_checkpoint: bool = true**

This parameter is for the message action discussed in *amr.message_int*. It controls whether an action will make a plot file as well when asked to make a checkpoint file.

amr.**run_log: string = [none]**

If this parameter is set, the run log will be enabled and this is the log file name.

amr.**run_log_terse: string = [none]**

If this parameter is set, the terse run log will be enabled and this is the log file name.

amr.**grid_log: string = [none]**

If this parameter is set, the grid log will be enabled and this is the log file name.

amr.**data_log: string = [none]**

If this parameter is set, the data log will be enabled and this is the log file name.

22.2 Basic Controls

amrex.**verbose: int = 1**

This controls the verbosity level of AMReX. Besides using ParmParse, you can also call amrex::SetVerbose(int) to set it.

amrex.**init_snan: bool = [build dependent]**

This controls whether MultiFab, FArrayBox, BaseFab<double|float>, PODVectors<double|float>, Gpu::DeviceVector<double|float>, etc. will be initialized to signaling NaNs at construction. The default value is true for debug builds. For non-debug builds, the default is false unless TEST=TRUE for GNU Make or AMReX_TESTING is enabled for CMake.

amrex.**abort_on_unused_inputs: bool = false**

If this is true and there are unused ParmParse parameters, AMReX will abort during amrex::Finalize.

amrex.parmparse.**verbose: int = amrex.verbose**

If this is greater than zero, unused ParmParse variables will be printed out during amrex::Finalize or ParmParse::QueryUnusedInputs. The parameter can also be set by calling amrex::ParmParse::SetVerbose(int).

amrex.device.**verbose: int = 0**

This controls whether AMReX prints out GPU device properties such name, vendor, total memory size, etc. This is only relevant for GPU runs.

amrex.**max_gpu_streams: int = 4**

This controls the number of GPU streams used by AMReX. It's only relevant for GPU runs.

amrex.**omp_threads: string = system**

If OpenMP is enabled, this can be used to set the default number of threads. Possible values are system, nosmt, or an integer string. The special value nosmt can be used to avoid using threads for virtual cores (aka Hyperthreading or SMT), as is default in OpenMP, and instead only spawns threads equal to the number of physical cores in the system. For the values system and nosmt, the environment variable OMP_NUM_THREADS takes precedence. If the string can be converted to an integer, OMP_NUM_THREADS is ignored.

amrex.**memory_log: string = memlog**

This is the name of the memory log file when memory profiling is enabled.

22.3 Communication

amrex.**use_gpu_aware_mpi: bool = false**

For GPU runs, this controls the memory type used for AMReX's communication buffers. When this is true, AMReX uses GPU device memory for communication data in MPI function calls. When this is false, the data are placed in pinned memory. Note that this flag does not enable GPU-aware MPI by itself. Enabling GPU-aware MPI is system dependent. Users should consult their system's documentation for instructions on setting up the environment and linking to GPU-aware MPI libraries.

22.4 Distribution Mapping

DistributionMapping.**verbose: int = 0**

This controls the verbosity level of DistributionMapping functions.

DistributionMapping.**strategy: string = SFC**

This is the default DistributionMapping strategy. Possible values are SFC, KNAPSACK, ROUNDROBIN, or RRSFC. Note that the default strategy can also be set by calling DistributionMapping::strategy(DistributionMapping::Strategy).

22.5 Embedded Boundary

eb2.**max_grid_size: int = 64**

This parameter specifies the maximum grid size in AMReX's internal EB database, not the user's data.

eb2.**extend_domain_face: bool = true**

This controls the behavior of the embedded boundary outside the domain. If this is true, the embedded boundary outside the domain is extended perpendicularly from the domain face. Otherwise, it's generated with the user provided implicit function or STL mesh. Note that this parameter can be overridden by the user when calling `amrex::EB2::Build` with the optional parameter `bool extend_domain_face`. The parameter applies to geometries built from an implicit function and from an STL file alike.

eb2.**num_coarsen_opt: int = 0**

If it is greater than 0, this parameter can speed up the EB generation. It indicates that the search for EB can be performed on grids coarsened by this factor and then the EB information details will be generated on the original grids. However, the user should be aware that setting this parameter too high could result in erroneous results. Also note that this parameter can be overridden by the user when calling `amrex::EB2::Build` with the optional parameter `int num_coarsen_opt`.

eb2.**geom_type: string = [none]**

There are two versions of the *amrex::EB2::Build* function that can be used to build EB. One version is a function template that takes a user provided `GeometryShop`, while the other uses `ParmParse` parameters to build EB. For the latter version, this parameter specifies the type of the EB. Possible values include the following.

all_regular
The entire domain is regular without any EB objects.

parser
The embedded boundary is describe by *eb2.parser_function*.

stl
The embedded boundary will be built using an STL file specified by *eb2.stl_file*.

eb2.**parser_function: string = [none]**

When `eb2.geom_type = parser`, this parameter is a parser function string that contains a math expression describing the surface of the EB.

> **See also**
>
> Section *Parser*.

eb2.**stl_file: string = [none]**

When `eb2.geom_type = stl`, this is a required string parameter specifying the STL file name.

eb2.**stl_scale: amrex:Real = 1**

When building EB using STL, the triangles in the STL file will be scaled by the given value of this optional parameter.

eb2.**stl_center: amrex::Real array = 0 0 0**

When building EB using STL, this optional parameter specifies the shifted center. The original coordinates in the STL file will be shifted by the provided values.

eb2.**stl_reverse_normal: bool = false**

When building EB using STL, the normal direction of the triangles in the STL file will be reversed if this optional parameter is set to true.

`eb2.small_volfrac: amrex::Real = [depend on the type of amrex::Real]`

This parameter specifies the threshold for small cells that will be converted to covered cells. The default value is `1.e-12` if `amrex::Real` is `double`, or `1.e-5` if `amrex::Real` is `float`.

`eb2.cover_multiple_cuts: bool = false`

If this parameter is set to true, multi-cut cells will be converted to covered cells.

> **Tip**
>
> Because AMReX currently does not support multi-cut cells, it would be a runtime error if multi-cut cells are left unfixed.

`eb2.maxiter: int = 32`

Fixing small and multi-cut cells is an iterative process. This parameter specifies the maximum number of iterations for the fix-up process.

22.6 Error Handling

By default AMReX installs a signal handler that will be run when a signal such as segfault is received. You can also enable floating point exception trapping. The signal handler will print out backtraces that can be useful for debugging.

> **Note**
>
> Floating point exception trapping is not enabled by default, because compilers might generate optimized SIMD code that raises the exceptions.

`amrex.signal_handling: bool = true`

This controls whether AMReX should handle signals.

`amrex.handle_sigsegv: bool = true`

If both this flag and `amrex.signal_handling` are true, `SIGSEGV` will be handled by AMReX.

`amrex.handle_sigterm: bool = false`

If both this flag and `amrex.signal_handling` are true, `SIGTERM` will be handled by AMReX. This flag is false by default because this could generate lots of backtrace files on some batch systems that issue `SIGTERM` for jobs running out of wall clock time.

`amrex.handle_sigint: bool = true`

If both this flag and `amrex.signal_handling` are true, `SIGINT` will be handled by AMReX.

`amrex.handle_sigabrt: bool = true`

If both this flag and `amrex.signal_handling` are true, `SIGABGT` will be handled by AMReX.

`amrex.handle_sigfpe: bool = true`

If both this flag and `amrex.signal_handling` are true, `SIGFPE` will be handled by AMReX.

> **See also**
>
> Use *amrex.fpe_trap_invalid*, *amrex.fpe_trap_zero* and *amrex.fpe_trap_overflow* to enable `FE_INVALID`, `FE_DIVBYZERO` and `FE_OVERFLOW` trapping, respectively.

amrex.**handle_sigill: bool = true**

If both this flag and amrex.signal_handling are true, SIGILL will be handled by AMReX.

amrex.**handle_crt_reports: bool = true**

Windows debug builds only: if true, failed assertions and run-time check failures of the debug C runtime are printed to stderr and the process breaks instead of opening a blocking dialog box. Unlike the flags above, this does not depend on amrex.signal_handling.

amrex.**throw_exception: bool = false**

If this flag is true and amrex.signal_handling is false, amrex::Abort and amrex::Error will throw std::runtime_error instead of aborting immediately. Note that according the C++ standard, if an exception is thrown and not caught, std::terminate will be called.

amrex.**fpe_trap_invalid: bool = false**

If SIGFPE is handled by AMReX and this flag is true, FE_INVALID (e.g., 0/0) trapping will be enabled. This flag has no effect on Windows.

amrex.**fpe_trap_zero: bool = false**

If SIGFPE is handled by AMReX and this flag is true, FE_DIVBYZERO (e.g., 1/0) trapping will be enabled. This flag has no effect on Windows.

amrex.**fpe_trap_overflow: bool = false**

If SIGFPE is handled by AMReX and this flag is true, FE_OVERFLOW (i.e., the result is too large to be representable) trapping will be enabled. This flag has no effect on Windows.

22.7 Extern

22.7.1 HYPRE

These parameters are relevant only when HYPRE support is enabled.

amrex.**init_hypre: bool = true**

This controls whether AMReX should call HYPRE_Init() during amrex::Initialize.

amrex.**hypre_spgemm_use_vendor: bool = false**

This controls whether HYPRE should use the vendor's SpGemm functionality.

amrex.**hypre_spmv_use_vendor: bool = false**

This controls whether HYPRE should use the vendor's SpMV functionality.

amrex.**hypre_sptrans_use_vendor: bool = false**

This controls whether HYPRE should use the vendor's SpTrans functionality.

22.8 Geometry

All these parameters are optional for constructing a *Geometry* object. They are only used if the information is not provided via function arguments.

geometry.**coord_sys: int = 0**

This specifies the coordinate system type with valid values being 0 (Cartesian), or 1 (cylindrical), or 2 (spherical).

geometry.**prob_lo: amrex::Real array = 0 0 0**

This specifies the position of the lower corner of the physical domain.

geometry.**prob_hi: amrex::Real array = [none]**

This specifies the position of the upper corner of the physical domain. If this is provided, *geometry.prob_extent* will be ignored.

geometry.**prob_extent: amrex::Real array = [none]**

This specifies the length of the physical domain. If *geometry.prob_hi* is provided, this will be ignored.

geometry.**is_periodic: int array = 0 0 0**

These integer parameters are boolean flags to indicate whether the domain is periodic in each direction. It's considered true (i.e., periodic) if its value is non-zero, and false (i.e., non-periodic) if its value is zero.

22.9 I/O

amrex.**async_out: bool = false**

If this is true, AMReX's native mesh and particle plotfiles will be written asynchronously by a background thread.

amrex.**async_out_nfiles: into = 64**

This is the maximum number of binary files on each AMR level that will be used when AMReX writes a plotfile asynchronously.

vismf.**verbose: int = 0**

This controls the verbosity level of `VisMF` functions.

22.10 Memory

amrex.**the_arena_init_size: long = [system dependent]**

This controls the main memory arena's initial size in bytes. For CPU runs, the default is 0, whereas for GPU runs, the default is set at run time to 3/4 of the system's device memory, or 3/8 on integrated GPUs that share memory with the host.

Tip

Since `amrex v24.08`, instead of `amrex.the_arena_init_size=10000000000`, one can use `amrex.the_arena_init_size=10'000'000'000` or `amrex.the_arena_init_size=1e10` to set `ParmParse` integer parameters like this one.

amrex.**the_device_arena_init_size: long = 8388608 [8 MB]**

This controls the GPU device arena's initial size in bytes. For CPU runs, this is ignored. If the main arena uses the device memory (as opposed to managed memory), this parameter is also ignored.

amrex.**the_managed_arena_init_size: long = 8388608 [8 MB]**

This controls the managed device arena's initial size in bytes. For CPU runs, this is ignored. If the main arena uses the managed memory (as opposed to device memory), this parameter is also ignored.

amrex.**the_pinned_arena_init_size: long = [system dependent]**

This controls the pinned host memory arena's initial size in bytes. The default is 8 MB for CPU runs. For GPU runs it's set to half of the GPU device memory by default.

amrex.**the_comms_arena_init_size: long = 8388608 [8 MB]**

This controls the MPI communication memory arena's initial size in bytes.

amrex.**the_arena_release_threshold: long = LONG_MAX**

This controls the release threshold of the main arena.

amrex.**the_device_arena_release_threshold: long = LONG_MAX**

This controls the release threshold of the device arena.

amrex.**the_managed_arena_release_threshold: long = LONG_MAX**

This controls the release threshold of the managed arena.

amrex.**the_pinned_arena_release_threshold: long = LONG_MAX**

This controls the release threshold of the pinned arena.

amrex.**the_comms_arena_release_threshold: long = LONG_MAX**

This controls the release threshold of the communication arena, which keeps the memory of MPI buffers for reuse. When the memory in use would reach this threshold, the arena first returns its unused memory to the system. A smaller value lowers the memory kept by the arena.

In GPU runs without GPU-aware MPI, MPI buffers come from the pinned arena, and *amrex.the_pinned_arena_release_threshold* applies instead. In GPU runs with GPU-aware MPI and managed memory for the main arena, they may come from the device arena, and then *amrex.the_device_arena_release_threshold* applies.

amrex.**the_arena_defragmentation: bool = true**

This controls the defragmentation behavior of the main arena.

amrex.**the_device_arena_defragmentation: bool = true**

This controls the defragmentation behavior of the device arena.

amrex.**the_managed_arena_defragmentation: bool = true**

This controls the defragmentation behavior of the managed arena.

amrex.**the_pinned_arena_defragmentation: bool = true**

This controls the defragmentation behavior of the pinned arena.

amrex.**the_comms_arena_defragmentation: bool = [build dependent]**

This controls the defragmentation behavior of the communication arena. The default is false for HIP builds and true for others.

amrex.**the_arena_is_managed: bool = false**

This controls if AMReX uses the managed memory for the main arena. This is only relevant for GPU runs.

amrex.**abort_on_out_of_gpu_memory: bool = false**

This controls if AMReX should throw `amrex::OutOfMemoryError` when the reported free device memory is less than the amount an arena is asked to allocate. Note that for managed memory it's possible to allocate more than the amount of free device memory available. However, the code will be very slow. This parameter is only relevant for GPU runs.

amrex.mf.**alloc_single_chunk: bool = false**

This controls if all the data in a `FabArray` (including `MultiFab`) are in a contiguous chunk of memory.

amrex.**vector_growth_factor: amrex::Real = 1.5**

This controls the growth factor of `amrex::PODVector` and its derived classes such as `amrex::Gpu::DeviceVector`, `amrex::Gpu::ManagedVector`, etc. A smaller value can avoid wasting memory, but it may result in a performance penalty during resizing.

22.11 Particles

particles.**do_tiling: bool = false**

This controls whether tiling is enabled for particle containers.

particles.**tile_size: int array = 1024000 8 8**

When tiling is enabled, this is the default tile size. Note that a big number like 1024000 effectively turns tiling off in that direction.

particles.**do_mem_efficient_sort: bool = true**

This parameter controls whether the more memory-efficient method will be used for sorting particles.

particles.**particles_nfiles: int = 256**

This is the maximum number of binary files per level for a particle container when writing checkpoint and plot files for particles. The special value of -1 indicates one file per process.

particles.**redistribute_use_mask: bool = true**

Added in version 26.08: Runtime parameter `particles.redistribute_use_mask`.

On CPU builds, this controls whether local `Redistribute()` may use a level-0 mask for particle-to-grid lookup when the call is otherwise eligible.

particles.**redistribute_mask_max_ratio: double = 8.0**

Added in version 26.08: Runtime parameter `particles.redistribute_mask_max_ratio`.

This is the maximum ratio of grown mask cells to valid cells for using the CPU local `Redistribute()` lookup mask. Non-positive values disable this ratio limit.

particles.**redistribute_mask_max_bytes: long = 268435456**

Added in version 26.08: Runtime parameter `particles.redistribute_mask_max_bytes`.

This is the maximum estimated per-rank storage, in bytes, for using the CPU local `Redistribute()` lookup mask. Negative values disable this byte limit.

particles.**redistribute_early_out: bool = true**

Added in version 26.12: Runtime parameter `particles.redistribute_early_out`.

This controls whether `Redistribute()` works in place, without communication buffers, on a single process with a single level, box and tile.

22.12 Tiling

fabarray.**mfiter_tile_size: int array = [build dependent]**

This is the default size for *tiling*. For GPU runs, it is disabled by default. For CPU runs, it is disabled by default in 1D and 2D, but enabled in 3D with a tile size of 8 in the y and z-directions.

fabarray.**comm_tile_size: int array = [build dependent]**

This is the default tiling size used in moving data in and out of the MPI communication buffer . It is disabled by default for GPU runs, but enabled for CPU runs with a tile size of 8 in the y and z-directions (if they exist).

22.13 Time Integration

All these parameters are optional for constructing or configuring a *TimeIntegrator* object.

`integration.type: string = [none]`

This parameter sets the type of time integrator to use. The supported values are:

- ForwardEuler
- RungeKutta
- SUNDIALS

`integration.time_step: amrex::Real = [none]`

This parameter sets the fixed time step size to use.

For SUNDIALS methods, this parameter sets a fixed step size for single rate methods (e.g., ERK) or a fixed slow time scale step size for multirate methods (e.g., EX-MRI).

22.13.1 Runge–Kutta Methods

These parameters are relevant only when *integration.type* is "RungeKutta".

`integration.rk.type: string = [none]`

This parameter sets the Runge–Kutta method to use. The supported values are:

- User
- ForwardEuler
- Trapezoid
- SSPRK3
- RK4

User-specified Runge–Kutta Method

When *integration.rk.type* is "User", the following parameters can be used to set a user-specified explicit Butcher tableau,

$$B \equiv \begin{array}{c|c} c & A \\ \hline & b \\ & \tilde{b} \end{array},$$

where, for a method with s stages, c, and b, $\tilde{b}$ are arrays of s values and A is a lower triangular $s \times s$ matrix.

`integration.rk.nodes: amrex::Real array = [none]`

The c values of the Butcher tableau.

`integration.rk.tableau: amrex::Real array = [none]`

The A values of the Butcher tableau.

`integration.rk.weights: amrex::Real array = [none]`

The b values of the Butcher tableau.

`integration.rk.extended_weights: amrex::Real array = [none]`

The $\tilde{b}$ values of the Butcher tableau. These values are only required if the method has an embedding.

22.13.2 SUNDIALS

These parameters are relevant only when support for SUNDIALS time integrators is enabled (see *Using SUNDIALS*) and *integration.type* is "SUNDIALS".

Methods

`integration.sundials.type: string = ERK`

This parameter sets type of SUNDIALS time integrator to use. See the table below for supported values.

Parameter Value	SUNDIALS Method Type
ERK	Explicit Runge-Kutta method
DIRK	Diagonally Implicit Runge-Kutta method
IMEX-RK	Implicit-Explicit Additive Runge-Kutta method
EX-MRI	Explicit Multirate Infinitesimal method
IM-MRI	Implicit Multirate Infinitesimal method
IMEX-MRI	Implicit-Explicit Multirate Infinitesimal method

`integration.sundials.fast_type: string = ERK`

When using a multirate method, this parameter sets the type of time integrator to use at the fast time scale. Currently, ERK and DIRK methods are supported.

`integration.sundials.method: string = [none]`

This parameter sets the name of the specific time integration method to use. See the sections listed below in the SUNDIALS documentation for valid method names. Note, IMEX method must be specified using the *integration.sundials.method_i* and *integration.sundials.method_e* parameters. If a method name is not provided, the SUNDIALS default method is used.

- ERK methods
- DIRK methods
- MRI methods

`integration.sundials.method_i: string = [none]`

When using an IMEX method, both the implicit and explicit methods must be specified. This parameter can be used to select the *implicit* portion of the IMEX method. See the ImEx methods section in the SUNDIALS documentation for valid method names. If a method name is not provided, the SUNDIALS default method is used.

`integration.sundials.method_e: string = [none]`

When using an IMEX method, both the implicit and explicit methods must be specified. This parameter can be used to select the *explicit* portion of the IMEX method. See the ImEx methods section in the SUNDIALS documentation for valid method names. If a method name is not provided, the SUNDIALS default method is used.

`integration.sundials.fast_method: string = [none]`

When using a multirate method, this parameter sets the method to use at the fast time scale. If a method name is not provided, the SUNDIALS default method is used.

Step Sizes

Note

The parameter *integration.time_step* is used to set a fixed step size with single rate methods (e.g., ERK) or a fixed slow time scale step size with multirate methods (e.g., EX-MRI).

integration.**fast_time_step: amrex::Real = [none]**

When using a multirate method, this parameter sets the fixed step size to use at the fast time scale. If it is not set, the fast step size is chosen adaptively with default tolerances. To control those tolerances with *integration.fast_rel_tol* and *integration.fast_abs_tol*, set *integration.use_adaptive_fast_time_step* to true.

integration.**use_adaptive_time_step: bool = false**

This parameter enables adaptive time step sizes with single rate methods (e.g., ERK) or adaptive time step sizes at the slow time scale with multirate methods (e.g., EX-MRI). Adaptive slow time steps with multirate methods require SUNDIALS 7.2 or later.

integration.**use_adaptive_fast_time_step: bool = false**

This parameter enables adaptive time step sizes at the fast time scale with multirate methods (e.g., EX-MRI).

integration.sundials.**max_num_steps: int = 500**

When using the `evolve` method, which allows the integrator to take multiple time steps to reach the specified output time, this parameter sets the maximum number of time steps allowed before reaching the output time. If this limit is reached before the output time, SUNDIALS will return an error.

integration.sundials.**stop_time: amrex::Real = [none]**

When using the `evolve` method (especially with adaptive time step sizes), the SUNDIALS integrator may step past the requested output time and return an interpolated solution at the requested time. This parameter will force the integrator to stop and return the solution at the requested time.

Tolerances

When using adaptive time step sizes or an implicit method (e.g., DIRK), selecting appropriate tolerances for the target application is a critical factor in method performance. For advice on selecting tolerances see the SUNDIALS documentation.

integration.**rel_tol: amrex::Real = 1.e-4**

Relative tolerance for temporal error control.

integration.**abs_tol: amrex::Real = [depend on the type of amrex::Real]**

Absolute tolerance for temporal error control. The default value is `1.e-9` if `amrex::Real` is `double`, or `1.e-6` if `amrex::Real` is `float`.

integration.**fast_rel_tol: amrex::Real = 1.e-4**

Relative tolerance for the temporal error at the fast time scale with multrate methods.

integration.**fast_abs_tol: amrex::Real = [depend on the type of amrex::Real]**

Absolute tolerance for the temporal error at the fast time scale with multrate methods. The default value is `1.e-9` if `amrex::Real` is `double`, or `1.e-6` if `amrex::Real` is `float`.

Algebraic Solvers

The parameters provide control over the nonlinear and linear solvers utilized with implicit methods (e.g., DIRK).

integration.sundials.**nonlinear_solver: string = Newton**

The nonlinear solver used with single rate methods (e.g., DIRK) or at the slow time scale with multirate methods (e.g., IM-MRI). The supported values are:

- Newton
- fixed-point

integration.sundials.**max_nonlinear_iters: int = 3**

The maximum number of nonlinear iterations allowed per solve with single rate methods (e.g., DIRK) or at the slow time scale with multirate methods (e.g., IM-MRI).

integration.sundials.**linear_solver: string = GMRES**

The linear solver used with Newton's method for single rate methods (e.g., DIRK) or at the slow time scale with multirate methods (e.g., IM-MRI).

integration.sundials.**max_linear_iters: int = 5**

The maximum number of linear iterations allowed per solve with single rate methods (e.g., DIRK) or at the slow time scale with multirate methods (e.g., IM-MRI).

integration.sundials.**fast_nonlinear_solver: string = Newton**

The nonlinear solver used at the fast time scale with multirate methods (e.g., when the fast method is DIRK). The supported values are:

- Newton
- fixed-point

integration.sundials.**fast_max_nonlinear_iters: int = 3**

The maximum number of nonlinear iterations allowed per solve at the fast time scale (e.g., when the fast method is DIRK).

integration.sundials.**fast_linear_solver: string = GMRES**

The linear solver used with Newton's method at the fast time scale with multirate methods (e.g., when the fast method is DIRK).

integration.sundials.**fast_max_linear_iters: int = 5**

The maximum number of linear iterations allowed at the fast time scale per solve with multirate methods (e.g., when the fast method is DIRK).

22.14 Tiny Profiler

These parameters are ignored unless profiling with `TinyProfiler` is enabled.

tiny_profiler.**verbose: int = 0**

If this value is greater than 0, messages about entering or leaving profiled regions will be printed on the I/O process.

tiny_profiler.**print_threshold: double = 1.0**

In the profiling report, regions with very small run times are not listed individually. Instead, they are included in a section named "Other". This parameter specifies the maximum inclusive run time that the "Other" section can take in percent relative to the total run time.

tiny_profiler.**device_synchronize_around_region: bool = false**

This parameter is only relevant for GPU runs. If it is set to true, the current GPU stream is synchronized when entering and leaving a profiling region. Because GPU kernels are asynchronous, time measurements without synchronization could be misleading. Enabling this parameter can provide more accurate measurements. However, the added synchronization points, which are unnecessary for correctness, could potentially degrade the performance. When this parameter is enabled, the GPU-synchronized profiling macros described in *GPU-synchronized TinyProfiler timers* do not switch TinyProfiler to their focused report.

tiny_profiler.**enabled: bool = true**

Added in version 24.09: Runtime parameter *tiny_profiler.enabled* `.

This parameter can be used to disable tiny profiling including `CArena` memory profiling at run time.

tiny_profiler.**memprof_enabled: bool = true**

Added in version 24.09: Runtime parameter `tiny_profiler.memprof_enabled`.

This parameter can be used to disable `CArena` memory profiling at run time. If `tiny_profiler.enabled` is false, this parameter has no effect.

tiny_profiler.**output_file: string = stdout**

Added in version 24.09: Runtime parameter `tiny_profiler.output_file`.

Changed in version 26.09: Special names `stdout` and `stderr`; the default value is now `stdout` (same behavior as the previous default, the empty string).

This parameter specifies the destination of the tiny profiling output. Note that `stdout`, `stderr` and `/dev/null` are special names that mean the default output stream, the default error stream, and no output, respectively. An empty value is treated the same as `stdout`. Any other value specifies the file name for the output.

CHAPTER

TWENTYTHREE

AMREX-BASED PROFILING TOOLS

AMReX-based application codes can be instrumented using AMReX-specific performance profiling tools that take into account the hierarchical nature of the mesh in most AMReX-based applications. These codes can be instrumented for varying levels of profiling detail.

Here are links to short courses (slides) on how to use the profiling tools. More details can be found in the documentation below.

Lecture 1: Introduction and TINYPROFILER

Lecture 2: Introduction to Full Profiling

Lecture 3: Using ProfVis – GUI Features

Lecture 4: Batch Options and Advanced Profiling Flags

23.1 Types of Profiling

AMReX's built-in profiling works through objects that start and stop timers based on user-placed macros or an object's constructor and destructor. The results from these timers are stored in a global list that is consolidated and printed during finalization, or at a user-defined flush point.

Currently, AMReX has two options for built-in profiling: *Tiny Profiling* and *Full Profiling*.

23.1.1 Tiny Profiling

To enable "Tiny Profiling" with GNU Make, edit the options in the file `GNUMakefile` as follows:

```
TINY_PROFILE = TRUE
PROFILE      = FALSE
```

If building with CMake, set the following CMake flags:

```
AMReX_TINY_PROFILE = ON
AMReX_BASE_PROFILE = OFF
```

Note

If you set `PROFILE = TRUE` (or `AMReX_BASE_PROFILE = ON`) to enable full profiling then this will override the `TINY_PROFILE` flag and tiny profiling will be disabled.

Output

At the end of a run, a summary of exclusive and inclusive function times will be written to `stdout`. This output includes the minimum and maximum (over processes) time spent in each routine as well as the average and the maximum percentage of total run time. See the sample output below.

```
TinyProfiler total time across processes [min...avg...max]: 1.765...1.765...1.765
--------------------------------------------------------------------------------
Name                            NCalls   Excl. Min   Excl. Avg   Excl. Max   Max  %
--------------------------------------------------------------------------------
mfix_level::EvolveFluid         1        1.602       1.668       1.691       95.83%
FabArray::FillBoundary()        11081    0.02195     0.03336     0.06617      3.75%
FabArrayBase::getFB()           22162    0.02031     0.02147     0.02275      1.29%
PC<...>::WriteAsciiFile()       1        0.00292     0.004072    0.004551     0.26%


--------------------------------------------------------------------------------
Name                            NCalls   Incl. Min   Incl. Avg   Incl. Max    Max  %
--------------------------------------------------------------------------------
mfix_level::Evolve()            1        1.69        1.723       1.734        98.23%
mfix_level::EvolveFluid         1        1.69        1.723       1.734        98.23%
FabArray::FillBoundary()        11081    0.04236     0.05485     0.08826       5.00%
FabArrayBase::getFB()           22162    0.02031     0.02149     0.02275       1.29%
```

The tiny profiler automatically writes the results to `stdout` at the end of your code, when `amrex::Finalize();` is reached. However, you may want to write partial profiling results to ensure your information is saved when you may fail to converge or if you expect to run out of allocated time. Partial results can be written at user-defined points in the code by inserting the line:

```
BL_PROFILE_TINY_FLUSH();
```

Any timers that have not reached their `BL_PROFILE_VAR_STOP` call or exited their scope and been destroyed will not be included in these partial outputs. (e.g., a properly instrumented `main()` should show a time of zero in all partial outputs.) Therefore, it is recommended to place these flush calls in easily identifiable regions of your code and outside of as many profiling timers as possible, such as immediately before or after writing a checkpoint.

Also, since flush calls will print multiple, similar-looking outputs to `stdout`, it is also recommended to wrap any `BL_PROFILE_TINY_FLUSH();` calls in informative `amrex::Print()` lines to ensure accurate identification of each set of timers.

The results can also be redirected, e.g. to a file or to `stderr`, with the runtime parameter `tiny_profiler.output_file` (see *Runtime Parameters*).

The name of the innermost active profiler can be queried while the code runs with `amrex::TinyProfiler::CurrentName()`, e.g., to report where a simulation currently is from a watchdog thread or a signal handler. Without `AMReX_TINY_PROFILE=ON`, it returns `nullptr`.

Hot Spots and Load Balance

The output of TinyProfiler can help identify hot spots. For example, the following output shows the top three hot spots of a linear solver test running on 4 MPI processes.

```
--------------------------------------------------------------------------------------------
↪---
Name                                    NCalls  Excl. Min  Excl. Avg  Excl. Max  ␣
↪Max %
```

(continues on next page)

(continued from previous page)

```
------------------------------------------------------------------------------------------
↪---
MLPoisson::Fsmooth()                          560     0.4775     0.4793     0.4815  34.
↪97%
MLPoisson::Fapply()                           114     0.1103      0.113     0.1167   8.
↪48%
FabArray::Xpay()                              109        0.1     0.1013     0.1038   7.
↪54%
```

In this test, there are 16 boxes evenly distributed among 4 MPI processes. The output above shows that the load is perfectly balanced. However, if the load is not balanced, the results can be very different and sometimes misleading. For example, if we put 2, 2, 6 and 6 boxes on processes 0, 1, 2 and 3, respectively, the top three hot spots now include two MPI communication functions, `FillBoundary` and `ParallelCopy`.

```
------------------------------------------------------------------------------------------
↪---
Name                                    NCalls  Excl. Min  Excl. Avg  Excl. Max  ␣
↪Max %
------------------------------------------------------------------------------------------
↪---
FillBoundary_finish()                      607    0.01568     0.3367     0.6574  41.
↪97%
MLPoisson::Fsmooth()                       560     0.2133     0.4047     0.5973  38.
↪13%
FabArray::ParallelCopy_finish()            231   0.002977    0.09748     0.1895  12.
↪10%
```

The reason that the MPI communication appears slow is that the lightly loaded processes have to wait for messages sent by the heavily loaded processes. See also *Profiling Options* for a diagnostic option that may provide more insight on the load imbalance.

23.1.2 Full Profiling

If you set `PROFILE = TRUE` then a `bl_prof` directory will be written that contains detailed per-task timings for each processor. This will be written in `nfiles` files (where `nfiles` is specified by the user). The information in the directory can be analyzed by the *AMRProfParser* tool within *Amrvis*. In addition, an exclusive-only set of function timings will be written to `stdout`.

Trace Profiling

If you set `TRACE_PROFILE = TRUE` in addition to `PROFILE = TRUE`, then the profiler keeps track of when each profiled function is called and the `bl_prof` directory will include the function call stack. This is especially useful when core functions, such as `FillBoundary` can be called from many different regions of the code. Using trace profiling allows one to specify regions in the code that can be analyzed for profiling information independently from other regions.

Communication Profiling

If you set `COMM_PROFILE = TRUE` in addition to `PROFILE = TRUE`, then the `bl_prof` directory will contain additional information about MPI communication (point-to-point timings, data volume, barrier/reduction times, etc.). `TRACE_PROFILE = TRUE` and `COMM_PROFILE = TRUE` can be set together.

The AMReX-specific profiling tools are currently under development and this documentation will reflect the latest status in the development branch.

23.2 Instrumenting C++ Code

AMReX profiler objects are created and managed through BL_PROF macros.

To start, you must at least instrument main(), i.e.:

```
int main(...)
{
  amrex::Initialize(argc,argv);
  BL_PROFILE_VAR("main()",pmain);

  <AMReX code block>

  BL_PROFILE_VAR_STOP(pmain);
  amrex::Finalize();
}
```

Or:

```
void main_main()
{
    BL_PROFILE("main()");

    <AMReX code block>
}

int main(...)
{
    amrex::Initialize(argc,argv);
    main_main();
    amrex::Finalize();
}
```

You can then instrument any of your functions or code blocks. There are four general profiler macro types available:

23.2.1 1) A scoped timer, BL_PROFILE:

These timers generate their own object names, so they can't be controlled after being defined. However, they are the cleanest and easiest to work with in many situations. They time from the point where the macro is called until the end of the enclosing scope. This macro is ideal for timing an entire function. For example:

```
void YourClass::YourFunction()
{
  BL_PROFILE("YourClass::YourFunction()");   // Timer starts here.

  < Your Function Code Block>

}    // <------ Timer goes out of scope here, calling stop and returning the function␣
↪time.
```

Note that all AMReX timers are scoped and will call "stop" when the corresponding object is destroyed. This macro is unique because it can *only* stop when it goes out of scope.

23.2.2 2) A named, scoped timer, BL_PROFILE_VAR:

In some cases, using scopes to control a timer is not ideal. In such cases, you can use the _VAR_ macros to create a named timer that can be controlled through _START_ and _STOP_ macros. _VAR_ signifies that the macro takes a variable name. For example, to time a function without scoping:

```
BL_PROFILE_VAR("Flaten::FORT_FLATENX()", anyname);  // Create and start "anyname".
  FORT_FLATENX(arg1, arg2);
BL_PROFILE_VAR_STOP(anyname);   // Stop the "anyname" timer object.
```

This can also be used to selectively time within the same scope. For example, to include Func_0 and Func_2, but not Func_1:

```
BL_PROFILE_VAR("MyFuncs()", myfuncs);  // the first one
  MyFunc_0(args);
BL_PROFILE_VAR_STOP(myfuncs);

  MyFunc_1(args);

BL_PROFILE_VAR_START(myfuncs);
  MyFunc_2(arg);
BL_PROFILE_VAR_STOP(myfuncs);
```

Remember, these are still scoped. So, the scoped timer example can be reproduced exactly with named timers by just using the _VAR macro:

```
void YourClass::YourFunction()
{
  BL_PROFILE_VAR("YourClass::YourFunction()",  pmain);   // Timer starts here.

  < Your Function Code Block>

}    // <------ Timer goes out of scope here correctly, without a STOP call.
```

23.2.3 3) A named, scoped timer that doesn't auto-start, BL_PROFILE_VAR_NS:

Sometimes, a complicated scope may mean the profiling object needs to be defined before it's started. To create a named AMReX timer that doesn't start automatically, use the _NS_ macros. ("NS" stands for "no start"). For example, this implementation times MyFunc0 and MyFunc1 but not any of the "Additional Code" blocks:

```
{
    BL_PROFILE_VAR_NS("MyFuncs()", myfuncs);  // don't start the timer

    <Additional Code A>

    {
       BL_PROFILE_VAR_START(myfuncs);
         MyFunc_0(arg);
       BL_PROFILE_VAR_STOP(myfuncs);
    }

    <Additional Code B>

    {
```

(continues on next page)

(continued from previous page)

```
        BL_PROFILE_VAR_START(myfuncs);
          MyFunc_1(arg);
        BL_PROFILE_VAR_STOP(myfuncs);

        <Additional Code C>
    }
}
```

Note

The _NS_ macro must, by necessity, also be a _VAR_ macro. Otherwise, you would never be able to turn the timer on!

23.2.4 4) Designate a sub-region to profile, BL_PROFILE_REGION:

Often, it's helpful to look at a subset of timers separately from the complete profile. For example, you may want to view the timing of a specific time step or isolate everything inside the "Chemistry" part of the code. This can be accomplished by designating profile regions. All timers within a named region will be included both in the full analysis, as well as in a separate sub-analysis.

Regions are meant to be large contiguous blocks of code, and should be used sparingly and purposefully to produce useful profiling reports. As such, the possible region options are purposefully limited.

Scoped Regions

When using the Tiny Profiler, the only available region macro is the scoped macro. To create a region that profiles the *MyFuncs* code block, including all timers in the "Additional Code" regions, add macros in the following way:

```
{
    BL_PROFILE_REGION("MyFuncs");

    <Additional Code A>

    {
        BL_PROFILE("MyFunc0");

        MyFunc_0(arg);
    }

    <Additional Code B>

    {
        BL_PROFILE("MyFunc1");

        MyFunc_1(arg);
        <Additional Code C>
    }
}
```

The MyFuncs region appears in the Tiny Profiler output as an additional table. The following output example mimics the above code. In it, the region is indicated by REG::MyFuncs.

```
BEGIN REGION MyFuncs

----------------------------------------------------------------
Name          NCalls  Excl. Min  Excl. Avg  Excl. Max   Max %
----------------------------------------------------------------
MyFunc0         1000      4.402      4.402      4.402  14.19%
MyFunc1         1000       4.39       4.39       4.39  14.15%
REG::MyFuncs    1000     0.0168     0.0168     0.0168   0.05%
----------------------------------------------------------------

----------------------------------------------------------------
Name          NCalls  Incl. Min  Incl. Avg  Incl. Max   Max %
----------------------------------------------------------------
REG::MyFuncs    1000      8.809      8.809      8.809  28.39%
MyFunc0         1000      4.402      4.402      4.402  14.19%
MyFunc1         1000       4.39       4.39       4.39  14.15%
----------------------------------------------------------------

END REGION MyFuncs
```

Named Regions

If using the Full Profiler, named region objects are also available. Named regions allow control of start and stop points without relying on scope. These macros use slightly modified _VAR_, _START_ and _STOP_ formatting. The first argument is the name, followed by the profile variable. Names for each section can differ, but because the profiler variable will be used to group the sections into a region, it must be the same. Consider the following example:

```
{
    BL_PROFILE_REGION_VAR("RegionAC",reg_ac);
    <Code Block A>
    BL_PROFILE_REGION_VAR_STOP("RegionAC", reg_ac);

    {

       MyFunc_0(arg);
    }

    BL_PROFILE_REGION_VAR("RegionB", reg_b)
    <Code Block B>
    BL_PROFILE_REGION_VAR_STOP("RegionB", reg_b);

    {

       MyFunc_1(arg);

       BL_PROFILE_REGION_VAR_START("SecondRegionAC", reg_ac);
       <Code Block C>
       BL_PROFILE_REGION_VAR_STOP("SecondRegionAC", reg_ac);
    }
}
```

Here, <Code Block A> and <Code Block C> are grouped into one region labeled "RegionAC" for profiling. <Code Block B> is isolated in its own group. Any timers inside MyFunc_0 and MyFunc_1 are not included in the region

groupings.

23.2.5 GPU-synchronized TinyProfiler timers

GPU kernels normally launch asynchronously, so a host timer does not necessarily measure the work launched inside its scope. On GPU builds, TinyProfiler provides opt-in variants that synchronize the current AMReX GPU stream immediately before recording both the start and stop timestamps. Work on other GPU streams is not waited for:

- `BL_PROFILE_GPU_SYNC(name)`
- `BL_PROFILE_VAR_GPU_SYNC(name, variable)`
- `BL_PROFILE_VAR_NS_GPU_SYNC(name, variable)`
- `BL_PROFILE_REGION_GPU_SYNC(name)`

Variables created with `BL_PROFILE_VAR_NS_GPU_SYNC` are started and stopped with the ordinary `BL_PROFILE_VAR_START` and `BL_PROFILE_VAR_STOP` macros. On CPU builds these four macros are aliases for their ordinary variants. They are also ordinary aliases when the full profiler is selected, and are no-ops when profiling is disabled.

After any GPU-synchronized timer starts, TinyProfiler uses a focused report for the remainder of that AMReX initialization cycle. The focused report contains every timer that ran at the top level since `amrex::Initialize`, including those that finished before the first synchronized timer, timers created by the GPU-synchronized macros, and tables created by `BL_PROFILE_REGION_GPU_SYNC`. A synchronized timer that shares its name with an ordinary timer in the same table is shown with a `[sync]` suffix. Ordinary nested timers and ordinary region tables are omitted from the timer tables of focused flushes and the final report; memory profiler tables are not filtered. Activation is combined across all MPI processes, so every process uses the same report even if only one process executes a synchronized timer. Percentages remain relative to the total application run time.

The runtime parameter `tiny_profiler.device_synchronize_around_region` synchronizes all TinyProfiler timer boundaries. When it is enabled, the complete report is kept, even if GPU-synchronized macros are used.

23.3 Instrumenting Fortran90 Code

When using the full profiler, Fortran90 functions can also be instrumented with the following calls:

```
call bl_proffortfuncstart("my_function")
...
call bl_proffortfuncstop("my_function")
```

Note that the start and stop calls must be matched before leaving the scope of the corresponding start. Moreover, it is necessary to take into account all possible code paths. Therefore, you may need to add `bl_proffortfuncstop` in multiple locations, such as before any returns, at the end of the function and at the point in the function where you want to stop profiling. The profiling output will only warn of any `bl_proffortfuncstart` calls that were not stopped with `bl_proffortfuncstop` calls when in debug mode.

For functions with a high number of calls, there is a lighter-weight interface:

```
call bl_proffortfuncstart_int(n)
...
call bl_proffortfuncstop_int(n)
```

where `n` is an integer in the range `[1,mFortProfsIntMaxFuncs]`. `mFortProfsIntMaxFuncs` is currently set to 32. The profiled function will be named `FORTFUNC_n` in the profiler output, unless you rename it with `BL_PROFILE_CHANGE_FORT_INT_NAME(fname, int)` where `fname` is a `std::string` and `int` is the integer `n` in the `bl_proffortfuncstart_int/bl_proffortfuncstop_int` calls. `BL_PROFILE_CHANGE_FORT_INT_NAME` should be called in `main()`.

Warning

Fortran functions cannot be profiled when using the Tiny Profiler. You will need to turn on the Full Profiler to receive the results from fortran instrumentation.

23.4 Profiling Options

AMReX's communication algorithms are often regions of code that increase in wall-clock time when the application is load imbalanced, due to the MPI_Wait calls in these functions. To better understand if this is occurring and by how much, you can turn on an AMReX timed synchronization with the runtime variable `amrex.use_profiler_syncs=1`. This adds named timers beginning with `SyncBeforeComms` immediately prior to the start of the FillBoundary, ParallelCopy and particle Redistribute functions, isolating any prior load imbalance to that timer before beginning the comm operation.

This is a diagnostic tool and may slow your code down, so it is not recommended to turn this on for production runs.

Note

The `SyncBeforeComms` timer is not equal to your load imbalance. It only captures imbalance between the comm functions and the previous sync point; there may be other load imbalances captured elsewhere. Also, the timer reports in terms of MPI rank, so if the most imbalanced rank changes throughout the simulation, the timer will be an underestimation.

The effect on the communication timers may be more helpful: they will show the time to complete communications if there was no load imbalance. This means the difference between a case with and without this profiler sync may be a more useful metric for analysis.

23.5 AMRProfParser

`AMRProfParser` is a tool for processing and analyzing the `bl_prof` database. It is a command line application that can create performance summaries, plotfiles showing point-to-point communication and timelines, HTML call trees, communication call statistics, function timing graphs, and other data products. The parser's data services functionality can be called from an interactive environment such as *Amrvis*, from a sidecar for dynamic performance optimization, and from other utilities such as the command line version of the parser itself. It has been integrated into Amrvis for visual interpretation of the data, allowing Amrvis to open the `bl_prof` database like a plotfile but with interfaces appropriate to profiling data. AMRProfParser and Amrvis can be run in parallel both interactively and in batch mode.

CHAPTER

TWENTYFOUR

EXTERNAL PROFILING TOOLS

AMReX is compatible with the most commonly used profiling tools. This chapter provides selected documentation on implementing a few of these tools with AMReX. For additional details on running these tools, please refer to the official documentation of the tools.

24.1 CrayPat

The profiling suite available on Cray XC systems is Cray Performance Measurement and Analysis Tools ("CrayPat")[1]. Most CrayPat functionality is supported for all compilers available in the Cray "programming environments" (modules which begin "`PrgEnv-`"); however, a few features, chiefly the "Reveal" tool, are supported only on applications compiled with Cray's compiler CCE[2][3].

CrayPat supports both high-level profiling tools, as well as fine-grained performance analysis, such as reading hardware counters. The default behavior uses sampling to identify the most time-consuming functions in an application.

24.1.1 High-level application profiling

The simplest way to obtain a high-level overview of an application's performance consists of the following steps:

1. Load the `perftools-base` module, then the `perftools-lite` module. (The modules will not work if loaded in the opposite order.)
2. Compile the application with the Cray compiler wrappers `cc`, `CC`, and/or `ftn`. This works with any of the compilers available in the `PrgEnv-` modules. For example, on the Cori system at NERSC, one can use the Intel, GCC, or CCE compilers. No extra compiler flags are necessary in order for CrayPat to work. CrayPat instruments the application, so the `perftools-` modules must be loaded before one compiles the application.
3. Run the application as normal. No special flags are required. Upon application completion, CrayPat will write a few files to the directory from which the application was launched. The profiling database is a single file with the `.ap2` suffix.
4. One can query the database in many different ways using the `pat_report` command on the `.ap2` file. `pat_report` is available on login nodes, so the analysis need not be done on a compute node. Querying the database with no arguments to `pat_report` prints several different profiling reports to STDOUT, including a list of the most time-consuming regions in the application. The output of this command can be long, so it can be convenient to pipe the output to a pager or a file. A portion of the output from `pat_report <file>.ap2` is shown below:

```
Table 1:  Profile by Function
```

(continues on next page)

[1] https://pubs.cray.com/content/S-2376/6.4.6/cray-performance-measurement-and-analysis-tools-user-guide-646-s-2376
[2] https://pubs.cray.com/content/S-2179/8.5/cray-c-and-c++-reference-manual-85
[3] https://pubs.cray.com/content/S-3901/8.5/cray-fortran-reference-manual-85

(continued from previous page)

```
  Samp% |     Samp |  Imb. |  Imb. | Group
        |          |  Samp | Samp% |  Function
        |          |       |       |   PE=HIDE

 100.0% | 5,235.5 |    -- |    -- | Total
|-------------------------------------------------------------------------------
|  50.2% | 2,628.5 |    -- |    -- | USER
||------------------------------------------------------------------------------
||   7.3% |   383.0 |  15.0 |  5.0% | eos_module_mp_iterate_ne_
||   5.7% |   300.8 | 138.2 | 42.0% | amrex_deposit_cic
||   5.1% |   265.2 |  79.8 | 30.8% | update_dm_particles
||   2.8% |   147.2 |   5.8 |  5.0% | fort_fab_setval
||   2.6% |   137.2 |  48.8 | 34.9% | amrex::ParticleContainer<>::Where
||   2.6% |   137.0 |  11.0 |  9.9% | ppm_module_mp_ppm_type1_
||   2.5% |   133.0 |  24.0 | 20.4% | eos_module_mp_nyx_eos_t_given_re_
||   2.1% |   107.8 |  33.2 | 31.4% | amrex::ParticleContainer<>::IncrementWithTotal
||   1.7% |    89.2 |  19.8 | 24.2% | f_rhs_
||   1.4% |    74.0 |   7.0 | 11.5% | riemannus_
||   1.1% |    56.0 |   2.0 |  4.6% | amrex::VisMF::Write
||   1.0% |    50.5 |   1.5 |  3.8% | amrex::VisMF::Header::CalculateMinMax
||===============================================================================
|  28.1% | 1,471.0 |    -- |    -- | ETC
||------------------------------------------------------------------------------
||   7.4% |   388.8 |  10.2 |  3.4% | __intel_mic_avx512f_memcpy
||   6.9% |   362.5 |  45.5 | 14.9% | CVode
||   3.1% |   164.5 |   8.5 |  6.6% | __libm_log10_l9
||   2.9% |   149.8 |  29.2 | 21.8% | _INTERNAL_25_______src_kmp_barrier_cpp_
↪5de9139b::__kmp_hyper_barrier_gather
||===============================================================================
|  16.8% |   879.8 |    -- |    -- | MPI
||------------------------------------------------------------------------------
||   5.1% |   266.0 | 123.0 | 42.2% | MPI_Allreduce
||   4.2% |   218.2 | 104.8 | 43.2% | MPI_Waitall
||   2.9% |   151.8 |  78.2 | 45.4% | MPI_Bcast
||   2.6% |   135.0 |  98.0 | 56.1% | MPI_Barrier
||   2.0% |   105.8 |   5.2 |  6.3% | MPI_Recv
||===============================================================================
|   1.9% |    98.2 |    -- |    -- | IO
||------------------------------------------------------------------------------
||   1.8% |    93.8 |   6.2 |  8.3% | read
||===============================================================================
```

24.2 IPM - Cross-Platform Integrated Performance Monitoring

IPM provides portable profiling capabilities across HPC platforms, including support on selected Cray and IBM machines (cori and (TODO: verify it works on) summit). Running an IPM instrumented binary generates a summary of number of calls and time spent on MPI communication library functions. In addition, hardware performance counters can also be collected through PAPI.

Detailed instructions can be found at[4] and[5].

[4] http://ipm-hpc.sourceforge.net/userguide.html
[5] https://www.nersc.gov/users/software/performance-and-debugging-tools/ipm/

24.2.1 Building with IPM on Cori

Steps:

1. Run module load ipm.
2. Build code as normal with make.
3. Re-run the link command (e.g. cut-and-paste) with `$IPM` added to the end of the line.

24.2.2 Running with IPM on Cori

1. Set environment variables: `export IPM_REPORT=full IPM_LOG=full IPM_LOGDIR= <dir>`
2. Results will be printed to stdout, and an XML file will be generated in the directory specified by `IPM_LOGDIR`.
3. Post-process the XML with `ipm_parse -html <xmlfile>`, which produces a directory with HTML.

24.2.3 Summary MPI Profile

Example MPI profile output:

```
##IPMv2.0.5############################################################
#
# command   : /global/cscratch1/sd/cchan2/projects/lbl/BoxLib/Tests/LinearSolvers/C_
↪CellMG/./main3d.intel.MPI.OMP.ex.ipm inputs.3d.25600
# start     : Tue Aug 15 17:34:23 2017   host      : nid11311
# stop      : Tue Aug 15 17:34:35 2017   wallclock : 11.54
# mpi_tasks : 128 on 32 nodes            %comm     : 32.51
# mem [GB]  : 126.47                     gflop/sec : 0.00
#
#           :       [total]        <avg>          min          max
# wallclock :       1188.42         9.28         8.73        11.54
# MPI       :        386.31         3.02         2.51         4.78
# %wall     :
#   MPI     :                      32.52        24.36        41.44
# #calls    :
#   MPI     :       5031172        39306        23067        57189
# mem [GB]  :        126.47         0.99         0.98         1.00
#
#                              [time]        [count]        <%wall>
# MPI_Allreduce                225.72         567552          18.99
# MPI_Waitall                   92.84         397056           7.81
# MPI_Recv                      29.36            193           2.47
# MPI_Isend                     25.04        2031810           2.11
# MPI_Irecv                      4.35        2031810           0.37
# MPI_Allgather                  2.60            128           0.22
# MPI_Barrier                    2.24            512           0.19
# MPI_Gatherv                    1.70            128           0.14
# MPI_Comm_dup                   1.23            256           0.10
# MPI_Bcast                      1.14            256           0.10
# MPI_Send                       0.06            319           0.01
# MPI_Reduce                     0.02            128           0.00
# MPI_Comm_free                  0.01            128           0.00
# MPI_Comm_group                 0.00            128           0.00
# MPI_Comm_size                  0.00            256           0.00
# MPI_Comm_rank                  0.00            256           0.00
```

(continues on next page)

(continued from previous page)

```
# MPI_Init                      0.00            128           0.00
# MPI_Finalize                  0.00            128           0.00
```

The total, average, minimum, and maximum wallclock and MPI times across ranks is shown. The memory footprint is also collected. Finally, results include number of calls and total time spent in each type of MPI call.

24.2.4 PAPI Performance Counters

To collect performance counters, set `IPM_HPM=<list>`, where the list is a comma-separated list of PAPI counters. For example: `export IPM_HPM=PAPI_L2_TCA,PAPI_L2_TCM`.

For reference, here is the list of available counters on Cori, which can be found by running `papi_avail`:

```
    Name        Code    Avail Deriv Description (Note)
PAPI_L1_DCM  0x80000000  Yes   No   Level 1 data cache misses
PAPI_L1_ICM  0x80000001  Yes   No   Level 1 instruction cache misses
PAPI_L1_TCM  0x80000006  Yes   Yes  Level 1 cache misses
PAPI_L2_TCM  0x80000007  Yes   No   Level 2 cache misses
PAPI_TLB_DM  0x80000014  Yes   No   Data translation lookaside buffer misses
PAPI_L1_LDM  0x80000017  Yes   No   Level 1 load misses
PAPI_L2_LDM  0x80000019  Yes   No   Level 2 load misses
PAPI_STL_ICY 0x80000025  Yes   No   Cycles with no instruction issue
PAPI_BR_UCN  0x8000002a  Yes   Yes  Unconditional branch instructions
PAPI_BR_CN   0x8000002b  Yes   No   Conditional branch instructions
PAPI_BR_TKN  0x8000002c  Yes   No   Conditional branch instructions taken
PAPI_BR_NTK  0x8000002d  Yes   Yes  Conditional branch instructions not taken
PAPI_BR_MSP  0x8000002e  Yes   No   Conditional branch instructions mispredicted
PAPI_TOT_INS 0x80000032  Yes   No   Instructions completed
PAPI_LD_INS  0x80000035  Yes   No   Load instructions
PAPI_SR_INS  0x80000036  Yes   No   Store instructions
PAPI_BR_INS  0x80000037  Yes   No   Branch instructions
PAPI_RES_STL 0x80000039  Yes   No   Cycles stalled on any resource
PAPI_TOT_CYC 0x8000003b  Yes   No   Total cycles
PAPI_LST_INS 0x8000003c  Yes   Yes  Load/store instructions completed
PAPI_L1_DCA  0x80000040  Yes   Yes  Level 1 data cache accesses
PAPI_L1_ICH  0x80000049  Yes   No   Level 1 instruction cache hits
PAPI_L1_ICA  0x8000004c  Yes   No   Level 1 instruction cache accesses
PAPI_L2_TCH  0x80000056  Yes   Yes  Level 2 total cache hits
PAPI_L2_TCA  0x80000059  Yes   No   Level 2 total cache accesses
PAPI_REF_CYC 0x8000006b  Yes   No   Reference clock cycles
```

Due to hardware limitations, there is a limit to which counters can be collected simultaneously in a single run. Some counters may map to the same registers and thus cannot be collected at the same time.

24.2.5 Example HTML Performance Summary

Running `ipm_parse -html <xmlfile>` on the generated xml file will produce an HTML document that includes summary performance numbers and automatically generated figures. Some examples are shown here.

6470883

- Load Balance
- Communication Balance
- Message Buffer Sizes
- Communication Topology
- Switch Traffic
- Memmory Usage
- Executable Info
- Host List
- Environment
- Developer Info

powered by IPM

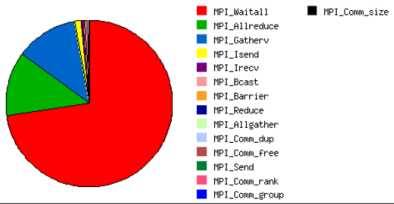
command: /global/cscratch1/sd/cchan2/projects/lbl/BoxLib/Tests/LinearSolvers/C_CellMG//main3d.intel.MPI.OMP.ex.ipm inputs.3d.15456			
codename:	unknown	state:	unknown
username:	cchan2	group:	
host:	nid04720 (x86_64_Linux)	mpi_tasks:	4 on 1 hosts
start:	08/22/17/12:04:29	wallclock:	5.20833e+01 sec
stop:	08/22/17/12:05:20	%comm:	4.99170175468912
total memory:	28.82264 gbytes	total gflop/sec:	0
switch(send):	0 gbytes	switch(recv):	0 gbytes

Computation

Event	Count	Pop
PAPI_L2_TCA	1884011012	*
PAPI_L2_TCM	447288209	*

Communication

% of MPI Time



HPM Counter Statistics

Event	Ntasks	Avg	Min(rank)	Max(rank)
PAPI_L2_TCA	*	471002753.00	418954739 (2)	558450816 (3)
PAPI_L2_TCM	*	111822052.25	83963410 (2)	172440952 (3)

Communication Event Statistics (100.00% detail, 6.9785e-06 error)

	Buffer Size	Ncalls	Total Time	Min Time	Max Time	%MPI	%Wall
MPI_Waitall	0	5360	5.774	1.788e-05	2.530e-02	55.53	2.77
MPI_Gatherv	114688	4	1.257	4.390e-01	4.390e-01	12.09	0.60
MPI_Allreduce	16	1908	0.684	1.288e-05	1.238e-02	6.58	0.33
MPI_Allreduce	8	10900	0.611	6.914e-06	3.187e-03	5.88	0.29
MPI_Waitall	4194304	302	0.435	1.044e-03	9.893e-03	4.18	0.21
MPI_Waitall	3670016	151	0.338	8.841e-04	1.869e-01	3.25	0.16
MPI_Waitall	2621440	151	0.176	8.039e-04	2.057e-02	1.69	0.08
MPI_Waitall	917504	135	0.160	3.550e-04	4.294e-02	1.54	0.08
MPI_Waitall	1048576	270	0.157	2.749e-04	3.719e-03	1.51	0.08
MPI_Waitall	65536	1838	0.112	5.198e-05	4.921e-04	1.07	0.05
MPI_Waitall	57344	919	0.095	5.293e-05	1.865e-02	0.92	0.05
MPI_Waitall	655360	135	0.082	2.880e-04	1.567e-02	0.79	0.04
MPI_Waitall	229376	135	0.066	1.240e-04	2.626e-02	0.64	0.03
MPI_Waitall	262144	270	0.063	1.280e-04	1.812e-03	0.61	0.03
MPI_Waitall	40960	919	0.062	4.482e-05	1.561e-02	0.60	0.03
MPI_Bcast	320	4	0.049	1.639e-02	1.639e-02	0.47	0.02
MPI_Waitall	163840	135	0.043	9.584e-05	1.981e-02	0.41	0.02
MPI_Isend	20480	3676	0.032	1.907e-06	9.990e-05	0.31	0.02
MPI_Barrier	0	16	0.027	4.540e-04	5.476e-03	0.26	0.01

Fig. 24.1: Sample performance summary generated by IPM

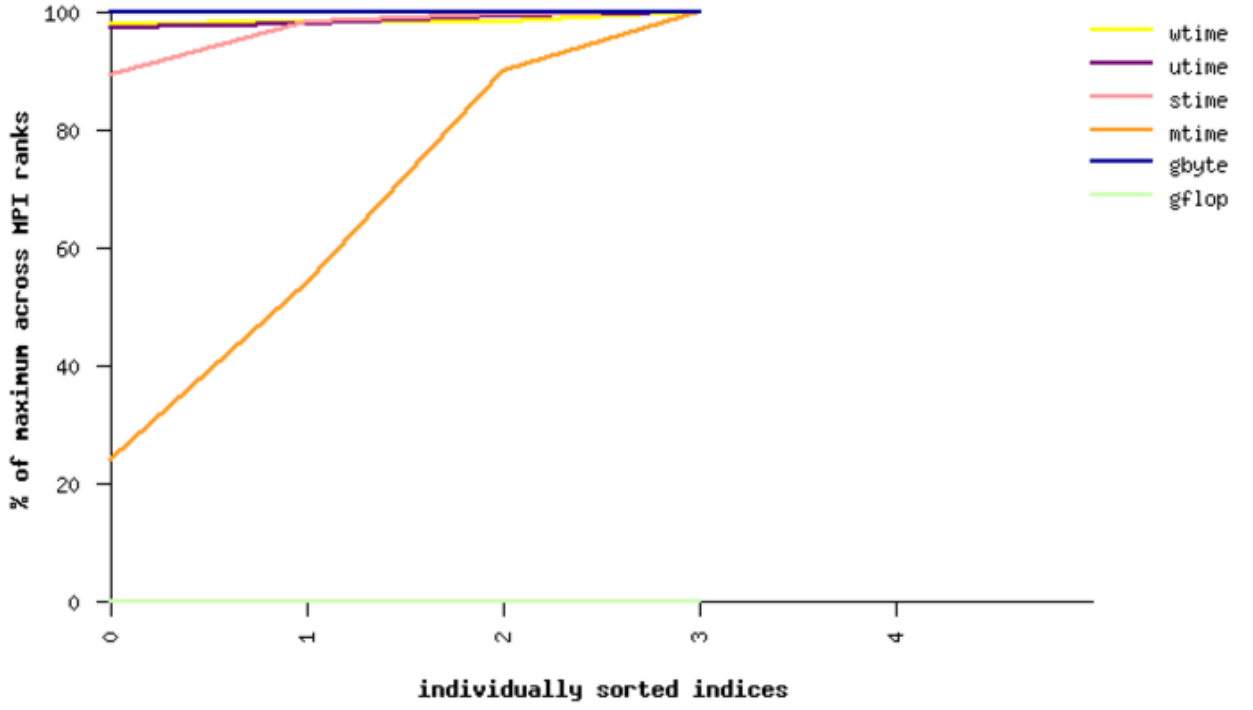
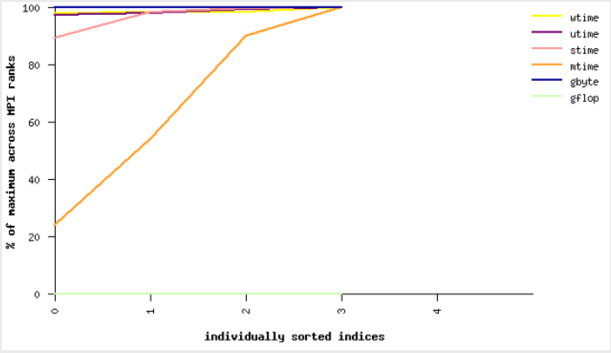
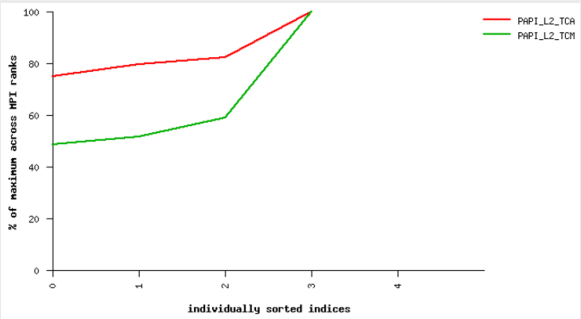


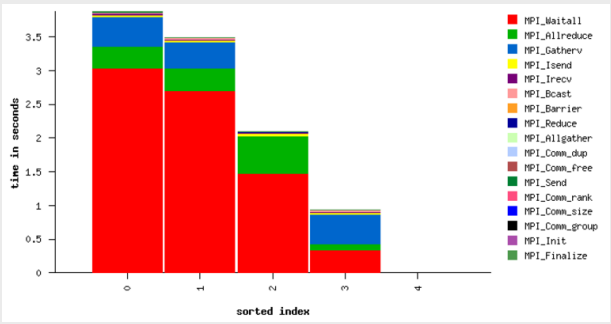
Table 24.1: Example of performance graphs generated by IPM



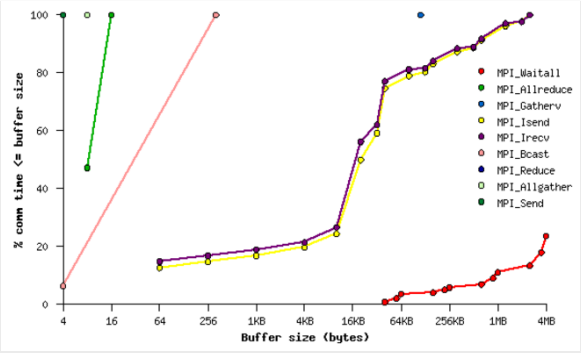
Timings



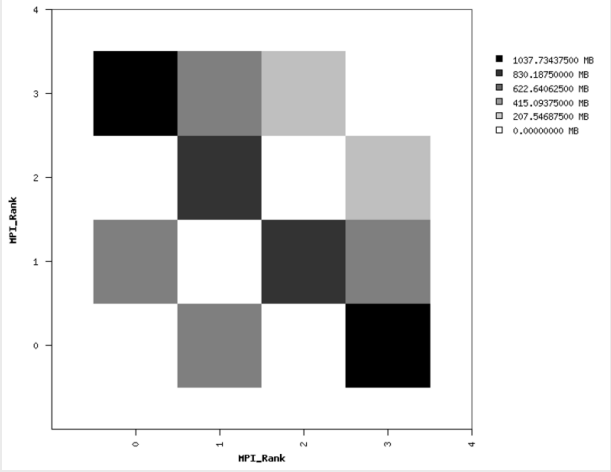
PAPI Counters



MPI Time by Function



MPI Time by Message Size



(left) Point-to-Point Communication Volume

24.3 Nsight Systems

The Nsight Systems tool provides a high-level overview of your code, displaying the kernel launches, API calls, NVTX regions and more in a timeline for a clear, visual picture of the overall runtime patterns. It analyzes CPU codes or CUDA-based GPU codes and is available on Summit and Cori in a system module.

Nsight Systems provides a variety of profiling options. This documentation will cover the most commonly used options for AMReX users to keep track of useful flags and analysis patterns. For the complete details of using Nsight Systems,

refer to the Nsight Systems official documentation.

24.3.1 Profile Analysis

The most common use case of Nsight Systems for AMReX users is the creation of a qdrep file that is viewed in the Nsight Systems GUI, typically on a local workstation or machine.

To generate a qdrep file, run nsys with the `-o` option:

```
nsys profile -o <file_name> ${EXE} ${INPUTS}
```

AMReX's lambda-based launch system often makes these timelines difficult to parse, as the kernels are mangled and difficult to decipher. AMReX's Tiny Profiler includes NVTX region markers, which can be used to mark the respective section of the Nsight Systems timeline. To include AMReX's built-in Tiny Profiler NVTX regions in Nsight Systems outputs, compile AMReX with `TINY_PROFILE=TRUE`.

Nsight Systems timelines only profile a single, contiguous block of time. There are a variety of methods to specify the specific region you would like to analyze. The most common options that AMReX users may find helpful are:

1. **Specify an NVTX region as the starting point of the analysis.**

This is done using `-c nvtx -p "region_name@*" -e NSYS_NVTX_PROFILER_REGISTER_ONLY=0`, where `region_name` is the identification string for the NVTX region. The additional environment variable, `-e ...` is needed because AMReX's NVTX region names currently do not use a registered string. TinyProfiler's built-in NVTX regions use the same identification string as the timer itself. For example, to start an analysis at the `do_hydro` NVTX region, run:

```
nsys profile -o <file_name> -c nvtx -p "do_hydro@*" -e NSYS_NVTX_PROFILER_REGISTER_
↪ONLY=0 ${EXE} ${INPUTS}
```

This will profile from the first instance of the specified NVTX region until the end of the application. In AMReX applications, this can be helpful to skip initialization and analyze the remainder of the code. To only analyze the specified NVTX region, add the flag `-x true`, which will end the analysis at the end of the region:

```
nsys profile -o <file_name> -c nvtx -p "do_hydro@*" -x true -e NSYS_NVTX_PROFILER_
↪REGISTER_ONLY=0 ${EXE} ${INPUTS}
```

Again, it's important to remember that Nsight Systems only analyzes a single contiguous block of time. So, this will only give you a profile for the first instance of the named region. Plan your Nsight Systems analyses accordingly.

2. **Specify a region with CUDA profiler function calls.**

This requires manually altering your source code, but can provide better specificity in what you analyze. Directly insert `cudaProfilerStart\Stop` around the region of code you want to analyze:

```
cudaProfilerStart();

// CODE TO PROFILE

cudaProfilerStop();
```

Then, run with `-c cudaProfilerApi`:

```
nsys profile -o <file_name> -c cudaProfilerApi ${EXE} ${INPUTS}
```

As with NVTX regions, Nsight Systems will only profile from the first call to `cudaProfilerStart()` to the first call to `cudaProfilerStop()`, so be sure to add these markers appropriately.

24.3.2 Nsight Systems GUI Tips

- When analyzing an AMReX application in the Nsight Systems GUI using NVTX regions or `TINY_PROFILE=TRUE`, AMReX users may find it useful to turn on the feature "Rename CUDA Kernels by NVTX". This will change the CUDA kernel names to match the inner-most NVTX region in which they were launched instead of the typical mangled compiler name. This will make identifying AMReX CUDA kernels in Nsight Systems reports considerably easier.

  This feature can be found in the GUI's drop-down menu, under:

```
Tools -> Options -> Environment -> Rename CUDA Kernels by NVTX.
```

24.4 Nsight Compute

The Nsight Compute tool provides a detailed, fine-grained analysis of your CUDA kernels, giving details about the kernel launch, occupancy, and limitations while suggesting possible improvements to maximize the use of the GPU. It analyzes CUDA-based GPU codes and is available on Summit and Cori in system modules.

Nsight Compute provides a variety of profiling options. This documentation will focus on the most commonly used options for AMReX users, primarily to keep track of useful flags and analysis patterns. For the complete details of using Nsight Compute, refer to the Nsight Compute official documentation.

24.4.1 Kernel Analysis

The standard way to run Nsight Compute on an AMReX application is to specify an output file that will be transferred to a local workstation or machine for viewing in the Nsight Compute GUI. Nsight Compute can be told to return a report file using the `-o` flag. In addition, when running with Nsight Compute on an AMReX application, it is important to turn off the floating-point exception trap, as it causes a runtime error. So, an entire AMReX application can be analyzed with Nsight Compute by running:

```
ncu -o <file_name> ${EXE} ${INPUTS} amrex.fpe_trap_invalid=0
```

However, this implementation should almost never be used by AMReX applications, as the analysis of every kernel would be extremely lengthy and unnecessary. To analyze a desired subset of CUDA kernels, AMReX users can use the Tiny Profiler's built-in NVTX regions to narrow the scope of the analysis. Nsight Compute allows users to specify which NVTX regions to include and exclude through the `--nvtx`, `--nvtx-include` and `--nvtx-exclude` flags. For example:

```
ncu --nvtx --nvtx-include "Hydro()/" --nvtx-exclude "StencilA(),StencilC()" -o kernels $
↪{EXE} ${INPUTS} amrex.fpe_trap_invalid=0
```

will return a file named `kernels` which contains an analysis of the CUDA kernels launched inside the `Hydro()` region, ignoring any kernels launched inside `StencilA()` and `StencilC()`. When using the NVTX regions built into AMReX's TinyProfiler, be aware that the application must be built with `TINY_PROFILE=TRUE` and the NVTX region names are identical to the TinyProfiler timer names. Note that the `/` must be appended to the TinyProfiler timer name specified with `--nvtx-include` because TinyProfiler sets NVTX push/pop regions, as described in the Nsight Compute official documentation on NVTX Filtering.

Another helpful flag for selecting a reasonable subset of kernels for analysis is the `-c` option. This flag specifies the total number of kernels to be analyzed. For example:

```
ncu --nvtx --nvtx-include "GravitySolve()/" -c 10 -o kernels ${EXE} ${INPUTS} amrex.fpe_
↪trap_invalid=0
```

will only analyze the first ten kernels inside the `GravitySolve()` NVTX region.

For further details on how to choose a subset of CUDA kernels to analyze, or to run a more detailed analysis, including CUDA hardware counters, refer to the Nsight Compute official documentation.

24.4.2 Roofline

As of version 2020.1.0, Nsight Compute has added the capability to perform roofline analysis on CUDA kernels to describe how well a given kernel is running on a given NVIDIA architecture. For details on the roofline capabilities in Nsight Compute, refer to the NVIDIA Kernel Profiling Guide.

To run a roofline analysis on an AMReX application, run `ncu` with the flag `--section SpeedOfLight_RooflineChart`. Again, using appropriate NVTX flags to limit the scope of the analysis will be critical to achieve results within a reasonable time. For example:

```
ncu --section SpeedOfLight_RooflineChart --nvtx --nvtx-include "MLMG()" -c 10 -o␣
↪roofline ${EXE} ${INPUTS} amrex.fpe_trap_invalid=0
```

will perform a roofline analysis of the first ten kernels inside the region `MLMG()`, and report their relative performance in the file `roofline`, which can be read by the Nsight Compute GUI.

For further information on the roofline model, refer to the scientific literature, Wikipedia overview, NERSC documentation and tutorials.

CHAPTER

TWENTYFIVE

EXTERNAL FRAMEWORKS

25.1 SUNDIALS

SUNDIALS stands for **SU**ite of **N**onlinear and **DI**fferential/**AL**gebraic equation **S**olvers. It consists of the following six solvers:

- CVODE, for initial value problems for ODE systems
- CVODES, solves ODE systems and includes sensitivity analysis
- ARKODE, solves initial value ODE problems with Runge-Kutta methods
- IDA, solves initial value problems for differential-algebraic equation systems
- IDAS, solves differential-algebraic equation systems and includes sensitivity analysis
- KINSOL, solves nonlinear algebraic systems

AMReX provides interfaces to the SUNDIALS suite. For time integration, users can refer to *Using SUNDIALS* for more information. In addition, an example code demonstrating time integration with SUNDIALS can be found in the SUNDIALS tutorial.

For more information on SUNDIALS, please see their readthedocs page.

CHAPTER

TWENTYSIX

REGRESSION TESTING

26.1 Continuous Compilation Testing

As a first line of testing, on every pull request and every commit to the repository, we verify that AMReX compiles and that its test suite passes for a common set of configuration options. This runs through GitHub Actions, covering GCC, Clang, Intel, CUDA, HIP, SYCL, macOS and Windows, and through HPSF GitLab CI on NVIDIA, AMD and Intel GPU hardware. This layer of testing is deliberately limited, so that it can be run quickly on every commit. For more extensive testing, we rely on the nightly regression results.

26.2 Nightly Regression Testing

Each night, we automatically run a suite of tests, both on AMReX itself and on most of the major application codes that use it as a framework. We use an in-house test runner script to manage this operation, originally developed by Michael Zingale for the Castro code, and later expanded to other application codes as well. The results for each night are collected and stored on a web page; see https://ccse.lbl.gov/pub/RegressionTesting/ for the latest set of results. The runtime option `amrex.abort_on_unused_inputs` (`0` or `1`; default is `0` for false) is useful for making sure that tests always stay up to date with API changes. It aborts the application after the test run if any unused input parameters are detected.

26.3 Running the test suite locally

The test suite is mostly used internally by AMReX developers. However, if you are making a pull request to AMReX, it can be useful to run the test suite on your local machine to reduce the likelihood that your changes break some existing functionality. To run the test suite locally, you must first obtain a copy of the test runner source, available on GitHub here: https://github.com/AMReX-Codes/regression_testing. The test runner requires Python 3.6 or later. Additional information on the test suite software can be found at https://amrex-codes.github.io/regression_testing/.

After obtaining the code, you will need a configuration file that defines which tests to run, which AMReX repository to test, which branch to use, etc. A sample configuration file for AMReX is distributed with the AMReX source code at `amrex/Tools/RegressionTesting/AMReX-tests.ini`. You will need to modify a few of the entries to, for example, point the test runner to the clone of AMReX on your local machine. Entries you will likely want to change include:

```
testTopDir = /path/to/test/output # the test results and benchmarks will be stored here
webTopDir  = /path/to/web/output  # a web page with the test results will be written here
```

to control where the generated output will be written, and

```
[AMReX]
dir = /path/to/amrex  # the path to the amrex repository you want to test
branch = "development"
```

to control which repository and branch to test.

The test runner is a Python script and can be invoked like so:

```
python regtest.py <options> AMReX-tests.ini
```

Before you can use it, you must first generate a set of "benchmarks", i.e., known "good" answers to the tests that will be run. If you are testing a pull request, you can generate these by running the script with a recent version of the `development` branch of AMReX. You can generate the benchmarks like so:

```
python regtest.py --make_benchmarks 'generating initial benchmarks' AMReX-tests.ini
```

Once that is finished, you can switch over to the branch you want to test in `AMReX-tests.ini`, and then re-run the script without the `--make_benchmarks` option:

```
python regtest.py AMReX-tests.ini
```

The script will generate a set of HTML pages in the directory specified in your `AMReX-tests.ini` file that you can examine using the browser of your choice.

For a complete set of script options, run

```
python regtest.py --help
```

A particularly useful option lets you run just a subset of the complete test suite. To run only one test, you can do:

```
python regtest.py --single_test <TestName> AMReX-tests.ini
```

To run an enumerated list of tests, do:

```
python regtest.py --tests '<TestName1> <TestName2> <TestName3>' AMReX-tests.ini
```

26.4 Adding a new test

New tests can be added to the suite by modifying the `AMReX-tests.ini` file. The easiest thing to do is start from an existing test and modify it. For example, this entry:

```
[MLMG_FI_PoisCom]
buildDir = Tests/LinearSolvers/ABecLaplacian_F
inputFile = inputs-rt-poisson-com
dim = 3
restartTest = 0
useMPI = 1
numprocs = 2
useOMP = 1
numthreads = 3
compileTest = 0
doVis = 0
outputFile = plot
testSrcTree = C_Src
```

defines a test called `MLMG_FI_PoisCom` by specifying the appropriate build directory, inputs file, and a set of configuration options. The above options are the most commonly changed; for a full list of options, see the example configuration file at https://github.com/AMReX-Codes/regression_testing/blob/main/example-tests.ini.

CHAPTER

TWENTYSEVEN

FREQUENTLY ASKED QUESTIONS

Q. Why am I getting a segmentation fault after my code runs?

A. Do you have `amrex::Initialize(); {` and `} amrex::Finalize();` at the beginning and end of your code? For all AMReX commands to function properly, including to release resources, they need to be contained between these two curly braces or in a separate function. In the Initialize and Finalize section, these commands are discussed in further detail.

Q. I want to use a different compiler with GNU Make to compile AMReX. How do I do this?

A. In the file `amrex/Tools/GNUMake/Make.local` you can specify your own compile commands by setting the variables `CXX`, `CC`, `FC`, and `F90`. An example can be found at Specifying your own compiler . Additional customizations are described in the file, `amrex/Tools/GNUMake/Make.local.template`. In the same directory, `amrex/Tools/GNUMake/README.md` contains detailed information on compiler commands.

Q. I'm having trouble compiling my code.

A. AMReX developers have found that running the command `make clean` can resolve many compilation issues.

If you are working in an environment that uses a module system, please ensure you have the correct modules loaded. Typically, to do this, type `module list` at the command prompt.

Q. When I profile my code that uses GPUs with `TINY_PROFILE=TRUE` or `PROFILE=TRUE` my timings are inconsistent.

A. Due to the asynchronous nature of GPU execution, profilers might only measure the runtime on the CPU if there is no explicit synchronization. For `TINY_PROFILE`, use the GPU-synchronized macros such as `BL_PROFILE_GPU_SYNC` for selected timers (see *GPU-synchronized TinyProfiler timers*), or the `ParmParse` parameter `tiny_profiler.device_synchronize_around_region=1` to synchronize all timers. Note that this may degrade performance.

Q. How do I know I am getting the right answer?

A. AMReX provides support for verifying output with several tools. To briefly mention a few:

- The `print_state` function can be used to output the data of a single cell.
- `VisMF::Write` can be used to write MultiFab data to disk that can be viewed with Amrvis.
- `amrex::Print()` and `amrex::AllPrint()` are useful for printing output when using multiple processes or threads as it prevents messages from getting mixed up.
- fcompare compares two plotfiles and reports absolute and relative error.

Additional tools and discussion on this topic is contained in the section Debugging.

Q. What's the difference between `Copy` and `ParallelCopy` for `MultiFab` data?

A. `MultiFab::Copy` is for two `MultiFabs` built with the same `BoxArray` and `DistributionMapping`, whereas `ParallelCopy` is for parallel communication of two `MultiFabs` with different `BoxArray` and/or `DistributionMapping`.

Q. How do I fill ghost cells?

A. See Ghost Cells in the AMReX Source Documentation.

Q. What's the difference between `AmrCore` and `AmrLevel`? How do I decide which to use?

A. The `AmrLevel` class is an abstract base class that holds data for a single AMR level. A vector of `AmrLevel` is stored in the `Amr` class, which is derived from `AmrCore`. An application code can derive from `AmrLevel` and override functions. `AmrCore` contains the meta-data for the AMR hierarchy, but it does not contain any floating point mesh data. Instead of using `Amr`/`AmrLevel`, an application can also derive from `AmrCore`. If you want flexibility, you might choose the `AmrCore` approach, otherwise the `AmrLevel` approach might be easier because it already has a lot of built-in capabilities that are common for AMR applications.

Q. For GPU usage, how can I perform explicit host to device and device to host copies without relying on managed memory?

A. Use `The_Pinned_Arena()` (See Memory Allocation in the AMReX Source Documentation.) and

```
void htod_memcpy (void* p_d, const void* p_h, const std::size_t sz);
void dtoh_memcpy (void* p_h, const void* p_d, const std::size_t sz);
void dtoh_memcpy (FabArray<FAB>& dst, FabArray<FAB> const& src, int scomp, int dcomp,␣
```

(continues on next page)

(continued from previous page)

```
↪int ncomp);
void htod_memcpy (FabArray<FAB>& dst, FabArray<FAB> const& src, int scomp, int dcomp,␣
↪int ncomp);
```

Q. How do I generate random numbers with AMReX? Can I set the seed? Are they thread safe with MPI and OpenMP?

A. (Thread safe) Yes, `amrex::Random()` is thread safe. When OpenMP is on, each thread will have its own dedicated Random Number Generator that is totally independent of the others. See *Random Numbers* for the available generators, how to set the seed, and how to draw random numbers inside a GPU kernel. Note in particular that `amrex::Random()` returns a value in `[0,1)` and can therefore return zero; use `amrex::RandomPositive()`, which returns a value in `(0,1]`, whenever the result is passed to something singular at zero such as `std::log`.

Q. Is Dirichlet boundary condition data loaded into cell-centered, or face-centered containers? How is it used in AMReX-based codes like MLMG and the advection routines in AMReX-Hydro?

A. In the cell-centered MLMG solver, the Dirichlet boundary data are stored in containers that have the information of the location of the data.

Q. How does coarse-grained OpenMP parallelism work in AMReX? How is it different from the fine-grained approach?

A. Our OpenMP strategy is explained in this paper, https://arxiv.org/abs/1604.03570.

Q. How do I avoid running into the `Formal parameter space overflowed` CUDA error while building complex EB geometries using AMReX implicit functions and CSG functionality?

A. AMReX enables logical operations and transformations to assemble basic shapes Implicit Functions into complex geometries. Each operation results in a more complex type which can eventually overflow the parameter space (4096 bytes on CUDA 11.4 for instance). To circumvent the problem, explicitly copy the object to the device and pass a device pointer function object *DevicePtrIF* into the *EB2* function using:

```
using IF_t = decltype(myComplexIF);
IF_t* dp = (IF_t*)The_Arena()->alloc(sizeof(myComplexIF));
Gpu::htod_memcpy_async(dp, &myComplexIF, sizeof(IF_t));
Gpu::streamSynchronize();
EB2::DevicePtrIF<IF_t> dp_myComplexIF{dp};
auto gshop = EB2::makeShop(dp_myComplexIF);
```

Q. I'm getting errors when running with GPU-aware MPI

A. While other problems may exist, one thing to check, if the machine is using Slurm, is the *cgroup.conf* file. If it contains *ConstrainDevices=yes*, then IPC can be impacted, which means bindings such as *–gpu-bind=closest* should not be used. Instead, try *–gpu-bind=none*. See *Assigning GPUs to MPI Ranks* for how to assign GPUs to MPI ranks in that case.

27.1 More Questions

If your question was not addressed here, you are encouraged to search and ask for help on the AMReX GitHub Discussions page.

The copyright notice for AMReX is included in the AMReX home directory as `README.md`. Your use of this software is subject to the 3-clause BSD license; the license agreement is included in the AMReX home directory as `LICENSE`.

For a PDF version of this documentation, click `here`.

BIBLIOGRAPHY

[Ruge1987] J. W. Ruge and K. Stüben, Algebraic multigrid, in *Multigrid Methods*, S. F. McCormick, ed., SIAM, Philadelphia, 1987, pp. 73-130, https://doi.org/10.1137/1.9781611971057.ch4.

[DeSterck2006] H. De Sterck, U. M. Yang and J. J. Heys, Reducing complexity in parallel algebraic multigrid preconditioners, *SIAM J. Matrix Anal. Appl.* 27 (2006), pp. 1019-1039, https://doi.org/10.1137/040615729.

[DeSterck2008] H. De Sterck, R. D. Falgout, J. W. Nolting and U. M. Yang, Distance-two interpolation for parallel algebraic multigrid, *Numer. Linear Algebra Appl.* 15 (2008), pp. 115-139, https://doi.org/10.1002/nla.559.

[Li2021] R. Li, B. Sjögreen and U. M. Yang, A new class of AMG interpolation methods based on matrix-matrix multiplications, *SIAM J. Sci. Comput.* 43 (2021), pp. S540-S564, https://doi.org/10.1137/20M134931X.

[Adams2003] M. Adams, M. Brezina, J. Hu and R. Tuminaro, Parallel multigrid smoothing: polynomial versus Gauss-Seidel, *J. Comput. Phys.* 188 (2003), pp. 593-610, https://doi.org/10.1016/S0021-9991(03)00194-3.

[Baker2011] A. H. Baker, R. D. Falgout, T. V. Kolev and U. M. Yang, Multigrid smoothers for ultraparallel computing, *SIAM J. Sci. Comput.* 33 (2011), pp. 2864-2887, https://doi.org/10.1137/100798806.

[Yang2010] U. M. Yang, On long-range interpolation operators for aggressive coarsening, *Numer. Linear Algebra Appl.* 17 (2010), pp. 453-472, https://doi.org/10.1002/nla.689.

INDEX

A

amr.blocking_factor (*built-in variable*), 248
amr.blocking_factor_x (*built-in variable*), 248
amr.blocking_factor_y (*built-in variable*), 248
amr.blocking_factor_z (*built-in variable*), 248
amr.check_file (*built-in variable*), 250
amr.check_input (*built-in variable*), 249
amr.check_int (*built-in variable*), 250
amr.check_per (*built-in variable*), 250
amr.checkpoint_files_output (*built-in variable*), 250
amr.checkpoint_nfiles (*built-in variable*), 251
amr.compute_new_dt_on_regrid (*built-in variable*), 250
amr.data_log (*built-in variable*), 252
amr.derive_plot_vars (*built-in variable*), 251
amr.derive_small_plot_vars (*built-in variable*), 252
amr.file_name_digits (*built-in variable*), 250
amr.force_regrid_level_zero (*built-in variable*), 250
amr.grid_eff (*built-in variable*), 248
amr.grid_log (*built-in variable*), 252
amr.initial_grid_file (*built-in variable*), 250
amr.max_grid_size (*built-in variable*), 248
amr.max_grid_size_x (*built-in variable*), 248
amr.max_grid_size_y (*built-in variable*), 248
amr.max_grid_size_z (*built-in variable*), 248
amr.max_level (*built-in variable*), 247
amr.message_int (*built-in variable*), 252
amr.n_cell (*built-in variable*), 247
amr.n_error_buf (*built-in variable*), 249
amr.n_error_buf_x (*built-in variable*), 249
amr.n_error_buf_y (*built-in variable*), 249
amr.n_error_buf_z (*built-in variable*), 249
amr.n_proper (*built-in variable*), 248
amr.no_box_split_dir (*built-in variable*), 249
amr.plot_file (*built-in variable*), 251
amr.plot_files_output (*built-in variable*), 251
amr.plot_int (*built-in variable*), 251
amr.plot_log_per (*built-in variable*), 251
amr.plot_max_level (*built-in variable*), 251
amr.plot_nfiles (*built-in variable*), 251
amr.plot_per (*built-in variable*), 251
amr.plot_vars (*built-in variable*), 251
amr.plotfile_on_restart (*built-in variable*), 250
amr.ref_ratio (*built-in variable*), 247
amr.ref_ratio_vect (*built-in variable*), 248
amr.refine_grid_layout (*built-in variable*), 249
amr.refine_grid_layout_x (*built-in variable*), 249
amr.refine_grid_layout_y (*built-in variable*), 249
amr.refine_grid_layout_z (*built-in variable*), 249
amr.refine_whole_domain_dir (*built-in variable*), 249
amr.regrid_file (*built-in variable*), 250
amr.regrid_int (*built-in variable*), 250
amr.regrid_on_restart (*built-in variable*), 250
amr.restart (*built-in variable*), 250
amr.run_log (*built-in variable*), 252
amr.run_log_terse (*built-in variable*), 252
amr.small_plot_file (*built-in variable*), 251
amr.small_plot_int (*built-in variable*), 251
amr.small_plot_log_per (*built-in variable*), 251
amr.small_plot_per (*built-in variable*), 251
amr.small_plot_vars (*built-in variable*), 252
amr.subcycling_mode (*built-in variable*), 250
amr.verbose (*built-in variable*), 247
amr.write_plotfile_with_checkpoint (*built-in variable*), 252
amrex.abort_on_out_of_gpu_memory (*built-in variable*), 258
amrex.abort_on_unused_inputs (*built-in variable*), 253
amrex.async_out (*built-in variable*), 257
amrex.async_out_nfiles (*built-in variable*), 257
amrex.device.verbose (*built-in variable*), 253
amrex.fpe_trap_invalid (*built-in variable*), 256
amrex.fpe_trap_overflow (*built-in variable*), 256
amrex.fpe_trap_zero (*built-in variable*), 256
amrex.handle_crt_reports (*built-in variable*), 256
amrex.handle_sigabrt (*built-in variable*), 255
amrex.handle_sigfpe (*built-in variable*), 255
amrex.handle_sigill (*built-in variable*), 255
amrex.handle_sigint (*built-in variable*), 255
amrex.handle_sigsegv (*built-in variable*), 255

amrex.handle_sigterm (*built-in variable*), 255
amrex.hypre_spgemm_use_vendor (*built-in variable*), 256
amrex.hypre_spmv_use_vendor (*built-in variable*), 256
amrex.hypre_sptrans_use_vendor (*built-in variable*), 256
amrex.init_hypre (*built-in variable*), 256
amrex.init_snan (*built-in variable*), 253
amrex.max_gpu_streams (*built-in variable*), 253
amrex.memory_log (*built-in variable*), 253
amrex.mf.alloc_single_chunk (*built-in variable*), 258
amrex.omp_threads (*built-in variable*), 253
amrex.parmparse.verbose (*built-in variable*), 253
amrex.signal_handling (*built-in variable*), 255
amrex.the_arena_defragmentation (*built-in variable*), 258
amrex.the_arena_init_size (*built-in variable*), 257
amrex.the_arena_is_managed (*built-in variable*), 258
amrex.the_arena_release_threshold (*built-in variable*), 257
amrex.the_comms_arena_defragmentation (*built-in variable*), 258
amrex.the_comms_arena_init_size (*built-in variable*), 257
amrex.the_comms_arena_release_threshold (*built-in variable*), 258
amrex.the_device_arena_defragmentation (*built-in variable*), 258
amrex.the_device_arena_init_size (*built-in variable*), 257
amrex.the_device_arena_release_threshold (*built-in variable*), 258
amrex.the_managed_arena_defragmentation (*built-in variable*), 258
amrex.the_managed_arena_init_size (*built-in variable*), 257
amrex.the_managed_arena_release_threshold (*built-in variable*), 258
amrex.the_pinned_arena_defragmentation (*built-in variable*), 258
amrex.the_pinned_arena_init_size (*built-in variable*), 257
amrex.the_pinned_arena_release_threshold (*built-in variable*), 258
amrex.throw_exception (*built-in variable*), 256
amrex.use_gpu_aware_mpi (*built-in variable*), 253
amrex.vector_growth_factor (*built-in variable*), 258
amrex.verbose (*built-in variable*), 252

D

DistributionMapping.strategy (*built-in variable*), 253
DistributionMapping.verbose (*built-in variable*), 253

E

eb2.cover_multiple_cuts (*built-in variable*), 255
eb2.extend_domain_face (*built-in variable*), 254
eb2.geom_type (*built-in variable*), 254
eb2.max_grid_size (*built-in variable*), 254
eb2.maxiter (*built-in variable*), 255
eb2.num_coarsen_opt (*built-in variable*), 254
eb2.parser_function (*built-in variable*), 254
eb2.small_volfrac (*built-in variable*), 254
eb2.stl_center (*built-in variable*), 254
eb2.stl_file (*built-in variable*), 254
eb2.stl_reverse_normal (*built-in variable*), 254
eb2.stl_scale (*built-in variable*), 254

F

fabarray.comm_tile_size (*built-in variable*), 259
fabarray.mfiter_tile_size (*built-in variable*), 259

G

geometry.coord_sys (*built-in variable*), 256
geometry.is_periodic (*built-in variable*), 257
geometry.prob_extent (*built-in variable*), 257
geometry.prob_hi (*built-in variable*), 256
geometry.prob_lo (*built-in variable*), 256

I

integration.abs_tol (*built-in variable*), 262
integration.fast_abs_tol (*built-in variable*), 262
integration.fast_rel_tol (*built-in variable*), 262
integration.fast_time_step (*built-in variable*), 261
integration.rel_tol (*built-in variable*), 262
integration.rk.extended_weights (*built-in variable*), 260
integration.rk.nodes (*built-in variable*), 260
integration.rk.tableau (*built-in variable*), 260
integration.rk.type (*built-in variable*), 260
integration.rk.weights (*built-in variable*), 260
integration.sundials.fast_linear_solver (*built-in variable*), 263
integration.sundials.fast_max_linear_iters (*built-in variable*), 263
integration.sundials.fast_max_nonlinear_iters (*built-in variable*), 263
integration.sundials.fast_method (*built-in variable*), 261
integration.sundials.fast_nonlinear_solver (*built-in variable*), 263
integration.sundials.fast_type (*built-in variable*), 261
integration.sundials.linear_solver (*built-in variable*), 263

integration.sundials.max_linear_iters (*built-in variable*), 263
integration.sundials.max_nonlinear_iters (*built-in variable*), 262
integration.sundials.max_num_steps (*built-in variable*), 262
integration.sundials.method (*built-in variable*), 261
integration.sundials.method_e (*built-in variable*), 261
integration.sundials.method_i (*built-in variable*), 261
integration.sundials.nonlinear_solver (*built-in variable*), 262
integration.sundials.stop_time (*built-in variable*), 262
integration.sundials.type (*built-in variable*), 261
integration.time_step (*built-in variable*), 260
integration.type (*built-in variable*), 259
integration.use_adaptive_fast_time_step (*built-in variable*), 262
integration.use_adaptive_time_step (*built-in variable*), 262

P

particles.do_mem_efficient_sort (*built-in variable*), 259
particles.do_tiling (*built-in variable*), 259
particles.particles_nfiles (*built-in variable*), 259
particles.redistribute_early_out (*built-in variable*), 259
particles.redistribute_mask_max_bytes (*built-in variable*), 259
particles.redistribute_mask_max_ratio (*built-in variable*), 259
particles.redistribute_use_mask (*built-in variable*), 259
particles.tile_size (*built-in variable*), 259

T

tiny_profiler.device_synchronize_around_region (*built-in variable*), 263
tiny_profiler.enabled (*built-in variable*), 263
tiny_profiler.memprof_enabled (*built-in variable*), 263
tiny_profiler.output_file (*built-in variable*), 264
tiny_profiler.print_threshold (*built-in variable*), 263
tiny_profiler.verbose (*built-in variable*), 263

V

vismf.verbose (*built-in variable*), 257